

КОНСПЕКТ ЛЕКЦИЙ

Комбинаторика и Теория Графов

(КиТГ)

Лектор: Казакевич В. Г.

Конспект: Котельников

Оглавление

1	Лекция 1. Бинарные отношения	2
2	Лекция 2. Отношения эквивалентности. Отношения порядка	4
3	Лекция 3. Строгий порядок. Замыкания. Основы теории графов	12
4	Лекция 4. Метрические характеристики графа. Способы представления. Связность	19
5	Лекция 5. Слабая и сильная связность. Граф Герца. Эйлеровость	28
6	Лекция 6. Гамильтоновы графы. Графы де Брёйна. Деревья	42
7	Лекция 7. Код Прюфера (обратная задача). Остовные деревья. Алгоритм Прима	54
8	Лекция 8. Алгоритм Краскала. Главные циклы. Цикловое пространство	61
9	Лекция 9. Двудольные графы. Паросочетания. Задача о максимальном потоке	68
10	Лекция 10. Алгоритм Форда—Фалкерсона	81
11	Лекция 11. Задача о максимальном потоке в сети	85
12	Лекция 12. Хроматический многочлен	92
13	Лекция 13. Поиск кратчайших путей во взвешенном графе	102
14	Лекция 14. Алгоритм Флойда. Алгоритм Джонсона	110
	Все определения	116
	Все теоремы и утверждения	132
	Все алгоритмы	142
	Все задачи	182

Лекция 1. Бинарные отношения

Определение 1. Бинарное отношение (через подмножество)

$M \neq \emptyset$. Бинарное отношение R на множестве M :

$$R \subseteq (M \times M).$$

Определение 2. Бинарное отношение (через функцию)

$M \neq \emptyset$. Бинарное отношение f на множестве M :

$$f: M^2 \rightarrow \mathbb{B}, \quad \text{где } 1 \text{ — принадлежность, } 0 \text{ — непринадлежность.}$$

Определения 1 и 2 эквивалентны: $f = \mathbb{I}_R$ — индикаторная функция.

Индикаторная функция. Пусть X — объемлющее множество, $A \subseteq X$. Тогда

$$\mathbb{I}_A: X \rightarrow \mathbb{B}, \quad \mathbb{I}_A(x) = \begin{cases} 1, & x \in A, \\ 0, & x \notin A. \end{cases}$$

Примеры (M — произвольное непустое множество):

1. $M = \mathbb{R}$, $\forall a, b \in M: f(a, b) = 1 \Leftrightarrow \{a\} = \{b\}$ (дробная часть).

Свойства: рефлексивно, симметрично, транзитивно.

2. $M = M_2(\mathbb{R})$ — матрицы 2×2 , $\forall a, b \in M: f(a, b) = 1 \Leftrightarrow \det A = \det B$.

Свойства: рефлексивно, симметрично, транзитивно.

3. M — имена присутствующих, $\forall a, b: f(a, b) = 1 \Leftrightarrow a \leq b$ (лексикографический порядок).

Свойства: рефлексивно, антисимметрично, транзитивно.

Простейшие свойства бинарных отношений.

Пусть $M \neq \emptyset$, $f: M^2 \rightarrow \mathbb{B}$ — б. о.

1. f на M — **рефлексивное** $\Leftrightarrow \forall a \in M: f(a, a) = 1$.

2. f на M — **арефлексивное** $\Leftrightarrow \forall a \in M: f(a, a) = 0$.

3. f на M — **симметричное** $\Leftrightarrow \forall a, b \in M: f(a, b) = 1 \Rightarrow f(b, a) = 1$.

4. f на M — **антисимметричное** $\Leftrightarrow \forall a, b \in M: (f(a, b) = 1 \wedge f(b, a) = 1) \Rightarrow a = b$.

5. f на M — **асимметричное** $\Leftrightarrow \forall a, b \in M: f(a, b) = 1 \Rightarrow f(b, a) = 0$.

6. f на M — **транзитивное** $\Leftrightarrow \forall a, b, c \in M: (f(a, b) = 1 \wedge f(b, c) = 1) \Rightarrow f(a, c) = 1$.

Упражнение: привести пример M , т. ч. не относится к свойствам 3–5 (не является ни симметричным, ни антисимметричным, ни асимметричным).

Пример (нетранзитивное отношение). $M = \{0, 1, 2, \dots, 20\}$, $f: M^2 \rightarrow \mathbb{B}$,

$$\forall a, b \in M: f(a, b) = 1 \Leftrightarrow |a - b| \leq 5.$$

Свойства: рефлексивно, симметрично, **не транзитивно** (контрпример: $f(a, b) = 1$, $f(b, c) = 1$, но $f(a, c) = 0$).

Далее условимся: M — дискретное и конечное множество.

Матричное представление бинарного отношения.

Пусть $M = \{a_1, a_2, \dots, a_n\}$, $f: M^2 \rightarrow \mathbb{B}$. Матрица A_f размера $n \times n$:

$$a_{ij} = f(a_i, a_j), \quad i \text{ — строка, } j \text{ — столбец.}$$

Пример. $M = \{07, 25, 52, 11, 20, 30\}$, $f(a, b) = 1 \Leftrightarrow |a - b| \leq 5$.

$$A_f =$$

$f(a, b)$	07	25	52	11	20	30
07	1	1	1	1	1	1
25	1	1	1	0	0	0
52	1	1	0	0	0	0
11	1	1	1	1	0	0
20	1	1	1	0	0	0
30	1	1	0	0	0	0

Строка i — элемент a , столбец j — элемент b ; на пересечении стоит $f(a, b)$ (единица, если $|a - b| \leq 5$).

Свойства б.о. через матрицу и граф.

$G = (V, E)$ — граф: V — множество вершин, E — рёбра (отношения между вершинами).

- **Р** (рефлексивность): на диагонали матрицы все единицы; в графе — петли у каждой вершины.
- **АР** (арефлексивность): на диагонали все нули; в графе — без петель.
- **С** (симметричность): $A = A^T$; в графе — если есть ребро (a, b) , то есть и (b, a) .
- **АнтиС** (антисимметричность): $a_{ij} = 1 \Rightarrow a_{ji} = 0$ (при $i \neq j$).
- **АС** (асимметричность): $a_{ij} = 1 \Rightarrow a_{ji} = 0$, и $\forall i: a_{ii} = 0$.
- **Т** (транзитивность): $(i, j) \in E, (j, k) \in E \Rightarrow (i, k) \in E$.

Упражнение: привести пример б.о., не удовлетворяющего ни одному свойству.

Упражнение: доказать, что асимметричность $\Rightarrow$ арефлексивность.

Лекция 2. Отношения эквивалентности. Отношения порядка

Определение 3. Отношение эквивалентности

$M \neq \emptyset, f: M^2 \rightarrow \mathbb{B}$.

f — **отношение эквивалентности** $\Leftrightarrow f$ рефлексивно, симметрично и транзитивно.

Пример. Список присутствующих: $f(a, b) = 1 \Leftrightarrow$ первая буква фамилии a совпадает с первой буквой фамилии b .

Все фамилии разбиваются на классы: А, Б, ...

Пример. $M = \mathbb{Z}$, операция сравнения по модулю n : $f(a, b) = 1 \Leftrightarrow a \equiv b \pmod{n}$.

Классы эквивалентности:

$$\mathcal{E} = \mathcal{E}_0 \cup \mathcal{E}_1 \cup \dots \cup \mathcal{E}_{n-1}.$$

Определение 4. Класс эквивалентности

$M \neq \emptyset, f$ — эквивалентность на M .

$\forall a \in M$ **класс эквивалентности** элемента a :

$$H_a := \{ c \in M \mid f(a, c) = 1 \}.$$

Теорема 1. Теорема о классификации

$M \neq \emptyset, f$ — эквивалентность на M . Тогда:

1. $\forall a \in M: a \in H_a$.
2. $\forall a, b \in M: H_a \cap H_b = \emptyset$ или $H_a = H_b$.
3. $M = \bigcup_i H_{a_i}$.

Доказательство. Напомним, что класс элемента a — это $H_a = \{x \in M: f(a, x) = 1\}$.

1) Каждый элемент лежит в своём классе. В силу рефлексивности $f(a, a) = 1$, поэтому $a \in H_a$; в частности, $H_a \neq \emptyset$.

2) Два класса либо не пересекаются, либо совпадают. Пусть $H_a \cap H_b \neq \emptyset$; покажем, что тогда $H_a = H_b$. Возьмём $c \in H_a \cap H_b$, то есть $f(a, c) = 1$ и $f(b, c) = 1$. Докажем включение $H_a \subseteq H_b$. Пусть $x \in H_a$, т. е. $f(a, x) = 1$. По симметричности из $f(a, c) = 1$ получаем $f(c, a) = 1$; тогда по транзитивности $f(c, a) = 1 \wedge f(a, x) = 1 \Rightarrow f(c, x) = 1$. Наконец, $f(b, c) = 1$ и $f(c, x) = 1$ дают по транзитивности $f(b, x) = 1$, то есть $x \in H_b$. Значит $H_a \subseteq H_b$; симметрично $H_b \subseteq H_a$, поэтому $H_a = H_b$. Итак, два класса либо совпадают, либо не пересекаются.

3) Классы покрывают всё множество. По пункту 1 каждый элемент $a \in M$ лежит в своём классе H_a , поэтому $M \subseteq \bigcup_{a \in M} H_a \subseteq M$, откуда $M = \bigcup_{a \in M} H_a$. Выбрав по одному представителю из каждого попарно различного класса (по пункту 2 различные классы не пересекаются), получаем разбиение $M = \bigsqcup_i H_{a_i}$ на непустые непересекающиеся классы. Это и есть искомая классификация. $\square$

Граф и матрица отношения эквивалентности.

Граф отношения эквивалентности распадается на полные подграфы (клики) — классы эквивалентности.

Пример. Вершины m_1, m_2 образуют один класс (полный подграф с петлями), вершины n_1, n_2, n_3, n_4 — другой класс (полный подграф с петлями).

Матрица смежности (в порядке $m_1, m_2, n_1, n_2, n_3, n_4$) имеет блочно-диагональный вид:

$$A_f = \begin{array}{c|cccccc} f & m_1 & m_2 & n_1 & n_2 & n_3 & n_4 \\ \hline m_1 & 1 & 1 & 0 & 0 & 0 & 0 \\ m_2 & 1 & 1 & 0 & 0 & 0 & 0 \\ \hline n_1 & 0 & 0 & 1 & 1 & 1 & 1 \\ n_2 & 0 & 0 & 1 & 1 & 1 & 1 \\ n_3 & 0 & 0 & 1 & 1 & 1 & 1 \\ n_4 & 0 & 0 & 1 & 1 & 1 & 1 \end{array}$$

Двойные линии выделяют блоки: матрица имеет блочно-диагональный вид, каждый блок — отдельный класс эквивалентности (вне блоков — нули).

Отношения порядка

Определение 5. Частичный (нестрогий) порядок

$M \neq \emptyset, f: M^2 \rightarrow \mathbb{B}$.

f — **частичный (нестрогий) порядок** $\Leftrightarrow f$ рефлексивно, антисимметрично и транзитивно.

Примеры частичного порядка.

1. $\mathbb{R}, \leq$ — обычный порядок на вещественных числах.
2. $M = \mathbb{B}^3$ (3-мерные двоичные векторы).

$$\alpha = a_1 a_2 a_3, \quad \beta = b_1 b_2 b_3.$$

$$\alpha \leq \beta \Leftrightarrow a_i \leq b_i, \quad 1 \leq i \leq 3.$$

3. $N = \{a, b\}$, M — все возможные подмножества N (т.е. $M = 2^N = \{\emptyset, \{a\}, \{b\}, \{a, b\}\}$).
 $f: M^2 \rightarrow \mathbb{B}, \quad \forall A, B \in M: f(A, B) = 1 \Leftrightarrow A \subseteq B$.

Отношение эквивалентности — для *классификации*, отношение порядка — для *иерархии*.

Граф (пример 3). $M = \{\emptyset, \{a\}, \{b\}, \{a, b\}\}$ — диаграмма Хассе. Независимые элементы ($\{a\}$ и $\{b\}$) — условия непонятно как соотносятся между собой.

Матрица смежности (в порядке $\emptyset, \{a\}, \{b\}, \{a, b\}$):

$$A_f = \begin{array}{c|cccc} f(a, b) & \emptyset & \{a\} & \{b\} & \{a, b\} \\ \hline \emptyset & 1 & 1 & 1 & 1 \\ \{a\} & 0 & 1 & 0 & 1 \\ \{b\} & 0 & 0 & 1 & 1 \\ \{a, b\} & 0 & 0 & 0 & 1 \end{array}$$

Определение 6. Линейный порядок

$M \neq \emptyset, f: M^2 \rightarrow \mathbb{B}$.

f — **линейный порядок** $\Leftrightarrow f$ — частичный порядок и $\forall a, b \in M: f(a, b) = 1 \vee f(b, a) = 1$.

Алгоритм дополнения частичного порядка до линейного.

Дано: $M \neq \emptyset$, $f: M^2 \rightarrow \mathbb{B}$, f — ч. п.

Построить: $\tilde{f}$ — линейный порядок на M , согласованный с f :

$$\forall a, b \in M: f(a, b) = 1 \Rightarrow \tilde{f}(a, b) = 1.$$

Способ не один!

DFS (поиск в глубину) — обход дерева/графа с использованием стека (FILO).

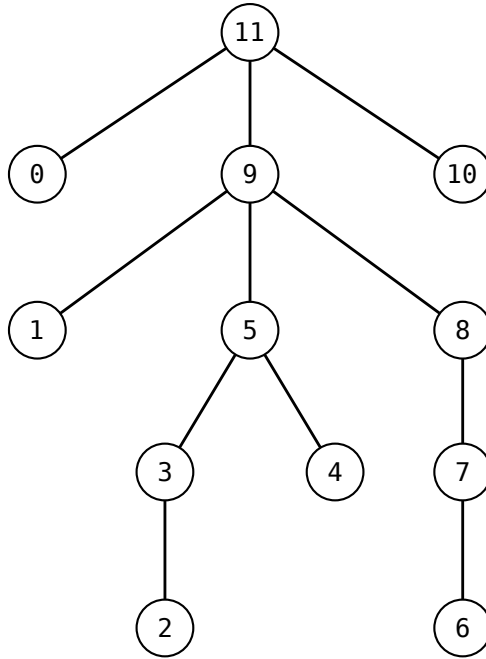


Рис. 1. Граф 12 вершин, нумерация DFS-обхода

BFS (поиск в ширину) — обход дерева/графа по уровням с использованием очереди (FIFO).

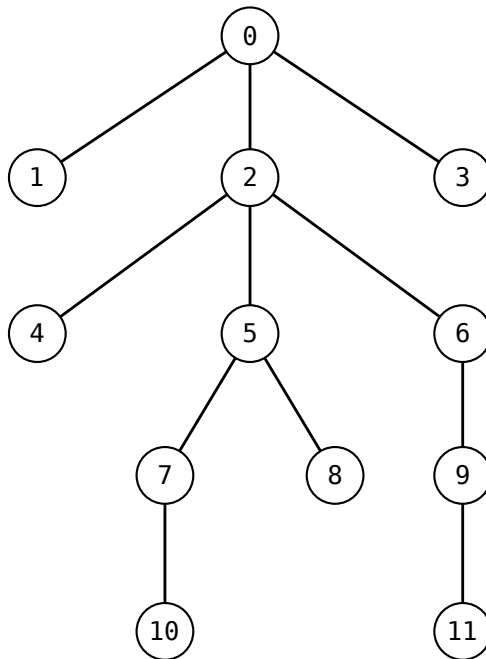


Рис. 2. Тот же граф, нумерация BFS-обхода

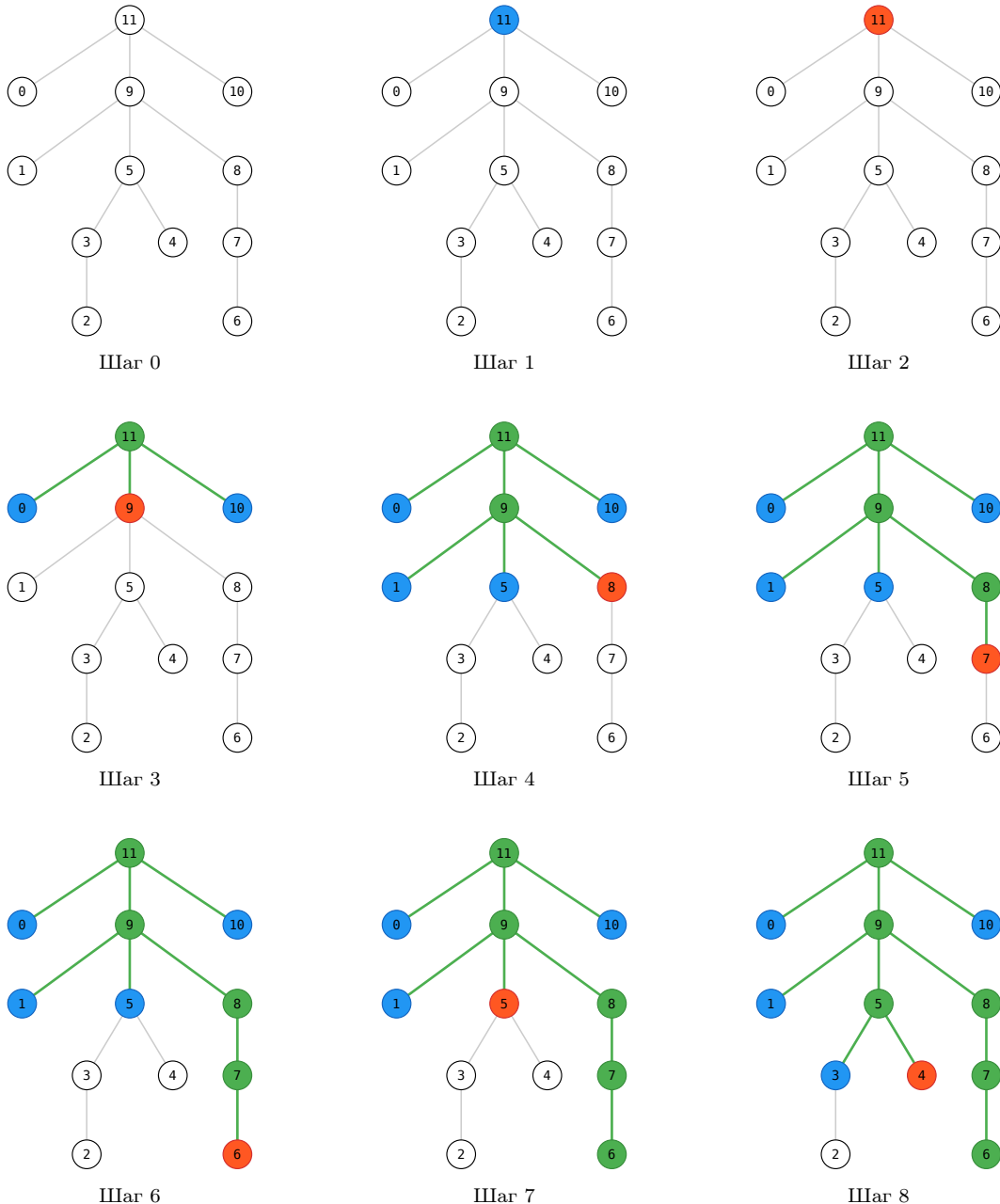
Обходы графа: поиск в глубину и в ширину

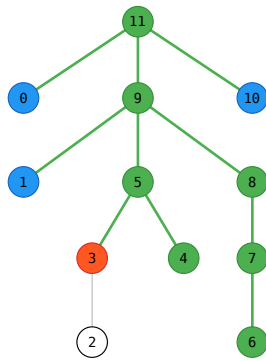
Алгоритм 1. Поиск в глубину (DFS) и поиск в ширину (BFS)

Поиск в глубину (DFS) использует стек: из текущей вершины уходим как можно глубже, и только упершись в тупик, возвращаемся назад. **Поиск в ширину (BFS)** использует очередь: сначала обрабатываются все соседи текущей вершины, затем соседи соседей — обход идёт «слоями».

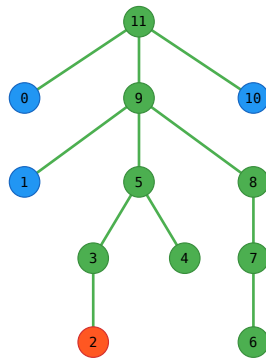
Цветовые обозначения на рисунках: текущая (активная) вершина — оранжевая, уже в стеке/очереди — синяя, обработанная — зелёная; рёбра дерева обхода выделены зелёным.

Пример работы DFS (старт из S):

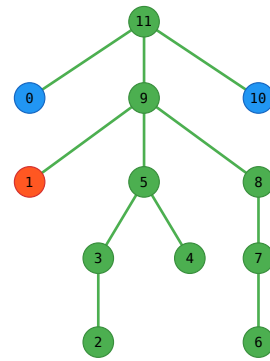




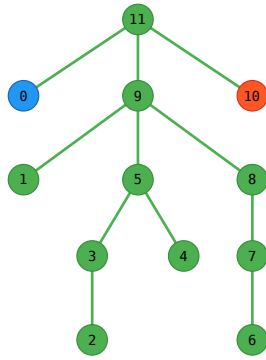
Шаг 9



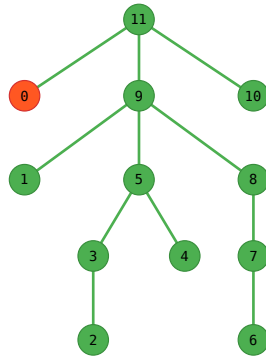
Шаг 10



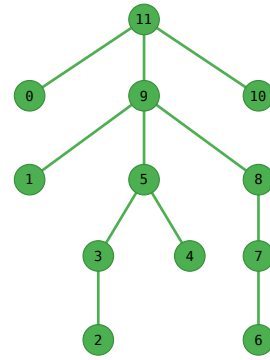
Шаг 11



Шаг 12

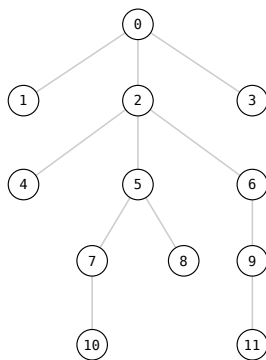


Шаг 13

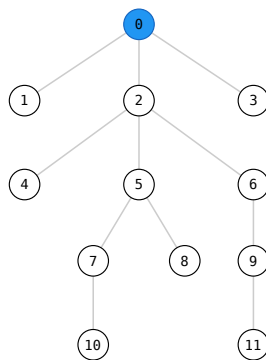


Шаг 14

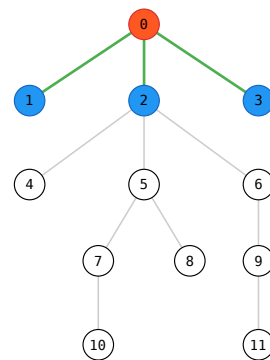
Пример работы BFS (старт из S):



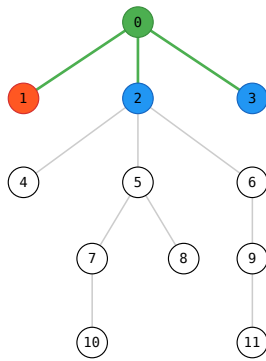
Шаг 0



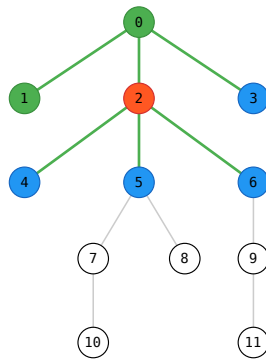
Шаг 1



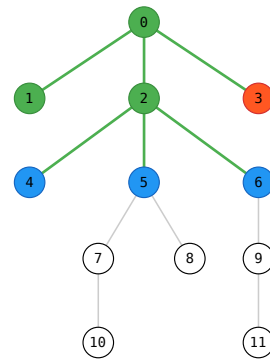
Шаг 2



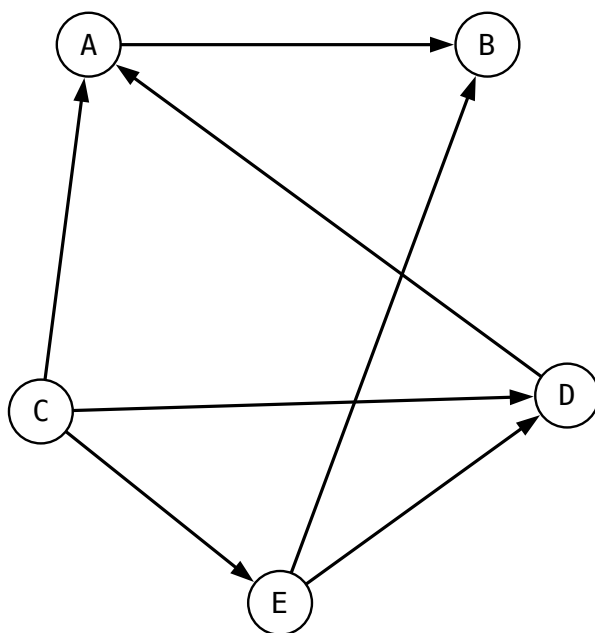
Шаг 3



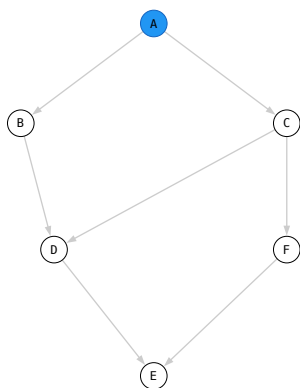
Шаг 4



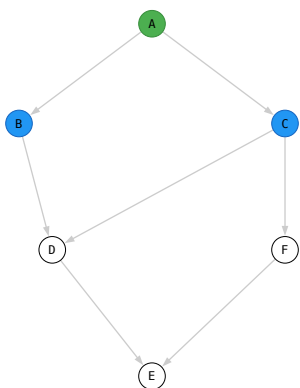
Шаг 5



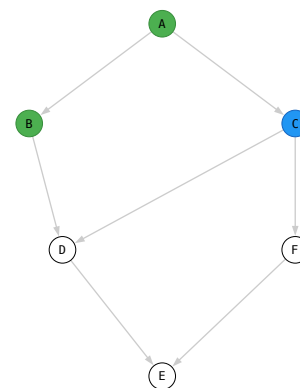
Активная ветка DFS — синяя, текущая вершина — оранжевая, тупиковые (уложенные в стек) — зелёные, рёбра дерева обхода — зелёные. Стек показан справа (верх стека — сверху).



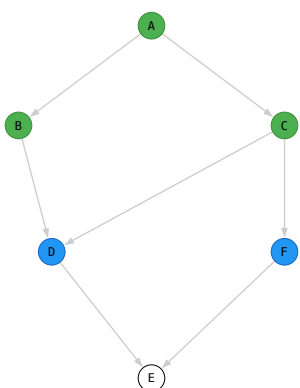
Шаг 0. Исходный граф, стек пуст



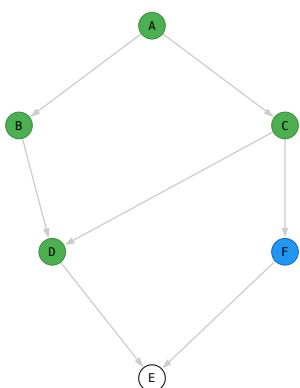
Шаг 1. DFS из A: идём $A \rightarrow B$



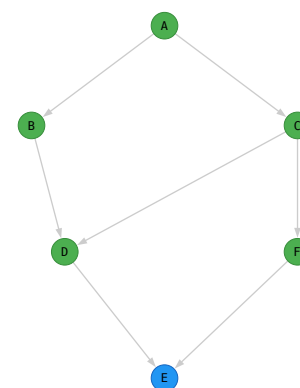
Шаг 2. B тупиковая $\rightarrow$ в стек



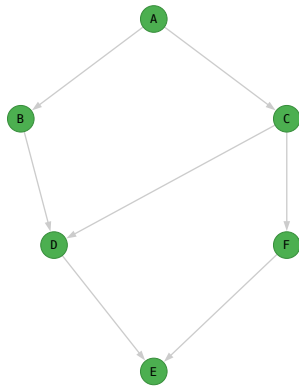
Шаг 3. A тупиковая $\rightarrow$ в стек



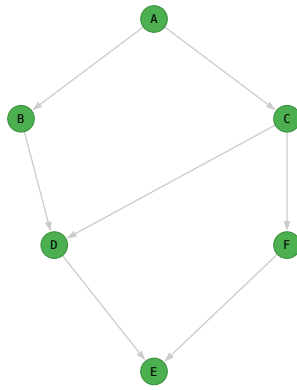
Шаг 4. DFS из C: $C \rightarrow D$; D $\rightarrow$ в стек



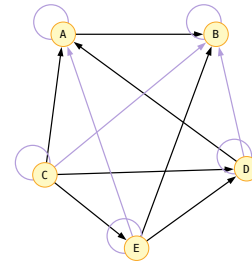
Шаг 5. C $\rightarrow$ E; E $\rightarrow$ в стек



Шаг 6. C тупиковая; шаг I завершён



Шаг 7. Нумерация по стеку + петли



Шаг 8. Недостающие рёбра

Стек:
$C = 5$
$E = 4$
$D = 3$
$A = 2$
$B = 1$

Комментарии к шагам. На шаге 1 запускаем DFS из A : единственное исходящее ребро ведёт в B . У B исходящих рёбер нет — она тупиковая, кладём её в стек (шаг 2); возвращаемся в A — все её рёбра обработаны, A тоже уходит в стек (шаг 3). Запускаем DFS из следующей непосещённой вершины C : ребро $C \rightarrow D$ приводит в D , откуда идёт только ребро $D \rightarrow A$ в уже посещённую вершину, поэтому D тупиковая (шаг 4). Далее $C \rightarrow E$: оба ребра из E ($E \rightarrow D$, $E \rightarrow B$) ведут в посещённые вершины — E в стек (шаг 5). Наконец тупиковой становится и C (шаг 6).

На шаге 7 нумеруем вершины по стеку снизу вверх: $B = 1$, $A = 2$, $D = 3$, $E = 4$, $C = 5$ — и дорисовываем рефлексивные петли (светло-фиолетовые). На шаге 8 добавляем недостающие рёбра из вершин с большими номерами в вершины с меньшими (тоже светло-фиолетовые: $D \rightarrow B$, $E \rightarrow A$, $C \rightarrow B$ и т. д.).

Стек (низ $\rightarrow$ верх): B, A, D, E, C .

Нумерация вершин: $B = 1$, $A = 2$, $D = 3$, $E = 4$, $C = 5$.

Линейный порядок: $B \leq A \leq D \leq E \leq C$.

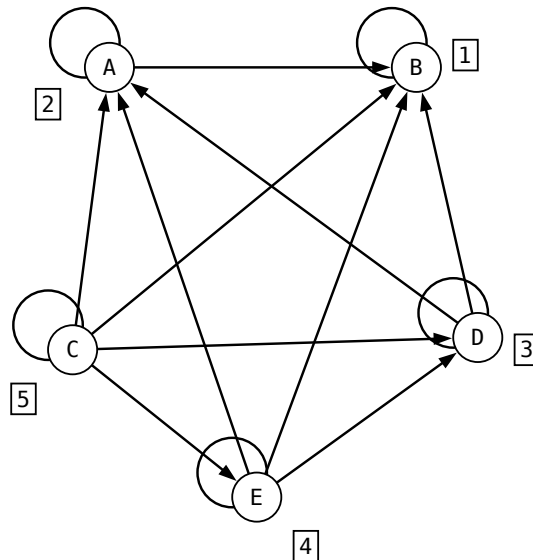


Рис. 3. Результат Topsort — линейный порядок $B \leq A \leq D \leq E \leq C$

Лекция 3. Строгий порядок. Замыкания. Основы теории графов

Определение 7. Строгий порядок

$M \neq \emptyset, f: M^2 \rightarrow \mathbb{B}$.

f — **строгий порядок** $\Leftrightarrow f$ асимметрично и транзитивно.

Пример: $\mathbb{R}, <$.

Задача 1.

Доказать, что асимметричность $\Rightarrow$ арефлексивность.

Решение. Асимметричность: $\forall a, b: f(a, b) = 1 \Rightarrow f(b, a) = 0$. Возьмём $b = a$: если бы нашлось a с $f(a, a) = 1$, то по асимметричности (при $b = a$) получили бы $f(a, a) = 0$ — противоречие. Значит $\forall a: f(a, a) = 0$, т. е. отношение арефлексивно. ■

Определение 8. Замыкание бинарного отношения

$M \neq \emptyset, f: M^2 \rightarrow \mathbb{B}, \tilde{f}: M^2 \rightarrow \mathbb{B}$.

Замыкание f относительно свойства $(*)$ — это $\tilde{f}$, т. ч.:

1. $\tilde{f}$ согласовано с f (т. е. $\forall a, b: f(a, b) = 1 \Rightarrow \tilde{f}(a, b) = 1$);
2. $\tilde{f}$ обладает свойством $(*)$;
3. $\tilde{f}$ имеет минимальный носитель среди всех б. о., удовлетворяющих 1) и 2) (минимальное количество рёбер).

Определение 9. Носитель функции

$f: A \rightarrow B$. **Носитель** f :

$$\text{supp}(f) := \{a \in A \mid f(a) \neq 0\}.$$

Замечание: замыкать можно только «вверх» (добавлять рёбра). Например, замыкание по арефлексивности невозможно — нельзя стирать рёбра.

Примеры замыканий.

1. **Рефлексивное замыкание:** граф $a \rightarrow b \rightarrow c$ — добавляем петли у каждой вершины.
2. **Симметричное замыкание:** $a \rightarrow b, b \rightarrow c$ — добавляем обратные рёбра ($b \rightarrow a, c \rightarrow b$).

ВВ: после рефлексивного и симметричного замыканий может нарушиться транзитивность: $a \rightarrow b$ и $b \rightarrow c$ есть, но $a \rightarrow c$ нет (транзитивная дуга отсутствует).

Алгоритм 3. Алгоритм Флойда–Уоршелла (построение транзитивного замыкания)

Вход: матрица смежности A_f б. о. f (размер $n \times n$).

Выход: матрица транзитивного замыкания $\tilde{A}$.

Шаг 1: $\forall v \in V$: рассматриваем v как «промежуточную» вершину.

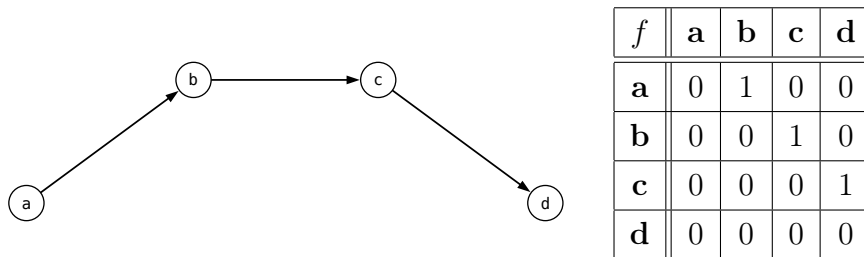
Шаг 2: Для каждой пары (i, j) : если $a_{iv} = 1$ и $a_{vj} = 1$, то полагаем $a_{ij} := 1$.

Повторяем для всех вершин v .

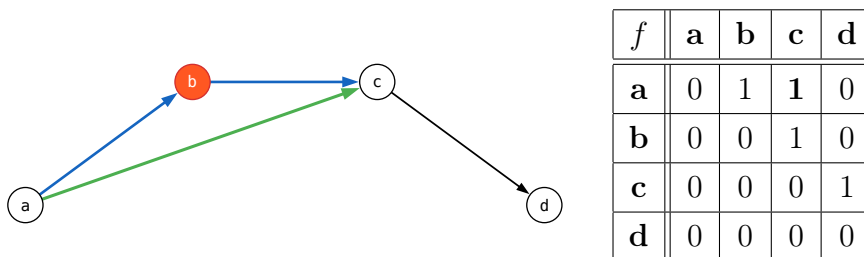
После дорисовывания графа получаем транзитивное замыкание.

Пример. Граф $a \rightarrow b \rightarrow c \rightarrow d$. Проследим работу алгоритма сразу в двух представлениях: на графе и на матрице смежности. Новые (дорисованные на текущем шаге) рёбра — зелёные, добавленные ранее — светло-фиолетовые; промежуточная вершина v — оранжевая. В матрицах новые единицы выделены жирным.

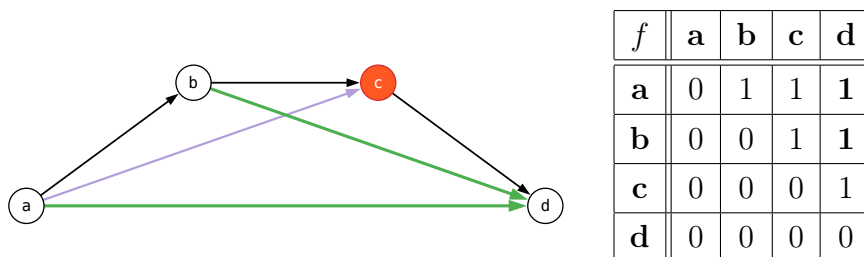
Исходный граф и его матрица ($v = a$ ничего не даёт: в a нет входящих рёбер):



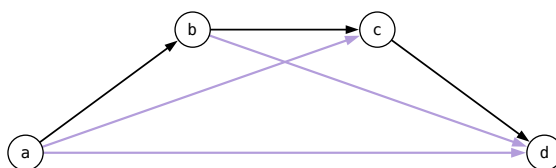
Промежуточная вершина $v = b$: есть $a \rightarrow b$ и $b \rightarrow c$, поэтому дорисовываем $a \rightarrow c$ (т. е. $a_{ac} := 1$):



Промежуточная вершина $v = c$: в c входят $a \rightarrow c$ и $b \rightarrow c$, из c выходит $c \rightarrow d$, поэтому дорисовываем $a \rightarrow d$ и $b \rightarrow d$:



Промежуточная вершина $v = d$ ничего не добавляет (из d нет исходящих рёбер). Итог — транзитивное замыкание $\tilde{f} = \{a \rightarrow b, b \rightarrow c, c \rightarrow d, a \rightarrow c, a \rightarrow d, b \rightarrow d\}$:



Основные понятия теории графов

$G(V, E)$ — **граф**, где V — множество вершин, E — множество рёбер.

Граф бывает:

- **неориентированный** (ненаправленный) — рёбра без направления;
- **ориентированный** (направленный) — хотя бы одно ориентированное ребро.

Рёбра: ориентированные / неориентированные. Также: петля, цикл, путь.

Связный граф. Компоненты связности. Дерево. Подграф.

Вершины:

- **Изолированная** — степени 0.
- **Висячая** — степени 1.

Рёбра:

- Петля — ребро из вершины в себя.
- Ориентированное ребро $a \rightarrow b$: a — родитель, b — потомок (только для ориент. графов).

Неориентированный ненаправленный граф — без дублирующих рёбер.

Ориентированный направленный граф — хотя бы одно ориент. ребро.

Смежность. Вершины a, b **смежные**, если между ними есть ребро.

e — ребро, **инцидентное** вершине a и вершине b .

Определение 10. Степень вершины

Степень вершины (для неор. графа) — количество инцидентных ей рёбер:

$$\deg(v).$$

Пусть $G(V, E)$, $|V| = n$. Тогда $0 \leq \deg(v) \leq n - 1$.

Регулярный граф — все вершины одинаковой степени.

В ориентированном графе:

- $\delta^+(v) = \#\{u \in V \mid \exists e: u \rightarrow v\}$ — количество входящих рёбер;
- $\delta^-(v) = \#\{u \in V \mid \exists e: v \rightarrow u\}$ — количество исходящих рёбер.

Путь из вершины A в вершину B — непрерывная последовательность рёбер, первое из которых начинается в A , последнее заканчивается в B .

Последовательность **непрерывная** — конец предыдущего ребра совпадает с началом следующего.

Простой путь — без повторяющихся рёбер.

Цепь — простой путь, в котором вершины тоже не повторяются.

Открытый путь — начало и конец различны. **Замкнутый путь** — начало и конец совпадают.

Цикл — замкнутый простой путь (без повторяющихся рёбер).

C_n — замкнутая цепь на n вершинах.

Определение 11. Полный граф

Полный граф K_n — граф, в котором любые две вершины смежные.

Определение 12. Дерево

Граф называется **деревом**, если он связный и ациклический.

T_n — дерево на n вершинах. При $n \geq 4$ дерево определено неоднозначно (существуют разные деревья с одинаковым числом вершин).

Определение 13. Лес

Граф называется **лесом**, если он ациклический (связность *не* требуется). Эквивалентно: лес — граф, каждая компонента связности которого является деревом; дерево — это связный лес.

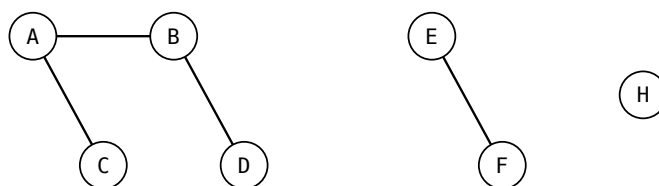


Рис. 4. Лес из трёх компонент: два дерева и изолированная вершина

Примеры. Граф на рисунке — лес из трёх деревьев: $\{A, B, C, D\}$, $\{E, F\}$ и одиночной вершины $\{H\}$ (изолированная вершина — тоже дерево). Если у леса n вершин и k компонент, то рёбер ровно $n - k$ (по $n_i - 1$ в каждой компоненте). Другой пример: любой ациклический подграф любого графа — лес; в частности, остовное дерево без одного ребра — лес из двух деревьев.

Определение 14. Связный граф

Связный граф — между любыми двумя вершинами $\exists$ путь.

Определение 15. Компонента связности

Компонента связности — максимальный по включению связный подграф.

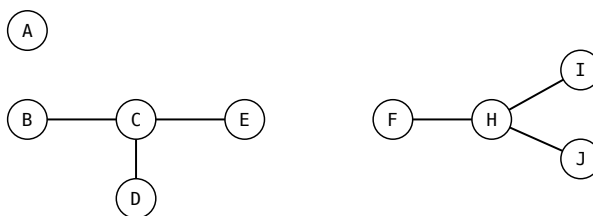


Рис. 5. 3 компоненты связности (9 вершин)

Теорема 2. Лемма о рукопожатиях

$G(V, E)$ — неориентированный граф. Тогда:

$$\sum_{v \in V} \deg(v) = 2|E|.$$

Доказательство. Посчитаем пары (v, e) , где ребро e инцидентно вершине v , двумя способами. С одной стороны, каждая вершина v участвует ровно в $\deg(v)$ таких парах, всего $\sum_{v \in V} \deg(v)$. С другой стороны, каждое ребро $e = \{u, w\}$ инцидентно ровно двум вершинам, т. е. участвует ровно в двух парах; всего $2|E|$. Значит, $\sum_v \deg(v) = 2|E|$. $\square$

Теорема 3. Лемма о чётности

$G(V, E)$ — неориентированный граф. Тогда количество вершин нечётной степени чётно:

$$\#\{v \in V \mid \deg(v) \not\equiv 2\} \equiv 2.$$

Доказательство. Разобьём вершины на два множества: $V_{\text{ч}}$ — вершины чётной степени и $V_{\text{н}}$ — вершины нечётной степени. По лемме о рукопожатиях

$$\underbrace{\sum_{v \in V_{\text{ч}}} \deg(v)}_{\text{чётно}} + \sum_{v \in V_{\text{н}}} \deg(v) = 2|E| \quad \text{— чётно,}$$

поэтому $\sum_{v \in V_{\text{н}}} \deg(v)$ тоже чётно. Но это сумма *нечётных* слагаемых, а сумма k нечётных чисел чётна тогда и только тогда, когда k чётно. Значит, $|V_{\text{н}}|$ чётно. $\square$

Теорема 4. Лемма о числе рёбер полного графа

$$|E_{K_n}| = \frac{n(n-1)}{2}.$$

Доказательство. В K_n любые две различные вершины соединены ровно одним ребром, поэтому рёбра взаимно однозначно соответствуют неупорядоченным парам вершин: $|E_{K_n}| = \binom{n}{2} = \frac{n(n-1)}{2}$. Иначе: каждая из n вершин имеет степень $n-1$, и по лемме о рукопожатиях $|E| = \frac{1}{2} \sum_v \deg(v) = \frac{n(n-1)}{2}$. $\square$

Задача 2.

Доказать: если $G(V, E)$ — связный граф, $|V| = n$, $|E| = n-1$, то G — дерево.

Решение. Дерево — связный ациклический граф, и связность дана, поэтому достаточно показать, что в G нет циклов. Предположим противное: в G есть цикл. Удалим любое ребро e этого цикла — граф $G' = G - e$ останется связным (любой путь, проходивший через e , можно провести в обход по оставшейся части цикла), и в нём $n-2$ ребра. Будем повторять: пока в графе остаются циклы, удаляем по одному ребру цикла, сохраняя связность. Процесс конечен и закончится связным ациклическим графом (деревом) T на тех же n вершинах, в котором *меньше* чем $n-1$ рёбер. Но любой связный граф на n вершинах содержит не менее $n-1$ ребра (см. лемму о связности ниже: каждая новая вершина при построении связного графа требует хотя бы одного нового ребра) — противоречие. Значит, циклов в G не было, и G — дерево. ■

Теорема 5. Лемма о связности

$G(V, E)$ — неориентированный связный граф, $|V| = n$. Тогда:

$$(n - 1) \leq |E| \leq \frac{n(n - 1)}{2}.$$

Доказательство. Верхняя оценка. Рёбер не может быть больше, чем всех неупорядоченных пар вершин, т. е. $|E| \leq \binom{n}{2} = \frac{n(n-1)}{2}$ (максимум достигается на полном графе K_n).

Нижняя оценка (индукция по n). База: при $n = 1$ имеем $|E| \geq 0 = n - 1$. Переход: пусть утверждение верно для всех связных графов с числом вершин меньше n . Возьмём связный граф G на $n \geq 2$ вершинах и удалим из него вершину v минимальной степени вместе с инцидентными ей рёбрами (их $\deg(v) \geq 1$, так как изолированных вершин в связном графе при $n \geq 2$ нет). Граф может распасться на $k \leq \deg(v)$ компонент связности с числом вершин $n_1, \dots, n_k$, $n_1 + \dots + n_k = n - 1$. По предположению индукции в i -й компоненте не менее $n_i - 1$ рёбер, итого

$$|E| \geq \deg(v) + \sum_{i=1}^k (n_i - 1) = \deg(v) + (n - 1) - k \geq n - 1,$$

поскольку $k \leq \deg(v)$. Оценка точна: у дерева ровно $n - 1$ ребро. □

Лекция 4. Метрические характеристики графа. Способы представления. Связность

$G(V, E)$ — неориентированный, связный граф.

Определение 16. Расстояние между вершинами

$\forall u, v \in V$:

$$\text{dist}(u, v) := \min l,$$

где l — длина пути $u \rightarrow \dots \rightarrow v$.

Свойства:

- $\text{dist}(u, u) = 0$;
- $\text{dist}(u, v) = \text{dist}(v, u)$.

Определение 17. Эксцентриситет вершины

$v \in V$:

$$\varepsilon(v) := \max_{u \in V} \{ \text{dist}(u, v) \}.$$

Определение 18. Радиус графа

$$R(G) := \min_{v \in V} \{ \varepsilon(v) \}.$$

Определение 19. Диаметр графа

$$D(G) := \max_{v \in V} \{ \varepsilon(v) \}.$$

Определение 20. Центр графа

$$\text{Cent}(G) := \{ u \in V \mid \varepsilon(u) = R(G) \}.$$

Пример 1.

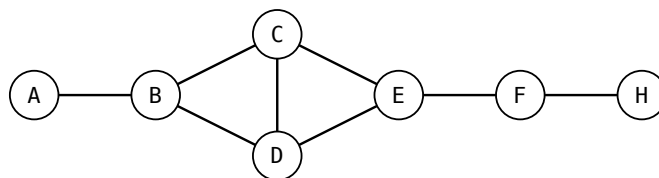


Рис. 6. Метрики: 7 вершин, $R = 3$, $D = 5$

Как считаются эксцентриситеты: для каждой вершины выписываем, какие вершины достижимы ровно за 1, 2, 3, ... шагов (поиск в ширину); номер последнего непустого столбца и есть $\varepsilon(v)$ — расстояние до самой дальней вершины.

верш.	за 1 шаг	за 2 шага	за 3 шага	за 4 шага	за 5 шагов	ε
A	B	C, D	E	F	H	5
B	A, C, D	E	F	H		4
C	B, D, E	A, F	H			3
D	B, C, E	A, F	H			3
E	C, D, F	B, H	A			3
F	E, H	C, D	B	A		4
H	F	E	C, D	B	A	5

Эксцентриситеты: $\varepsilon(A) = 5$, $\varepsilon(B) = 4$, $\varepsilon(C) = 3$, $\varepsilon(D) = 3$, $\varepsilon(E) = 3$, $\varepsilon(F) = 4$, $\varepsilon(H) = 5$.

Минимум эксцентриситетов даёт радиус, максимум — диаметр: $R(G) = 3$, $D(G) = 5$, $\text{Cent}(G) = \{C, D, E\}$.

Пример 2.

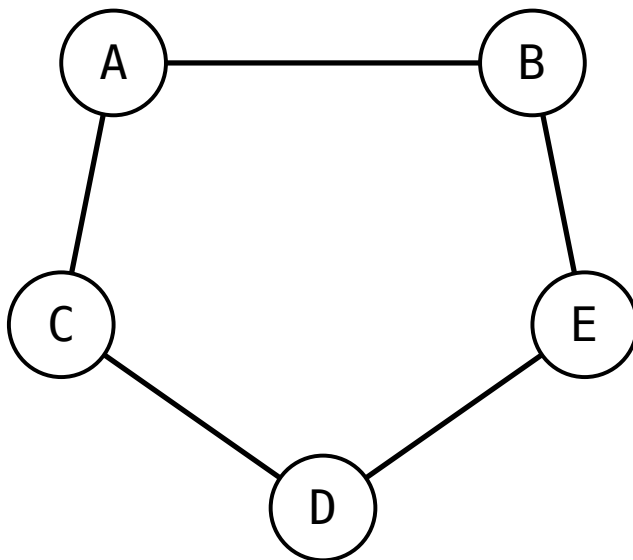


Рис. 7. Метрики: цикл C_5 , $R = D = 2$

Цикл C_5 : $A-B-E-D-C-A$.

Здесь всё симметрично, достаточно посчитать для одной вершины: например, от A за 1 шаг достижимы соседи B и C , за 2 шага — остальные две вершины E и D , поэтому $\varepsilon(A) = 2$; по симметрии цикла так же для всех остальных вершин.

Эксцентриситеты: $\varepsilon(A) = \varepsilon(B) = \varepsilon(C) = \varepsilon(D) = \varepsilon(E) = 2$.

$R(G) = 2$, $D(G) = 2$, $\text{Cent}(G) = \{A, B, C, D, E\} = V$.

Полный граф: $D(K_n) = 1$, $R(K_n) = 1$, $\text{Cent}(K_n) = V_{K_n}$.

Вычисление метрических характеристик графа.

Алгоритм (I): Для дерева (без циклов): $D(G) = \max \text{длина цепи}$, $R(G) = \left\lceil \frac{D(G)}{2} \right\rceil$.

Пример (дерево).

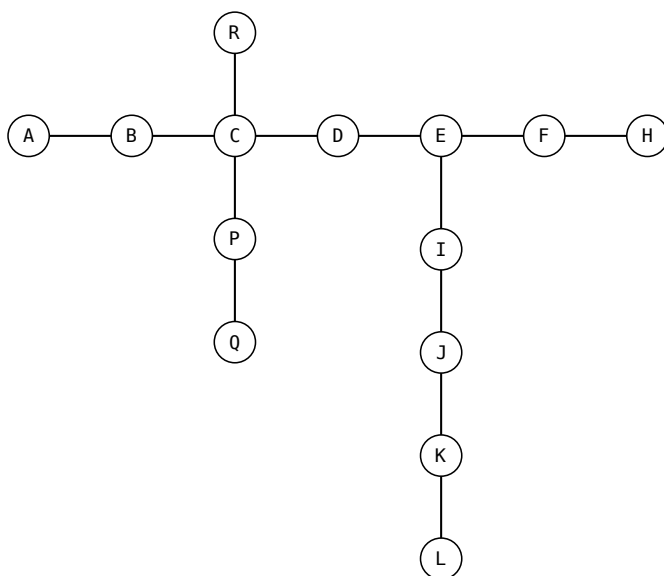


Рис. 8. Метрики: дерево 14 вершин

Для дерева не нужно считать все расстояния: ищем самую длинную цепь (*диаметральную цепь*). Здесь это, например,

$$A - B - C - D - E - I - J - K - L$$

длины 8 (другая такая же: $Q - P - C - D - E - I - J - K - L$). Следовательно $D(G) = 8$ и $R(G) = \lceil 8/2 \rceil = 4$, а центр — середина диаметральной цепи, вершина E : она удалена ровно на 4 от обоих концов цепи (A и L), и дальше чем на 4 от неё ничего нет (проверка: за 1 шаг от E достижимы D, F, I ; за 2 — C, H, J ; за 3 — B, R, P, K ; за 4 — A, Q, L , т.е. $\varepsilon(E) = 4$).

Эксцентриситеты (для проверки): $\varepsilon(A) = 8$, $\varepsilon(B) = 7$, $\varepsilon(C) = 6$, $\varepsilon(D) = 5$, $\varepsilon(E) = 4$, $\varepsilon(F) = 5$, $\varepsilon(H) = 6$, $\varepsilon(I) = 5$, $\varepsilon(J) = 6$, $\varepsilon(K) = 7$, $\varepsilon(L) = 8$, $\varepsilon(R) = 7$, $\varepsilon(P) = 7$, $\varepsilon(Q) = 8$.

$$R(G) = 4, \quad D(G) = 8, \quad \text{Cent}(G) = \{E\}.$$

(II) Граф содержит хотя бы один цикл.

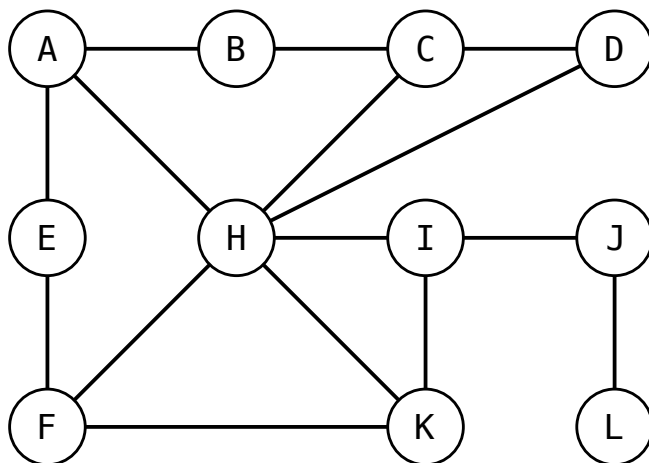


Рис. 9. Метрики: 11 вершин с циклами

Списки смежности:
верш. | смежные

A	B, E, H
B	A, C
C	B, D, H
D	C, H
E	A, F
F	E, H, K
H	A, C, D, F, I, K
I	H, J, K
J	I, L
K	F, H, I
L	J

Таблица достижимости:

верш.	за 1 шаг	за 2 шага	за 3 шага	за 4 шага	за 5 шагов
<i>A</i>	<i>B, E, H</i>	<i>C, D, F, I, K</i>	<i>J</i>	<i>L</i>	<i>L</i>
<i>B</i>	<i>A, C</i>	<i>D, E, H</i>	<i>F, I, K</i>	<i>J</i>	
<i>C</i>	<i>B, D, H</i>	<i>A, F, I, K</i>	<i>E, J</i>	<i>L</i>	
<i>D</i>	<i>C, H</i>	<i>A, B, F, I, K</i>	<i>E, J</i>	<i>L</i>	<i>L</i>
<i>E</i>	<i>A, F</i>	<i>B, H, K</i>	<i>C, D, I</i>	<i>J</i>	
<i>F</i>	<i>E, H, K</i>	<i>A, C, D, I</i>	<i>B, J</i>	<i>L</i>	
<i>H</i>	<i>A, C, D, F, I, K</i>	<i>B, E, J</i>	<i>L</i>	<i>B, E</i>	
<i>I</i>	<i>H, J, K</i>	<i>A, C, D, F, L</i>	<i>B, E</i>		
<i>J</i>	<i>I, L</i>	<i>H, K</i>	<i>A, C, D, F</i>		
<i>K</i>	<i>F, H, I</i>	<i>A, C, D, E, J</i>	<i>B, L</i>	<i>A, C, D, F</i>	<i>B, E</i>
<i>L</i>	<i>J</i>	<i>I</i>	<i>H, K</i>		

Эксцентриситеты: $\varepsilon(A) = 4$, $\varepsilon(B) = 5$, $\varepsilon(C) = 4$, $\varepsilon(D) = 4$, $\varepsilon(E) = 5$, $\varepsilon(H) = 3$, $\varepsilon(I) = 3$, $\varepsilon(J) = 4$, $\varepsilon(F) = 4$, $\varepsilon(K) = 3$, $\varepsilon(L) = 5$.

$R(G) = 3$, $D(G) = 5$, $\text{Cent}(G) = \{H, I, K\}$.

Способы представления графа

1. **Аналитический** (через $\mathbb{B}$): бинарная функция $f: V^2 \rightarrow \mathbb{B}$.
2. **Матрица смежности**.

Пример (неориент.): граф a – b – c :

$$A = \begin{array}{c|ccc} & A & a & b & c \\ \hline a & & 0 & 1 & 0 \\ b & & 1 & 0 & 1 \\ c & & 0 & 0 & 0 \end{array}$$

Для ориентированного графа: начало ребра = -1 , конец ребра = $+1$.

3. **Матрица инцидентности**.

Строки — вершины, столбцы — рёбра.

Для неориент. графа: инцидентная вершина = 1, иначе 0.

Для ориент. графа ($a \rightarrow b$, $b \rightarrow c$):

$$B = \begin{array}{c|cc} & B & ab & bc \\ \hline a & & -1 & 0 \\ b & & +1 & -1 \\ c & & 0 & +1 \end{array}$$

4. **Список инцидентности (список смежности)**.

a – b – c : $a: b$; $b: c, a$; $c: \dots$

Связность и k -связность. Мосты

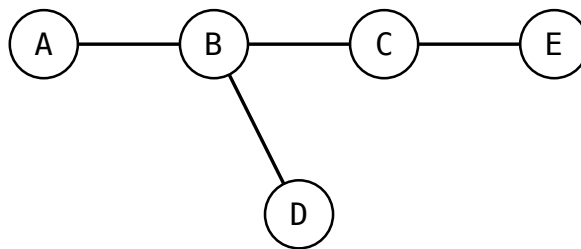
$G(V, E)$ — неориентированный граф.

Определение 21. k -связность

G — k -связный $\Leftrightarrow$:

1. $\forall e_{i_1}, \dots, e_{i_{k-1}} \in E$: $G(V, E \setminus \{e_{i_1}, \dots, e_{i_{k-1}}\})$ — связный (удаление любых $k - 1$ рёбер не нарушает связность);
2. $\exists e_{j_1}, \dots, e_{j_k} \in E$, т. ч. $G(V, E \setminus \{e_{j_1}, \dots, e_{j_k}\})$ — несвязный (существует набор из k рёбер, удаление которого нарушает связность).

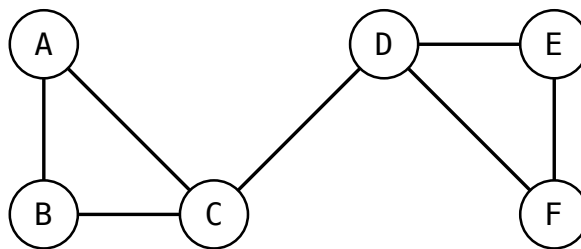
Примеры:



1.

Рис. 10. k -связность: 1-связный (дерево)

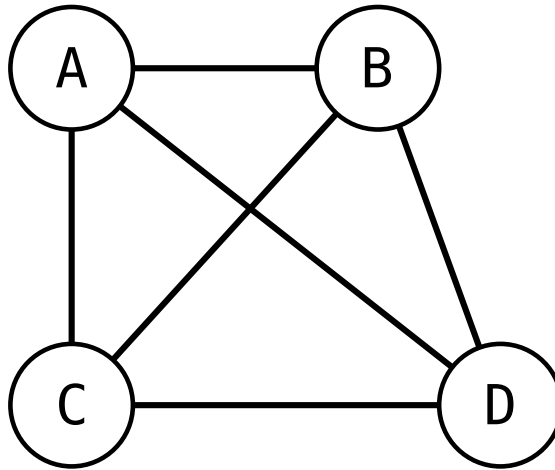
Дерево $A-B-C-E$, $B-D$ — **1-связный** (удаление любого ребра разрушает связность).



2.

Рис. 11. k -связность: 1-связный (6 верш.)

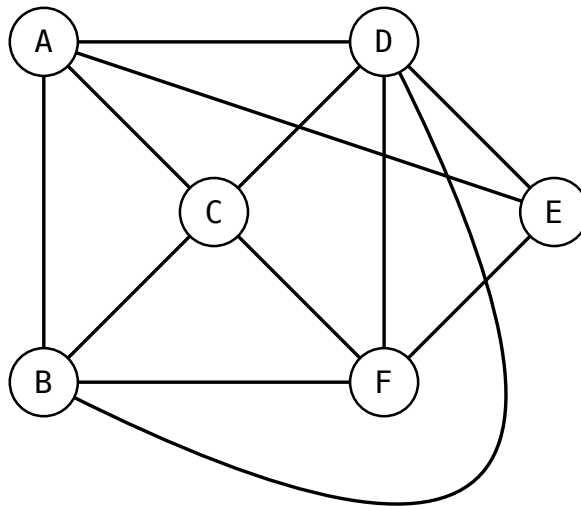
Два цикла $A-B-C-A$ и $D-E-F-D$, соединённые ребром $C-D$, — граф **1-связный**: единственный мост — $C-D$ (удаление его разделяет граф на два треугольника); остальные рёбра лежат на циклах и мостами не являются.



3.

Рис. 12. k -связность: 3-связный (K_4)

Полный граф K_4 на вершинах A, B, C, D — **3-связный**: у каждой вершины степень 3, поэтому, удалив любые 2 ребра, связность сохраняем, а удалив 3 ребра при одной вершине — изолируем её.



4.

Рис. 13. k -связность: 3-связный (6 верш.)

Плотный граф на 6 вершинах (12 рёбер, все степени ≥ 3) — тоже **3-связный**: минимальная степень равна 3 (вершина E), и любые два ребра удалить можно, а три ребра при E — нельзя.

Алгоритм 4. Алгоритм поиска мостов

Вход: $G(V, E)$ — связный, неориентированный.

Выход: полный список мостов.

Шаг 1. Серия DFS:

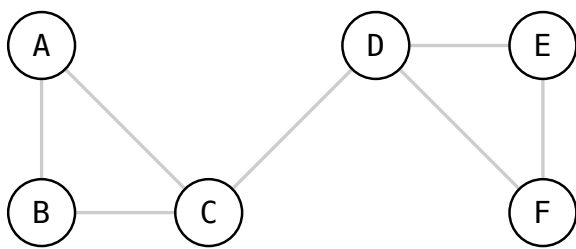
- а) при первом посещении вершины — добавить её в список;
- б) при первом посещении ребра — ориентировать его в направлении обхода.

Шаг 2. Инвертируем все рёбра: $\bar{E}$ — меняем направление каждого ребра на противоположное.

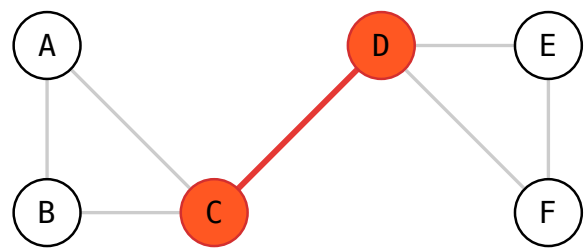
Шаг 3. Серия DFS по списку из шага 1 на графе $G(V, \bar{E})$; каждая серия (компонента) красится своим цветом.

Результат: мосты — это в точности рёбра, концы которых оказались *разного цвета* (в разных компонентах обратного обхода).

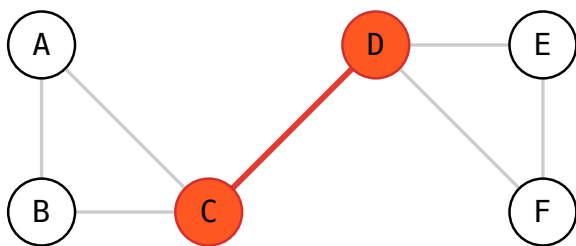
Работа алгоритма по шагам (граф из 12 вершин):



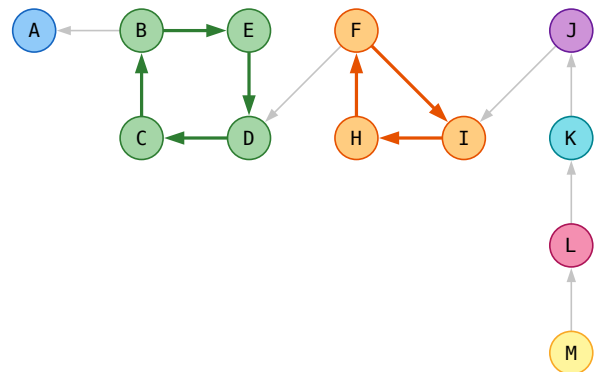
Шаг 0. Исходный граф



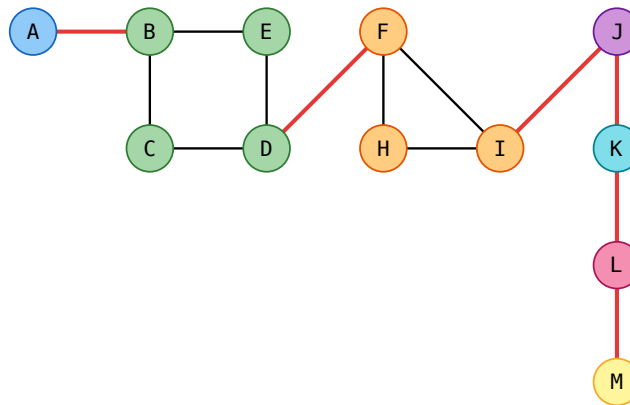
Шаг 1. DFS из A: ориентируем рёбра по обходу



Шаг 2. Инвертируем все рёбра



Шаг 3. Серия DFS: компоненты — цветами



Шаг 4. Мосты — рёбра между разными цветами (красные)

Комментарии к шагам. На шаге 1 поиск в глубину из A ориентирует рёбра в порядке обхода: $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$, ребро $E-B$ впервые посещается из E (ориентируем $E \rightarrow B$); далее $D \rightarrow F \rightarrow H \rightarrow I \rightarrow J \rightarrow K \rightarrow L \rightarrow M$ и ребро $I \rightarrow F$. На шаге 3 серия DFS на инвертированном графе (в порядке списка $A, B, C, \dots$) даёт компоненты $\{A\}$, $\{B, C, D, E\}$, $\{F, H, I\}$, $\{J\}$, $\{K\}$, $\{L\}$, $\{M\}$: из A по инвертированным рёбрам никуда не попасть; из B обходится цикл $B \rightarrow E \rightarrow D \rightarrow C \rightarrow B$; из F — цикл $F \rightarrow I \rightarrow H \rightarrow F$; вершины J, K, L, M остаются поодиночке.

Мосты данного графа (рёбра между разными компонентами): $A-B$, $D-F$, $I-J$, $J-K$, $K-L$, $L-M$.

Лекция 5. Слабая и сильная связность. Граф Герца. Эйлеровость

Ориентированный граф $G(V, E)$.

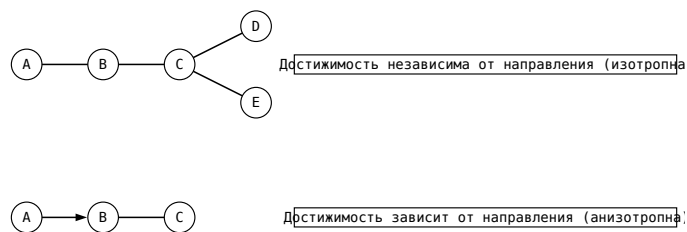


Рис. 14. Изотропия/анизотропия

Пример (изотропия): неориент. дерево $A-B-C-D$, $C-E$ — достижимость *не зависит* от направления рёбер.

Пример (анизотропия): $A \rightarrow B \rightarrow C$ — достижимость *зависит* от направления рёбер.

Определение 22. Слабая связность

$G(V, E)$ — ориентированный граф.

G — **слабо связан** $\Leftrightarrow G$ связан без учёта ориентации рёбер.

Определение 23. Сильная связность

$G(V, E)$ — ориентированный граф.

G — **сильно связан** $\Leftrightarrow \forall u, v \in V: \exists$ путь $u \rightarrow \dots \rightarrow v$.

Определение 24. Компонента сильной связности

Компонента сильной связности — максимальный по включению сильно связный подграф.

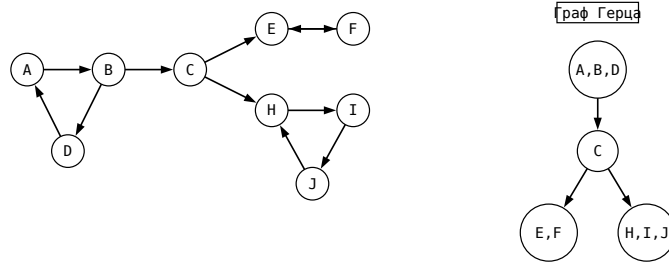


Рис. 15. Ориент. граф (SCC) + граф Герца

Пример. Ориентированный граф: $A \rightarrow B$, $B \rightarrow D$, $D \rightarrow A$ (цикл); $B \rightarrow C$; $C \rightarrow E$, $E \rightarrow F$, $F \rightarrow E$ (цикл); $C \rightarrow H$, $H \rightarrow I$, $I \rightarrow J$, $J \rightarrow H$ (цикл).

Компоненты сильной связности: $\{A, B, D\}$, $\{C\}$, $\{E, F\}$, $\{H, I, J\}$.

Определение 25. Граф Герца (конденсации)

$G(V, E)$ — орграф.

Граф Герца (граф конденсации) для $G = \tilde{G}(\tilde{V}, \tilde{E})$, где:

- $\tilde{V} \ni w \Leftrightarrow w$ — компонента сильной связности G ;
- $\tilde{e} \in \tilde{E} \Leftrightarrow$ между соответствующими компонентами сильной связности в $G \exists$ путь.

Граф Герца для примера выше:

$\{A, B, D\} \rightarrow \{C\}$, $\{C\} \rightarrow \{E, F\}$, $\{C\} \rightarrow \{H, I, J\}$.

Тривиальные свойства графа Герца:

1. $\forall G$: $\tilde{G}$ существует и единственен;
2. $\tilde{G}$ — строго ориентированный;
3. $\tilde{G}$ — ациклический.

Алгоритм 5. Алгоритм Косарайю (построение графа Герца)

Алгоритм для выделения компонент сильной связности.

Шаг I. Серия DFS: «тупиковая» вершина (все исходящие рёбра которой уже обработаны) кладётся в стек. Стек заполняется снизу вверх по порядку завершения обработки вершин.

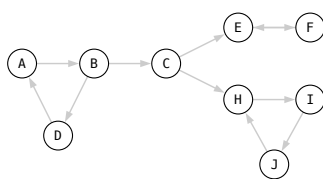
Зачем это нужно: если из компоненты X есть путь в компоненту Y (X «выше по течению»), то хотя бы одна вершина X окажется в стеке *выше* всех вершин Y . Поэтому на шаге III обход стартует из компонент, «выше по течению», и не «перетекает» в чужие компоненты.

Шаг II. $\text{inv } E$ — инвертируем все рёбра.

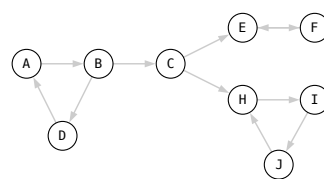
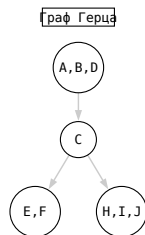
Шаг III. Серия DFS, согласованная со стеком: достаём вершины с верха стека; из каждой ещё не помеченной вершины запускаем DFS по *инвертированным* рёбрам — все достижимые непомеченные вершины образуют одну компоненту сильной связности (красим своим цветом).

Результат: компоненты сильной связности; стянув каждую в одну вершину, получаем граф Герца.

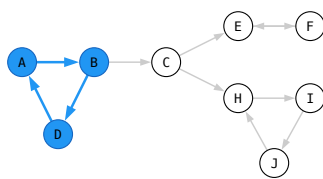
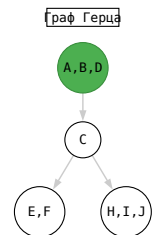
Работа алгоритма по шагам (граф из примера выше; стек показан справа, верх стека — сверху):



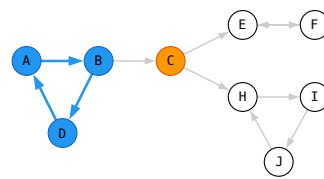
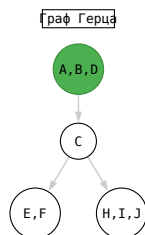
Шаг 0. Исходный граф



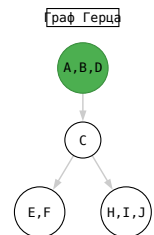
Шаг 1. Серия DFS из A: стек по временам выхода

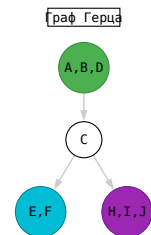
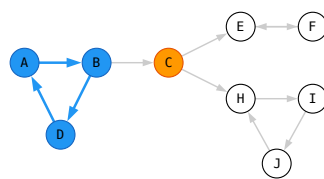
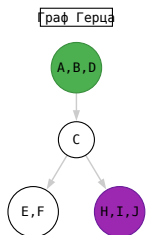
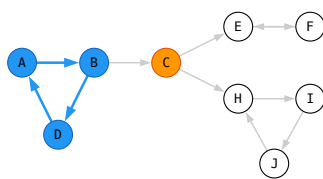


Шаг 2. Инвертируем все рёбра



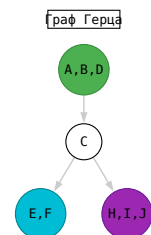
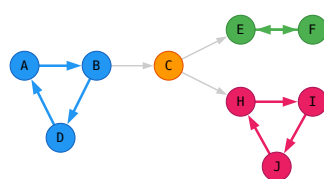
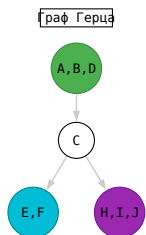
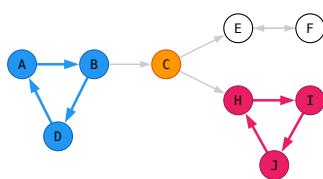
Шаг 3. Сверху стека A: DFS даёт {A, D, B}





Шаг 4. Следующая непомеченная $C: \{C\}$

Шаг 5. Из H : компонента $\{H, J, I\}$



Шаг 6. Из E : компонента $\{E, F\}$

Шаг 7. Итог: 4 компоненты сильной связности

Комментарии к шагам. На шаге 1 поиск в глубину из A обходит граф: $A \rightarrow B \rightarrow D$ (ребро $D \rightarrow A$ ведёт в посещённую вершину, D — тупиковая, кладём в стек); далее $B \rightarrow C \rightarrow E \rightarrow F$ (ребро $F \rightarrow E$ — в посещённую, F в стек, затем E); $C \rightarrow H \rightarrow I \rightarrow J$ ($J \rightarrow H$ — в посещённую, в стек уходят J, I, H); наконец в стек уходят C, B, A . Стек снизу вверх: $D, F, E, J, I, H, C, B, A$.

На шаге III достаём вершины сверху: из A по инвертированным рёбрам достижимы D и B — компонента $\{A, B, D\}$ (шаг 3); B уже помечена, следующая C — из неё по инвертированным рёбрам можно попасть только в помеченную B , компонента $\{C\}$ (шаг 4); из H достижимы J и I — $\{H, I, J\}$ (шаг 5); из E достижима F — $\{E, F\}$ (шаг 6). Все вершины помечены — алгоритм завершён (шаг 7).

Эйлеровость

Определение 26. Эйлеров цикл и эйлеров путь

Дан $G(V, E)$.

G содержит **эйлеров цикл** $\Leftrightarrow \exists$ в G цикл, содержащий $\forall e \in E$ ровно 1 раз.

G содержит **эйлеров путь** $\Leftrightarrow$ в $G \exists$ открытый простой путь, содержащий $\forall e \in E$.

Определение 27. Эйлеров граф

G — **эйлеров**:

- G — ориент. $\Leftrightarrow$ в $G \exists$ эйлеров цикл, G — слабо связан.
- G — неориент. $\Leftrightarrow$ в $G \exists$ эйлеров цикл, G — связный.

Определение 28. Полуэйлеров граф

G — **полуэйлеров**:

- G — ориент. $\Leftrightarrow$ в $G \exists$ эйлеров путь, G — слабо связан.
- G — неориент. $\Leftrightarrow$ в $G \exists$ эйлеров путь, G — связный.

Примеры (проверяем по степеням вершин):

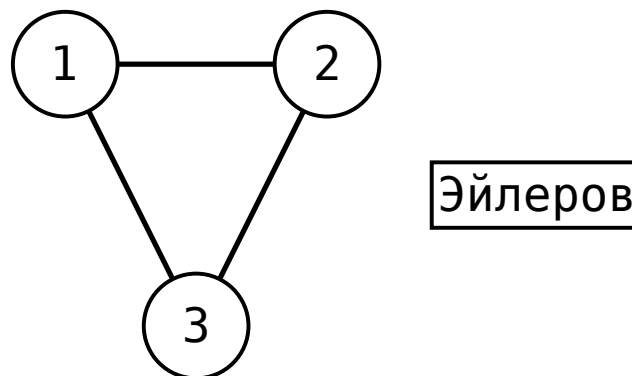


Рис. 16. Эйлеровость: пример 1 — эйлеров

Пример 1. Цикл 1–2–3–1: все степени равны 2 (чётные), граф связный $\Rightarrow$ **эйлеров**; эйлеров цикл — сам треугольник.

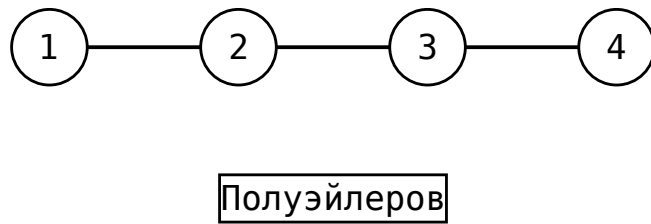


Рис. 17. Эйлеровость: пример 2 — полуэйлеров

Пример 2. Цепь 1–2–3–4: ровно две вершины нечётной степени ($\deg 1 = \deg 4 = 1$), у остальных степень 2 $\Rightarrow$ **полуэйлеров**; эйлеров путь — сама цепь от 1 до 4.

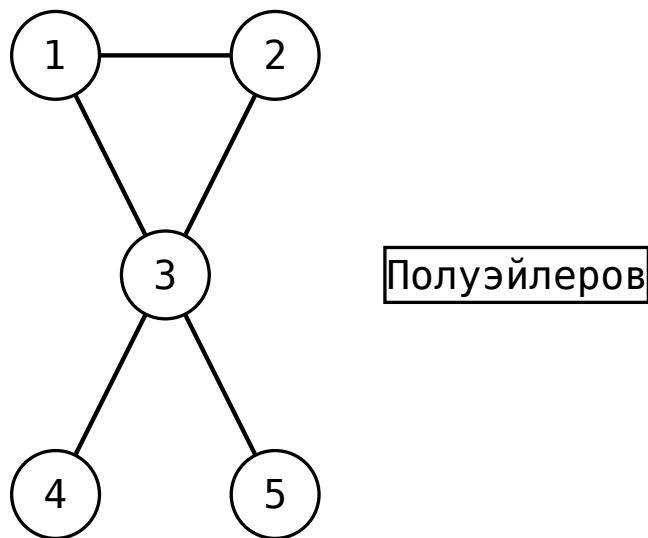


Рис. 18. Эйлеровость: пример 3 — полуэйлеров

Пример 3. Треугольник 1–2–3 с «усами» 3–4 и 3–5: нечётные степени ровно у двух вершин — $\deg 4 = \deg 5 = 1$ (при этом $\deg 3 = 4$, $\deg 1 = \deg 2 = 2$) $\Rightarrow$ **полуэйлеров**; эйлеров путь, например, 4–3–1–2–3–5.

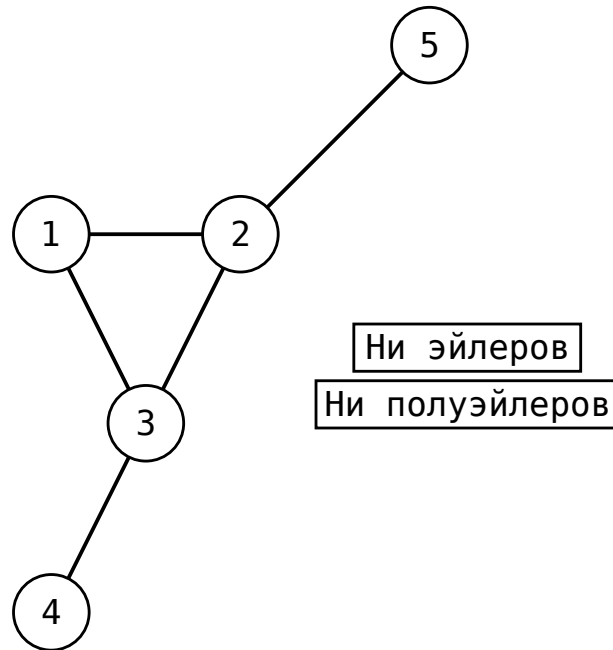


Рис. 19. Эйлеровость: пример 4 — ни Э, ни ПЭ

Пример 4. Треугольник 1–2–3 с «усами» 3–4 и 5–2: нечётная степень у *четырёх* вершин ($\deg 2 = \deg 3 = 3$, $\deg 4 = \deg 5 = 1$) $\Rightarrow$ **ни эйлеров, ни полуэйлеров**.

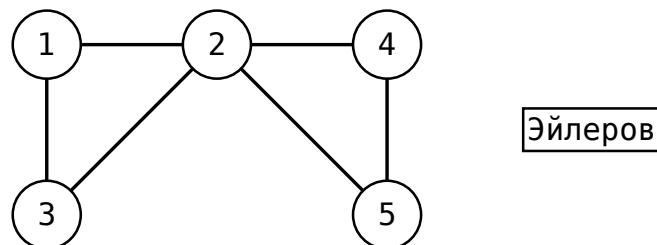


Рис. 20. Эйлеровость: пример 5 — эйлеров

Пример 5. Два цикла 1–2–3–1 и 2–4–5–2 с общей вершиной 2: все степени чётные ($\deg 2 = 4$, у остальных 2), граф связный $\Rightarrow$ **эйлеров**; эйлеров цикл: 1–2–4–5–2–3–1.

Ни эйлеров,
ни полуэйлеров

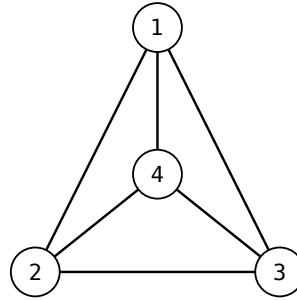


Рис. 21. Эйлеровость: пример 6 — ни Э, ни ПЭ

Пример 6. Полный граф K_4 : все четыре вершины имеют нечётную степень 3 $\Rightarrow$ **ни эйлеров, ни полуэйлеров**.

Теорема 6. Критерий Э и ПЭ

I. $G(V, E)$ — неориент., связный.

а) $G - \text{Э} \Leftrightarrow \forall v \in V: \deg(v) \vdots 2$

б) $G - \text{ПЭ} \Leftrightarrow \exists u \neq v \in V: \deg(u) \nmid 2, \deg(v) \nmid 2,$
 $\forall w \in V \setminus \{u, v\}: \deg(w) \vdots 2$

II. G — ориент., связный (слабо).

а) $G - \text{Э} \Leftrightarrow \forall v \in V: \delta^+(v) = \delta^-(v)$

б) $G - \text{ПЭ} \Leftrightarrow \exists u, v \in V: \delta^+(u) = \delta^-(u) + 1, \delta^-(v) = \delta^+(v) + 1,$
 $\forall w \in V \setminus \{u, v\}: \delta^+(w) = \delta^-(w)$

Доказательство (случай I). а) Необходимость ($\Rightarrow$). Пусть в G есть эйлеров цикл. Идя по циклу, мы в каждую вершину столько же раз «входим», сколько «выходим», и каждое инцидентное вершине ребро используется ровно один раз — либо для входа, либо для выхода. Значит, рёбра при каждой вершине разбиваются на пары «вход–выход», и все степени чётны.

а) Достаточность ($\Leftarrow$). Пусть все степени чётны и G связен; индукция по числу рёбер. Возьмём любую вершину v_0 и пойдём по непройденным рёбрам, никогда не останавливаясь: войдя в вершину $w \neq v_0$, мы заняли нечётное число её рёбер, а степень чётна — значит, есть свободное ребро для выхода. Застрясть можно только в v_0 , т. е. получаем цикл C . Удалим рёбра C из графа: степени снова чётны (в каждой вершине цикл занял чётное число рёбер). Каждая компонента оставшегося графа по предположению индукции имеет эйлеров цикл, причём из связности G каждая компонента пересекается с C по вершине. Вклеим эти циклы в C при прохождении общих вершин — получим эйлеров цикл всего графа.

б) Сводится к пункту а): если ровно две вершины u, v нечётны, добавим фиктивное ребро $u-v$ — все степени станут чётными, по пункту а) существует эйлеров цикл; удалив из него фиктивное

ребро, получим эйлеров путь из u в v . Обратно: эйлеров путь из u в v плюс фиктивное ребро $u-v$ дают эйлеров цикл, поэтому по пункту а) все вершины, кроме u и v , чётны, а u и v нечётны.

Случай II (ориентированный) доказывается дословно так же с заменой степени на полустепени: внутри цикла в каждую вершину входим столько же раз, сколько выходим, поэтому $\delta^+(v) = \delta^-(v)$, и наоборот. $\square$

Алгоритм поиска эйлерова пути / эйлерова цикла (не единственные!).

Алгоритм 6. Алгоритм Флёри

Вход: G — Э или ПЭ.

Выход: эйлеров цикл / эйлеров путь.

P.S. Граф не может быть Э и ПЭ одновременно.

Инициализация:

1. Э $\Rightarrow$ start $\in V$ — любая вершина.
2. ПЭ $\Rightarrow$ start $\in V$: $\deg(v) \not\equiv 2$.
3. (ориент.) $\Rightarrow$ start $\in V$: $\delta^-(v) = \delta^+(v) + 1$.

Шаг (step): на каждом шаге идём по ребру, которое **не является мостом** в оставшемся графе (если есть выбор). По мосту идём, только когда другого ребра из текущей вершины нет. Пройденное ребро удаляем.

Пример.

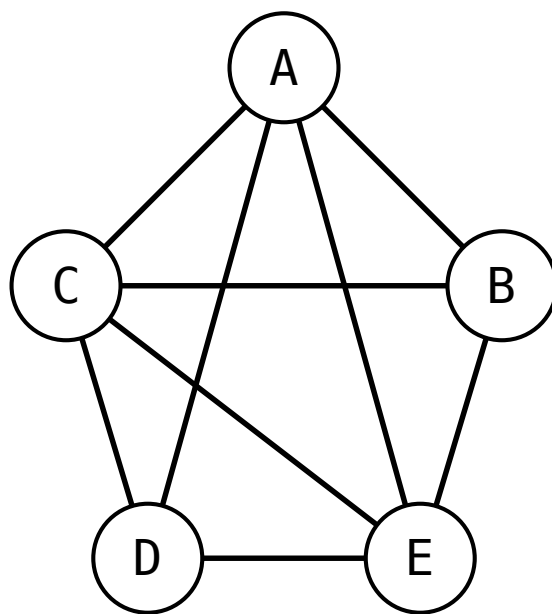


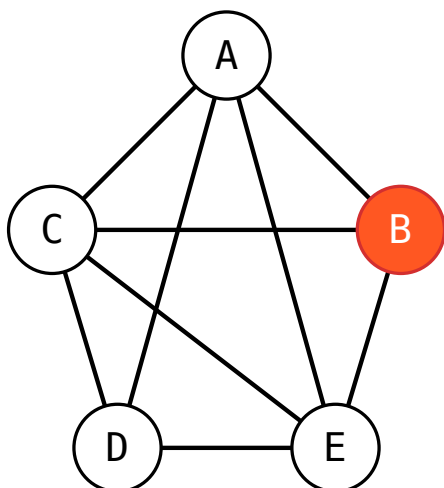
Рис. 22. Алг. Флёри: пример графа

Степени: $\deg A = \deg C = \deg E = 4$, $\deg B = \deg D = 3$ — ровно две нечётные вершины, граф полуэйлеров; стартуем из нечётной вершины B (закончим в D).

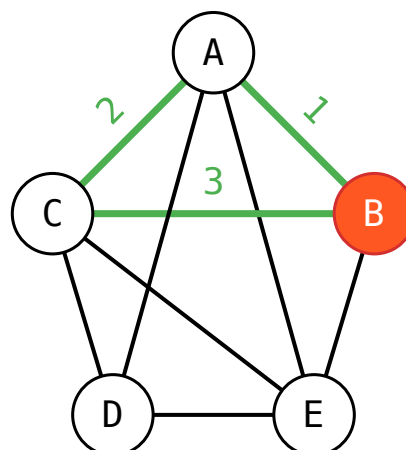
№	вершина	ребро	комментарий (мосты в оставшемся графе)
1	<i>B</i>	<i>B–A</i>	мостов нет, берём любое ребро
2	<i>A</i>	<i>A–C</i>	мостов нет
3	<i>C</i>	<i>C–B</i>	мостов нет
4	<i>B</i>	<i>B–E</i>	единственное оставшееся ребро из <i>B</i>
5	<i>E</i>	<i>E–C</i>	<i>E–C</i> не мост (цикл <i>C–D–E</i> цел)
6	<i>C</i>	<i>C–D</i>	единственное ребро из <i>C</i> (мост, но выбора нет)
7	<i>D</i>	<i>D–E</i>	<i>D–E</i> не мост, а <i>D–A</i> — мост: избегаем
8	<i>E</i>	<i>E–A</i>	единственное ребро из <i>E</i>
9	<i>A</i>	<i>A–D</i>	последнее ребро; финиш в <i>D</i>

Эйлеров путь: *B, A, C, B, E, C, D, E, A, D* — все 9 рёбер ровно по одному разу, от одной нечётной вершины (*B*) до другой (*D*).

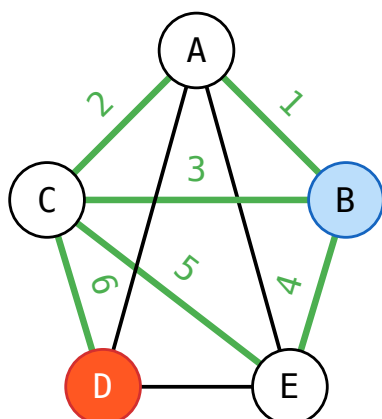
Работа алгоритма по шагам (номера на рёбрах — порядок прохождения; старт *B* — синий, текущая вершина — оранжевая):



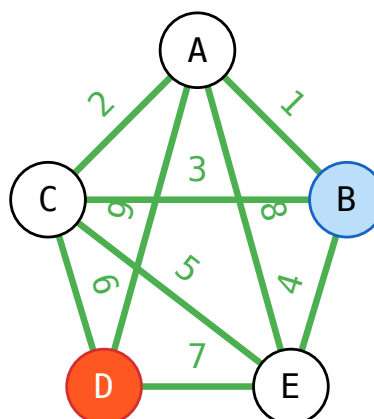
Шаг 0. Старт в нечётной вершине *B*



Шаги 1-3: *B–A–C–B*



Шаги 4-6: *B–E–C–D*



Шаги 7-9: *D–E–A–D*, финиш в *D*

Алгоритм 7. Алгоритм поиска эйлерова цикла (на списках инцидентности)

Идея. Граф задан списками инцидентности (для каждой вершины — список исходящих рёбер). Используются стек **temp** и список **result**:

1. кладём стартовую вершину в стек;
2. пока стек не пуст, смотрим на вершину v с верха стека:
 - если в списке v осталось неиспользованное ребро $v \rightarrow u$ — вычёркиваем его из списка и кладём u в стек;
 - если список v исчерпан — снимаем v со стека и дописываем её в **result**;
3. в конце **result**, прочитанный в обратном порядке, — эйлеров цикл (путь).

Пример.

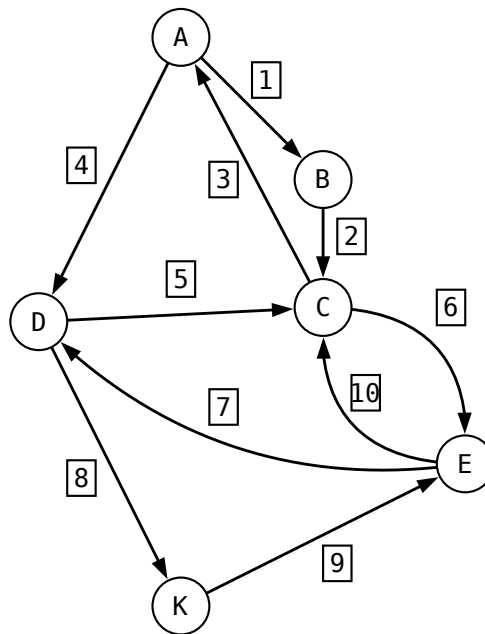


Рис. 23. Алг. на списках инцидентности

Полустепени: у вершины A выходов на один больше, чем входов ($\delta^-(A) = 2$, $\delta^+(A) = 1$), у C — наоборот ($\delta^-(C) = 2$, $\delta^+(C) = 3$), у остальных входы равны выходам — по критерию граф полуэйлеров; эйлеров путь идёт из A в C .

Списки инцидентности (исходящие рёбра):

A: B, D
B: C
C: A, E
D: C, K
E: D, C
K: E

Работа алгоритма: сначала стек только растёт — $A \rightarrow B \rightarrow C \rightarrow A \rightarrow D \rightarrow C \rightarrow E \rightarrow D \rightarrow K \rightarrow E \rightarrow C$ (каждый раз вычёркиваем первое неиспользованное ребро из списка текущей вершины). К этому моменту все 10 рёбер израсходованы, и стек начинает разбираться: вершины по очереди исчерпаны и уходят в **result**.

Состояния (**temp** — стек после фазы роста, читается снизу вверх; **result** — порядок выгрузки, сверху вниз):

temp (низ → верх)	result
A	C
B	E
C	K
A	D
D	E
C	C
E	D
D	A
K	C
E	B
C	A

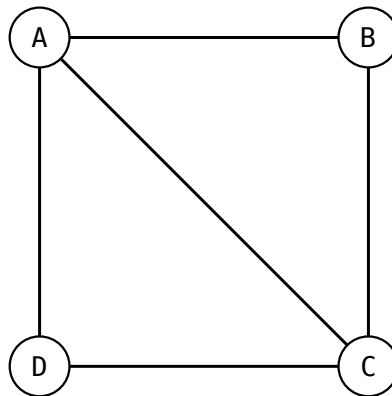
Эйлеров путь — это **result**, прочитанный снизу вверх:

$A, B, C, A, D, C, E, D, K, E, C$

— все 10 дуг ровно по одному разу, из A в C .

Задача 3. Эйлеровость графа

Условие. Дан связный граф на вершинах A, B, C, D с рёбрами AB, BC, CD, DA, AC . Выясните, является ли он эйлеровым или полуэйлеровым, и при возможности укажите эйлеров путь/цикл.



Решение. Применим критерий Эйлера: связный граф эйлеров (есть эйлеров цикл) $\Leftrightarrow$ все степени вершин чётны; полуэйлеров (есть эйлеров путь, но не цикл) $\Leftrightarrow$ ровно две вершины нечётной степени.

Подсчитаем степени:

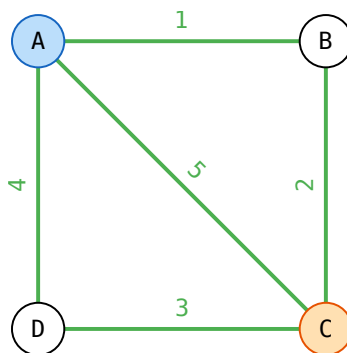
$$\deg A = 3 (B, D, C), \quad \deg B = 2 (A, C), \quad \deg C = 3 (B, D, A), \quad \deg D = 2 (C, A).$$

Нечётную степень имеют ровно две вершины — A и C . Следовательно, граф *полуэйлеров*: эйлерова цикла нет, но есть эйлеров путь, причём он обязан начинаться и заканчиваться в нечётных вершинах A и C .

Построим путь (например, алгоритмом Флёрри, не выбирая мост, пока есть альтернатива):

$$A \rightarrow B \rightarrow C \rightarrow D \rightarrow A \rightarrow C.$$

Он проходит по всем пяти рёбрам AB, BC, CD, DA, AC ровно по одному разу (номера на рёбрах — порядок прохождения):



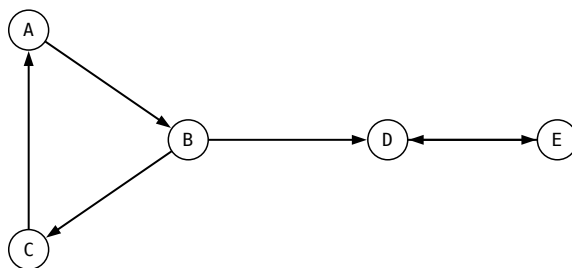
Ответ. Граф полуэйлеров; эйлеров путь $A-B-C-D-A-C$ (из A в C). Эйлерова цикла нет, так как есть вершины нечётной степени.

Задача 4. Компоненты сильной связности: алгоритм Косарайю

Условие. Дан ориентированный граф на вершинах A, B, C, D, E с дугами

$$A \rightarrow B, B \rightarrow C, C \rightarrow A, B \rightarrow D, D \rightarrow E, E \rightarrow D.$$

Найдите компоненты сильной связности алгоритмом Косарайю.



Решение. Алгоритм Косарайю состоит из трёх шагов: (1) обход в глубину исходного графа G с записью времён выхода; (2) построение транспонированного графа G^T (все дуги развёрнуты); (3) обход G^T в порядке убывания времён выхода — каждое дерево обхода даёт одну компоненту.

Шаг 1. DFS по G из A . Порядок выхода (по возрастанию):

$$C(1), E(2), D(3), B(4), A(5).$$

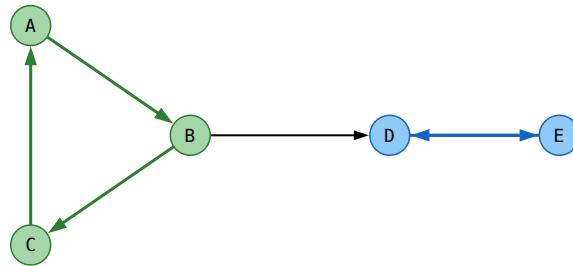
Порядок убывания времён выхода: A, B, D, E, C .

Шаг 2. Транспонированный граф G^T :

$$B \rightarrow A, C \rightarrow B, A \rightarrow C, D \rightarrow B, E \rightarrow D, D \rightarrow E.$$

Шаг 3. DFS по G^T в порядке A, B, D, E, C .

- Из A : $A \rightarrow C \rightarrow B \rightarrow A$ — собрана компонента $\{A, B, C\}$.
- Следующая непосещённая (по порядку) — D : $D \rightarrow E \rightarrow D$ — компонента $\{D, E\}$.



Ответ. Две компоненты сильной связности: $\{A, B, C\}$ (цикл $A \rightarrow B \rightarrow C \rightarrow A$) и $\{D, E\}$ (цикл $D \rightleftarrows E$). Дуга $B \rightarrow D$ соединяет компоненты в одном направлении, поэтому в одну компоненту они не объединяются.

Лекция 6. Гамильтоновы графы. Графы де Брёйна. Деревья

Гамильтоновы графы

Определение 29. Гамильтонов путь

$G(V, E)$.

G содержит **гамильтонов путь** $\Leftrightarrow \exists$ путь, проходящий через $\forall v \in V$ ровно 1 раз.

$\Rightarrow G$ — **полугамильтонов**.

Определение 30. Гамильтонов цикл

G содержит **гамильтонов цикл** $\Leftrightarrow \exists$ замкнутый простой путь через $\forall v \in V$ ровно 1 раз, кроме $v_{\text{start}} = v_{\text{finish}}$.

$\Rightarrow G$ — **гамильтонов**.

Примеры:

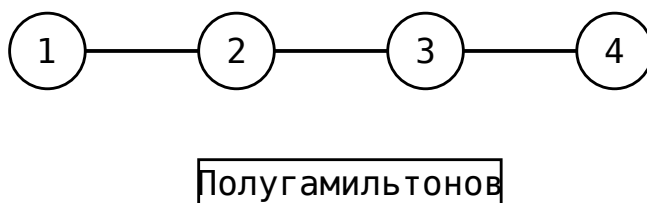


Рис. 24. Полугамильтонов: цепь 1–2–3–4

Пример 1. Цепь 1–2–3–4 сама является гамильтоновым путём (каждая вершина ровно один раз), но цикла нет — граф **полугамильтонов**.

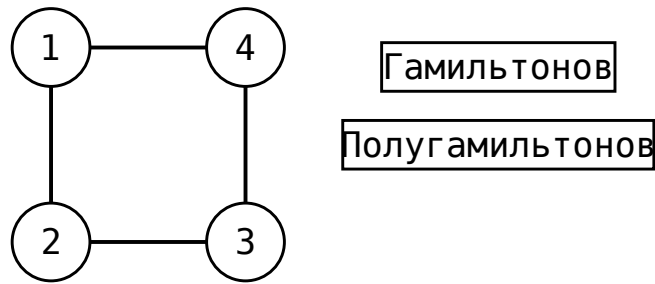


Рис. 25. Гамильтонов (и полугамильтонов): цикл 1–2–3–4–1

Пример 2. Цикл 1–2–3–4–1 — гамильтонов цикл, поэтому граф **гамильтонов** (а значит, и **полугамильтонов**: убрав последнее ребро цикла, получаем гамильтонов путь).

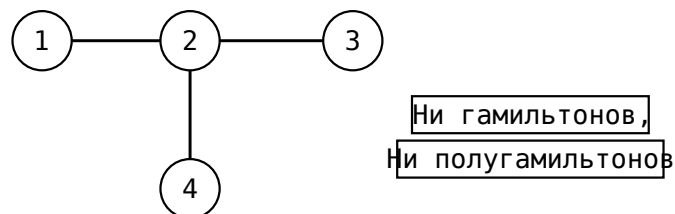


Рис. 26. Ни гамильтонов, ни полугамильтонов: 1–2–3, 2–4

Пример 3. «Звезда» 1–2–3 с дополнительным листом 2–4: любой путь, заходящий во все три листа 1, 3, 4, обязан проходить через центр 2 минимум дважды — граф **ни гамильтонов, ни полугамильтонов**.

Замечание. Теоремы ниже дают только *достаточные* условия гамильтоновости: если условие выполнено — граф **точно** гамильтонов, но если условие *не* выполнено, это ещё не значит, что граф не гамильтонов (например, цикл C_5 гамильтонов, хотя все степени равны $2 < 5/2$).

Условия гамильтоновости ($G(V, E)$ — неориент., связный):

Теорема 7. Теорема Оре

$|V| = n \geq 3$. Если $\forall u, v \in V$ т. ч. $\nexists e = uv$ (u, v — несмежные): $\deg(u) + \deg(v) \geq n$, то G — гамильтонов.

Доказательство. От противного. Пусть условие выполнено, но гамильтонова цикла нет. Будем добавлять в G рёбра, пока это возможно без появления гамильтонова цикла (условие Оре при добавлении рёбер тем более сохраняется); получим *максимальный* контрпример G' : гамильтонова цикла нет, но добавление любого ребра его создаёт.

В G' есть несмежные вершины u, v (иначе $G' = K_n$ — гамильтонов). Добавление ребра uv создаёт гамильтонов цикл, значит уже в G' есть гамильтонов *путь* $u = v_1, v_2, \dots, v_n = v$.

Рассмотрим множества индексов $S = \{i \mid u \sim v_{i+1}\}$ и $T = \{i \mid v \sim v_i\}$; $|S| = \deg(u)$, $|T| = \deg(v)$, причём $S, T \subseteq \{1, \dots, n-1\}$. Так как $|S| + |T| = \deg(u) + \deg(v) \geq n > n-1$, по принципу Дирихле $\exists i \in S \cap T$: одновременно $u \sim v_{i+1}$ и $v \sim v_i$. Тогда

$$u = v_1, v_2, \dots, v_i, v_n = v, v_{n-1}, \dots, v_{i+1}, v_1$$

— гамильтонов цикл (идём по пути до v_i , прыгаем по ребру $v_i v_n$ в конец, возвращаемся по пути назад до v_{i+1} и замыкаемся ребром $v_{i+1} u$). Противоречие с тем, что в G' цикла нет. $\square$

Теорема 8. Теорема Дирака

$|V| = n \geq 3$. Если $\forall u \in V: \deg(u) \geq \frac{n}{2}$, то G — гамильтонов.

Доказательство. Частный случай теоремы Оре: для любых несмежных u, v имеем $\deg(u) + \deg(v) \geq \frac{n}{2} + \frac{n}{2} = n$. $\square$

Определение 31. Турнир

Турнир — ориентированный граф, в котором каждая пара различных вершин соединена *ровно одной* дугой (в одном из двух направлений). Иначе говоря, турнир — это полный граф K_n , каждому ребру которого приписана ориентация.

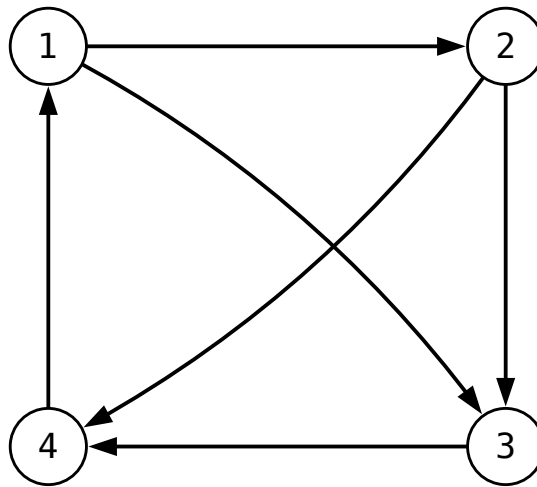


Рис. 27. Турнир на 4 вершинах: гамильтонов путь $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$

Примеры. 1) Результаты кругового спортивного турнира без ничьих: вершины — участники, дуга $u \rightarrow v$ означает « u выиграл у v » (отсюда и название). На рисунке: 1 победил всех, 4

проиграл всем, между 2 и 3 — победа 2. 2) Транзитивный турнир: вершины $1, \dots, n$, все дуги от меньшего номера к большему — это в точности строгий линейный порядок из лекции 3.

Теорема 9. Теорема Редди (о турнирах)

В любом турнире существует гамильтонов путь.

Доказательство. Индукция по числу вершин n . База $n = 1$ тривиальна. Переход: удалим из турнира вершину v ; на оставшихся $n - 1$ вершинах по предположению индукции есть гамильтонов путь $u_1 \rightarrow u_2 \rightarrow \dots \rightarrow u_{n-1}$. Вставим v :

- если $v \rightarrow u_1$ — ставим v в начало;
- иначе $u_1 \rightarrow v$; пусть i — наибольший индекс с $u_i \rightarrow v$. Если $i = n - 1$, ставим v в конец. Если же $i < n - 1$, то по максимальной $v \rightarrow u_{i+1}$, и v вставляется между u_i и u_{i+1} : $u_1 \rightarrow \dots \rightarrow u_i \rightarrow v \rightarrow u_{i+1} \rightarrow \dots \rightarrow u_{n-1}$.

В каждом случае получили гамильтонов путь на всех n вершинах (заметим: гамильтонов *цикл* в турнире существовать не обязан — в транзитивном турнире из последней вершины дуги не выходят). $\square$

Алгоритм 8. Построение гамильтонова пути в условиях теоремы Дирака

Инициализация: $v_1, v_2, \dots, v_n$ — произвольный порядок *всех* вершин графа.

Шаг (поворот): пока в последовательности есть несмежная соседняя пара (v_i, v_{i+1}) , ищем индекс $j > i + 1$, для которого *оба* ребра существуют:

$$\exists e_1 = v_i v_j \quad \text{и} \quad \exists e_2 = v_{i+1} v_{j+1};$$

(условие Дирака гарантирует, что такой j найдётся). Переворачиваем отрезок $v_{i+1} \dots v_j$:

$$v_1 \dots v_i \mid \underbrace{v_j v_{j-1} \dots v_{i+1}}_{\text{перевёрнут}} \mid v_{j+1} \dots v_n.$$

На стыках теперь стоят пары (v_i, v_j) и (v_{i+1}, v_{j+1}) — обе смежные, а внутри перевёрнутого отрезка смежность пар не изменилась. Число несмежных пар строго уменьшилось; повторяем, пока их не останется — получился гамильтонов путь.

Пример. Возьмём граф с рёбрами $A-C, C-E, E-F, F-D, D-B, B-A, A-D, C-F, B-E$ (к графу с рис. выше по тексту добавлено ребро $B-E$, чтобы выполнялось условие Дирака: теперь все степени равны $3 = \frac{6}{2} = \frac{n}{2}$).

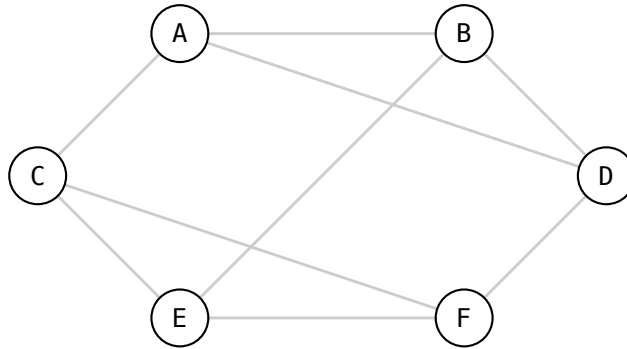


Рис. 28. Алг. Дирака: граф $A, \dots, F$, все степени = 3

Начальный порядок: E, A, B, D, F, C — т.е. $v_1 = E, v_2 = A, v_3 = B, v_4 = D, v_5 = F, v_6 = C$. Порядок *произвольный* (алгоритм стартует с любого); этот выбран для компактности примера — в нём всего одна несмежная соседняя пара. Проверяем пары: (E, A) — *несмежная* (ребра $E-A$ нет); остальные пары $(A, B), (B, D), (D, F), (F, C)$ — смежные.

Чиним пару $(v_i, v_{i+1}) = (v_1, v_2) = (E, A)$. По описанию шага ищем индекс $j > i + 1 = 2$, для которого существуют *оба* ребра $v_i v_j$ и $v_{i+1} v_{j+1}$, т.е. $E \sim v_j$ и $A \sim v_{j+1}$ — перебираем кандидатов $j = 3, 4, 5$ по порядку:

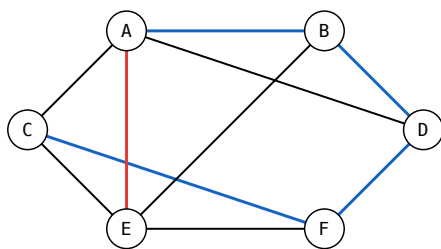
- $j = 3$: $v_3 = B, v_4 = D$. Проверяем: $E \sim B$? — да (ребро $B-E$); $A \sim D$? — да (ребро $A-D$). **Оба условия выполнены** — берём $j = 3$ (далее можно не перебирать; для сравнения: $j = 4$ не подошёл бы — уже первое условие $E \sim v_4 = D$ не выполнено, ребра $E-D$ в графе нет).

Переворачиваем отрезок $v_{i+1} \dots v_j = v_2 \dots v_3 = (A, B)$:

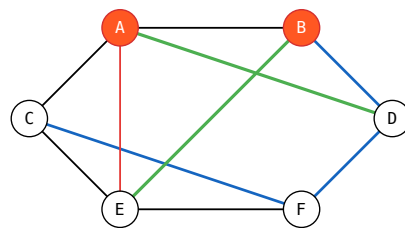
$$E, \underline{A}, \underline{B}, D, F, C \longrightarrow E, \underline{B}, \underline{A}, D, F, C.$$

На стыках перевёрнутого отрезка встали проверенные рёбра: слева — $v_1 v_3 = E-B$, справа — $v_2 v_4 = A-D$. Все соседние пары теперь смежные: $E-B, B-A, A-D, D-F, F-C$ — это **гамильтонов путь**. Более того, концы E и C тоже смежны ($C-E$ — ребро), так что путь замыкается в **гамильтонов цикл** E, B, A, D, F, C, E .

Работа алгоритма по шагам (синие рёбра — смежные соседние пары, красное — несмежная пара, зелёные — рёбра поворота и итоговый путь):



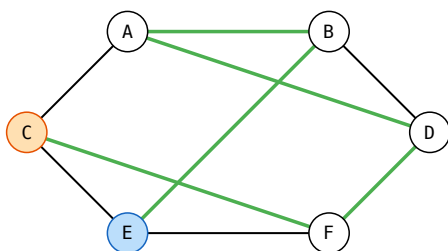
Последовательность: E, A, B, D, F, C
 Несмежная пара: (E, A)



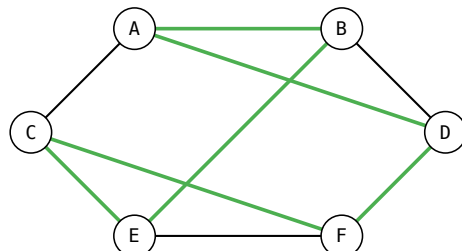
Поворот: $j = 3$, $v_j = B$, $v_{j+1} = D$
 E-B и A-D есть в графе → переворачиваем (A, B)

Шаг 0. Порядок E, A, B, D, F, C; пара (E, A) несмежная

Шаг 1. Поворот: $j = 3$, рёбра E-B и A-D есть



Гамильтонов путь: E, B, A, D, F, C



C и E смежны → гамильтонов цикл

Шаг 2. Отрезок (A, B) перевёрнут: гамильтонов путь

Шаг 3. $C \sim E$: замыкаем в гамильтонов цикл

Графы де Брёйна

Определение 32. Граф де Брёйна

$\Sigma = \{a_0, \dots, a_{k-1}\}$ — алфавит.

$\Sigma^m = \{a_{i_1} \dots a_{i_m} \mid a_{i_j} \in \Sigma\}$ — слова длины m .

$\Sigma^m = V$ (вершины — слова длины m).

$E = \Sigma^{m+1}$ (рёбра — слова длины $m+1$).

Рёбро $a_{i_1} a_{i_2} \dots a_{i_m} a_{i_{m+1}}$ соединяет вершину $(a_{i_1} \dots a_{i_m})$ с вершиной $(a_{i_2} \dots a_{i_{m+1}})$: суффикс первой вершины совпадает с префиксом второй; a_{i_1} и $a_{i_{m+1}}$ — любые.

$B(\Sigma, m)$ — граф де Брёйна.

Пример. $\Sigma = \{0, 1\}$, $m = 2$.

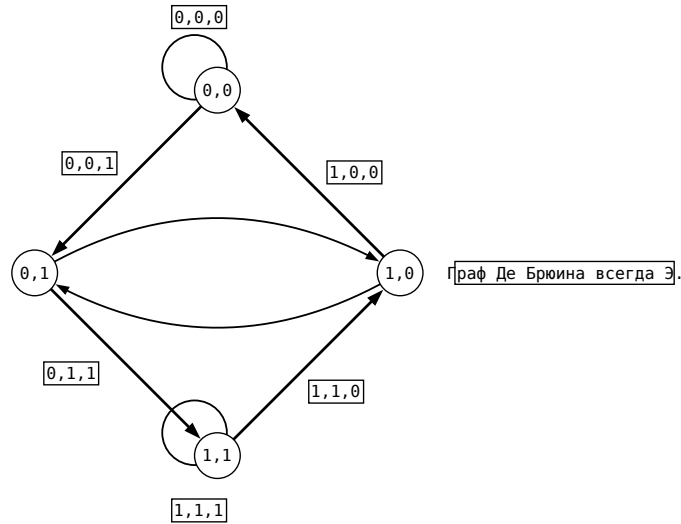


Рис. 29. Граф де Брюйна $B(\{0,1\}, 2)$

$\Sigma^2 = \{00, 01, 10, 11\}$ — вершины.

$B(\{0,1\}, 2)$: рёбра — все слова длины 3 (000, 001, 010, 011, 100, 101, 110, 111).

Теорема 10. Графы де Брюйна всегда эйлеровы

$\forall \Sigma (|\Sigma| = k), \forall m$: граф $B(\Sigma, m)$ — эйлеров.

Доказательство. Проверим критерий эйлеровости для ориентированного графа.

Полустепени равны. Из вершины $w = a_{i_1} \dots a_{i_m}$ выходит ровно k дуг — слова $w \cdot a$ для каждой буквы $a \in \Sigma$ (ведут в вершины $a_{i_2} \dots a_{i_m} a$). Входит в w тоже ровно k дуг — слова $a \cdot w$ для каждой $a \in \Sigma$. Значит, $\delta^-(v) = \delta^+(v) = k$ для всех вершин.

Связность. Из любой вершины $u = a_{i_1} \dots a_{i_m}$ можно дойти до любой вершины $v = b_{j_1} \dots b_{j_m}$: последовательно «вдвигаем» буквы слова v справа — за m шагов $u \rightarrow a_{i_2} \dots a_{i_m} b_{j_1} \rightarrow \dots \rightarrow b_{j_1} \dots b_{j_m} = v$. Граф даже сильно связан.

Оба условия критерия выполнены — эйлеров цикл существует. (Именно он даёт *последовательность де Брюйна* — кратчайшую циклическую строку, содержащую все слова длины $m+1$.)

□

Деревья

Определение 33. Дерево

Ациклический, связный граф.

Что знаем:

1. G — дерево $\Rightarrow |E| = |V| - 1$.
2. G — связный, $|E| = |V| - 1 \Rightarrow G$ — дерево.
3. G — дерево $\Rightarrow \forall u, v \in V: \exists!$ цепь.
4. Если $|V| \geq 4 \Rightarrow$ дерево, $|V|!, |E|!$ — не единственно!

Помеченные / непомеченные деревья.

Помеченные (3 вершины): $A-B-C$, $A-C-B$, $B-A-C$.

Непомеченные (3 вершины): $\circ-\circ-\circ$ (единственное).

Код Прюфера

Как компактно закодировать помеченное дерево: **код Прюфера**.

$G(V, E)$ — дерево, $|V| = n$.

$|\text{Pruf}(G)| = n - 2$.

Алгоритм 9. Код Прюфера (прямая задача)

Дано: $G(V, E)$ — дерево, $|V| = n$, порядок лексикографический.

Выход: $\text{Pruf}(G) = \{v_{i_1}, \dots, v_{i_{n-2}}\}$.

Шаг (step): удаление лексикографически минимального листа; в код записывается его родитель (единственный сосед). Повторяем, пока не останется 2 вершины: код имеет длину $n - 2$.

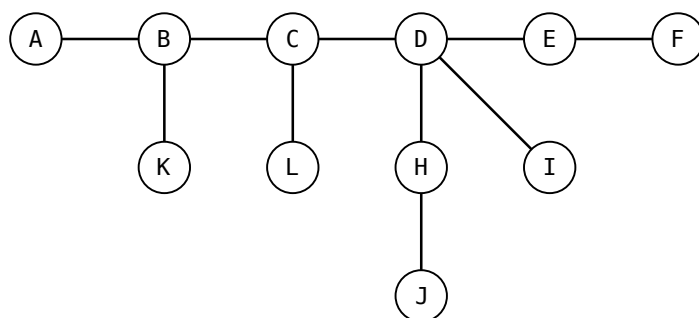


Рис. 30. Код Прюфера: дерево 11 вершин

Пример. Дерево (11 вершин): $A-B$, $B-C$, $C-D$, $D-E$, $E-F$, $D-H$, $H-J$, $D-I$, $B-K$, $C-L$.
Порядок: $A, B, C, D, E, F, H, I, J, K, L$.

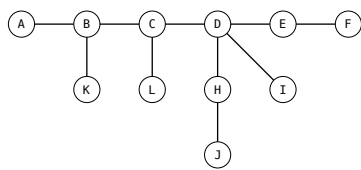
По шагам (лист $\rightarrow$ в код пишем его соседа):

1. листья $\{A, F, I, J, K, L\}$; минимальный A , сосед $B \rightarrow$ код (B) ;
2. минимальный лист F , сосед $E \rightarrow$ код (B, E) ;
3. E стала листом; минимальный лист E , сосед $D \rightarrow (B, E, D)$;
4. минимальный лист I , сосед $D \rightarrow (B, E, D, D)$;
5. минимальный лист J , сосед $H \rightarrow (B, E, D, D, H)$;
6. H стала листом; сосед $D \rightarrow (B, E, D, D, H, D)$;

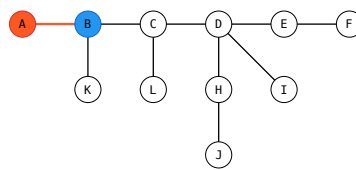
7. D стала листом; сосед $C \rightarrow (B, E, D, D, H, D, C)$;
8. минимальный лист K , сосед $B \rightarrow (B, E, D, D, H, D, C, B)$;
9. B стала листом; сосед $C \rightarrow (B, E, D, D, H, D, C, B, C)$; остались две вершины C и L — стоп.

$$\text{Pruf}(G) = (B, E, D, D, H, D, C, B, C), \quad |\text{Pruf}(G)| = 11 - 2 = 9.$$

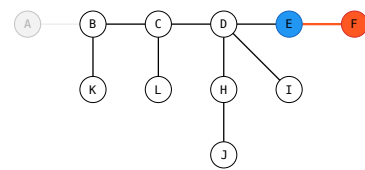
Работа алгоритма по шагам. На каждом шаге удаляется лист с минимальной меткой (оранжевый), в код дописывается его сосед (синий); удалённые вершины и рёбра показаны серым.



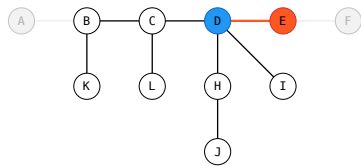
Шаг 0. Исходное дерево



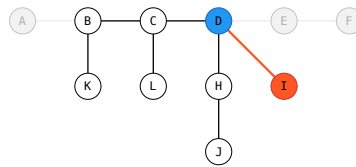
Шаг 1. $A \rightarrow$ код: B



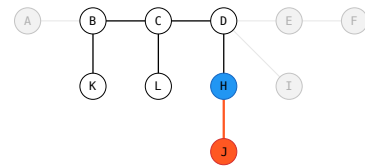
Шаг 2. $F \rightarrow$ код: E



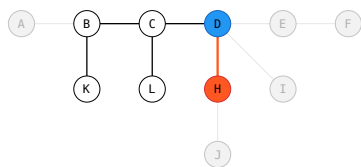
Шаг 3. $E \rightarrow$ код: D



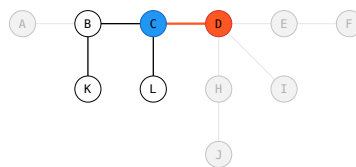
Шаг 4. $I \rightarrow$ код: D



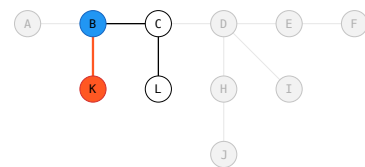
Шаг 5. $J \rightarrow$ код: H



Шаг 6. $H \rightarrow$ код: D



Шаг 7. $D \rightarrow$ код: C



Шаг 8. $K \rightarrow$ код: B

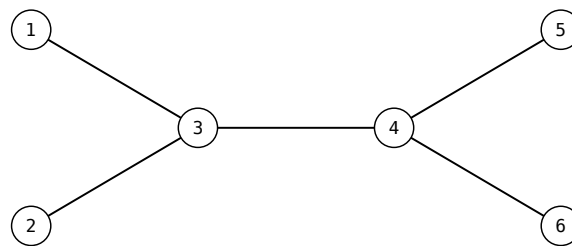


Шаг 9. $B \rightarrow$ код: C

Шаг 10. Остались C и L — стоп

Задача 5. Код Прюфера: прямая и обратная задача

Условие. (а) Постройте код Прюфера для дерева на вершинах $1, \dots, 6$ с рёбрами $1-3, 2-3, 3-4, 4-5, 4-6$. (б) Восстановите дерево по коду Прюфера $(3, 3, 4, 4)$.



Решение (а). На каждом шаге удаляем лист с наименьшим номером и записываем номер его единственного соседа; повторяем, пока не останется 2 вершины (для дерева на n вершинах код имеет длину $n - 2 = 4$).

1. Листья $\{1, 2, 5, 6\}$. Наименьший — 1, сосед 3. Записываем 3.
2. Листья $\{2, 5, 6\}$. Наименьший — 2, сосед 3. Записываем 3.
3. Теперь 3 стала листом (соединена только с 4). Листья $\{3, 5, 6\}$. Наименьший — 3, сосед 4. Записываем 4.
4. Листья $\{5, 6\}$. Наименьший — 5, сосед 4. Записываем 4. Осталось 2 вершины (4 и 6) — стоп.

Код Прюфера: $(3, 3, 4, 4)$.

Решение (б). Обратный алгоритм. Список степеней: каждая вершина встречается в коде $(\deg - 1)$ раз, поэтому степень = (число вхождений в код) + 1. Для кода $(3, 3, 4, 4)$ и вершин $1 \dots 6$:

$$\deg 3 = 3, \deg 4 = 3, \deg 1 = \deg 2 = \deg 5 = \deg 6 = 1.$$

На каждом шаге берём наименьший лист (вершину степени 1, не входящую в оставшийся код) и соединяем с первым элементом кода:

1. Код (3, 3, 4, 4); наименьший лист 1 $\rightarrow$ ребро 1–3; убираем 3 из кода.
2. Код (3, 4, 4); наименьший лист 2 $\rightarrow$ ребро 2–3; убираем 3.
3. Код (4, 4); теперь 3 стала листом, наименьший лист 3 $\rightarrow$ ребро 3–4; убираем 4.
4. Код (4); наименьший лист 5 $\rightarrow$ ребро 5–4; убираем 4.
5. Код пуст; остались вершины 4 и 6 $\rightarrow$ ребро 4–6.

Восстановленные рёбра: 1–3, 2–3, 3–4, 4–5, 4–6 — совпадают с исходным деревом из пункта (а) (см. рисунок выше), как и должно быть.

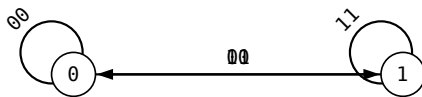
Ответ. Код Прюфера дерева — (3, 3, 4, 4); обратное восстановление даёт то же дерево.

Задача 6. Последовательность де Брёйна над алфавитом 0 и 1

Условие. Постройте граф де Брёйна для слов длины 2 над алфавитом $\{0, 1\}$ и найдите кратчайшую циклическую последовательность, содержащую все 2-буквенные слова в качестве подстрок.

Решение. В графе де Брёйна $B(\{0, 1\}, 2)$ вершины — слова длины 1 (то есть 0 и 1), а дуги — слова длины 2: дуга из u в v помечена словом uv . Получаем 4 дуги:

$$0 \xrightarrow{00} 0, \quad 0 \xrightarrow{01} 1, \quad 1 \xrightarrow{11} 1, \quad 1 \xrightarrow{10} 0.$$



Искомая последовательность — это *эйлеров цикл* (он проходит по каждой дуге-слову ровно один раз). У каждой вершины полустепени захода и исхода равны 2, граф связан — эйлеров цикл существует. Один из них:

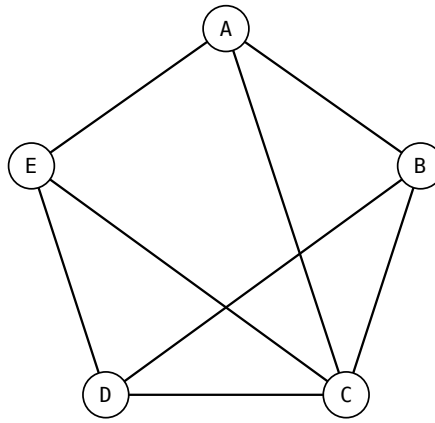
$$0 \xrightarrow{00} 0 \xrightarrow{01} 1 \xrightarrow{11} 1 \xrightarrow{10} 0,$$

что даёт циклическую последовательность **0011**. Читая её по кругу, получаем все 2-буквенные слова: 00, 01, 11, 10.

Ответ. Кратчайшая циклическая последовательность — 0011 (длина $4 = 2^2$), содержит все четыре слова длины 2.

Задача 7. Гамильтоновость: теорема Дирака

Условие. Граф на 5 вершинах A, B, C, D, E задан как полный граф K_5 без двух рёбер AD и BE . Является ли он гамильтоновым? Постройте гамильтонов цикл.



Решение. Теорема Дирака: если в графе на $n \geq 3$ вершинах степень каждой вершины не меньше $n/2$, то граф гамильтонов. Здесь $n = 5$, порог $n/2 = 2,5$. Подсчитаем степени (в K_5 каждая вершина имела степень 4; удаление AD и BE уменьшает на единицу степени A, D, B, E):

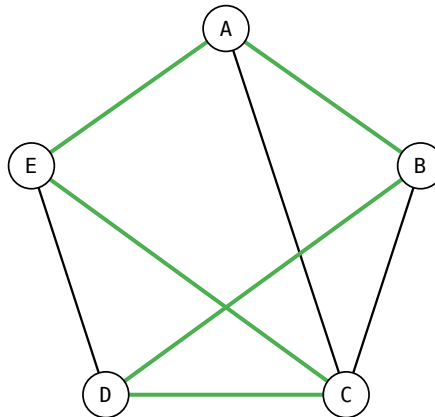
$$\deg A = \deg B = \deg D = \deg E = 3, \quad \deg C = 4.$$

Все степени $\geq 3 > 2,5$, поэтому по теореме Дирака граф гамильтонов.

Построим гамильтонов цикл, обходя все вершины по одному разу и избегая отсутствующих рёбер AD, BE :

$$A-B-D-C-E-A.$$

Проверим рёбра: AB, BD, DC, CE, EA — все присутствуют (среди них нет ни AD , ни BE).



Ответ. Граф гамильтонов (условие Дирака выполнено); гамильтонов цикл $A-B-D-C-E-A$.

Лекция 7. Код Прюфера (обратная задача). Остовные деревья. Алгоритм Прима

Обратная задача кода Прюфера

Дано: $V = \{v_1, \dots, v_n\}$, $\text{Pruf}(G) = \{v_{i_1}, \dots, v_{i_{n-2}}\}$.

Восстановить G .

Step: выбор $\min$ из $(V \setminus \text{Pruf}(G)) = v_k$; соединяем: $e = v_k v_{i_s}$, удаляем v_k из V , v_{i_s} из $\text{Pruf}(G)$ (v_{i_s} — s -й элемент кода, s — номер шага).

Замечание: в процессе не обязательно должно быть дерево, а в конце — обязательно.

Пример.

$V = \{A, B, C, D, E, F, H, I, J\}$, $\text{Pruf}(G) = \{D, A, A, F, H, A, J\}$.

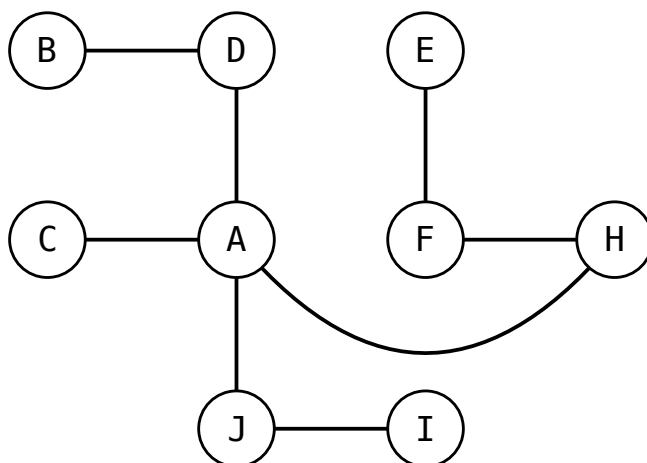


Рис. 31. Обратная задача Прюфера

Теорема Кэли

$V = \{v_1, \dots, v_n\}$; сколько $\exists$ различных помеченных деревьев?

$|\text{Pruf}(G)| = n - 2 \Rightarrow n^{n-2}$ помеченных деревьев.

Теорема 11. Теорема Кэли (о количестве помеченных деревьев)

Количество различных помеченных деревьев на n вершинах равно n^{n-2} .

Доказательство. Построим биекцию между помеченными деревьями на n вершинах и последовательностями длины $n - 2$ из элементов V — это в точности код Прюфера.

Код корректно определён. Прямой алгоритм (лекция 6) на каждом шаге удаляет минимальный лист (у дерева с ≥ 2 вершинами лист всегда есть) и делает ровно $n - 2$ записи — любому дереву отвечает ровно одна последовательность $\text{Pruf}(G) \in V^{n-2}$.

Инъективность и сюръективность. Обратный алгоритм (выше) по *любой* последовательности $p \in V^{n-2}$ строит граф: на каждом шаге к минимальной вершине, не встречающейся в остатке кода и ещё не использованной, проводится ребро к первому элементу кода. Получается

ровно $n - 1$ ребро, и построенный граф связан и ациклический (каждый шаг присоединяет «новый лист» к остальной части), т. е. дерево. Прямой проверкой видно, что обратный алгоритм восстанавливает исходное дерево по его коду, а прямой алгоритм, применённый к дереву, построенному по последовательности p , возвращает p . Значит, соответствие — биекция.

Последовательностей длины $n - 2$ из n символов ровно n^{n-2} , следовательно, столько же и помеченных деревьев. $\square$

Рёберно-взвешенные графы. Минимальное остовное дерево

Определение 34. Рёберно-взвешенный граф

$G(V, E)$ — (рёберно-)взвешенный:

$\exists \sigma: E \rightarrow W$,

где W — некоторое упорядоченное мн-во с операциями, зависящими от задачи.

NB: W — нижняя упорядоченная мн-во.

$G(V, E)$ — неориент., н. у. о. — связный.

$\sigma: E \rightarrow \mathbb{R}$.

Определение 35. Остов (каркас)

Остов (скелет, каркас) — связный подграф в G , являющийся деревом и содержащий все вершины G .

Определение 36. Остовное дерево

$G(V, E)$ — **остовное дерево** G : T — дерево и остов.

Если граф — не дерево, то остовов несколько.

Определение 37. Минимальное остовное дерево

$G(V, E)$ — связный, взвешенный.

$\sigma: E \rightarrow \mathbb{R}$.

Минимальное остовное дерево G : $T_G: \sum_{\substack{(v,\hat{v}) \in T \\ e \in \hat{E}}} \sigma(e) \rightarrow \min$ по всем T для G .

Жадные алгоритмы — алгоритмы, оптимальные только в рамках одного шага.

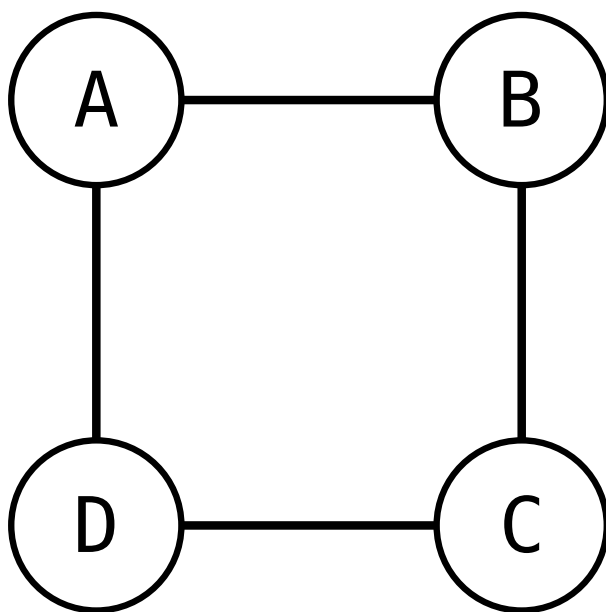


Рис. 32. Жадный алг.: контрпример 1

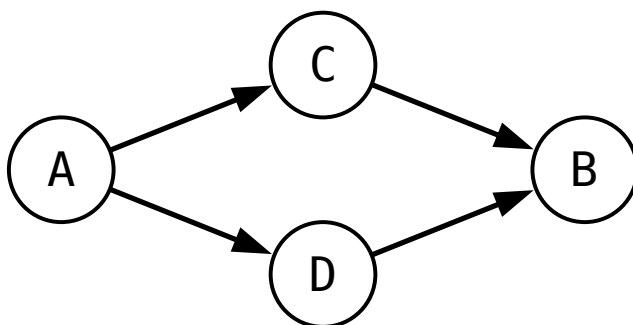


Рис. 33. Жадный алг.: контрпример 2

Алгоритм построения min остовного дерева

$G(V, E)$, $\sigma: E \rightarrow \mathbb{R}$, н. у. о. — связный.

Алгоритм 10. Алгоритм Прима

Инициализация: $\text{start} \in V$ (любая вершина); дерево $T = \{\text{start}\}$.

Step: среди всех рёбер, соединяющих уже выбранную вершину с ещё не выбранной, берём ребро e с минимальным весом $\sigma(e)$; добавляем его (и новую вершину) в T . Новая вершина присоединяется не обязательно к последней — к *любой* из уже выбранных. Повторяем, пока T не охватит все вершины ($n - 1$ ребро).

Пример (старт из J):

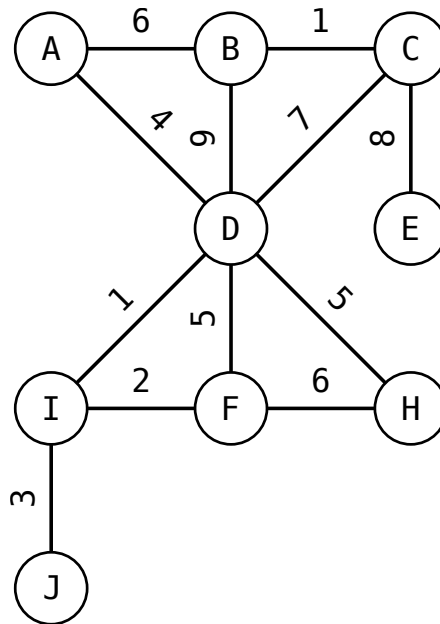
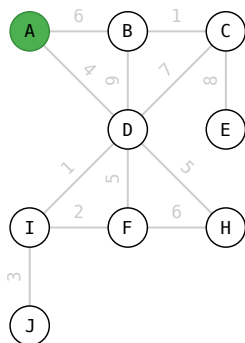


Рис. 34. Алг. Прима: взвешенный граф 9 вершин

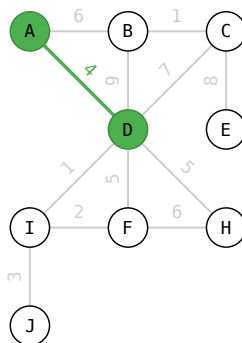
V	e	$\sum \sigma(e)$	комментарий
J	—	0	старт
I	$IJ(3)$	3	единственное ребро из J
D	$DI(1)$	4	$DI(1) < IF(2)$
F	$IF(2)$	6	$IF(2) < AD(4) < DH(5) = DF(5) < \dots$
A	$AD(4)$	10	$AD(4) < DH(5) < FH(6) < \dots$
H	$DH(5)$	15	$DH(5) < AB(6) = FH(6) < DC(7)$
B	$AB(6)$	21	$AB(6) < DC(7) < BD(9)$
C	$BC(1)$	22	после прихода B открылось ребро $BC(1)$
E	$CE(8)$	30	единственное ребро к последней вершине E

Минимальное остовное дерево: $IJ, DI, IF, AD, DH, AB, BC, CE$; суммарный вес $\sum \sigma = 30$.

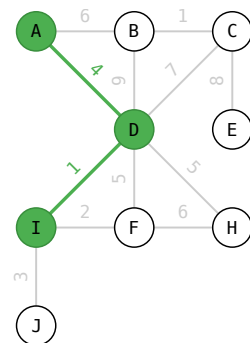
Работа алгоритма по шагам (выбранные рёбра — зелёные, текущее добавляемое — оранжевое, недоступные пока рёбра — серые):



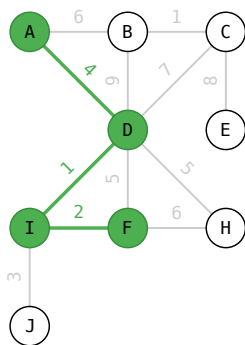
Шаг 0. Старт: $T = \{J\}$



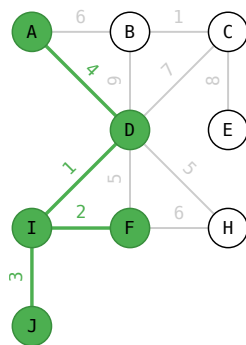
Шаг 1. $+IJ(3)$, $\Sigma = 3$



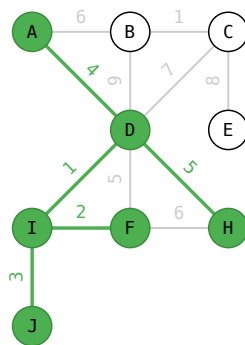
Шаг 2. $+DI(1)$, $\Sigma = 4$



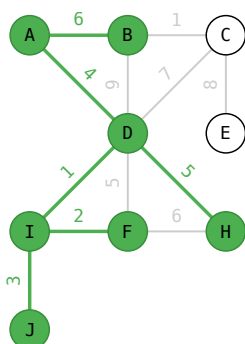
Шаг 3. $+IF(2)$, $\Sigma = 6$



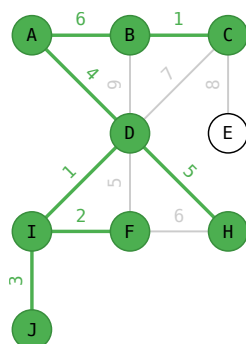
Шаг 4. $+AD(4)$, $\Sigma = 10$



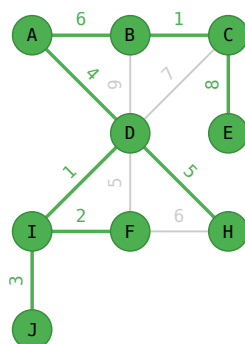
Шаг 5. $+DH(5)$, $\Sigma = 15$



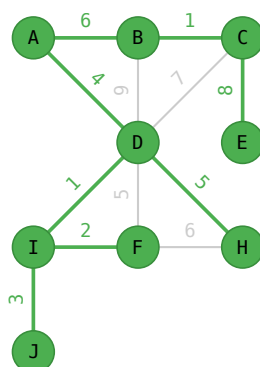
Шаг 6. $+AB(6)$, $\Sigma = 21$



Шаг 7. $+BC(1)$, $\Sigma = 22$



Шаг 8. $+CE(8)$, $\Sigma = 30$



Шаг 9. Остов построен, $\Sigma = 30$

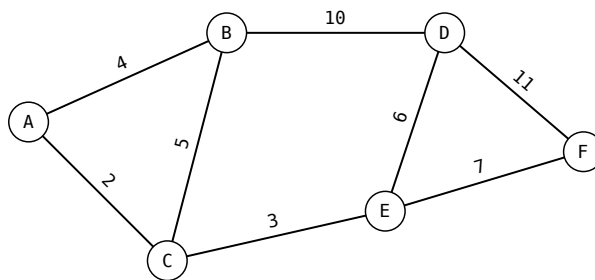
Замечание. Если все веса рёбер различны, минимальное остовное дерево единственно — и алгоритм Прима находит именно его независимо от выбора стартовой вершины.

Задача 8. Минимальное остовное дерево: алгоритм Прима

Условие. Дан неориентированный взвешенный граф на вершинах A, B, C, D, E, F с рёбрами

$$AB = 4, \quad AC = 2, \quad BC = 5, \quad BD = 10, \quad CE = 3, \quad ED = 6, \quad EF = 7, \quad DF = 11.$$

Постройте минимальное остовное дерево алгоритмом Прима, начиная с вершины A . Укажите порядок включения рёбер и суммарный вес.

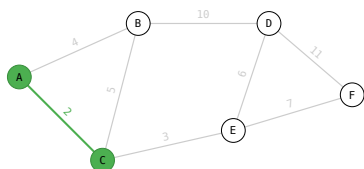


Решение. Алгоритм Прима выращивает дерево из одной вершины, добавляя на каждом шаге ребро наименьшего веса, ведущее из уже построенного дерева в ещё не включённую вершину.

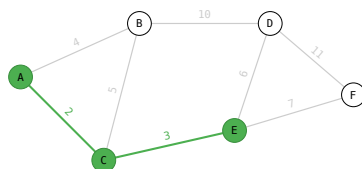
1. Дерево $\{A\}$. Из A выходят $AC = 2$ и $AB = 4$; минимум — $AC = 2$, включаем вершину C .
2. Дерево $\{A, C\}$. Доступны $CE = 3$, $AB = 4$, $BC = 5$; минимум — $CE = 3$, включаем E .
3. Дерево $\{A, C, E\}$. Доступны $AB = 4$, $BC = 5$, $ED = 6$, $EF = 7$; минимум — $AB = 4$, включаем B .
4. Дерево $\{A, C, E, B\}$. Доступны $ED = 6$, $EF = 7$, $BD = 10$; минимум — $ED = 6$, включаем D .
5. Дерево $\{A, C, E, B, D\}$. Доступны $EF = 7$, $DF = 11$; минимум — $EF = 7$, включаем F . Дерево охватило все вершины.

Порядок включения рёбер: $AC(2)$, $CE(3)$, $AB(4)$, $ED(6)$, $EF(7)$.

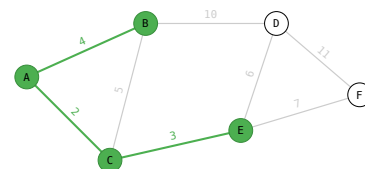
Шаги построения:



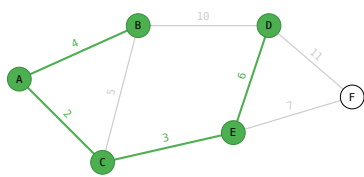
$+AC(2)$



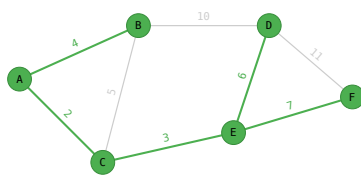
$+CE(3)$



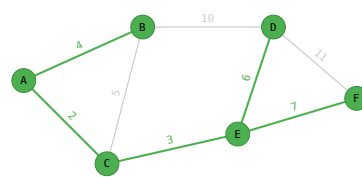
$+AB(4)$



+ED(6)



+EF(7)



Остов построен

Ответ. Минимальное остовное дерево состоит из рёбер AC, CE, AB, ED, EF ; его суммарный вес равен $2 + 3 + 4 + 6 + 7 = 22$.

Лекция 8. Алгоритм Краскала. Главные циклы. Цикловое пространство

$G(V, E)$ — связный, $\sigma: E \rightarrow \mathbb{R}$.
 $\Rightarrow T_G(V, \hat{E})$ — остовное дерево ($\min \sum \sigma(e), e \in \hat{E}$).

Алгоритм 11. Алгоритм Краскала

Step: рёбра перебираются в порядке возрастания веса; очередное ребро e с минимальным $\sigma(e)$ добавляется в остов, если при этом *не появляется цикл* (т.е. концы e лежат в разных компонентах уже построенного леса); иначе ребро пропускается.

Пример (граф той же структуры, что у алгоритма Прима, с другими весами):

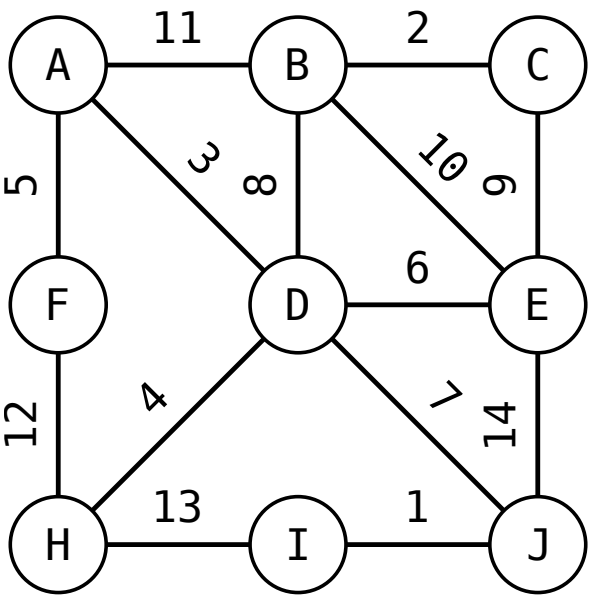


Рис. 35. Алг. Краскала: взвешенный граф 9 вершин

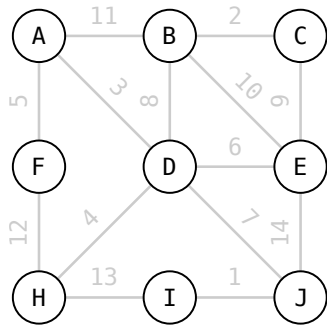
V	e	$\sum \sigma(e)$	комментарий
—	—	0	лес из 9 изолированных вершин
J, I	$JI(1)$	1	соединяем $\{J\}$ и $\{I\}$
B, C	$BC(2)$	3	соединяем $\{B\}$ и $\{C\}$
A, D	$AD(3)$	6	соединяем $\{A\}$ и $\{D\}$
H	$HD(4)$	10	H присоединяется к $\{A, D\}$
F	$AF(5)$	15	F присоединяется к $\{A, D, H\}$
E	$DE(6)$	21	E присоединяется к $\{A, D, H, F\}$
—	$JD(7)$	28	сливаем $\{J, I\}$ и $\{A, D, H, F, E\}$
—	$BD(8)$	36	сливаем $\{B, C\}$ с остальными; 8 рёбер — стоп

Оставшиеся рёбра $EC(9), BE(10), AB(11), FH(12), HI(13), EJ(14)$ замыкали бы циклы и пропускаются. Минимальное остовное дерево: $JI, BC, AD, HD, AF, DE, JD, BD$, суммарный вес $\sum \sigma = 36$.

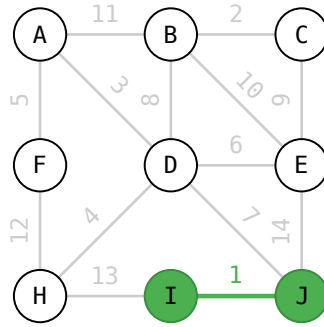
Упр. Почему алгоритм сойдётся (построит остов)?

Ответ. Каждое добавленное ребро уменьшает число компонент леса ровно на 1: начали с $|V|$ компонент, дерево получится после $|V| - 1$ добавлений. Застыть алгоритм не может: пока компонент больше одной, в связном графе обязательно найдётся ребро между разными компонентами — и при просмотре списка оно не будет пропущено.

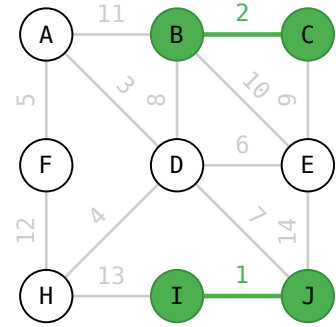
Работа алгоритма по шагам (добавленные рёбра — зелёные, текущее — оранжевое, пропущенные — бледно-серые):



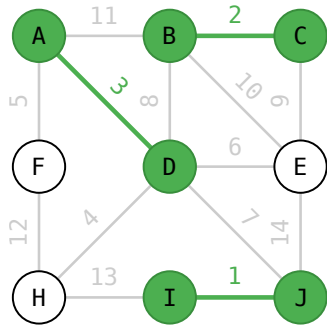
Шаг 0. Лес из изолированных вершин



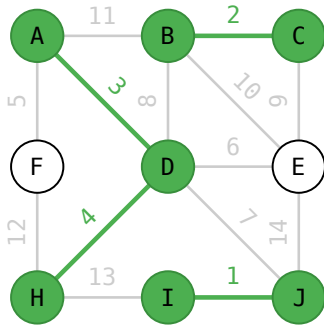
Шаг 1. $+JI(1)$



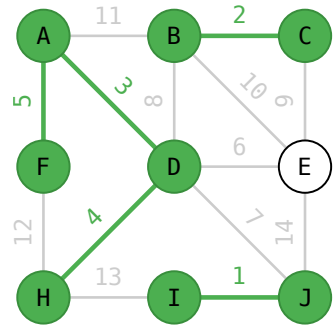
Шаг 2. $+BC(2)$



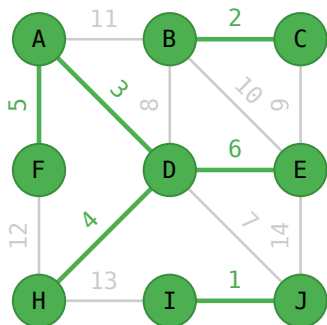
Шаг 3. $+AD(3)$



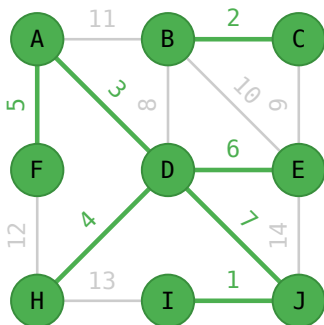
Шаг 4. $+HD(4)$



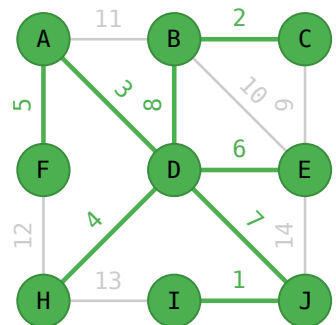
Шаг 5. $+AF(5)$



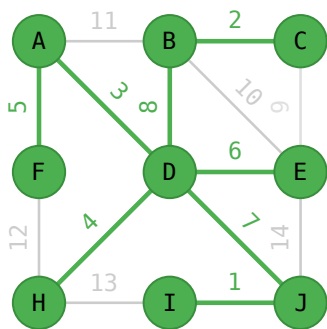
Шаг 6. $+DE(6)$



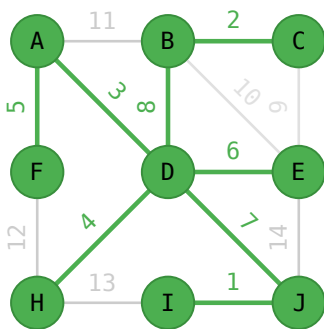
Шаг 7. $+JD(7)$



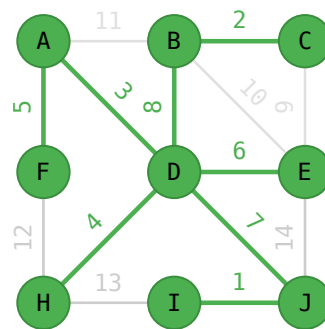
Шаг 8. $+BD(8)$: остов готов



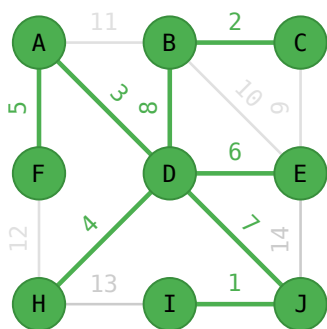
Шаг 9. $EC(9)$ — цикл, пропуск



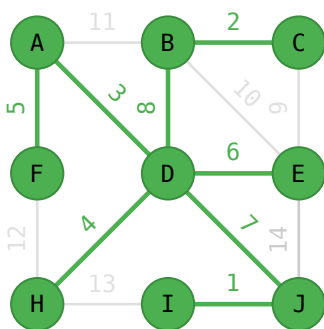
Шаг 10. $BE(10)$ — пропуск



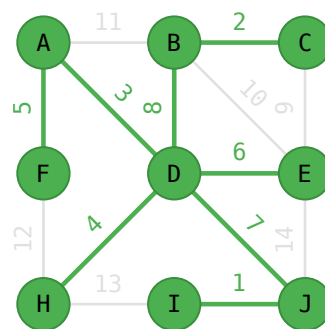
Шаг 11. $AB(11)$ — пропуск



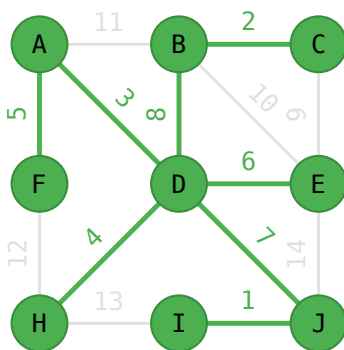
Шаг 12. $FH(12)$ — пропуск



Шаг 13. $HI(13)$ — пропуск



Шаг 14. $EJ(14)$ — пропуск



Шаг 15. Остов построен, $\Sigma = 36$

Главные циклы

Определение 38. Хорда

$G(V, E)$ — связный, невзвешенный.

$T_G(V, \hat{E})$ — остовное дерево G .

Хорда $\leftrightarrow e \in (E \setminus \hat{E})$.

Зафиксируем в графе остовное дерево; все рёбра вне остова — хорды:

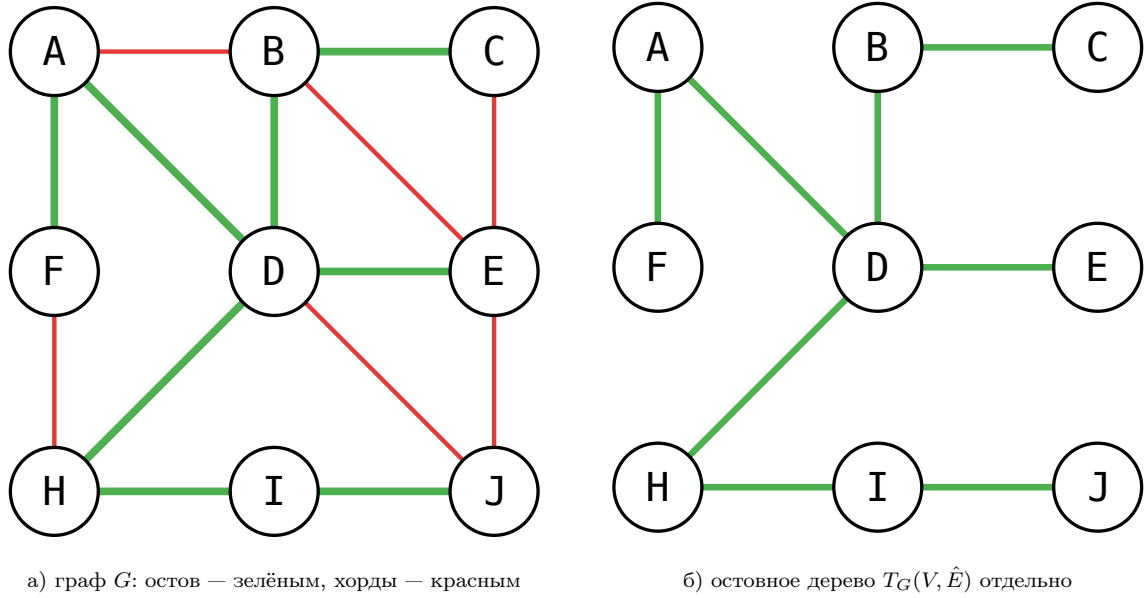


Рис. 36. Главные циклы: граф с хордами и его остов

$G(V, E): T_G(V, \hat{E}), E \setminus \hat{E} = \{e_1, \dots, e_s\}$ — набор хорд.

Добавление к дереву любой хорды e_i создаёт ровно один цикл: $\forall 1 \leq i \leq s: T_G(V, \hat{E}) \cup \{e_i\}$ содержит цикл $\mathcal{E}_i$ — **главный** (хорда e_i плюс единственная цепь в дереве между её концами).

$\mathcal{E}_1, \dots, \mathcal{E}_s$ — набор из s главных циклов.

Пример. Остов $\hat{E} = \{BC, AD, BD, AF, DH, HI, IJ, DE\}$ (зелёные рёбра на рисунке), хорды $E \setminus \hat{E} = \{AB, DJ, BE, FH, EJ, CE\}$ (красные), $s = 6$. Для каждой хорды выписываем цепь в дереве между её концами и замыкаем:

1. $e_1 = AB$: цепь в дереве $A-D-B \Rightarrow \mathcal{E}_1 = ABDA$
2. $e_2 = DJ$: цепь $D-H-I-J \Rightarrow \mathcal{E}_2 = DH IJD$
3. $e_3 = BE$: цепь $B-D-E \Rightarrow \mathcal{E}_3 = DBED$
4. $e_4 = FH$: цепь $F-A-D-H \Rightarrow \mathcal{E}_4 = DAFHD$
5. $e_5 = EJ$: цепь $E-D-H-I-J \Rightarrow \mathcal{E}_5 = DH IJED$
6. $e_6 = CE$: цепь $C-B-D-E \Rightarrow \mathcal{E}_6 = CBDEC$

Цикловое пространство

Определение 39. Сумма циклов

$G(V, E)$, l_1, l_2 — 2 цикла (замкнутых пути) в графе.

$l_1 \oplus l_2 := (e \in l_1) \Delta (e \in l_2)$

(симметрическая разность рёбер).

$A \Delta B = (A \cup B) \setminus (B \cap A) = \{a \in A \& a \notin B\} \cup \{a \notin A \& a \in B\}$: общие рёбра «сокращаются», остальные остаются.

Пример 1. $\mathcal{E}_1 = ABDA$ (рёбра AB, BD, DA), $\mathcal{E}_6 = CBDEC$ (рёбра CB, BD, DE, EC). Общее ребро BD сокращается:

$$\mathcal{E}_1 \oplus \mathcal{E}_6 = \{AB, DA, CB, DE, EC\} = ABCEDA$$

— снова цикл $(A-B-C-E-D-A)$.

Пример 2. $\mathcal{E}_5 = DH I J E D$ (рёбра DH, HI, IJ, JE, ED), $\mathcal{E}_4 = D A F H D$ (рёбра DA, AF, FH, HD).
Общее ребро DH сокращается:

$$\mathcal{E}_5 \oplus \mathcal{E}_4 = \{HI, IJ, JE, ED, DA, AF, FH\} = ADEJIHFA$$

— цикл $A-D-E-J-I-H-F-A$.

Пример 3. Сложим все четыре:

$$l = \mathcal{E}_1 \oplus \mathcal{E}_6 \oplus \mathcal{E}_5 \oplus \mathcal{E}_4 = (\mathcal{E}_1 \oplus \mathcal{E}_6) \oplus (\mathcal{E}_5 \oplus \mathcal{E}_4).$$

У циклов $ABCEDA$ и $ADEJIHFA$ общие рёбра AD и DE сокращаются:

$$l = \{AB, BC, CE, EJ, JI, IH, HF, FA\} = ABCEJIHFA$$

— цикл по «внешнему контуру» графа, составленный из главных циклов $\mathcal{E}_1, \mathcal{E}_6, \mathcal{E}_5, \mathcal{E}_4$.

Замечание.

- а) Множество всех «обобщённых циклов» (рёберно-непересекающихся объединений циклов, включая пустое множество рёбер) с операцией $\oplus$ образует векторное пространство над полем $\mathbb{F}_2$ — **цикловое пространство** графа;
- б) главные циклы $\mathcal{E}_1, \dots, \mathcal{E}_s$ — базис этого пространства: они линейно независимы (каждый содержит «свою» хорду, которой нет в остальных), и любой цикл l раскладывается по ним — l есть сумма главных циклов тех хорд, которые входят в l ;
- в) размерность циклового пространства равна числу хорд: $s = |E| - |\hat{E}| = |E| - (|V| - 1) = |E| - |V| + 1$ (*цикломатическое число* графа). В примере выше: $s = 14 - 9 + 1 = 6$.

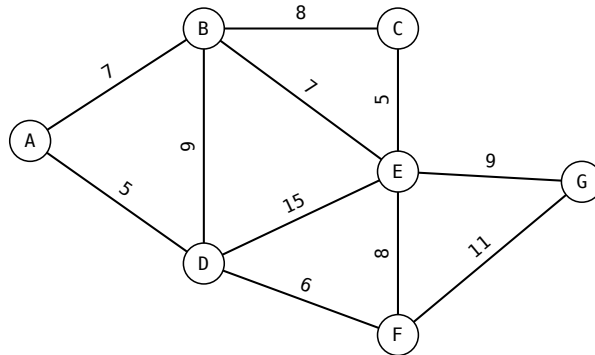
Упр. $G(V, E)$ — связный, fix $T_G(V, \hat{E})$, $\mathcal{E}_1, \dots, \mathcal{E}_s$ — главные циклы. Доказать аккуратно пункты а) и б) замечания: что $(\{\text{обобщённые циклы}\}, \oplus)$ — векторное пространство над $\mathbb{F}_2$ и что $\mathcal{E}_1, \dots, \mathcal{E}_s$ — его базис.

Задача 9. Минимальное остовное дерево: алгоритм Краскала

Условие. Дан неориентированный взвешенный граф на вершинах A, B, C, D, E, F, G с рёбрами

$$\begin{aligned} AB = 7, \quad AD = 5, \quad BC = 8, \quad BD = 9, \quad BE = 7, \quad CE = 5, \\ DE = 15, \quad DF = 6, \quad EF = 8, \quad EG = 9, \quad FG = 11. \end{aligned}$$

Постройте минимальное остовное дерево алгоритмом Краскала. Укажите порядок включения рёбер и суммарный вес.



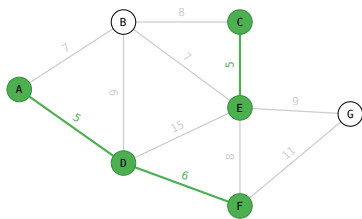
Решение. Краскал сортирует рёбра по возрастанию веса и добавляет ребро, если его концы лежат в разных компонентах (что отслеживается системой непересекающихся множеств) — иначе ребро замкнуло бы цикл.

Отсортированный список рёбер:

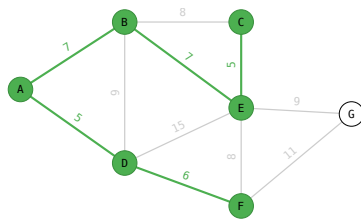
$AD(5), CE(5), DF(6), AB(7), BE(7), BC(8), EF(8), BD(9), EG(9), FG(11), DE(15)$.

1. $AD(5)$ — компоненты $\{A\}, \{D\}$ различны, добавляем $\Rightarrow \{A, D\}$.
2. $CE(5)$ — добавляем $\Rightarrow \{C, E\}$.
3. $DF(6)$ — F присоединяется к $\{A, D\}$, добавляем $\Rightarrow \{A, D, F\}$.
4. $AB(7)$ — B присоединяется, добавляем $\Rightarrow \{A, B, D, F\}$.
5. $BE(7)$ — сливаем $\{A, B, D, F\}$ и $\{C, E\}$, добавляем $\Rightarrow \{A, B, C, D, E, F\}$.
6. $BC(8)$ — B и C уже в одной компоненте: *цикл, пропускаем*.
7. $EF(8)$ — цикл, пропускаем.
8. $BD(9)$ — цикл, пропускаем.
9. $EG(9)$ — G присоединяется, добавляем. Все 7 вершин в одной компоненте, набрано $6 = 7 - 1$ рёбер — остов построен.
10. $FG(11), DE(15)$ — цикл, пропускаем.

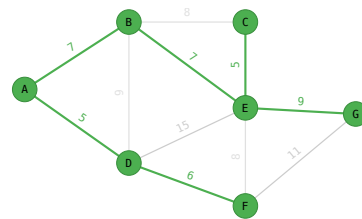
Ключевые шаги построения:



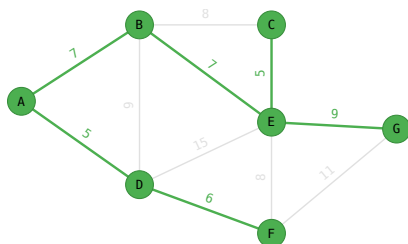
$+AD, +CE, +DF$



$+AB, +BE$



пропуски BC, EF, BD ; $+EG$



Остов построен, $\Sigma = 39$

Ответ. Остовное дерево: рёбра AD, CE, DF, AB, BE, EG ; суммарный вес $5 + 5 + 6 + 7 + 7 + 9 = 39$.

Лекция 9. Двудольные графы. Паросочетания. Задача о максимальном потоке

Двудольные графы

Определение 40. Двудольный граф

$G(V, E)$ — двудольный:

$V = V_1 \sqcup V_2$ т. ч. $\forall e \in E: u \in V_1, v \in V_2$ (начальная вершина — любая).

Примеры (доля V_1 — синяя, доля V_2 — оранжевая):

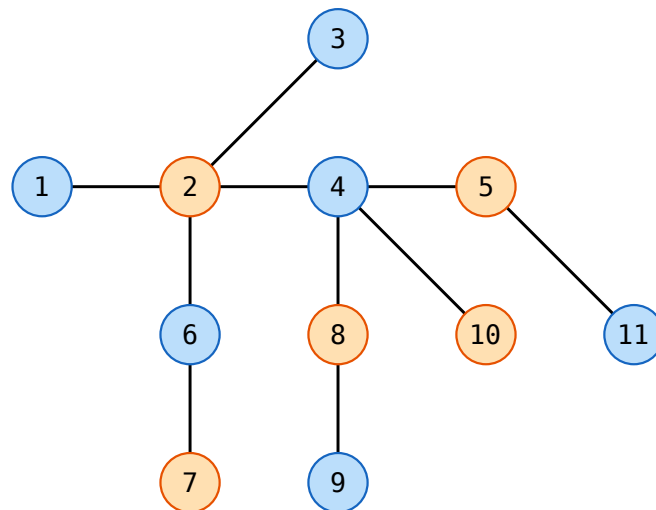


Рис. 37. Двудольный: дерево 11 вершин

Пример 1. Любое дерево двудольно: раскрашиваем вершины по чётности уровня. Здесь $V_1 = \{1, 3, 4, 6, 9, 11\}$ (чётные уровни), $V_2 = \{2, 5, 7, 8, 10\}$ (нечётные) — каждое ребро соединяет соседние уровни, т. е. разные доли.

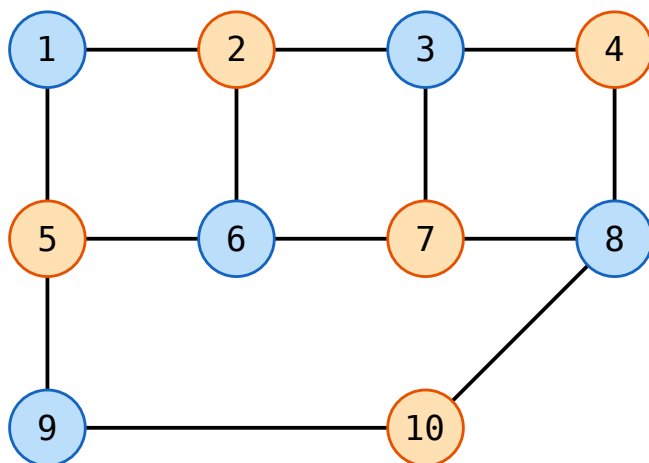


Рис. 38. Двудольный: решётка 10 вершин

Пример 2. Решётка двудольна («шахматная» раскраска): $V_1 = \{1, 3, 6, 8, 9\}$, $V_2 = \{2, 4, 5, 7, 10\}$ — у каждого ребра концы разного цвета.

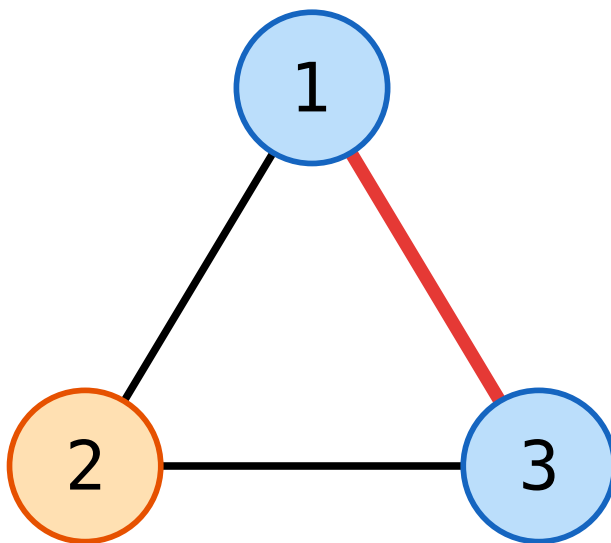


Рис. 39. НЕ двудольный: треугольник

Пример 3. Треугольник не двудолен: положив $1 \in V_1$, получаем $2 \in V_2$, $3 \in V_1$ — но тогда ребро 3–1 (красное) соединяет две вершины одной доли. Куда ни относи вершины, одно ребро всегда останется внутри доли.

Теорема 12. Критерий двудольности графа

G — двудольный $\Leftrightarrow G$ не содержит циклов нечётной длины.

Доказательство. ($\Rightarrow$) Если граф двудолен, нечётных циклов нет. Пусть $V = V_1 \sqcup V_2$ — доли, и все рёбра идут между ними. Любой цикл $v_0 v_1 \dots v_k v_0$ при движении по рёбрам

поочерёдно меняет долю: $v_0 \in V_1 \Rightarrow v_1 \in V_2 \Rightarrow v_2 \in V_1$ и так далее. Чтобы вернуться в исходную долю (в вершину v_0), число шагов должно быть чётным, поэтому длина любого цикла чётна — нечётных циклов нет.

($\Leftarrow$) **Если нечётных циклов нет, граф двудолен.** Достаточно доказать утверждение для каждой связной компоненты по отдельности. Зафиксируем в компоненте вершину s и для каждой вершины v рассмотрим расстояние $d(s, v)$ (длину кратчайшего пути, BFS-нумерация). Положим

$$V_1 = \{v: d(s, v) \text{ чётно}\}, \quad V_2 = \{v: d(s, v) \text{ нечётно}\}.$$

Покажем, что любое ребро uv соединяет вершины из разных долей. Предположим противное: пусть u, v лежат в одной доле, то есть $d(s, u)$ и $d(s, v)$ одной чётности. Рассмотрим замкнутый маршрут: кратчайший путь $s \rightarrow u$, ребро uv и кратчайший путь $v \rightarrow s$. Его длина равна $d(s, u) + 1 + d(s, v)$ — сумма двух чисел одной чётности плюс единица, то есть *нечётна*. Замкнутый маршрут нечётной длины обязательно содержит простой цикл нечётной длины, что противоречит условию. Значит концы каждого ребра попадают в разные доли, и разбиение $V = V_1 \sqcup V_2$ задаёт двудольность.

Иллюстрация. Цикл *чётной* длины (C_8 слева) раскрашивается чередованием $V_1, V_2, V_1, V_2, \dots$ — двудолен. В цикле *нечётной* длины (C_7 справа) при таком чередовании последняя вершина оказывается смежной с первой вершиной той же доли (красное ребро) — разбить на две доли невозможно.

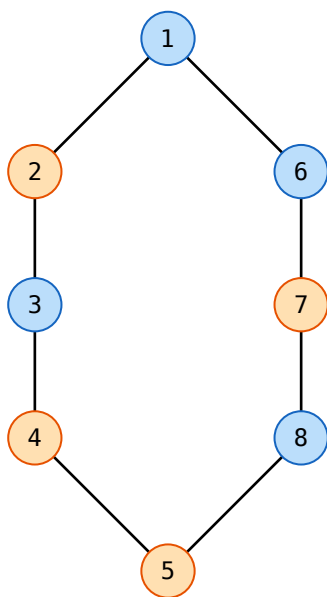


Рис. 40. Критерий: чётный цикл C_8 (двудольный)

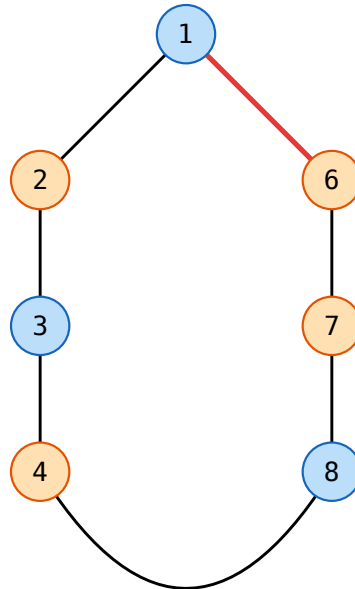


Рис. 41. Критерий: нечётный цикл C_7 (не двудольный)

□

Паросочетания

Определение 41. Паросочетание

$G(V, E)$ — двудольный, $V = V_1 \sqcup V_2$.

Паросочетание для G : $\hat{E} \subseteq E$ т. ч. $\forall e_i, e_j \in \hat{E}$ — инцидентны не более чем одной общей вершине (т. е. рёбра паросочетания попарно не имеют общих вершин).

Определение 42. Максимальное паросочетание

Максимальное паросочетание — максимальное по включению: к нему нельзя добавить ни одно ребро графа, не нарушив определение.

Определение 43. Наибольшее паросочетание

Наибольшее паросочетание — наибольшее количество рёбер для данного графа.

Примеры. Один и тот же граф — цепь 1–2–3–4; рёбра паросочетания — зелёные жирные, насыщенные ими вершины — закрашены:

не макс., не наиб.

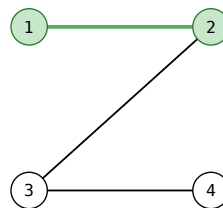


Рис. 42. Паросочетание $\{1-2\}$: не макс., не наиб.

Пример 1. $\hat{E} = \{1-2\}$ — **не максимальное**: можно добавить ребро 3–4 (его концы свободны); значит, и **не наибольшее**.

макс., не наиб.

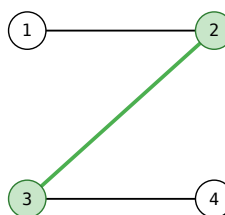


Рис. 43. Паросочетание $\{2-3\}$: макс., но не наиб.

Пример 2. $\hat{E} = \{2-3\}$ — **максимальное**: оба оставшихся ребра (1–2 и 3–4) задевают занятые вершины 2 и 3, добавить нечего. Но **не наибольшее**: в примере 3 есть паросочетание из двух рёбер.

макс. и наиб.

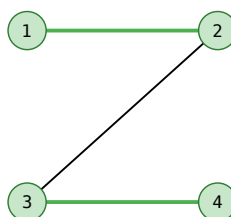


Рис. 44. Паросочетание $\{1-2, 3-4\}$: макс. и наиб.

Пример 3. $\hat{E} = \{1-2, 3-4\}$ — **максимальное и наибольшее**: все 4 вершины насыщены, больше двух непересекающихся рёбер в графе из 4 вершин быть не может.

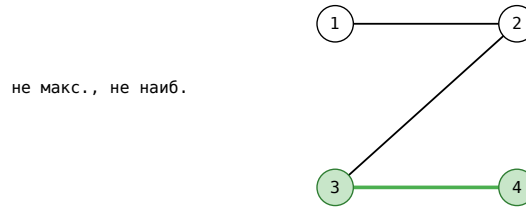


Рис. 45. Паросочетание $\{3-4\}$: не макс., не наиб.

Пример 4. $\hat{E} = \{3-4\}$ — симметричный к примеру 1 случай: можно добавить ребро 1–2, поэтому паросочетание **не максимальное** (и не наибольшее).

НВ: $\max |\hat{E}| \leq \min(|V_1|, |V_2|)$ — каждое ребро паросочетания занимает по одной вершине в каждой доле, поэтому рёбер не больше, чем вершин в меньшей доле.

Упр. Доказать, что:

- наибольшее $\Rightarrow$ максимальное;
- максимальное $\nRightarrow$ наибольшее.

Решение. Если бы наибольшее паросочетание не было максимальным, к нему можно было бы добавить ребро и получить паросочетание большего размера — противоречие с «наибольшестью». Обратная импликация неверна: контрпример — примеры 2 и 3 выше ($\{2-3\}$ максимально, но меньше наибольшего).

Упр. Пример G с наибольшим паросочетанием $\hat{E}$, т. ч. $|\hat{E}| < \min(|V_1|, |V_2|)$.

Решение. «Звезда»: $V_1 = \{a, b\}$, $V_2 = \{x, y\}$, рёбра $a-x$, $a-y$. Все рёбра проходят через a , поэтому любое паросочетание содержит не более одного ребра: $|\hat{E}| = 1 < 2 = \min(|V_1|, |V_2|)$.

Алгоритм 12. Алгоритм поиска наибольшего паросочетания

Дано: $G(V, E)$ — двудольный, $V = V_1 \sqcup V_2$.

Найти: наибольшее паросочетание $\hat{E}$.

Инициализация. Добавляем фиктивные вершины S и F :

$$\forall v_i \in V_1: \text{ дуга } S \rightarrow v_i; \quad \forall w_j \in V_2: \text{ дуга } w_j \rightarrow F,$$

а все рёбра графа ориентируем слева направо: $v_i \rightarrow w_j$.

Step. Пока существует путь $l: S \rightarrow \dots \rightarrow F$:

1. находим такой путь (любым обходом);
2. *инвертируем* на нём все рёбра между долями: пройденное ребро $v_i \rightarrow w_j$ разворачивается в $w_j \rightarrow v_i$ — развёрнутые рёбра и есть текущее паросочетание;

3. крайние дуги пути $(S \rightarrow v_i \text{ и } w_j \rightarrow F)$ удаляем — вершины v_i и w_j насыщены.

Когда путей из S в F больше нет, развёрнутые рёбра $w_j \rightarrow v_i$ образуют наибольшее паросочетание.

Почему это работает: путь $S \rightarrow v_i \rightarrow w_j \rightarrow v_k \rightarrow \dots \rightarrow F$, проходящий по развёрнутому ребру $w_j \rightarrow v_k$, — это *увеличивающий путь*: он отбирает w_j у вершины v_k и тут же находит ей новую пару, а в конце насыщает одну новую вершину каждой доли. Число рёбер паросочетания каждый раз растёт на 1.

Пример. $V_1 = \{A, B, C, D\}$, $V_2 = \{\alpha, \beta, \gamma\}$, рёбра: $A\alpha, A\beta, B\alpha, B\beta, C\beta, C\gamma, D\alpha, D\gamma$.

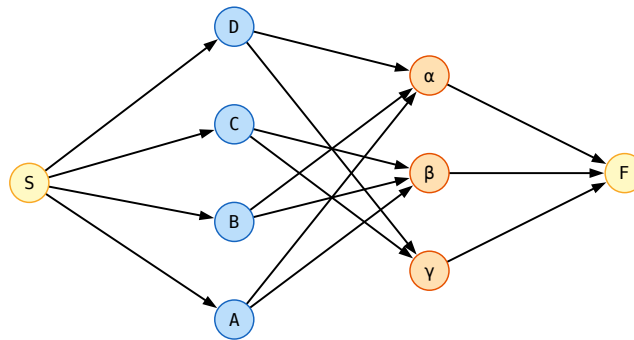


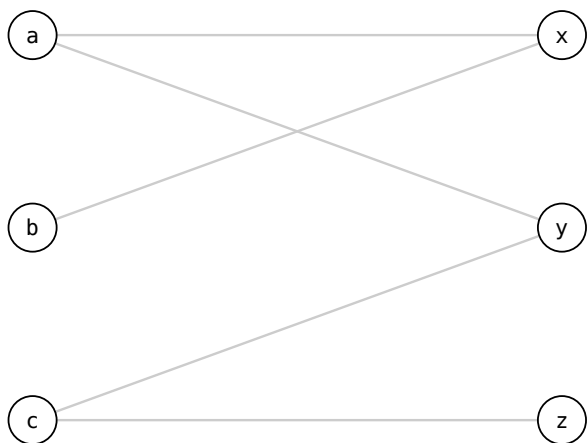
Рис. 46. Алг. наиб. паросочетания: S , доли $\{A, B, C, D\}$ и $\{\alpha, \beta, \gamma\}$, F

Находим пути и инвертируем (стрелки «туда-обратно»):

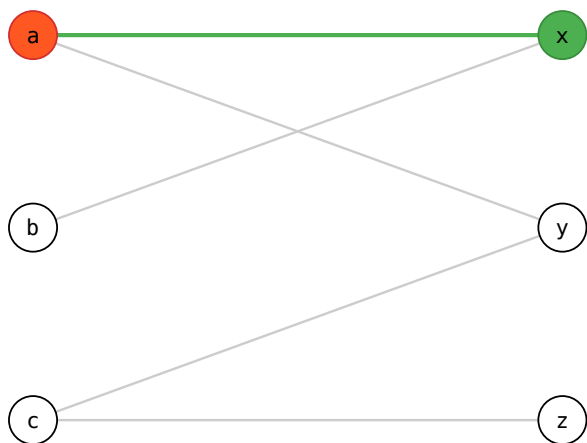
1. S, B, β, F — ребро $B\beta$ разворачивается: $\beta \rightarrow B$; паросочетание $\{B\beta\}$;
2. $S, C, \beta, B, \alpha, F$ — путь идёт по развёрнутому ребру $\beta \rightarrow B$ (отбираем β у B): теперь $\beta \rightarrow C$, $\alpha \rightarrow B$; паросочетание $\{C\beta, B\alpha\}$;
3. $S, A, \beta, C, \gamma, F$ — аналогично β переходит к A , а C получает γ ; паросочетание $\{A\beta, B\alpha, C\gamma\}$.

Итог: наибольшее паросочетание $\hat{E} = \{A\beta, B\alpha, C\gamma\}$, $|\hat{E}| = 3 = \min(|V_1|, |V_2|)$ — вершина D осталась свободной.

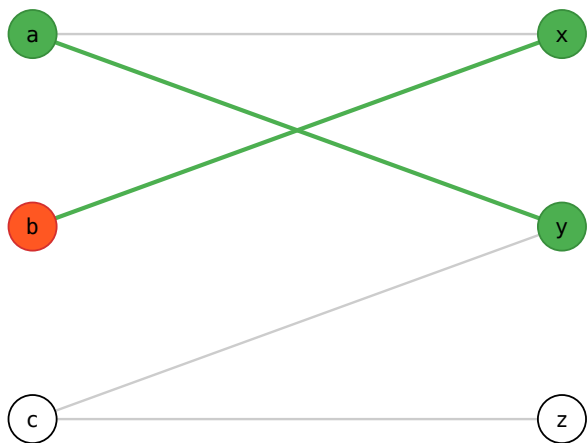
Работа алгоритма по шагам (текущий путь — оранжевый, развёрнутые рёбра паросочетания — зелёные, удалённые дуги — бледные):



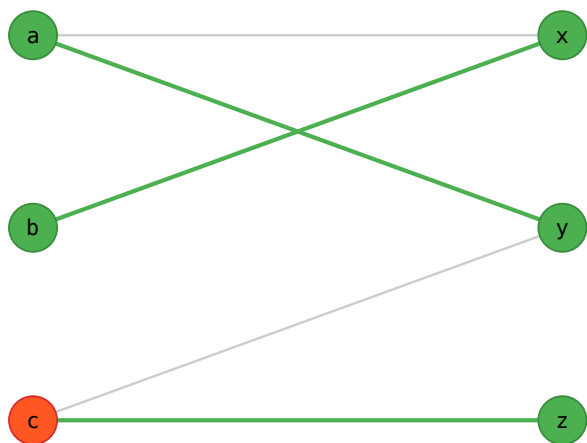
Шаг 0. Сеть с фиктивными S и F



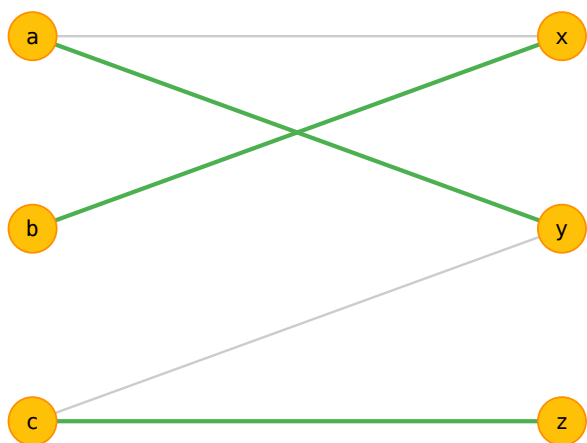
Шаг 1. Путь 1: S, B, β, F



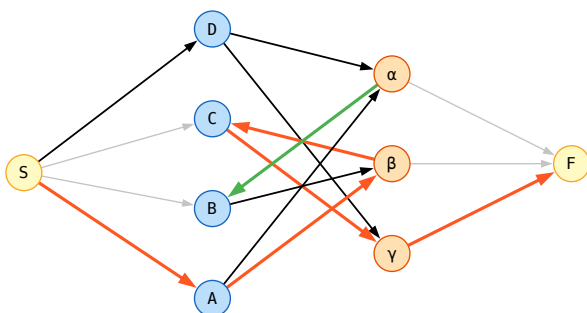
Шаг 2. $B\beta$ развёрнуто: $\{B\beta\}$



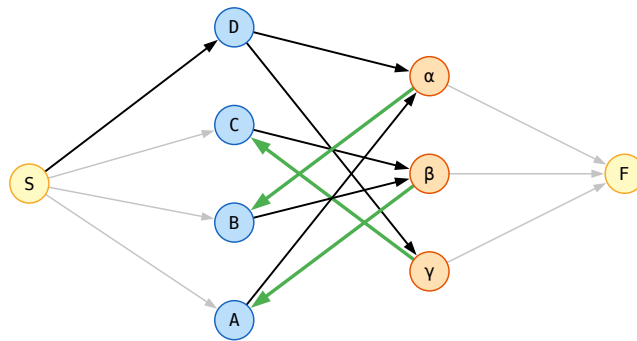
Шаг 3. Путь 2: $S, C, \beta, B, \alpha, F$



Шаг 4. β перешла к C , B получил α



Шаг 5. Путь 3: $S, A, \beta, C, \gamma, F$



Шаг 6. Итог: $\{A\beta, B\alpha, C\gamma\}$, путей $S \rightarrow F$ больше нет

Упр. Почему паросочетание наибольшее?

Ответ (идея, лемма Бернса). Паросочетание наибольшее $\Leftrightarrow$ для него нет увеличивающего пути (чередующегося пути между двумя свободными вершинами). Алгоритм останавливается ровно тогда, когда пути $S \rightarrow F$ нет, т. е. нет и увеличивающего пути. Если бы существовало паросочетание $\hat{E}'$ с $|\hat{E}'| > |\hat{E}|$, то в симметрической разности $\hat{E} \Delta \hat{E}'$ (каждая вершина задета не более чем двумя рёбрами — это набор путей и чётных циклов) нашлась бы компонента-путь, где рёбер $\hat{E}'$ больше, чем рёбер $\hat{E}$, — это и есть увеличивающий путь для $\hat{E}$. Противоречие.

Задача о максимальном потоке в сети

Определение 44. Сеть

$G(V, E)$ — **сеть**: (орграф)

- **исток**: S — нет входящих рёбер!
- **сток/устье**: F — нет исходящих рёбер!

$\sigma: E \rightarrow \mathbb{R}_{\geq 0}$ (пропускная способность, пока на рёбрах).

Определение 45. Поток

Поток: $f: E \rightarrow \mathbb{R}$, свойства:

1. $0 \leq f(e) \leq \sigma(e)$

2. **Условие балансировки**: $\forall v \in V \setminus \{S, F\}$: $\sum_{\substack{e: u \rightarrow v \\ u \in V}} f(e) = \sum_{\substack{e: v \rightarrow w \\ w \in V}} f(e)$

(поток входящий = поток исходящий)

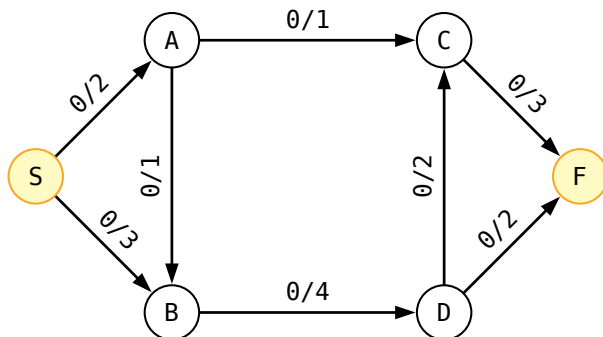
Определение 46. Мощность потока

Мощность потока $|f|$ = суммарное количество перемещаемого в единицу времени (всё, что вытекает из S).

$|f| \rightarrow \max$

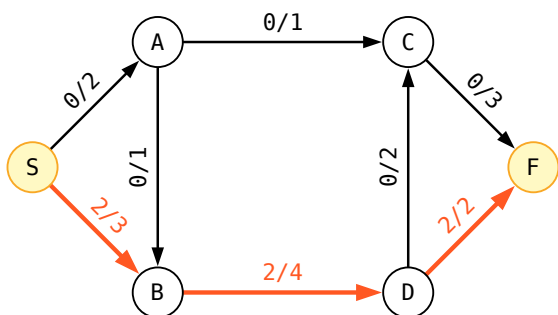
Поток — функция, НЕ число!

Пример. Сеть с пропускными способностями (подпись ребра — f/σ , т. е. «поток / пропускная способность»):

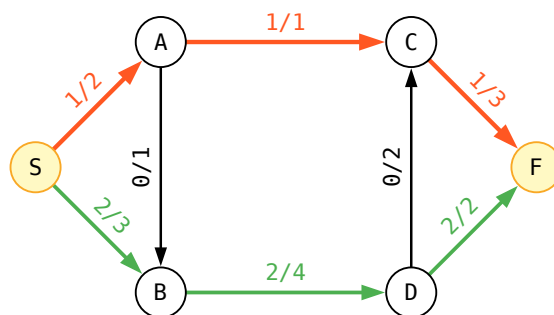


Ищем пути из S в F и пускаем по каждому максимум возможного — *минимум остаточных пропускных способностей рёбер пути* (самое «узкое горлышко» пути ограничивает весь путь):

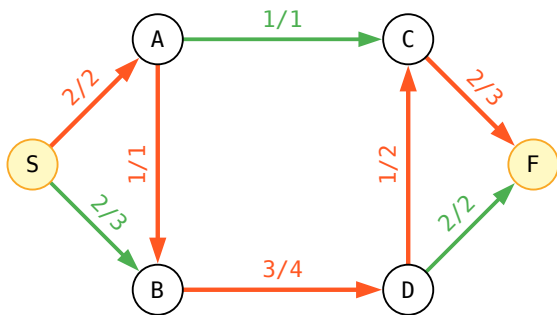
1. путь S, B, D, F : остатки рёбер 3, 4, 2, узкое место — DF , $\min = 2$ — пускаем 2;
2. путь S, A, C, F : остатки 2, 1, 3, узкое место AC , $\min = 1$ — пускаем 1;
3. путь S, A, B, D, C, F : остатки 1, 1, 2, 2, 2, $\min = 1$ — пускаем 1;
4. путь S, B, D, C, F : остатки 1, 2, 1, 1, $\min = 1$ — пускаем 1.



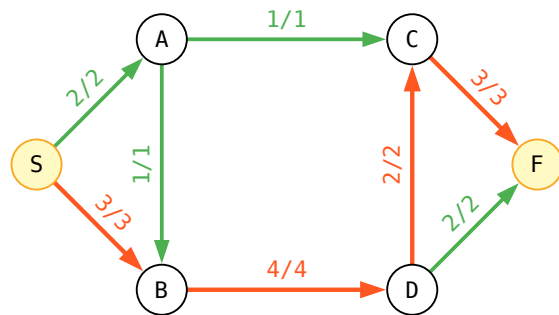
Путь 1: S, B, D, F , $\min = 2$, $|f| = 2$



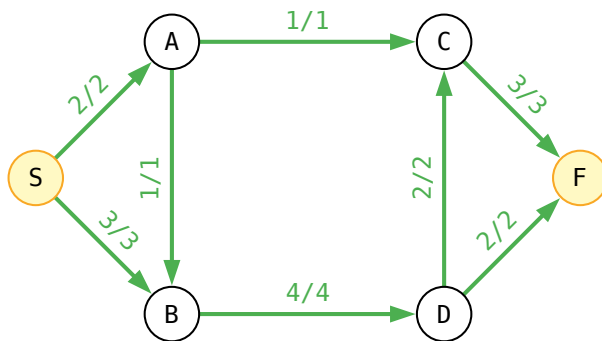
Путь 2: S, A, C, F , $\min = 1$, $|f| = 3$



Путь 3: S, A, B, D, C, F , $\min = 1$, $|f| = 4$



Путь 4: S, B, D, C, F , $\min = 1$, $|f| = 5$



Итог: $|f| = 5$, оба ребра из S насыщены

После каждого пути остаточные веса пересчитываются; при насыщении ребра в общем случае добавляется фиктивное обратное ребро с остаточной ёмкостью (подробно — в лекции про алгоритм Форда–Фалкерсона).

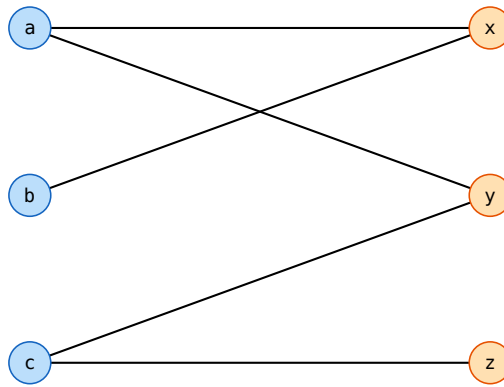
Итог: условие балансировки выполняется в каждой внутренней вершине, и $|f| = 2 + 1 + 1 + 1 = 5$ — максимум: оба ребра из истока ($S \rightarrow A$ и $S \rightarrow B$) насыщены, больше из S вытечь не может.

Задача 10. Наибольшее паросочетание в двудольном графе

Условие. Дан двудольный граф с долями $\{a, b, c\}$ и $\{x, y, z\}$ и рёбрами

$$a-x, \quad a-y, \quad b-x, \quad c-y, \quad c-z.$$

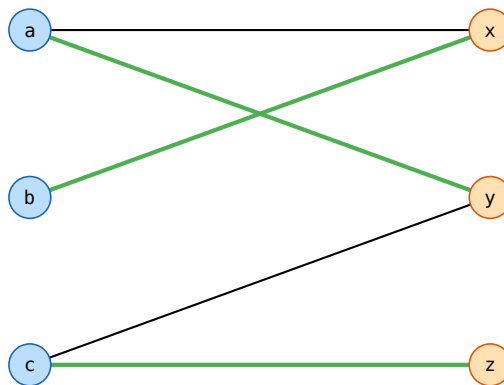
Найдите наибольшее паросочетание (алгоритм Куна, поиск увеличивающих путей).



Решение. Паросочетание — набор попарно несмежных рёбер. Алгоритм Куна по очереди пытается насытить каждую вершину левой доли, ища увеличивающий путь (чередующий путь из свободной вершины в свободную).

1. a : свободно ребро $a-x \Rightarrow$ паросочетание $\{a-x\}$.
2. b : единственное ребро $b-x$, но x занят a . Ищем увеличивающий путь: $b-x$, x занят a , у a есть альтернатива $a-y$ (y свободна). Перекладываем: $a-y$, $b-x$. Паросочетание $\{a-y, b-x\}$.
3. c : ребро $c-y$ ведёт к занятому y ; пробуем $c-z$ (z свободна) $\Rightarrow$ добавляем $c-z$. Паросочетание $\{a-y, b-x, c-z\}$.

Все три вершины левой доли насыщены — паросочетание совершенное, его размер 3 (больше быть не может, так как доли по 3 вершины).



Ответ. Наибольшее паросочетание: $\{a-y, b-x, c-z\}$, размер 3 (совершенное).

Лекция 10. Алгоритм Форда—Фалкерсона

Постановка задачи

Дано: сеть $G(V, E)$, $\sigma: E \rightarrow \mathbb{R}$ — пропускная способность рёбер.

Найти: один из максимальных потоков $f: E \rightarrow \mathbb{R}_{\geq 0}$, т. е. поток наибольшей мощности $|f|$.

Напомним (лекция 9): поток f удовлетворяет ограничению $0 \leq f(e) \leq \sigma(e)$ и условию балансировки во всех вершинах, кроме истока S и стока F .

Остаточная сеть

Определение 47. Остаточная пропускная способность

Для ребра $e = \overline{uv}$ и текущего потока f **остаточная пропускная способность** $\tilde{\sigma}(e)$ показывает, сколько ещё можно пропустить по e . Вводится обратное (фиктивное) ребро $\text{inv}(e) = \overline{vu}$, и

$$\tilde{\sigma}(e) + \tilde{\sigma}(\text{inv}(e)) = \sigma(e).$$

Изначально $\tilde{\sigma}(e) := \sigma(e)$, $\tilde{\sigma}(\text{inv}(e)) := 0$.

Определение 48. Остаточная сеть

Остаточная сеть — сеть на тех же вершинах, рёбрами которой служат все рёбра с положительной остаточной пропускной способностью $\tilde{\sigma}(e) > 0$ (включая обратные рёбра $\text{inv}(e)$).

Определение 49. Исчерпанное (насыщенное) ребро

Исчерпанное ребро — ребро с нулевой остаточной пропускной способностью $\tilde{\sigma}(e) = 0$ (поток по нему равен пропускной способности: $f(e) = \sigma(e)$).

Разрезы

Определение 50. Разрез в сети

Разрез в сети — такой набор рёбер $\tilde{E} \subseteq E$, что удаление его разрывает все пути $S \rightarrow F$, т. е. граф $G(V, E \setminus \tilde{E})$ теряет связность между S и F .

НВ. Любой мост — разрез. Обычно рассматривают $\tilde{E}$, минимальный по включению.

Определение 51. Главный разрез

Главный разрез — разрез, при котором граф делится ровно на две части.

Определение 52. Пропускная способность разреза

Пропускная способность разреза $\tilde{E}$ — сумма пропускных способностей его рёбер, направленных из части истока в часть стока: $\sigma(\tilde{E}) = \sum_{e \in \tilde{E}} \sigma(e)$.

Алгоритм Форда—Фалкерсона

Алгоритм 13. Форда—Фалкерсона (поиск максимального потока)

Инициализация:

$$\forall e \in E: f(e) := 0, \quad \forall e \in E: \tilde{\sigma}(e) := \sigma(e).$$

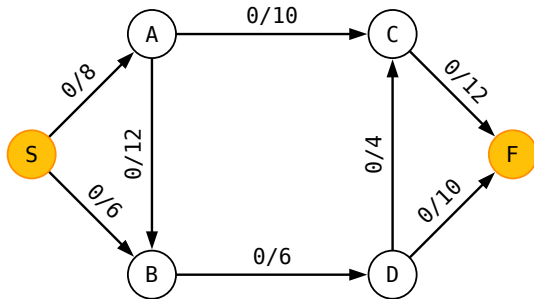
Шаг. Пока в остаточной сети существует путь $S \xrightarrow{e_1} \dots \xrightarrow{e_t} F$ (дополняющий путь):

1. Находим узкое место пути — $\delta := \min_{1 \leq i \leq t} \tilde{\sigma}(e_i)$, и увеличиваем поток вдоль всего пути:
 $\forall e_i: f(e_i) += \delta$.
2. Перестраиваем остаточную сеть: для каждого ребра пути

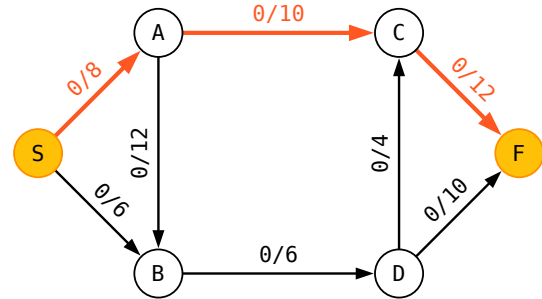
$$\tilde{\sigma}_{\text{new}}(e_i) = \tilde{\sigma}_{\text{old}}(e_i) - \delta, \quad \tilde{\sigma}_{\text{new}}(\text{inv}(e_i)) = \tilde{\sigma}_{\text{old}}(\text{inv}(e_i)) + \delta.$$

Выход: когда дополняющих путей больше нет, текущий поток f — максимальный.

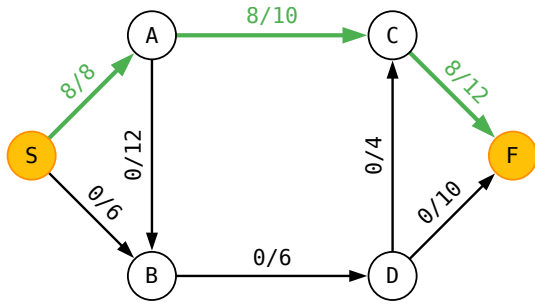
Пример. Сеть с вершинами S, A, B, C, D, F (та же, что в лекции 9). Подпись каждого ребра — f/σ (поток / пропускная способность). Дополняющий путь выделен оранжевым, рёбра с положительным потоком — зелёным. Пути ищем поиском в ширину (кратчайший по числу рёбер — вариант Эдмондса—Карпа).



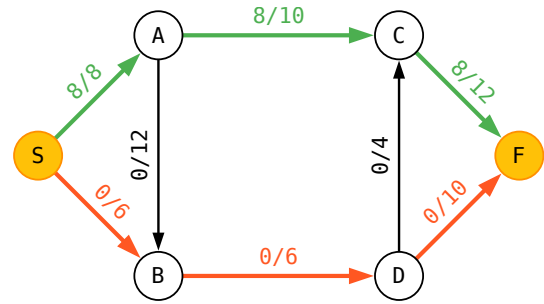
Шаг 0. Инициализация: $f \equiv 0$, $\tilde{\sigma} = \sigma$



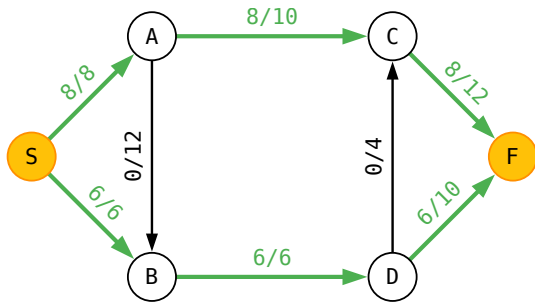
Шаг 1. Путь $S \rightarrow A \rightarrow C \rightarrow F$, $\delta = \min(2, 1, 3) = 1$



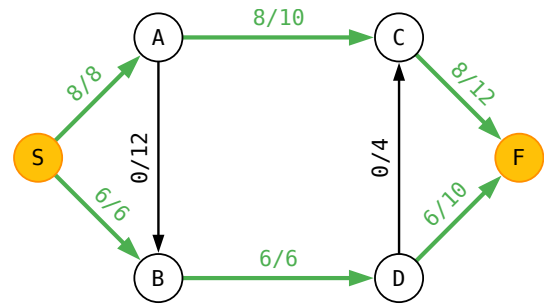
Шаг 2. Поток проведён: $|f| = 1$; $A \rightarrow C$ исчерпано



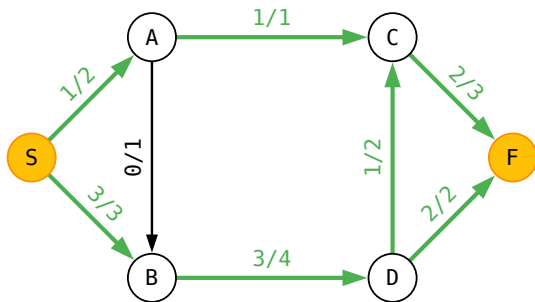
Шаг 3. Путь $S \rightarrow B \rightarrow D \rightarrow F$, $\delta = \min(3, 4, 2) = 2$



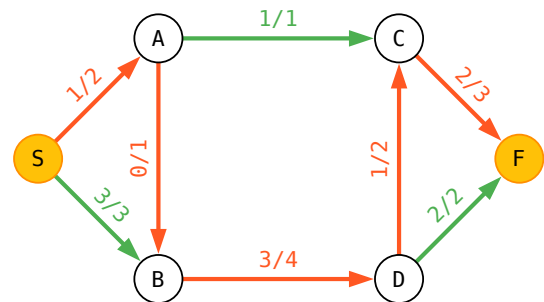
Шаг 4. $|f| = 3$; ребро $D \rightarrow F$ исчерпано



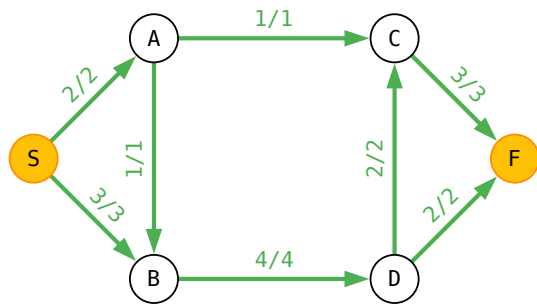
Шаг 5. Путь $S \rightarrow B \rightarrow D \rightarrow C \rightarrow F$, $\delta = 1$



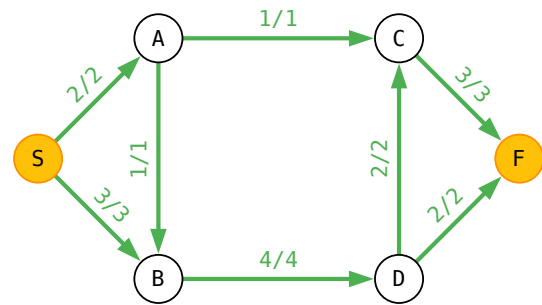
Шаг 6. $|f| = 4$; ребро $S \rightarrow B$ исчерпано



Шаг 7. Путь $S \rightarrow A \rightarrow B \rightarrow D \rightarrow C \rightarrow F$, $\delta = 1$



Шаг 8. $|f| = 5$; ребро $S \rightarrow A$ исчерпано



Шаг 9. Путь $S \rightarrow F$ в остаточной сети нет: $|f|_{\max} = 5$

Комментарии к шагам. На каждом шаге величина потока увеличивается на узкое место δ найденного пути: $1 + 2 + 1 + 1 = 5$. После шага 8 оба ребра из истока ($S \rightarrow A$ и $S \rightarrow B$) исчерпаны: в остаточной сети из S не выходит ни одного ребра, дополняющих путей нет — алгоритм останавливается.

Проверка через разрез. Набор рёбер $\tilde{E} = \{S \rightarrow A, S \rightarrow B\}$ — главный разрез ($\{S\}$ против остальных вершин) с пропускной способностью $\sigma(\tilde{E}) = 2 + 3 = 5$. Мощность любого потока не превосходит пропускной способности любого разреза, поэтому $|f| = 5 = \sigma(\tilde{E})$ — поток максимален, а разрез минимален (теорема Форда–Фалкерсона: $\max |f| = \min \sigma(\tilde{E})$).

Лекция 11. Задача о максимальном потоке в сети

Метод (алгоритм) проталкивания предпотока

Идея метода: в отличие от Форда–Фалкерсона, который наращивает поток целыми путями $S \rightarrow F$, здесь мы разрешаем вершинам временно *накапливать* лишний втекающий поток (избыток) и затем «проталкивать» его дальше — как вода, стекающая по каскаду резервуаров разной высоты.

Определение 53. Предпоток

Пусть G — сеть, σ — пропускная способность рёбер.

Предпоток — это функция $\tilde{f}: V \times V \rightarrow \mathbb{R}$, удовлетворяющая условиям:

1. **Ограниченность:** $\forall e = uv: -\sigma(e) \leq \tilde{f}(u, v) \leq \sigma(e)$;
2. **Кососимметричность:** $\forall u, v \in V: \tilde{f}(u, v) = -\tilde{f}(v, u)$;
3. $\forall \nexists e = uv, e = vu \Rightarrow \tilde{f}(u, v) = 0$.

В отличие от потока, условие балансировки *не требуется*: в вершину может втекать больше, чем вытекает.

Определение 54. Эксцесс вершины

Эксцесс вершины $\forall v \in V$:

$$\varepsilon(v) := \sum_{w \in P(v)} \tilde{f}(wv) - \sum_{u \in \text{Ch}(v)} \tilde{f}(vu),$$

где $P(v)$ — родители, $\text{Ch}(v)$ — дети (т.е. эксцесс — это «втекло минус вытекло»).

НВ. Если $\forall v \in V \setminus \{S, F\}: \varepsilon(v) = 0$, то $\tilde{f}$ — поток.

Определение 55. Высота вершины

Высота вершины — это функция $h: V \rightarrow \mathbb{N}_0$. Инициализируется как $h(S) = |V|$, $h(v) = 0$ для остальных, и в процессе работы только растёт. Проталкивать избыток разрешается только «вниз»: из вершины v в вершину u с $h(u) = h(v) - 1$.

Алгоритм 14. Метод проталкивания предпотока (Push-Relabel)

Инициализация:

1. $h(S) := |V|$; $\forall u \in V \setminus \{S\}: h(u) := 0$.
2. $\tilde{f}_{\text{start}}: \forall v \in \text{Ch}(S): \tilde{f}(S, v) := \sigma(\overline{Sv}), \quad \tilde{f}(v, S) := -\sigma(\overline{Sv});$
 $\forall u, \hat{v} \in V \setminus \{S\}: \tilde{f}(u, v) := 0$ (рёбра из истока насыщаются полностью).

Основной цикл: Пока $\exists v \in V \setminus \{S, F\}$ т.ч. $\varepsilon(v) > 0$ (*активная* вершина):

Push (проталкивание): если $\exists u \in \text{Ch}(v)$ (в остаточной сети) т.ч. $h(u) < h(v)$:

$$\delta_{\text{new}} := \min\{\varepsilon(v), \tilde{\sigma}(\overline{vu})\},$$

$$\begin{aligned}\tilde{f}_{\text{new}}(v, u) &:= \tilde{f}_{\text{old}}(v, u) + \delta_{\text{new}}, & \tilde{f}_{\text{new}}(u, v) &:= -\tilde{f}_{\text{new}}(v, u), \\ \tilde{\sigma}_{\text{new}}(v, u) &:= \tilde{\sigma}_{\text{old}}(v, u) - \delta_{\text{new}}, & \tilde{\sigma}_{\text{new}}(u, v) &:= \tilde{\sigma}_{\text{old}}(u, v) + \delta_{\text{new}}.\end{aligned}$$

Здесь $\tilde{\sigma}(u, v) + \tilde{\sigma}(v, u) = \sigma(v, u)$ для $vu = e$ (остаточные пропускные способности, как в алгоритме Форда–Фалкерсона; проталкивание по обратному ребру означает *возврат* ранее протолкнутого потока).

Lift (подъём): если $\nexists u \in \text{Ch}(v)$ т. ч. $h(u) < h(v)$, то

$$h(v) := \min_{u \in \text{Ch}(v)} h(u) + 1$$

(минимум — по соседям в остаточной сети; после подъёма проталкивание из v снова возможно).

Выход: когда активных вершин не осталось, предпоток стал потоком (все внутренние эксцессы равны нулю), и этот поток — максимальный; $|f| = \varepsilon(F)$.

Пример. Сеть S, A, B, C, D, F с пропускными способностями (та же топология, что в лекциях 9–10):

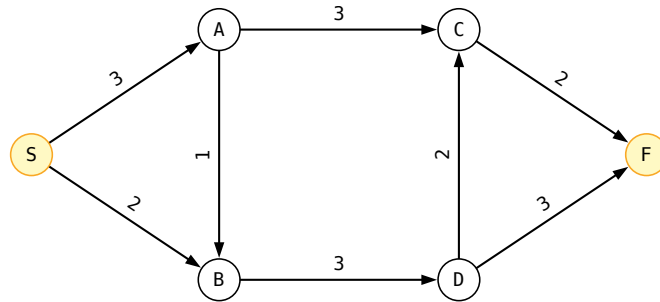


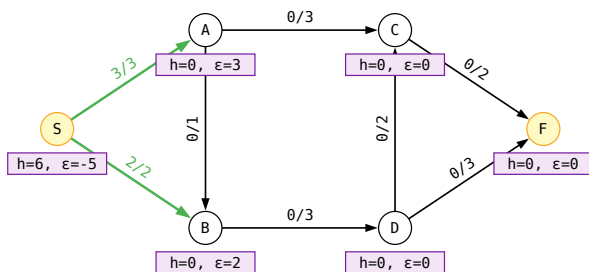
Рис. 47. Сеть Push-Relabel: S, A, B, C, D, F , σ на рёбрах

Протокол работы (каждая строка — одна операция; δ — объём проталкивания):

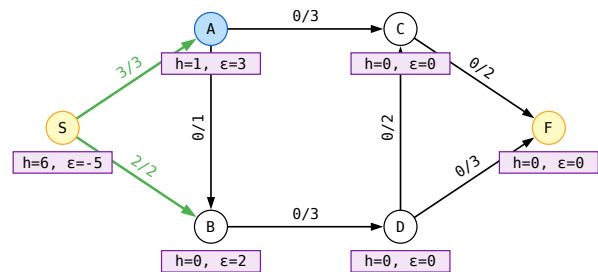
№	операция	δ	комментарий
0	инициализация	—	$S \rightarrow A, S \rightarrow B$ насыщены: $\varepsilon(A)=3, \varepsilon(B)=2$
1	lift $A: h(A):=1$	—	соседи A имеют $h = 0$, проталкивать некуда
2	push $A \rightarrow C$	3	$h(C) = 0 < 1; \varepsilon(A)=0, \varepsilon(C)=3$
3	lift $B: h(B):=1$	—	
4	push $B \rightarrow D$	2	$\varepsilon(B)=0, \varepsilon(D)=2$
5	lift $C: h(C):=1$	—	
6	push $C \rightarrow F$	2	ребро $C \rightarrow F$ насыщено; $\varepsilon(C)=1$
7	lift $C: h(C):=2$	—	вниз идти некуда: остаточный сосед — только A ($h=1$)
8	push $C \rightarrow A$	1	возврат по обратному ребру: $A \rightarrow C$ теперь $2/3; \varepsilon(A)=1$
9	lift $A: h(A):=2$	—	остаточные соседи: C ($h=2$), B ($h=1$), S ($h=6$)
10	push $A \rightarrow B$	1	$h(B) = 1 < 2; \varepsilon(B)=1$
11	push $B \rightarrow D$	1	$\varepsilon(D)=3$
12	lift $D: h(D):=1$	—	сосед F имеет $h = 0$
13	push $D \rightarrow F$	3	$\varepsilon(D)=0$; активных вершин нет — стоп

Итог: $|f| = \varepsilon(F) = 2 + 3 = 5$; поток по рёбрам: $S \rightarrow A = 3, S \rightarrow B = 2, A \rightarrow C = 2, A \rightarrow B = 1, B \rightarrow D = 3, C \rightarrow F = 2, D \rightarrow F = 3, D \rightarrow C = 0$ — условие балансировки выполнено во всех внутренних вершинах.

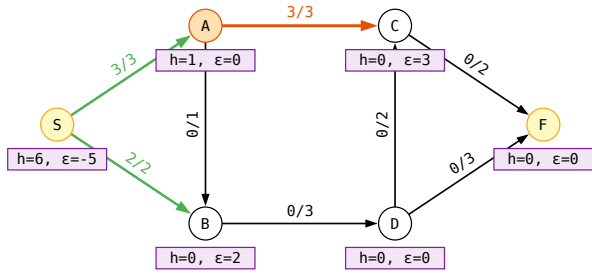
Работа алгоритма по шагам. Подпись ребра — $\tilde{f}/\sigma$; под каждой вершиной — её h и ε . Активная вершина операции — оранжевая, поднимаемая — синяя, ребро проталкивания — оранжевое, рёбра с положительным потоком — зелёные.



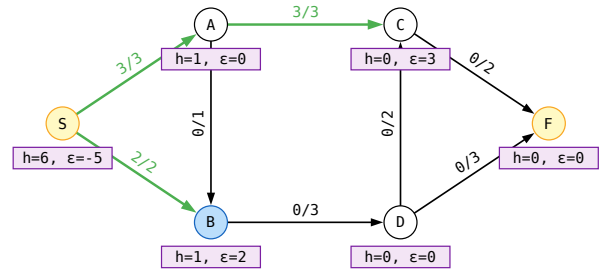
Шаг 0. Инициализация: рёбра из S насыщены



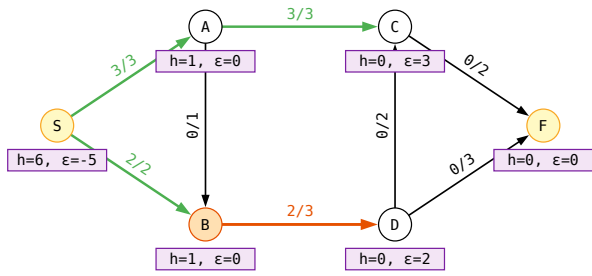
Шаг 1. Lift $A: h(A) := 1$



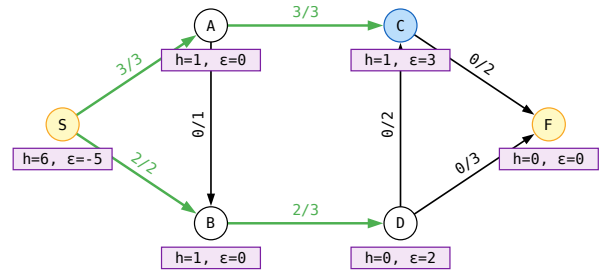
IIIar 2. Push $A \rightarrow C$, $\delta = 3$



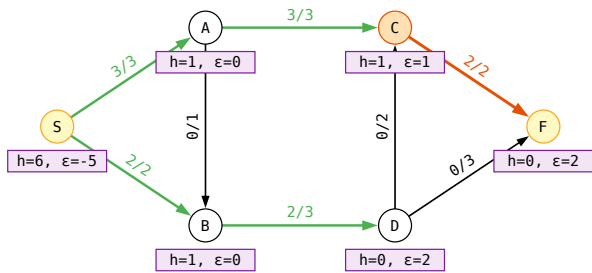
IIIar 3. Lift B : $h(B) := 1$



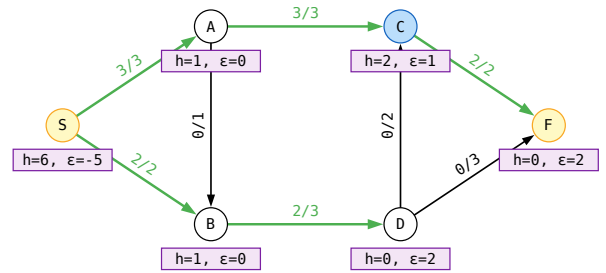
IIIar 4. Push $B \rightarrow D$, $\delta = 2$



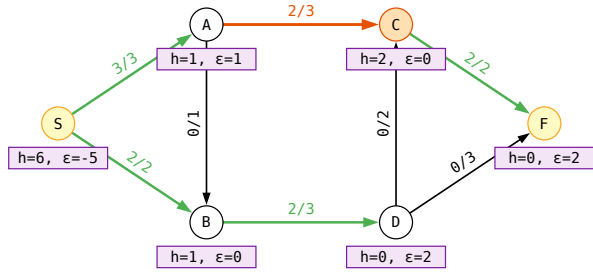
IIIar 5. Lift C : $h(C) := 1$



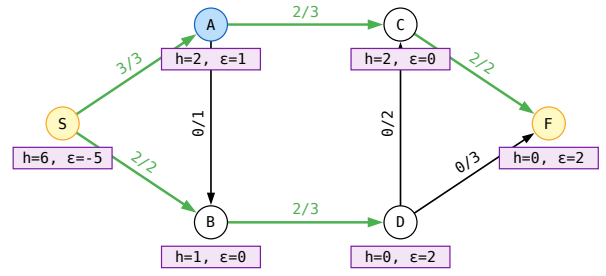
IIIar 6. Push $C \rightarrow F$, $\delta = 2$: ребро насыщено



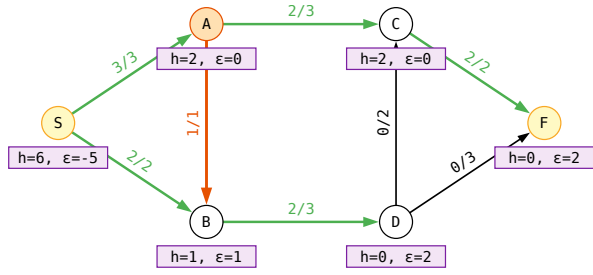
IIIar 7. Lift C : $h(C) := 2$ (выше A)



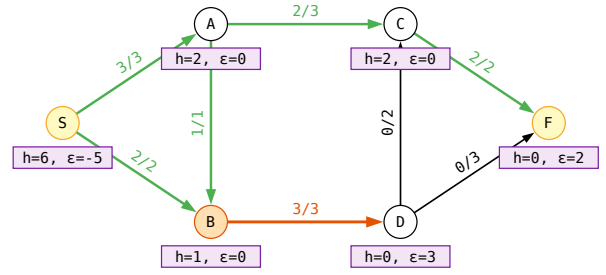
IIIar 8. Push $C \rightarrow A$, $\delta = 1$: возврат, $A \rightarrow C$ стало $2/3$



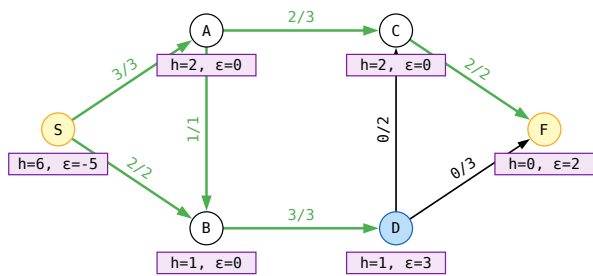
IIIar 9. Lift A : $h(A) := 2$



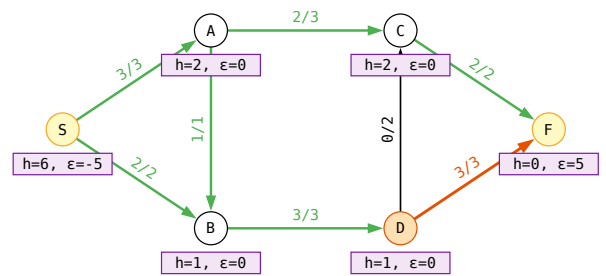
IIIar 10. Push $A \rightarrow B$, $\delta = 1$



IIIar 11. Push $B \rightarrow D$, $\delta = 1$



IIIar 12. Lift D : $h(D) := 1$



IIIar 13. Push $D \rightarrow F$, $\delta = 3$: $|f| = 5$, конец

Замечание (сумма эксцессов). В любой момент работы $\sum_{v \in V} \varepsilon(v) = 0$: каждая единица предпотока на ребре uv входит в $\varepsilon(v)$ со знаком «плюс» и в $\varepsilon(u)$ со знаком «минус», поэтому при суммировании всё сокращается. В частности, в конце работы $\varepsilon(F) = -\varepsilon(S) = |f|$.

Хроматика (раскраска графов)

НВ. Пока рассматриваем *вершинные* раскраски.

Определение 56. Правильная раскраска графа

Раскраска графа называется **правильной**, если любые две смежные вершины окрашены в разные цвета.

Определение 57. Хроматическое число

Хроматическое число графа $\chi(G)$ — минимальное число цветов, в которые можно правильно раскрасить граф G .

Примеры:

1. $|E| = 0 \Rightarrow \chi(G) = 1$: изолированные вершины можно красить одним цветом — конфликтовать нечему.
2. $|E| > 0 \Rightarrow \chi(G) \geq 2$: у любого ребра концы обязаны различаться.
3. У любого двудольного графа (с рёбрами) $\chi(G) = 2$: красим долю V_1 первым цветом, долю V_2 — вторым; все рёбра идут между долями, конфликтов нет.
4. $\chi(K_n) = n$: все вершины попарно смежны, поэтому все цвета обязаны быть различны (рис. ниже: K_4 , $\chi = 4$).
5. $\chi(C_{2n+1}) = 3$: двух цветов не хватает (нечётный цикл не двудольный — лекция 9), а три цвета есть: чередуем 1, 2 по циклу, последней вершине даём цвет 3 (рис. ниже: C_5). Для чётных циклов $\chi(C_{2n}) = 2$.

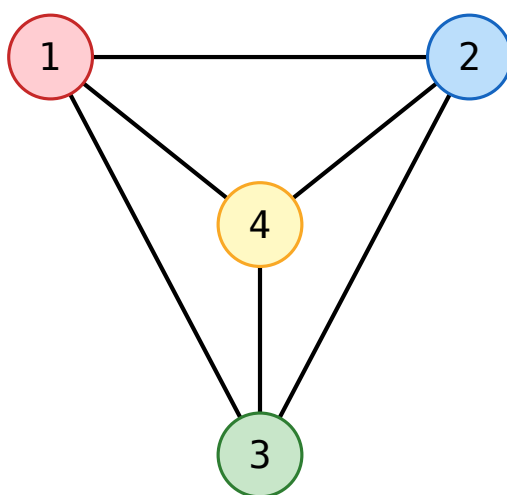


Рис. 48. Хром. число: K_4 , $\chi = 4$ — все цвета различны

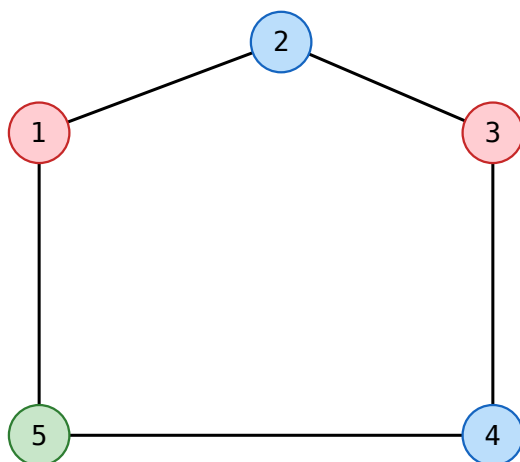


Рис. 49. Хром. число: C_5 , $\chi = 3$ — чередование 1, 2 и третий цвет

Определение 58. Клика

Клика — максимальный полный подграф графа G .

Лемма 13. О клике

Если K_n — наибольшая клика в G , то $\chi(G) \geq n$.

Доказательство. Любая правильная раскраска G является правильной и на подграфе K_n . Но в K_n все n вершин попарно смежны, поэтому уже им требуется n различных цветов: $\chi(G) \geq \chi(K_n) = n$. $\square$

Лекция 12. Хроматический многочлен

Определение 59. Хроматический многочлен

Пусть $G(V, E)$ — граф. **Хроматический многочлен** графа G :

$$P_G: \mathbb{N}_0 \rightarrow \mathbb{N}_0, \quad P_G(t) = a_1 t + a_2 t^2 + \dots + a_n t^n,$$

т. е. это многочлен, значение которого $P_G(k)$ равно числу правильных раскрасок G в k цветов ($\forall k \in \mathbb{N}_0$).

Примеры:

1. Пустой граф (n изолированных вершин): $P_G(t) = t^n$ — каждая вершина красится независимо любым из t цветов.
2. K_n : $P_{K_n}(t) = t(t-1)(t-2) \cdots (t-(n-1))$ — красим вершины по очереди, каждая обязана отличаться от всех предыдущих.
3. Дерево T_n (n вершин): $P_{T_n}(t) = t(t-1)^{n-1}$ — красим от корня: корень t способами, каждую следующую вершину (по мере удаления от корня) — $t-1$ способами, лишь бы она отличалась от единственного уже покрашенного соседа (родителя).

Иллюстрация на двух деревьях с $n = 4$ (показана одна из правильных раскрасок в 2 цвета):

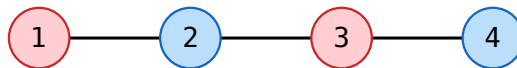


Рис. 50. Путь P_4 : $P(t) = t(t-1)^3$; в 2 цвета — ровно 2 раскраски

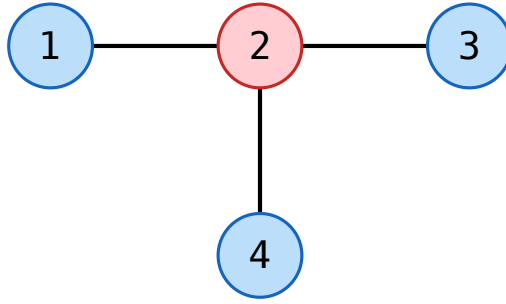


Рис. 51. Звезда: тоже $P(t) = t(t-1)^3$ — дерево с тем же n

Заметим: путь и звезда — разные графы с одинаковым хроматическим многочленом (см. свойство 4 ниже).

Тривиальные свойства (с обоснованиями):

- 0) P_G — многочлен. *Почему:* сгруппируем раскраски по числу i реально использованных цветов: если α_i — число разбиений V на i непустых независимых (попарно несмежных внутри) множеств, то $P_G(t) = \sum_i \alpha_i \cdot t(t-1) \cdots (t-i+1)$ — выбираем, какие именно i из t цветов достанутся классам разбиения. Это конечная сумма многочленов.
- 1) $\deg(P_G) = |V|$. *Почему:* старшее слагаемое в сумме выше отвечает $i = n$ (все вершины разных цветов, $\alpha_n = 1$) и равно $t(t-1) \cdots (t-n+1)$ — многочлен степени n .
- 2) $P_G(0) = 0$ для $\forall G$ т. ч. $|V| > 0$: в 0 цветов нельзя раскрасить даже одну вершину.
- 3) $P_G(t) = t^k \cdot g(t)$, где $g(0) \neq 0$, k — число компонент связности графа G . *Почему:* компоненты красятся независимо, поэтому P_G — произведение многочленов компонент, и каждый сомножитель делится на t ровно один раз.
- 4) G по P_G не восстанавливается однозначно. *Контрпример:* все деревья на n вершинах имеют один и тот же многочлен $t(t-1)^{n-1}$, но при $n \geq 4$ дерево не единственно (путь P_4 и звезда на рисунках выше).
- 5) $a_n = 1$ (старший коэффициент) — см. обоснование свойства 1.
- 6) $\forall 1 \leq i \leq n-1: a_i \cdot a_{i+1} \leq 0$ (знакопереживание). *Идея:* индукция по числу рёбер с теоремой о редукции в форме $P_G = P_{G \setminus e} - P_{G/e}$: у $P_{G \setminus e}$ и $P_{G/e}$ знаки чередуются по предположению, степени отличаются на 1, и при вычитании чередование сохраняется.
- 7) $|E_G| > 0 \Rightarrow P_G(1) = 0$: при одном цвете концы любого ребра совпадут по цвету. Более точно, $P_G(t) = (t-1)^d \cdot g(t)$, где $g(1) \neq 0$, d — число блоков (компонент вершинной двусвязности) в G .

Как найти P_G ?

Теорема 14. Теорема о редукции

Пусть $G(V, E)$, и $u, v \in V$ — не смежные вершины. Тогда:

$$P_G(t) = P_{G \vee uv}(t) + P_{G * uv}(t),$$

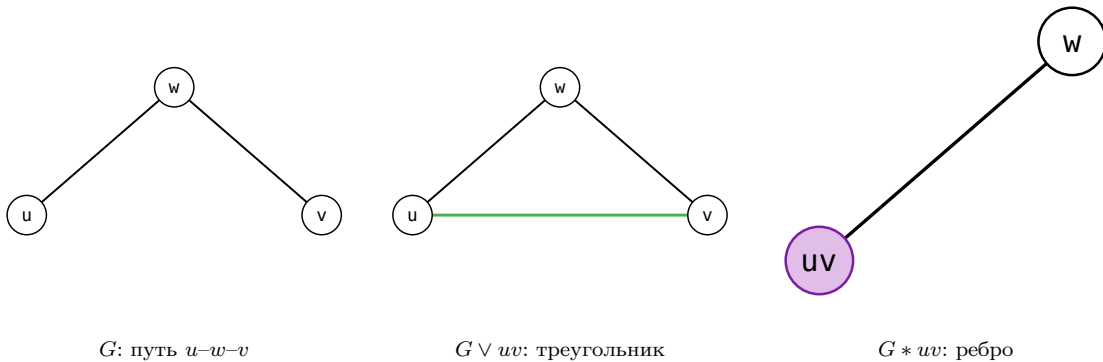
где $G \vee uv$ — граф с добавленным ребром uv (приближаем к полному), а $G * uv$ — граф со стянутыми вершинами u и v .

Доказательство. Разобьём все правильные раскраски G в t цветов на два класса по тому, как покрашены u и v (они не смежны, поэтому возможно и то, и другое):

- **цвета u и v различны** — такие раскраски взаимно однозначно соответствуют правильным раскраскам графа $G \vee uv$: добавленное ребро uv как раз и требует различия цветов, остальные ограничения совпадают;
- **цвета u и v совпадают** — такие раскраски взаимно однозначно соответствуют правильным раскраскам графа $G * uv$: склеенная вершина получает этот общий цвет, и её смежности — объединение смежностей u и v .

Классы не пересекаются и исчерпывают все раскраски, поэтому $P_G = P_{G \vee uv} + P_{G * uv}$. $\square$

Иллюстрация: u и v не смежны в G ; слева направо — G , $G \vee uv$ (зелёное ребро добавлено), $G * uv$ (вершины склеены):



Мини-пример (проверка теоремы). Для пути $G = u-w-v$ на рисунке: $G \vee uv = K_3$, $G * uv = K_2$,

$$P_{G \vee uv} + P_{G * uv} = t(t-1)(t-2) + t(t-1) = t(t-1)^2$$

— в точности $P_{T_3}(t) = t(t-1)^2$ по формуле для дерева. $\checkmark$

Следствие. Если $\exists e = uv$ (смежные), то:

$$P_G(t) = P_{G \setminus e}(t) - P_{G * uv}(t).$$

Почему: применим теорему к графу $G \setminus e$ (в нём u, v уже не смежны): $P_{G \setminus e} = P_{(G \setminus e) \vee uv} + P_{(G \setminus e) * uv} = P_G + P_{G * uv}$, и перенесём слагаемое.

Пример: Цикл C_n :

$$P_{C_n}(t) = (t-1)^n + (-1)^n(t-1).$$

Доказательство. Воспользуемся теоремой об удалении и стягивании: для любого ребра $e = uv$

$$P_G(t) = P_{G \setminus e}(t) - P_{G/e}(t),$$

где $G \setminus e$ — граф с удалённым ребром, а G/e — граф со стянутым ребром (вершины u и v отождествлены). Понадобится также хроматический многочлен простого пути (дерева) на n вершинах: $P_{T_n}(t) = t(t-1)^{n-1}$ (первую вершину красим t способами, каждую следующую — $t-1$ способами, лишь бы она отличалась от предыдущей).

База ($n = 3$). $C_3 = K_3$, поэтому $P_{C_3}(t) = t(t-1)(t-2)$. Проверим формулу:

$$(t-1)^3 + (-1)^3(t-1) = (t-1)[(t-1)^2 - 1] = (t-1)(t^2 - 2t) = t(t-1)(t-2).$$

Переход. Возьмём в цикле C_n произвольное ребро $e = uv$. Его удаление размыкает цикл в путь: $C_n \setminus e = T_n$, поэтому $P_{C_n \setminus e}(t) = t(t-1)^{n-1}$. Стягивание ребра e склеивает u и v и превращает цикл из n вершин в цикл из $n-1$ вершины: $C_n/e = C_{n-1}$. По теореме об удалении и стягивании

$$P_{C_n}(t) = t(t-1)^{n-1} - P_{C_{n-1}}(t).$$

Подставляя предположение индукции $P_{C_{n-1}}(t) = (t-1)^{n-1} + (-1)^{n-1}(t-1)$, получаем

$$P_{C_n}(t) = t(t-1)^{n-1} - (t-1)^{n-1} - (-1)^{n-1}(t-1) = (t-1)^{n-1}(t-1) + (-1)^n(t-1),$$

то есть $P_{C_n}(t) = (t-1)^n + (-1)^n(t-1)$, что и требовалось. $\square$

Теорема о разделяющей клике

Лемма 15. Лемма 1 (висячая вершина)

Если в графе G есть висячая вершина u , соединённая с подграфом G_1 , то:

$$P_G(t) = P_{G_1}(t) \cdot (t-1).$$

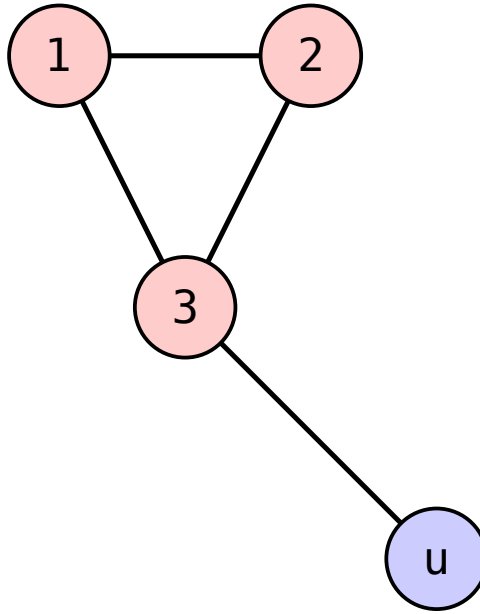


Рис. 52. Лемма 1: висячая вершина $\rightarrow P_G = P_{G_1} \cdot (t-1)$

Доказательство. Сначала правильно красим G_1 — $P_{G_1}(t)$ способами. У вершины u ровно один сосед, и какой бы цвет он ни получил, для u остаётся $t-1$ допустимых цветов; выбор независим от остального. По правилу произведения $P_G(t) = P_{G_1}(t)(t-1)$. $\square$

Лемма 16. Лемма 2 (дерево T_{n+1})

Если к подграфу G_1 присоединено дерево T_{n+1} (через одну вершину), то:

$$P_G(t) = P_{G_1}(t) \cdot (t - 1)^n.$$

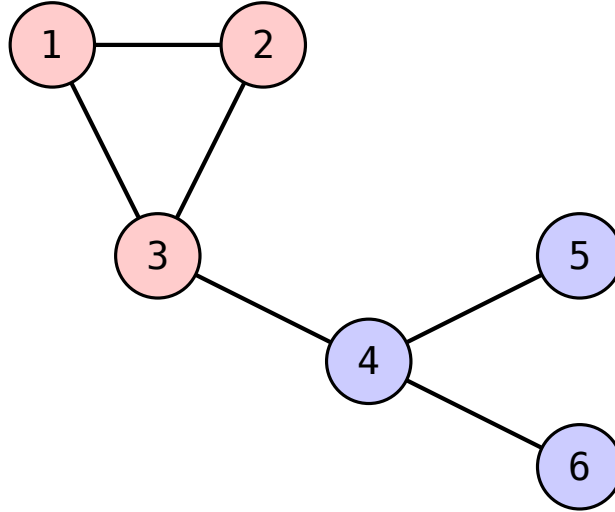


Рис. 53. Лемма 2: дерево $T_{n+1} \rightarrow P_G = P_{G_1} \cdot (t-1)^n$

Доказательство. Дерево T_{n+1} имеет $n + 1$ вершину (одна из них — общая с G_1) и n рёбер. Присоединяем его вершины по одной, начиная от общей вершины и удаляясь по дереву: каждая новая вершина — висячая относительно уже покрашенной части, и по лемме 1 даёт множитель $(t - 1)$. Всего n новых вершин: $P_G = P_{G_1} \cdot (t - 1)^n$. $\square$

Лемма 17. Лемма 3 (вершина при ребре)

Если вершина u присоединена к G_1 рёбрами к *обоим концам* некоторого ребра $vw \in G_1$ (т. е. через клику K_2), то:

$$P_G(t) = P_{G_1}(t) \cdot (t - 2).$$

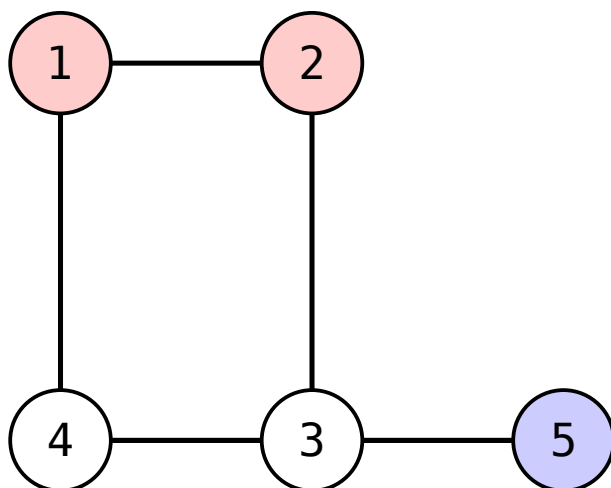


Рис. 54. Лемма 3: вершина при ребре $\rightarrow P_G = P_{\{G_1\}} \cdot (t-2)$

Доказательство. В любой правильной раскраске G_1 концы ребра vw покрашены в *два различных* цвета. Вершине u запрещены ровно эти два цвета, остаётся $t - 2$ вариантов — независимо от того, какая именно раскраска G_1 выбрана. $\square$

Лемма 18. Лемма 4 (вершина при K_n)

Если вершина u с $\deg u = n$ присоединена к подграфу G_1 через K_n (все n соседей u попарно смежны в G_1), то:

$$P_G(t) = P_{G_1}(t) \cdot (t - n).$$

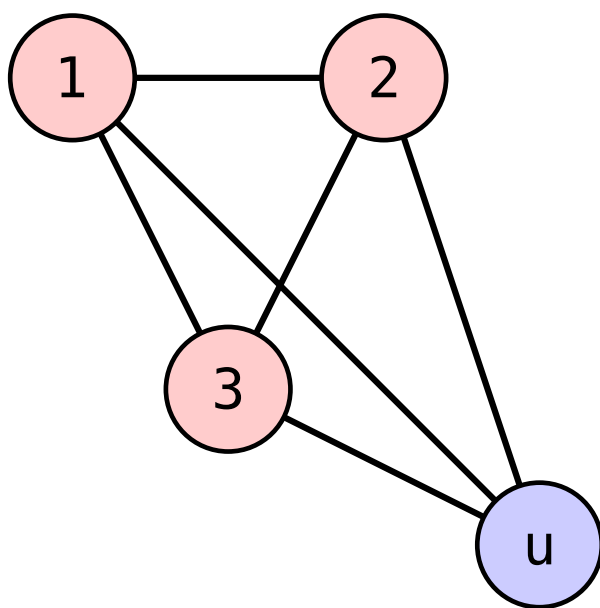


Рис. 55. Лемма 4: вершина при $K_n \rightarrow P_G = P_{\{G_1\}} \cdot (t-n)$

Доказательство. Соседи u образуют клику, поэтому в любой правильной раскраске G_1 они покрашены в n *попарно различных* цветов. Для u запрещены ровно n цветов — остаётся $t - n$ вариантов. Леммы 1 и 3 — частные случаи при $n = 1$ и $n = 2$. $\square$

Лемма 19. Лемма 5 (комбинация подвесок)

Если к G_1 присоединены висячая вершина и вершина при ребре (как на рисунке), множители перемножаются:

$$P_G(t) = P_{G_1}(t) \cdot (t-1)(t-2).$$

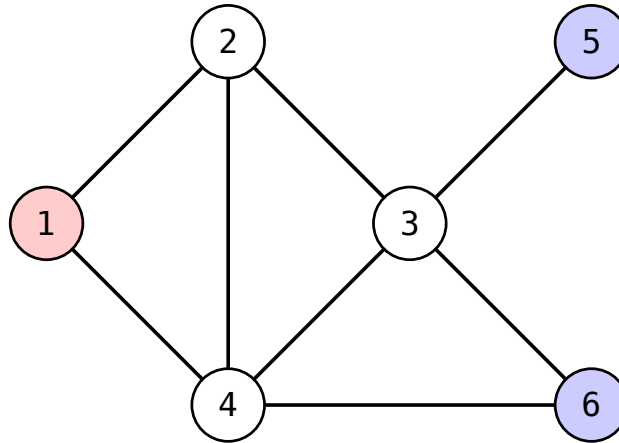


Рис. 56. Лемма 5: висячая вершина + вершина при ребре

Доказательство. Применяем леммы по очереди: сначала к $G' = G$ без висячей вершины 5 — по лемме 3 (вершина 6 при ребре 3–4) имеем $P_{G'} = P_{G_1}(t-2)$; затем возвращаем висячую вершину 5 — по лемме 1 получаем $P_G = P_{G'}(t-1) = P_{G_1}(t-1)(t-2)$. Порядок применения не важен: каждая подвеска даёт свой множитель независимо. $\square$

Лемма 20. Лемма 6 (общая, разделяющая K_n)

Если граф G разделяется кликой K_n на подграфы G_1 и G_2 (т. е. $G = G_1 \cup G_2$, $G_1 \cap G_2 = K_n$), то:

$$P_G(t) = \frac{P_{G_1}(t) \cdot P_{G_2}(t)}{P_{K_n}(t)}.$$

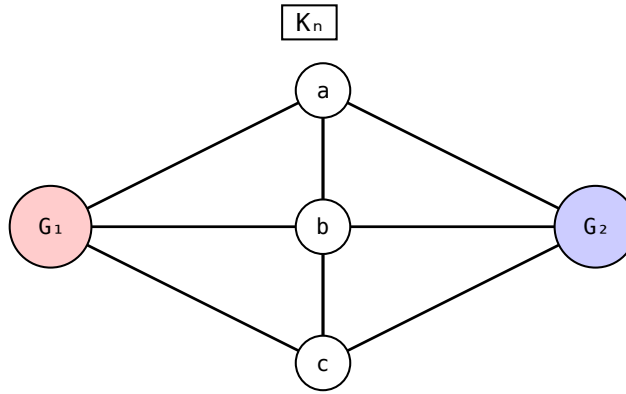


Рис. 57. Лемма 6: общая, $K_n \rightarrow P_G = P_{\{G_1\}} \cdot P_{\{G_2\}} / P_{\{K_n\}}$

Доказательство. Ключевое наблюдение: *число продолжений* фиксированной правильной раскраски клики K_n до правильной раскраски всего G_1 не зависит от того, какая именно раскраска клики выбрана. Действительно, любые две правильные раскраски K_n (n попарно различных цветов) переводятся друг в друга перестановкой цветов, а перестановка цветов — биекция на множестве продолжений. Поэтому продолжений ровно $P_{G_1}(t)/P_{K_n}(t)$ (всего раскрасок G_1 , делённое на число раскрасок клики); аналогично для G_2 .

Раскраска G — это раскраска клики ($P_{K_n}(t)$ способов) плюс независимые продолжения на G_1 и на G_2 (общих вершин вне клики нет, рёбер между $G_1 \setminus K_n$ и $G_2 \setminus K_n$ нет):

$$P_G = P_{K_n} \cdot \frac{P_{G_1}}{P_{K_n}} \cdot \frac{P_{G_2}}{P_{K_n}} = \frac{P_{G_1} \cdot P_{G_2}}{P_{K_n}}.$$

Леммы 1–5 — частные случаи (например, лемма 4: $G_2 = K_{n+1}$, $P_{G_2}/P_{K_n} = t - n$). □

Рёберная раскраска

Определение 60. Правильная рёберная раскраска

Рёберная раскраска графа называется **правильной**, если $\forall e_1, e_2 \in E$, имеющие общую вершину, окрашены в разные цвета.

Определение 61. Хроматически двойственный граф

Пусть $G(V, E)$. **Хроматически двойственный** граф $G_\chi(V_\chi, E_\chi)$ строится так:

1. $V_\chi = E$ (вершины G_χ — это рёбра G);
2. $\hat{u}, \hat{v} \in V_\chi: \exists \hat{e} = \hat{u}\hat{v} \Leftrightarrow$ рёбра $u, v \in E$ имеют общую вершину в G .

Правильные *рёберные* раскраски G — это в точности правильные *вершинные* раскраски G_χ , поэтому рёберное хроматическое число $\chi'(G) = \chi(G_\chi)$.

Пример (рис. p28-example-1 $\rightarrow$ p28-chromatic-dual):

Граф G : вершины A, B, C, D, E ; рёбра AB, BC, CA, CD, BE, DB .

Хроматически двойственный G_χ : вершины AB, BC, CA, CD, BE, DB ; рёбра — пары, имеющие общую вершину в G .

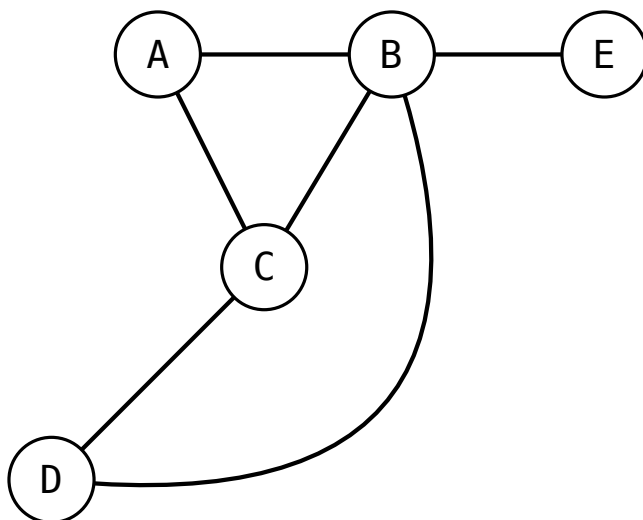


Рис. 58. Рёберная раскраска: исходный граф G (A, B, C, D, E)

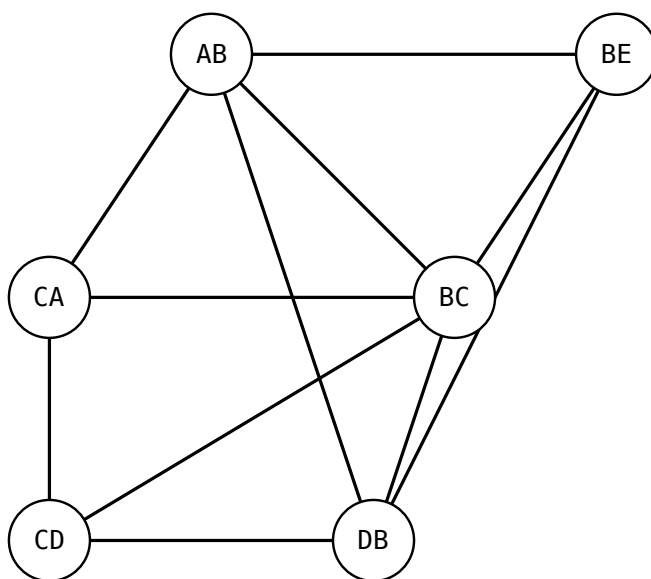


Рис. 59. Хром. двойственный G_χ (AB, BC, CA, CD, BE, DB)

Заметим: четыре ребра AB, BC, BE, DB инцидентны вершине B ($\deg B = 4$), поэтому в G_χ они образуют клику K_4 — по лемме о клике $\chi'(G) = \chi(G_\chi) \geq 4$.

Оценки на хроматическое число

Обозначим $\Delta(G) = \max_{v \in V} \deg(v)$.

Теорема 21. Жадная оценка

$$\chi(G) \leq \Delta(G) + 1.$$

Доказательство. Красим вершины в произвольном порядке, выделив $\Delta + 1$ цветов. Когда очередь доходит до вершины v , покрашено не более $\deg(v) \leq \Delta$ её соседей — они запрещают не более Δ цветов, и хотя бы один из $\Delta + 1$ цветов свободен. Оценка точна: для K_n и для нечётных циклов достигается равенство (теорема Брукса утверждает, что во всех остальных связных графах $\chi \leq \Delta$). $\square$

Теорема 22. Теорема Визинга (о рёберной раскраске)

$$\Delta(G) \leq \chi'(G) \leq \Delta(G) + 1.$$

Нижняя оценка очевидна: все $\deg(v)$ рёбер при вершине v попарно смежны и требуют разных цветов. Верхняя — содержательная часть теоремы Визинга (доказательство выходит за рамки курса).

Замечание (плоские графы). Для любого плоского графа $\chi(G) \leq 5$ (теорема о пяти красках; более того, верна знаменитая теорема о четырёх красках: $\chi(G) \leq 4$).

Задача 11. Хроматический многочлен цикла и графа с подвеской

Условие. Найдите хроматические многочлены: (а) цикла C_4 на вершинах $A-B-C-D-A$; (б) графа G на вершинах A, B, C, D с рёбрами AB, BC, CA (треугольник) и подвешенным ребром CD .

Решение (а). По доказанной формуле для цикла $P_{C_n}(t) = (t-1)^n + (-1)^n(t-1)$. При $n = 4$:

$$P_{C_4}(t) = (t-1)^4 + (t-1) = t^4 - 4t^3 + 6t^2 - 4t + 1 + t - 1 = t^4 - 4t^3 + 6t^2 - 3t.$$

Проверка: $P_{C_4}(2) = 16 - 32 + 24 - 6 = 2$ (чётный цикл имеет ровно 2 правильные раскраски в 2 цвета); $P_{C_4}(3) = 81 - 108 + 54 - 9 = 18$.

Решение (б). Треугольник — это K_3 , поэтому $P_{K_3}(t) = t(t-1)(t-2)$. Подвешенная вершина D соединена единственным ребром CD : по лемме 1 (висячая вершина) добавление висячего ребра умножает многочлен на $(t-1)$:

$$P_G(t) = P_{K_3}(t) \cdot (t-1) = t(t-1)(t-2)(t-1) = t(t-1)^2(t-2).$$

Проверка: $P_G(3) = 3 \cdot 4 \cdot 1 = 12$ — треугольник красится $3! = 6$ способами, а D — 2 способами, итого $6 \cdot 2 = 12$.

Ответ. (а) $P_{C_4}(t) = t^4 - 4t^3 + 6t^2 - 3t$; (б) $P_G(t) = t(t-1)^2(t-2)$.

Лекция 13. Поиск кратчайших путей во взвешенном графе

Пусть $G(V, E)$ — ориентированный связный (слабо) граф, $\sigma: E \rightarrow \mathbb{R}$ — весовая функция.

Пример (рис. p29-example-1): граф A, B, C, D, K с рёбрами $AB(1)$, $BC(-2)$, $CD(-2)$, $DB(3)$, $BK(8)$. Цикл $B \rightarrow C \rightarrow D \rightarrow B$ имеет суммарный вес $(-2) + (-2) + 3 = -1 < 0$ — отрицательный цикл.

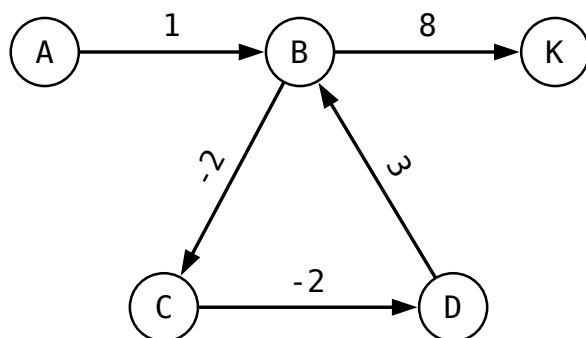


Рис. 60. Граф с отриц. циклом: A, B, C, D, K

Критерий (необходимость и достаточность): кратчайшие пути существуют тогда и только тогда, когда в G отсутствуют циклы отрицательного суммарного веса.

Почему. ($\Leftarrow$) Если отрицательных циклов нет, то из любого маршрута можно выбрасывать циклы, не увеличивая длину, — значит, среди кратчайших маршрутов есть *простой* путь; простых путей конечное число, и минимум достигается. ($\Rightarrow$) Если же отрицательный цикл достижим, то, прокручивая его сколько угодно раз, длину маршрута можно сделать сколь угодно малой — минимума нет (в примере выше путь $A \rightarrow B \rightarrow K$ можно «удешевлять» обходами цикла $B \rightarrow C \rightarrow D \rightarrow B$ по -1 за круг).

Отдельные рёбра отрицательного веса в общем случае возможны.

Задачи:

⊕ Кратчайшие пути от фиксированной вершины $v \in V$ до всех $w \in V$.

⊙ Кратчайшие пути для каждой пары $u, v \in V$.

Кратчайший путь $\neq$ его длина:

$$S = A_0 \rightarrow A_1 \rightarrow A_2 \rightarrow \dots \rightarrow A_k \rightarrow \dots \rightarrow A_{n-1} \rightarrow A_n = F.$$

$P(A_i) = A_{i-1}$ (предок), $G(A_i) = A_{i+1}$ (потомок).

Алгоритм 15. Алгоритм Дейкстры

Требование: $\sigma: E \rightarrow \mathbb{R}_{\geq 0}$ (только неотрицательные веса!).

Инициализация: $\mu(\text{start}) := 0$, $\mu(v) := \infty$ для всех остальных вершин.

Шаг: Выбираем v с минимальной меткой $\mu(v)$ среди непосещённых.

Для каждого $w \in \text{Ch}(v)$:

$$\text{если } \mu(w) > \mu(v) + \sigma(\overline{vw}), \text{ то } \begin{cases} \mu(w) := \mu(v) + \sigma(\overline{vw}), \\ P(w) := v. \end{cases}$$

Это **динамическое программирование** (жадный выбор + оптимальная подструктура).

Почему жадный выбор корректен: пусть v — непосещённая вершина с минимальной меткой $\mu(v)$. Любой путь из старта в v обязан в какой-то момент впервые покинуть множество посещённых вершин по ребру $x \rightarrow y$ (y не посещена); его длина не меньше $\mu(y) \geq \mu(v)$, а дальнейшие рёбра *неотрицательны* — короче $\mu(v)$ не получится. Значит, метку $\mu(v)$ можно зафиксировать как окончательную. (Именно здесь существенно $\sigma \geq 0$: с отрицательными рёбрами «дешёвый обход позже» возможен, и жадность ломается — нужен Форд–Беллман.)

Пример (рис. p29-dijkstra-example-1): граф A, B, C, D . Start = A . Рёбра: BA (2), CB (5), AC (10), DC (3), AD (2), BD (7).

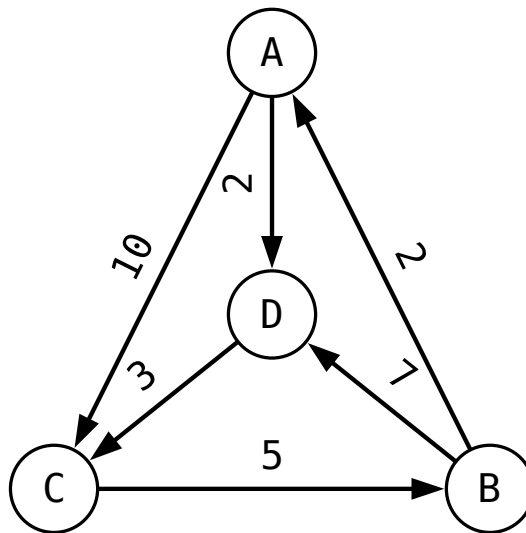


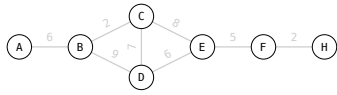
Рис. 61. Алг. Дейкстры: A, B, C, D ($AD=2, AC=10, DC=3, BD=7, CB=5, BA=2$)

	A	B	C	D
Иниц.	0	∞	∞	∞
A	—	∞	10, A	2, A
D	—	∞	5, D	—
C	—	10, C	—	—

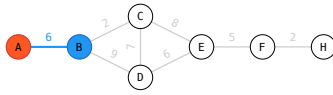
$D \rightarrow C$: $\mu(D) = 2$, $\sigma(DC) = 3$, итого $2 + 3 = 5 < 10$, обновляем.

Дерево кратчайших путей: $A \xrightarrow{2} D \xrightarrow{3} C \xrightarrow{5} B$.

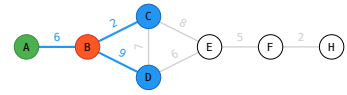
Работа алгоритма по шагам. Для разнообразия демонстрация выполнена на другом графе (7 вершин, рис. p9-example-0, веса проставлены детерминированно), старт — A .



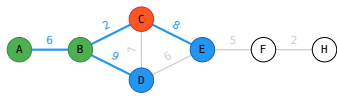
Шаг 0



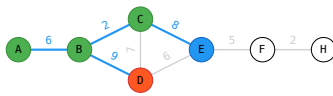
Шаг 1



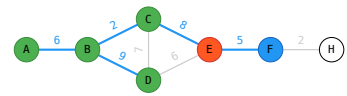
Шаг 2



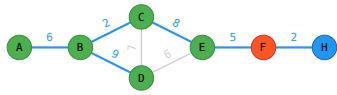
Шаг 3



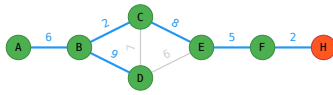
Шаг 4



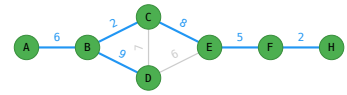
Шаг 5



Шаг 6



Шаг 7



Шаг 8

Алгоритм 16. Алгоритм Форда-Беллмана

Допускается: $\sigma: E \rightarrow \mathbb{R}$ (в том числе отрицательные веса).

Инициализация: Произвольное упорядочивание рёбер. Стартовая вершина v : $\mu(v) := 0$, $\mu(w) := \infty$ для остальных.

Шаг (1 шаг = полный просмотр списка рёбер):

$\forall e_i \in \text{list}(E), e_i = uv$:

$$\text{если } \mu(v) > \mu(u) + \sigma(e_i), \text{ то } \begin{cases} \mu(v) := \mu(u) + \sigma(e_i), \\ P(v) := u. \end{cases}$$

Конец алгоритма: шаги k и $k + 1$ дают одинаковый результат. Если n вершин, то алгоритм сойдётся не позже чем за $n - 1$ шагов.

Почему работает: индукцией по k проверяется инвариант — после k -го прохода по списку рёбер метка $\mu(w)$ не превосходит длины кратчайшего пути из старта в w , содержащего $\leq k$ рёбер (релаксация последнего ребра такого пути произойдёт не позже k -го прохода). Кратчайшие пути просты, т.е. содержат не более $n - 1$ ребра, поэтому $n - 1$ проходов

достаточно. Если же обновления случаются и на n -м проходе — в графе есть достижимый отрицательный цикл (так его и детектируют).

Пример (рис. p30-fb-example-1): граф A, B, C, D . Start = B . Отличие от Дейкстры: ребро BA имеет вес -2 .

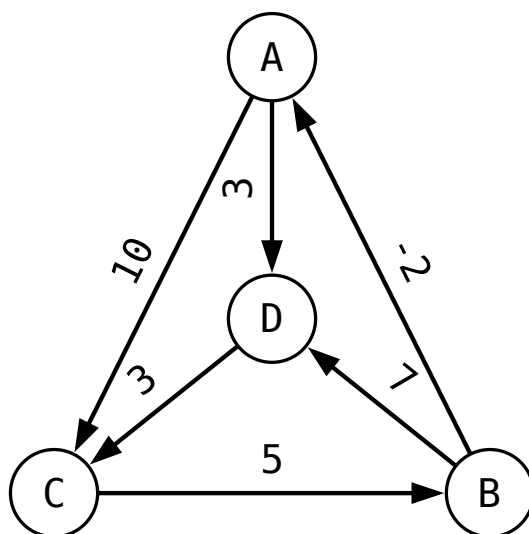


Рис. 62. Алг. Форда-Беллмана: тот же граф, $BA = -2$

Список рёбер с весами:

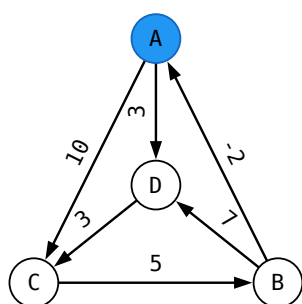
Ребро	Вес
AC	10
AD	2
BA	-2
BD	7
CB	5
DC	3

Протокол:

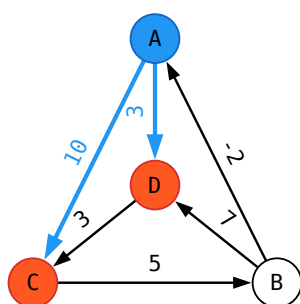
	A	B	C	D
Иниц.	∞	0	∞	∞
Шаг 1	-2, B	0	$7 + 3 = 10$, D	7, B
Шаг 2	-2, B	0	8, $A \rightarrow 3$, D	0, A

Корневое дерево кратчайших путей: $B \xrightarrow{-2} A \xrightarrow{2} D \xrightarrow{3} C$.

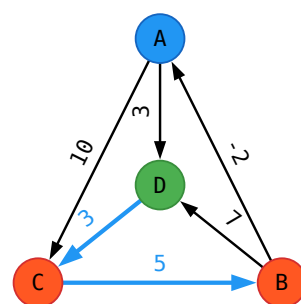
Работа алгоритма по шагам:



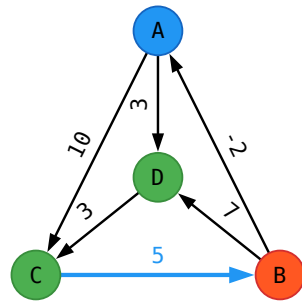
Шаг 0



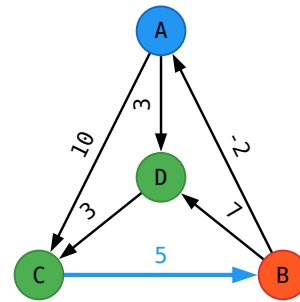
Шаг 1



Шаг 2



Шаг 3



Шаг 4

Алгоритм 17. Алгоритм Флойда-Уоршелла

Задача $\odot$: кратчайшие пути для каждой пары $u, v \in V$.

Идея: аналог алгоритма Уоршелла (транзитивное замыкание), но для взвешенного графа.

Инициализация: Матрица $\mu(u, w)$ — по весам рёбер. Диагональ: $\mu(v, v) = 0$. Нет ребра: $\mu(u, w) = \infty$.

Шаг: v — ведущая вершина. На i -м шаге выделяем i -ю строку и i -й столбец и смотрим пересечения:

$\forall (u, w)$, где $u \neq v, w \neq v$:

$$\text{если } \mu(u, w) > \mu(u, v) + \mu(v, w), \text{ то } \begin{cases} \mu(u, w) := \mu(u, v) + \mu(v, w), \\ P(w) := \begin{cases} v, & \text{если } P(v) = \emptyset, \\ P(v), & \text{если } P(v) \neq \emptyset. \end{cases} \end{cases}$$

Пример (рис. p31-fu-example-1): тот же граф A, B, C, D .

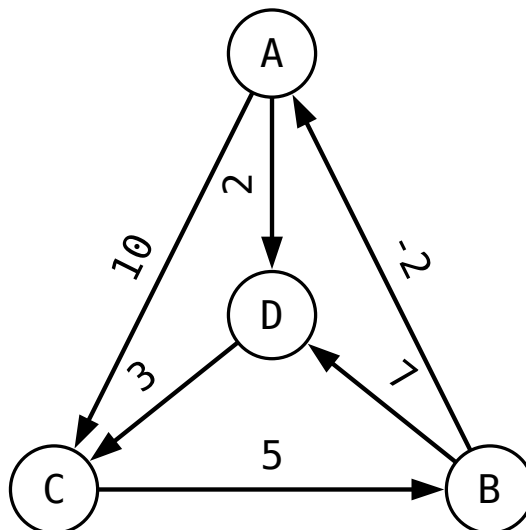


Рис. 63. Алг. Флойда-Уоршелла: тот же граф ($BA=2$)

Шаг 0 (инициализация):

	A	B	C	D
A	0	∞	10	2
B	-2	0	∞	7
C	∞	5	0	∞
D	∞	∞	3	0

Шаг 1 (ведущая A): $\mu(B, C): \infty > (-2) + 10 = 8(A)$; $\mu(B, D): 7 > (-2) + 2 = 0(A)$.

	A	B	C	D
A	0	∞	10	2
B	-2	0	8, A	0, A
C	∞	5	0	∞
D	∞	∞	3	0

Шаг 2 (ведущая B): $\mu(C, A): \infty > 5 + (-2) = 3(B)$; $\mu(C, D): \infty > 5 + 0 = 5(B)$.

	A	B	C	D
A	0	∞	10	2
B	-2	0	8, A	0, A
C	3, B	5	0	5, B
D	∞	∞	3	0

Шаг 3 (ведущая C): $\mu(A, B): \infty > 10 + 5 = 15(C)$; $\mu(D, A): \infty > 3 + 3 = 6(C)$; $\mu(D, B): \infty > 3 + 5 = 8(C)$.

	A	B	C	D
A	0	15, C	10	2
B	-2	0	8, A	0, A
C	3, B	5	0	5, B
D	6, C	8, C	3	0

Шаг 4 (ведущая D): $\mu(A, B): 15 > 2 + 8 = 10(D)$ — путь $A \rightarrow D \rightarrow C \rightarrow B$; $\mu(A, C): 10 > 2 + 3 = 5(D)$; $\mu(B, C): 8 > 0 + 3 = 3(A)$ — путь $B \rightarrow A \rightarrow D \rightarrow C$, первая путевая точка A .

	A	B	C	D
A	0	10, D	5, D	2
B	-2	0	3, A	0, A
C	3, B	5	0	5, B
D	6, C	8, C	3	0

Восстановление: $A \rightarrow D$: путевая точка не записана (прямое ребро), путь $A \rightarrow D$, длина = 2. $C \rightarrow B$: путевая точка тоже не записана — кратчайший путь так и остался прямым ребром $C \rightarrow B$ длины 5 (обходы $C \rightarrow A \rightarrow \dots$ дороже: $\mu(C, A) = 3$, а ребра $A \rightarrow B$ нет, $\mu(A, B) = 10$). $A \rightarrow B$: $P_{AB} = D$, далее $\mu(D, B): P_{DB} = C$, далее прямое ребро $C \rightarrow B$; итог $A \xrightarrow{2} D \xrightarrow{3} C \xrightarrow{5} B$, длина = 10.

Циклы отрицательного суммарного веса $\Leftrightarrow$ отрицательные значения на главной диагонали.

Задача 12. Кратчайшие пути: алгоритм Дейкстры

Условие. Дан неориентированный взвешенный граф на вершинах S, A, B, C, D, F с рёбрами

$$SA = 2, SB = 5, AB = 1, AC = 4, BD = 3, CD = 1, CF = 6, DF = 2.$$

Найдите кратчайшие расстояния от вершины S до всех остальных алгоритмом Дейкстры.

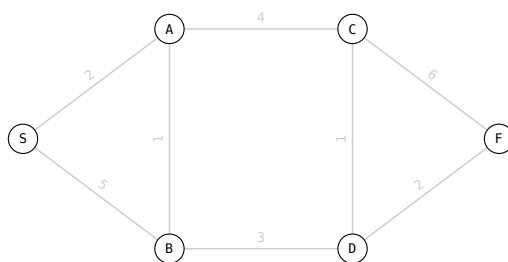


Рис. 64. Граф задачи (условие)

Решение. Дейкстра поддерживает множество вершин с окончательно найденным расстоянием. На каждом шаге фиксируется вершина с наименьшей текущей оценкой d , после чего релаксируются её рёбра.

1. Старт: $d[S] = 0$, остальные $= \infty$. Фиксируем S : $d[A] = 2$, $d[B] = 5$.
2. Минимум среди нефиксированных — A (2). Фиксируем A : $d[B] = \min(5, 2 + 1) = 3$, $d[C] = 2 + 4 = 6$.
3. Минимум — B (3). Фиксируем B : $d[D] = 3 + 3 = 6$.
4. Минимум — C (6). Фиксируем C : D ($6 + 1 = 7 > 6$ — без изменений), $d[F] = 6 + 6 = 12$.
5. Минимум — D (6). Фиксируем D : $d[F] = \min(12, 6 + 2) = 8$.
6. Фиксируем F (8). Все вершины обработаны.

Ответ. $d[S] = 0$, $d[A] = 2$, $d[B] = 3$, $d[C] = 6$, $d[D] = 6$, $d[F] = 8$.

Задача 13. Кратчайшие пути с отрицательными рёбрами: Форд—Беллман

Условие. Дан *ориентированный* взвешенный граф на вершинах S, A, B, C, D, F с дугами $S \rightarrow A = 3$, $S \rightarrow B = 2$, $A \rightarrow C = 2$, $B \rightarrow A = -2$, $C \rightarrow D = 1$, $B \rightarrow D = 5$, $D \rightarrow F = 3$, $C \rightarrow F = 6$.
Найдите кратчайшие расстояния от S алгоритмом Форда—Беллмана.

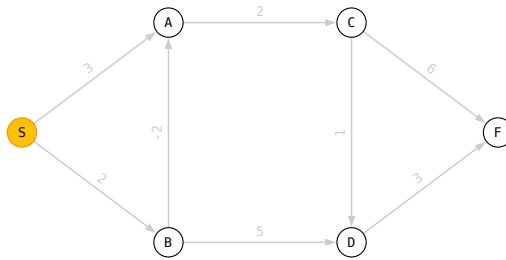


Рис. 65. Граф задачи (условие)

Решение. Форд—Беллман выполняет до $|V| - 1$ итераций, на каждой релаксируя все дуги; отрицательные веса допустимы (при отсутствии отрицательных циклов).

- **Итерация 1.** $d[A] = 3$, $d[B] = 2$, $d[C] = 5$; затем дуга $B \rightarrow A$ улучшает $d[A] = 2 - 2 = 0$; далее $d[D] = 6$, $d[F] = 9$.
- **Итерация 2.** Так как $d[A]$ упало до 0, пересчёт даёт $d[C] = 0 + 2 = 2$, $d[D] = 2 + 1 = 3$, $d[F] = 3 + 3 = 6$.
- **Итерация 3.** Изменений нет — расстояния стабилизировались.

Заметим: кратчайший путь до A идёт *не* напрямую $S \rightarrow A$ (вес 3), а через $S \rightarrow B \rightarrow A$ (вес $2 + (-2) = 0$). Именно из-за отрицательной дуги здесь нужен Форд—Беллман, а не Дейкстра.

Ответ. $d[S] = 0$, $d[A] = 0$, $d[B] = 2$, $d[C] = 2$, $d[D] = 3$, $d[F] = 6$.

Лекция 14. Алгоритм Флойда. Алгоритм Джонсона

Алгоритм 18. Алгоритм Флойда

$G(V, E)$ — слабо связный ориентированный граф, $\sigma: E \rightarrow \mathbb{R}$.

Алгоритм аналогичен Флойду–Уоршеллу. На каждом шаге выделяем ведущую вершину (строку и столбец), проверяем все пересечения.

Хранение пути: для пары (u, w) храним P_{uw} — первую путевую точку. Для прямого ребра $A \rightarrow B$ — ничего хранить не надо. Остаток пути: $B \rightarrow \dots \rightarrow L$.

Если $\mu(u, w) > \mu(u, v) + \mu(v, w)$, то $\mu(u, w) := \mu(u, v) + \mu(v, w)$, $P_{uw} := P_{uv}$ (или v , если P_{uv} пуст).

Пример (рис. p32-f-example-1): граф A, B, C, D .

Список рёбер с весами:

Ребро	Вес
AB	1
BC	1
BD	3
CA	-2
CD	1
DA	4

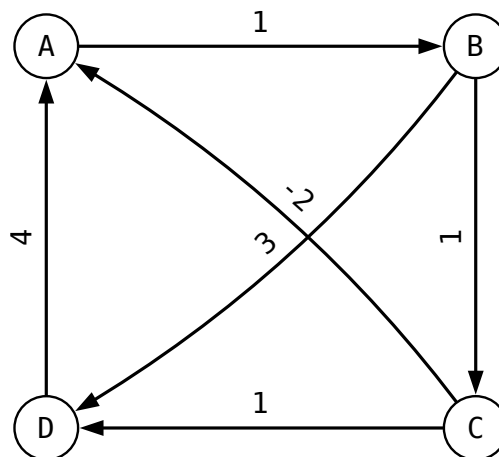


Рис. 66. Алг. Флойда: A, B, C, D ($AB=1$, $BC=1$, $BD=3$, $CA=-2$, $CD=1$, $DA=4$)

Шаг 0 (инициализация):

	A	B	C	D
A	0	1	∞	∞
B	∞	0	1	3
C	-2	∞	0	1
D	4	∞	∞	0

Шаг 1 (ведущая A): выделяем строку A и столбец A . Обновления: $\mu(C, B): \infty > (-2)+1 = -1$; $\mu(D, B): \infty > 4+1 = 5$.

	A	B	C	D
A	0	1	∞	∞
B	∞	0	1	3
C	-2	-1, A	0	1
D	4	5, A	∞	0

Шаг 2 (ведущая B): обновления: $\mu(A, C): \infty > 1 + 1 = 2$; $\mu(A, D): \infty > 1 + 3 = 4$; $\mu(D, C): \infty > 5 + 1 = 6$.

	A	B	C	D
A	0	1	2, B	4, B
B	∞	0	1	3
C	-2	-1, A	0	1
D	4	5, A	6, A	0

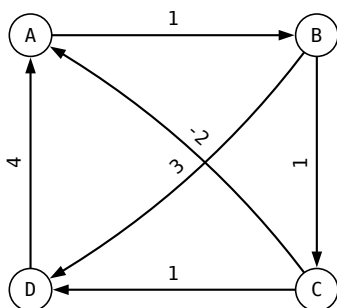
Шаг 3 (ведущая C): обновления: $\mu(A, D): 4 > 2 + 1 = 3$; $\mu(B, A): \infty > 1 + (-2) = -1$; $\mu(B, D): 3 > 1 + 1 = 2$.

	A	B	C	D
A	0	1	2, B	3, B
B	-1, C	0	1	2, C
C	-2	-1, A	0	1
D	4	5, A	6, A	0

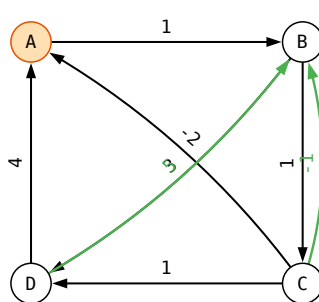
Шаг 4 (ведущая D): нет изменений (все пути через D дороже найденных).

	A	B	C	D
A	0	1	2, B	3, B
B	-1, C	0	1	2, C
C	-2	-1, A	0	1
D	4	5, A	6, A	0

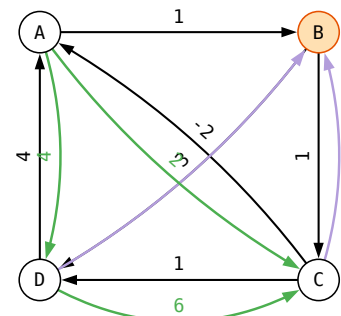
Работа алгоритма по шагам. Ведущая вершина — оранжевая; найденные на текущем шаге кратчайшие обходы — зелёные дуги с новой длиной, найденные ранее — светло-фиолетовые:



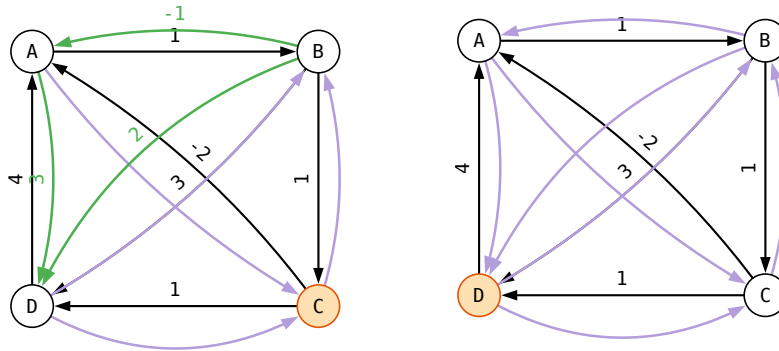
Шаг 0. Исходный граф



Шаг 1. Через A: $CB=-1$, $DB=5$



Шаг 2. Через B: $AC=2$, $AD=4$, $DC=6$



Шаг 3. Через C: $AD=3$, $BA=-1$, $BD=2$ Шаг 4. Через D обновлений нет — итог

Восстановление путей:

$A \rightarrow D$: $P_{AD} = B$. $A \xrightarrow{1} B$, $P_{BD} = C$, $B \xrightarrow{1} C$, P_{CD} пуст, $C \xrightarrow{1} D$. Итого: $A \xrightarrow{1} B \xrightarrow{1} C \xrightarrow{1} D$, длина = 3.

$C \rightarrow B$: $P_{CB} = A$. $C \xrightarrow{-2} A$, P_{AB} пуст, $A \xrightarrow{1} B$. Итого: $C \xrightarrow{-2} A \xrightarrow{1} B$, длина = -1.

Циклы отрицательного суммарного веса $\Leftrightarrow$ отрицательные значения на главной диагонали.

Почему: диагональный элемент $\mu(v, v)$ — длина кратчайшего замкнутого маршрута через v ; он станет отрицательным в точности тогда, когда через v проходит цикл отрицательного веса. Здесь диагональ нулевая: например, цикл $A \rightarrow B \rightarrow C \rightarrow A$ весит $1 + 1 - 2 = 0$, отрицательных циклов нет.

Алгоритм 19. Алгоритм Джонсона

Преобразование графа к виду с неотрицательными весами рёбер с сохранением набора кратчайших путей.

$G(V, E)$ — слабо связный, $\sigma: E \rightarrow \mathbb{R}$.

Идея. Дейкстра быстрее Флойда ($|V|$ запусков Дейкстры дешевле при разреженных графах), но не работает с отрицательными рёбрами. Джонсон «выпрямляет» веса потенциалами так, что все веса становятся неотрицательными, а кратчайшие пути не меняются.

Шаг 1. Джонсоновский потенциал.

Добавляем фиктивную вершину S : $\forall u \in V: e_u = Su$, $\sigma(e_u) = 0$.

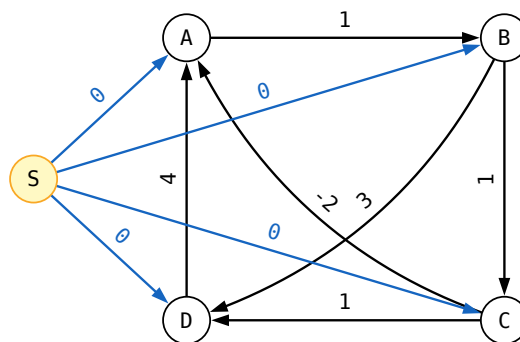


Рис. 67. Алг. Джонсона: тот же граф + фиктивная S (синие дуги веса 0)

Запускаем **алгоритм Форда–Беллмана** от S (Дейкстру пока нельзя — веса ещё отрицательные); $h(u) :=$ найденное расстояние от S до u .

Результат Форда–Беллмана (потенциалы):

$$h(A) = -2, \quad h(B) = -1, \quad h(C) = h(D) = 0.$$

(Проверка: до C и D дешевле всего прямые нулевые дуги из S , т.е. $h(C) = h(D) = 0$; до A выгоден путь $S \rightarrow C \rightarrow A$ весом $0 + (-2) = -2$; до B — путь $S \rightarrow C \rightarrow A \rightarrow B$ весом $0 - 2 + 1 = -1$.)

Шаг 2. Пересчёт весов.

Новая весовая функция:

$$\tilde{\sigma}(\overline{uv}) = \sigma(\overline{uv}) + h(u) - h(v) \geq 0.$$

Почему $\tilde{\sigma} \geq 0$: h — корректные расстояния от S , поэтому для любого ребра uv выполнено неравенство треугольника $h(v) \leq h(u) + \sigma(uv)$, что и означает $\sigma(uv) + h(u) - h(v) \geq 0$.

Ребро	σ	$\tilde{\sigma}$	Вычисление
AB	1	0	$1 + (-2) - (-1) = 0$
BC	1	0	$1 + (-1) - 0 = 0$
BD	3	2	$3 + (-1) - 0 = 2$
CA	-2	0	$-2 + 0 - (-2) = 0$
CD	1	1	$1 + 0 - 0 = 1$
DA	4	6	$4 + 0 - (-2) = 6$

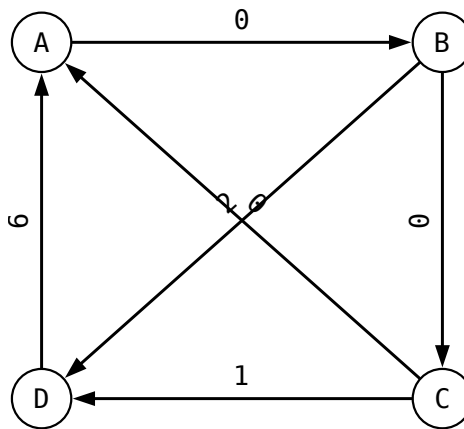


Рис. 68. Перевешенный граф: все $\tilde{\sigma} \geq 0$

Все $\tilde{\sigma} \geq 0$ — можно применять Дейкстру!

Шаг 3. Дейкстра от каждой вершины с весами $\tilde{\sigma}$.

Матрица расстояний в весах $\tilde{\sigma}$:

$\text{dist}_{\tilde{\sigma}}$	A	B	C	D
A	0	0	0	1
B	0	0	0	1
C	0	0	0	1
D	6	6	6	0

(Например, из D : единственный выход — дуга DA веса 6, дальше всё бесплатно по нулевым дугам $A \rightarrow B \rightarrow C$; поэтому вся строка D равна 6.)

Шаг 4. Обратный пересчёт.

Для получения истинных расстояний:

$$\text{dist}_\sigma(u, v) = \text{dist}_{\tilde{\sigma}}(u, v) - h(u) + h(v).$$

Почему пути не изменились: для любого пути $u = x_0 \rightarrow x_1 \rightarrow \dots \rightarrow x_k = v$ его новая длина равна

$$\sum_i \tilde{\sigma}(x_i x_{i+1}) = \sum_i \sigma(x_i x_{i+1}) + h(u) - h(v)$$

— телескопическая сумма: все промежуточные потенциалы сокращаются. Все пути из u в v сдвинулись на *одну и ту же* константу $h(u) - h(v)$, поэтому порядок путей по длине (и сами кратчайшие пути) сохранился, а формула пересчёта очевидна.

Итоговая матрица истинных расстояний ($\text{dist}_\sigma = \text{dist}_{\tilde{\sigma}} - h(u) + h(v)$):

dist_σ	A	B	C	D
A	0	1	2	3
B	-1	0	1	2
C	-2	-1	0	1
D	4	5	6	0

Например, $\text{dist}_\sigma(D, A) = 6 - h(D) + h(A) = 6 - 0 - 2 = 4$; $\text{dist}_\sigma(B, A) = 0 - (-1) + (-2) = -1$. Матрица совпала с результатом алгоритма Флойда выше — проверка пройдена. ✓

Свойства:

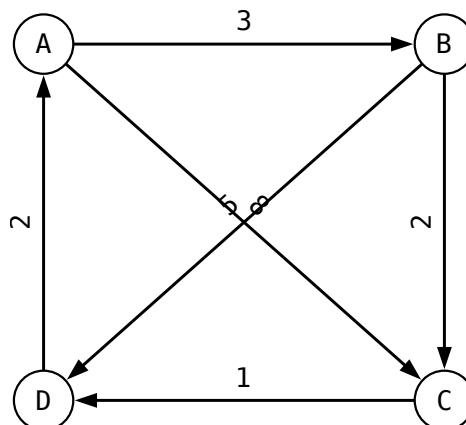
1. Набор кратчайших путей для G в терминах σ и $\tilde{\sigma}$ совпадает (пути не меняются, только длины) — телескопическая сумма выше.
2. $\forall e \in E: \tilde{\sigma}(e) \geq 0$ (неравенство треугольника для корректных потенциалов).
3. Если $\tilde{\sigma}(e) < 0$ для какого-то ребра — значит, потенциалы найдены неправильно.

Задача 14. Кратчайшие пути между всеми парами: алгоритм Флойда

Условие. Дан ориентированный взвешенный граф на вершинах A, B, C, D с дугами

$$A \rightarrow B = 3, A \rightarrow C = 8, B \rightarrow C = 2, B \rightarrow D = 5, C \rightarrow D = 1, D \rightarrow A = 2.$$

Найдите кратчайшие расстояния между всеми парами вершин алгоритмом Флойда.



Решение. Флойд последовательно разрешает в качестве промежуточных вершины A, B, C, D . На шаге k обновляем $d[i][j] \leftarrow \min(d[i][j], d[i][k] + d[k][j])$.

Начальная матрица $D^{(0)}$ (на диагонали 0, отсутствующие дуги — ∞):

$D^{(0)}$	A	B	C	D
A	0	3	8	∞
B	∞	0	2	5
C	∞	∞	0	1
D	2	∞	∞	0

Через A: обновляются пути из D ($D \rightarrow A = 2$): $D \rightarrow B = 2 + 3 = 5$, $D \rightarrow C = 2 + 8 = 10$.

Через B: $A \rightarrow C = \min(8, 3 + 2) = 5$, $A \rightarrow D = 3 + 5 = 8$; $D \rightarrow C = \min(10, 5 + 2) = 7$.

Через C: $A \rightarrow D = \min(8, 5 + 1) = 6$, $B \rightarrow D = \min(5, 2 + 1) = 3$.

Через D: замыкаются обратные пути: $B \rightarrow A = 3 + 2 = 5$, $C \rightarrow A = 1 + 2 = 3$, $C \rightarrow B = 1 + 2 + 3 = 6$ и т. д.

Итоговая матрица кратчайших расстояний $D^{(4)}$:

$D^{(4)}$	A	B	C	D
A	0	3	5	6
B	5	0	2	3
C	3	6	0	1
D	2	5	7	0

Проверка: $B \rightarrow A$ идёт по маршруту $B \rightarrow C \rightarrow D \rightarrow A = 2 + 1 + 2 = 5$; $C \rightarrow B = C \rightarrow D \rightarrow A \rightarrow B = 1 + 2 + 3 = 6$ — совпадает с таблицей.

Ответ. Матрица кратчайших расстояний $D^{(4)}$ приведена выше.

Все определения

Определение 1. Бинарное отношение (через подмножество) (см. стр. 2)

$M \neq \emptyset$. Бинарное отношение R на множестве M :

$$R \subseteq (M \times M).$$

Примеры (дополнительные).

1. На множестве людей: $R = \{(x, y) : x \text{ — родитель } y\}$. Это отношение не симметрично и не рефлексивно.
2. На $\mathbb{Z}$: отношение делимости $R = \{(a, b) : a \mid b\}$ — например, $(3, 12) \in R$, но $(12, 3) \notin R$.

Определение 2. Бинарное отношение (через функцию) (см. стр. 2)

$M \neq \emptyset$. Бинарное отношение f на множестве M :

$$f: M^2 \rightarrow \mathbb{B}, \quad \text{где } 1 \text{ — принадлежность, } 0 \text{ — непринадлежность.}$$

Определения 1 и 2 эквивалентны: $f = \mathbb{I}_R$ — индикаторная функция.

Индикаторная функция. Пусть X — объемлющее множество, $A \subseteq X$. Тогда

$$\mathbb{I}_A: X \rightarrow \mathbb{B}, \quad \mathbb{I}_A(x) = \begin{cases} 1, & x \in A, \\ 0, & x \notin A. \end{cases}$$

Примеры (дополнительные).

1. Сравнение строк: $f(s_1, s_2) = 1 \Leftrightarrow s_1$ является префиксом s_2 . Например, $f(\mathbf{ab}, \mathbf{abc}) = 1$.
2. Взаимная простота на $\mathbb{N}$: $f(a, b) = 1 \Leftrightarrow \gcd(a, b) = 1$; $f(8, 15) = 1$, а $f(8, 12) = 0$.

Определение 3. Отношение эквивалентности (см. стр. 4)

$M \neq \emptyset$, $f: M^2 \rightarrow \mathbb{B}$.

f — **отношение эквивалентности** $\Leftrightarrow f$ рефлексивно, симметрично и транзитивно.

Примеры (дополнительные).

1. Сравнение по модулю m на $\mathbb{Z}$: $a \sim b \Leftrightarrow m \mid (a - b)$. Классы — вычеты $\overline{0}, \dots, \overline{m-1}$.
2. «Иметь одинаковую длину» на множестве слов: разбивает все слова на классы по числу букв.

Определение 4. Класс эквивалентности (см. стр. 4)

$M \neq \emptyset$, f — эквивалентность на M .

$\forall a \in M$ **класс эквивалентности** элемента a :

$$H_a := \{c \in M \mid f(a, c) = 1\}.$$

Примеры (дополнительные).

1. Для сравнения по модулю 3: класс $\bar{1} = \{\dots, -2, 1, 4, 7, \dots\}$ — все числа вида $3k + 1$.
2. Для отношения «подобия» треугольников: класс — все треугольники, подобные данному (с точностью до масштаба).

Определение 5. Частичный (нестрогий) порядок (см. стр. 5)

$M \neq \emptyset, f: M^2 \rightarrow \mathbb{B}$.

f — **частичный (нестрогий) порядок** $\Leftrightarrow f$ рефлексивно, антисимметрично и транзитивно.

Примеры (дополнительные).

1. Включение множеств $\subseteq$ на булеане 2^X : например, $\{1\} \subseteq \{1, 2\}$, но $\{1\}$ и $\{2\}$ несравнимы.
2. Покомпонентный порядок на $\mathbb{R}^2$: $(a_1, a_2) \preceq (b_1, b_2) \Leftrightarrow a_1 \leq b_1$ и $a_2 \leq b_2$; точки $(1, 0)$ и $(0, 1)$ несравнимы.

Определение 6. Линейный порядок (см. стр. 5)

$M \neq \emptyset, f: M^2 \rightarrow \mathbb{B}$.

f — **линейный порядок** $\Leftrightarrow f$ — частичный порядок и $\forall a, b \in M: f(a, b) = 1 \vee f(b, a) = 1$.

Примеры (дополнительные).

1. Обычный порядок $\leq$ на $\mathbb{Q}$: любые два числа сравнимы.
2. Лексикографический порядок на словах (как в словаре): $\text{cat} < \text{dog}$; любые два слова сравнимы.

Определение 7. Строгий порядок (см. стр. 12)

$M \neq \emptyset, f: M^2 \rightarrow \mathbb{B}$.

f — **строгий порядок** $\Leftrightarrow f$ асимметрично и транзитивно.

Примеры (дополнительные).

1. Строгое включение $\subsetneq$ на множествах: $\{1\} \subsetneq \{1, 2\}$, но $\{1\} \not\subsetneq \{1\}$.
2. Отношение «быть строго моложе» на множестве людей — антирефлексивно и транзитивно.

Определение 8. Замыкание бинарного отношения (см. стр. 12)

$M \neq \emptyset, f: M^2 \rightarrow \mathbb{B}, \tilde{f}: M^2 \rightarrow \mathbb{B}$.

Замыкание f относительно свойства $(*)$ — это $\tilde{f}$, т. ч.:

1. $\tilde{f}$ согласовано с f (т. е. $\forall a, b: f(a, b) = 1 \Rightarrow \tilde{f}(a, b) = 1$);
2. $\tilde{f}$ обладает свойством $(*)$;

3. $\tilde{f}$ имеет минимальный носитель среди всех б. о., удовлетворяющих 1) и 2) (минимальное количество рёбер).

Примеры (дополнительные).

1. Транзитивное замыкание отношения «прямой рейс между городами» — отношение «можно долететь с пересадками».
2. Рефлексивно-транзитивное замыкание отношения « x ссылается на y » на веб-страницах — достижимость по гиперссылкам.

Определение 9. Носитель функции (см. стр. 12)

$f: A \rightarrow B$. Носитель f :

$$\text{supp}(f) := \{a \in A \mid f(a) \neq 0\}.$$

Примеры (дополнительные).

1. Для функции $f(x) = \max(0, 1 - |x|)$ носитель — отрезок $[-1, 1]$ (вне него $f = 0$).
2. Для последовательности $a = (0, 0, 5, 0, 7, 0, \dots)$ носитель — множество индексов $\{2, 4\}$.

Определение 10. Степень вершины (см. стр. 15)

Степень вершины (для неор. графа) — количество инцидентных ей рёбер:

$$\deg(v).$$

Пусть $G(V, E)$, $|V| = n$. Тогда $0 \leq \deg(v) \leq n - 1$.

Примеры (дополнительные).

1. В графе-звезде $K_{1,4}$ центр имеет степень 4, а каждый из четырёх лучей — степень 1.
2. В кубическом графе (например, графе вершин куба Q_3) все 8 вершин имеют степень 3.

Определение 11. Полный граф (см. стр. 15)

Полный граф K_n — граф, в котором любые две вершины смежные.

Примеры (дополнительные).

1. K_5 — пять вершин, все $\binom{5}{2} = 10$ рёбер; это наименьший непланарный полный граф.
2. K_2 — одно ребро; простейший нетривиальный полный граф.

Определение 12. Дерево (см. стр. 15)

Граф называется **деревом**, если он связный и ациклический.

Примеры (дополнительные).

1. Двоичное дерево поиска из информатики — связный ациклический граф с выделенным корнем.
2. Граф-путь P_5 $(1-2-3-4-5)$ — дерево, в котором ровно две вершины степени 1.

Определение 13. Лес (см. стр. 15)

Граф называется **лесом**, если он ациклический (связность *не* требуется). Эквивалентно: лес — граф, каждая компонента связности которого является деревом; дерево — это связный лес.

Примеры (дополнительные).

1. Несколько изолированных деревьев в базе данных каталогов (например, файловые системы нескольких дисков) — лес.
2. Граф из двух непересекающихся треугольников — *не* лес (есть циклы); а вот два непересекающихся пути — лес.

Определение 14. Связный граф (см. стр. 16)

Связный граф — между любыми двумя вершинами $\exists$ путь.

Примеры (дополнительные).

1. Граф автомобильных дорог между городами одного острова, если из любого города можно доехать в любой другой.
2. Цикл C_6 связан; а граф из C_3 и отдельной вершины — нет.

Определение 15. Компонента связности (см. стр. 16)

Компонента связности — максимальный по включению связный подграф.

Примеры (дополнительные).

1. В социальной сети — группа людей, связанных цепочками знакомств, не связанная с остальными.
2. В графе из двух треугольников и одной изолированной вершины — ровно три компоненты связности.

Определение 16. Расстояние между вершинами (см. стр. 19)

$\forall u, v \in V$:

$$\text{dist}(u, v) := \min l,$$

где l — длина пути $u \rightarrow \dots \rightarrow v$.

Свойства:

- $\text{dist}(u, u) = 0$;
- $\text{dist}(u, v) = \text{dist}(v, u)$.

Примеры (дополнительные).

1. В графе-цикле C_6 расстояние между противоположными вершинами равно 3.
2. В дереве расстояние между двумя листьями — длина единственной цепи, их соединяющей.

Определение 17. Эксцентриситет вершины (см. стр. 19)

$v \in V$:

$$\varepsilon(v) := \max_{u \in V} \{\text{dist}(u, v)\}.$$

Примеры (дополнительные).

1. В звезде $K_{1,n}$ эксцентриситет центра равен 1, а любого луча — 2.
2. В пути P_5 эксцентриситет концевой вершины равен 4, средней — 2.

Определение 18. Радиус графа (см. стр. 19)

$$R(G) := \min_{v \in V} \{\varepsilon(v)\}.$$

Примеры (дополнительные).

1. У звезды $K_{1,n}$ радиус равен 1 (центр близок ко всем).
2. У цикла C_7 радиус равен 3 (как и диаметр).

Определение 19. Диаметр графа (см. стр. 19)

$$D(G) := \max_{v \in V} \{\varepsilon(v)\}.$$

Примеры (дополнительные).

1. У полного графа K_n диаметр равен 1 — все вершины смежны.
2. У пути P_6 диаметр равен 5 (между концами).

Определение 20. Центр графа (см. стр. 19)

$$\text{Cent}(G) := \{ u \in V \mid \varepsilon(u) = R(G) \}.$$

Примеры (дополнительные).

1. Центр звезды $K_{1,n}$ — единственная центральная вершина.
2. У чётного цикла C_6 центр — все вершины (граф вершинно-транзитивен).

Определение 21. k -связность (см. стр. 24)

G — k -связный $\Leftrightarrow$:

1. $\forall e_{i_1}, \dots, e_{i_{k-1}} \in E$: $G(V, E \setminus \{e_{i_1}, \dots, e_{i_{k-1}}\})$ — связный (удаление любых $k - 1$ рёбер не нарушает связность);
2. $\exists e_{j_1}, \dots, e_{j_k} \in E$, т. ч. $G(V, E \setminus \{e_{j_1}, \dots, e_{j_k}\})$ — несвязный (существует набор из k рёбер, удаление которого нарушает связность).

Примеры (дополнительные).

1. Цикл C_n ($n \geq 3$) является 2-связным: нужно удалить не менее двух вершин, чтобы разорвать его.
2. Полный граф K_4 является 3-связным.

Определение 22. Слабая связность (см. стр. 28)

$G(V, E)$ — ориентированный граф.

G — слабо связан $\Leftrightarrow G$ связан без учёта ориентации рёбер.

Примеры (дополнительные).

1. Орграф $a \rightarrow b \leftarrow c$ слабо связан (как неориентированный — связан), но не сильно.
2. Дерево с произвольно ориентированными рёбрами слабо связано.

Определение 23. Сильная связность (см. стр. 28)

$G(V, E)$ — ориентированный граф.

G — сильно связан $\Leftrightarrow \forall u, v \in V$: $\exists$ путь $u \rightarrow \dots \rightarrow v$.

Примеры (дополнительные).

1. Ориентированный цикл $a \rightarrow b \rightarrow c \rightarrow a$ сильно связан.
2. Двухнаправленная дорожная сеть (каждое ребро в обе стороны) — сильно связна, если связна как неориентированная.

Определение 24. Компонента сильной связности (см. стр. 28)

Компонента сильной связности — максимальный по включению сильно связный подграф.

Примеры (дополнительные).

1. В орграфе $a \rightarrow b \rightarrow a$, $b \rightarrow c$ компонента $\{a, b\}$ сильно связна, а $\{c\}$ — отдельная.
2. В ациклическом орграфе каждая вершина — своя собственная компонента сильной связности.

Определение 25. Граф Герца (конденсации) (см. стр. 29)

$G(V, E)$ — оргграф.

Граф Герца (граф конденсации) для $G = \tilde{G}(\tilde{V}, \tilde{E})$, где:

- $\tilde{V} \ni w \Leftrightarrow w$ — компонента сильной связности G ;
- $\tilde{e} \in \tilde{E} \Leftrightarrow$ между соответствующими компонентами сильной связности в $G \exists$ путь.

Примеры (дополнительные).

1. Для оргграфа $a \rightleftarrows b$, $b \rightarrow c$ граф конденсации — $\{a, b\} \rightarrow \{c\}$ (две вершины, одна дуга).
2. Конденсация любого сильно связного графа — одна вершина.

Определение 26. Эйлеров цикл и эйлеров путь (см. стр. 32)

Дан $G(V, E)$.

G содержит **эйлеров цикл** $\Leftrightarrow \exists$ в G цикл, содержащий $\forall e \in E$ ровно 1 раз.

G содержит **эйлеров путь** $\Leftrightarrow$ в $G \exists$ открытый простой путь, содержащий $\forall e \in E$.

Примеры (дополнительные).

1. У «конверта» (квадрат с двумя диагоналями сверху) есть эйлеров путь — классическая задача «нарисовать не отрывая руки».
2. Полный граф K_5 имеет эйлеров цикл (все степени равны 4).

Определение 27. Эйлеров граф (см. стр. 32)

G — **эйлеров**:

- G — ориент. $\Leftrightarrow$ в $G \exists$ эйлеров цикл, G — слабо связан.
- G — неориент. $\Leftrightarrow$ в $G \exists$ эйлеров цикл, G — связный.

Примеры (дополнительные).

1. K_5 — эйлеров (все степени чётны).
2. Любой цикл C_n — эйлеров граф.

Определение 28. Полуэйлеров граф (см. стр. 32)

G — **полуэйлеров**:

- G — ориент. $\Leftrightarrow$ в $G \exists$ эйлеров путь, G — слабо связан.
- G — неориент. $\Leftrightarrow$ в $G \exists$ эйлеров путь, G — связный.

Примеры (дополнительные).

1. Путь P_4 — полуэйлеров (две вершины нечётной степени).

2. «Конверт» — полуэйлеров: ровно две вершины степени 3.

Определение 29. Гамильтонов путь (см. стр. 42)

$G(V, E)$.

G содержит **гамильтонов путь** $\Leftrightarrow \exists$ путь, проходящий через $\forall v \in V$ ровно 1 раз.

$\Rightarrow G$ — **полугамильтонов**.

Примеры (дополнительные).

1. В графе шахматного коня 5×5 существует гамильтонов путь («обход конём»).
2. В полном графе K_n гамильтонов путь — любая перестановка вершин.

Определение 30. Гамильтонов цикл (см. стр. 42)

G содержит **гамильтонов цикл** $\Leftrightarrow \exists$ замкнутый простой путь через $\forall v \in V$ ровно 1 раз, кроме $v_{\text{start}} = v_{\text{finish}}$.

$\Rightarrow G$ — **гамильтонов**.

Примеры (дополнительные).

1. Граф додекаэдра имеет гамильтонов цикл (историческая «игра в кругосветное путешествие» Гамильтона).
2. Любой цикл C_n сам является гамильтоновым циклом.

Определение 31. Турнир (см. стр. 44)

Турнир — ориентированный граф, в котором каждая пара различных вершин соединена *ровно одной* дугой (в одном из двух направлений). Иначе говоря, турнир — это полный граф K_n , каждому ребру которого приписана ориентация.

Примеры (дополнительные).

1. Итоги шахматного кругового турнира без ничьих: дуга от победителя к проигравшему.
2. Транзитивный турнир на $\{1, 2, 3\}$: $1 \rightarrow 2$, $1 \rightarrow 3$, $2 \rightarrow 3$ — кодирует линейный порядок.

Определение 32. Граф де Брёйна (см. стр. 47)

$\Sigma = \{a_0, \dots, a_{k-1}\}$ — алфавит.

$\Sigma^m = \{a_{i_1} \dots a_{i_m} \mid a_{i_j} \in \Sigma\}$ — слова длины m .

$\Sigma^m = V$ (вершины — слова длины m).

$E = \Sigma^{m+1}$ (рёбра — слова длины $m+1$).

Ребро $a_{i_1} a_{i_2} \dots a_{i_m} a_{i_{m+1}}$ соединяет вершину $(a_{i_1} \dots a_{i_m})$ с вершиной $(a_{i_2} \dots a_{i_{m+1}})$: суффикс первой вершины совпадает с префиксом второй; a_{i_1} и $a_{i_{m+1}}$ — любые.

$B(\Sigma, m)$ — **граф де Брёйна**.

Примеры (дополнительные).

1. $B(\{0, 1\}, 3)$: вершины — двоичные слова длины 3, дуги — слова длины 4; эйлеров цикл даёт последовательность де Брёйна длины 16.

2. Над алфавитом $\{A, C, G, T\}$ графы де Брёйна применяются для сборки геномов из коротких прочтений.

Определение 33. Дерево (см. стр. 48)

Ациклический, связный граф.

Примеры (дополнительные).

1. Дерево разбора арифметического выражения в компиляторе.
2. Турнирная сетка плей-офф (одиночное выбывание) — двоичное дерево.

Определение 34. Рёберно-взвешенный граф (см. стр. 55)

$G(V, E)$ — (рёберно-)взвешенный:

$\exists \sigma: E \rightarrow W$,

где W — некоторое упорядоченное мн-во с операциями, зависящими от задачи.

NB: W — нижняя упорядоченная мн-во.

Примеры (дополнительные).

1. Карта авиаперелётов, где вес ребра — стоимость билета.
2. Электрическая схема, где вес ребра — сопротивление провода.

Определение 35. Остов (каркас) (см. стр. 55)

Остов (скелет, каркас) — связный подграф в G , являющийся деревом и содержащий все вершины G .

Примеры (дополнительные).

1. У цикла C_4 четыре различных остова (убираем любое из рёбер).
2. У полного графа K_n число остовов равно n^{n-2} (теорема Кэли).

Определение 36. Остовное дерево (см. стр. 55)

$G(V, E)$ — **остовное дерево** G : T — дерево и остов.

Если граф — не дерево, то остовов несколько.

Примеры (дополнительные).

1. Дерево обхода в ширину (BFS-дерево) — остовное дерево графа.
2. В сетке 2×2 (C_4) любой из четырёх путей P_4 — остовное дерево.

Определение 37. Минимальное остовное дерево (см. стр. 55)

$G(V, E)$ — связный, взвешенный.

$\sigma: E \rightarrow \mathbb{R}$.

Минимальное остовное дерево G : $T_G: \sum_{\substack{(v,\hat{v}) \in T \\ e \in \hat{E}}} \sigma(e) \longrightarrow \min$ по всем T для G .

Примеры (дополнительные).

1. Прокладка кабеля между зданиями с минимальной суммарной длиной — классическая задача о минимальном остове.
2. Если все веса различны, минимальное остовное дерево единственно.

Определение 38. Хорда (см. стр. 63)

$G(V, E)$ — связный, невзвешенный.

$T_G(V, \hat{E})$ — остовное дерево G .

Хорда $\leftrightarrow e \in (E \setminus \hat{E})$.

Примеры (дополнительные).

1. В цикле C_5 с выбранным остовом-путём единственное оставшееся ребро — хорда.
2. У графа K_4 с остовом-звездой три ребра являются хордами.

Определение 39. Сумма циклов (см. стр. 64)

$G(V, E)$, l_1, l_2 — 2 цикла (замкнутых пути) в графе.

$l_1 \oplus l_2 := (e \in l_1) \Delta (e \in l_2)$

(симметрическая разность рёбер).

Примеры (дополнительные).

1. Сумма двух смежных граней-циклов в плоском графе — их внешний контур (общее ребро сокращается).
2. Сумма цикла с самим собой — пустое множество рёбер (нейтральный элемент).

Определение 40. Двудольный граф (см. стр. 68)

$G(V, E)$ — **двудольный**:

$V = V_1 \sqcup V_2$ т. ч. $\forall e \in E: u \in V_1, v \in V_2$ (начальная вершина — любая).

Примеры (дополнительные).

1. Граф «студенты–курсы» (ребро = студент записан на курс) — двудольен по построению.
2. Гиперкуб Q_n двудольен: доли — слова чётного и нечётного веса Хэмминга.

Определение 41. Паросочетание (см. стр. 71)

$G(V, E)$ — двудольный, $V = V_1 \sqcup V_2$.

Паросочетание для G : $\hat{E} \subseteq E$ т. ч. $\forall e_i, e_j \in \hat{E}$ — инцидентны не более чем одной общей вершине (т. е. рёбра паросочетания попарно не имеют общих вершин).

Примеры (дополнительные).

1. Распределение работников по задачам «один к одному» — паросочетание в двудольном графе.
2. В графе-цикле C_6 три попарно несмежных ребра образуют совершенное паросочетание.

Определение 42. Максимальное паросочетание (см. стр. 71)

Максимальное паросочетание — максимальное по включению: к нему нельзя добавить ни одно ребро графа, не нарушив определение.

Примеры (дополнительные).

1. В пути P_5 паросочетание $\{1-2, 3-4\}$ максимально (вершина 5 остаётся, но добавить нечего).
2. Любое совершенное паросочетание автоматически максимально.

Определение 43. Наибольшее паросочетание (см. стр. 71)

Наибольшее паросочетание — наибольшее количество рёбер для данного графа.

Примеры (дополнительные).

1. В пути P_5 наибольшее паросочетание содержит 2 ребра.
2. В K_4 наибольшее паросочетание совершенно и состоит из 2 рёбер.

Определение 44. Сеть (см. стр. 76)

$G(V, E)$ — **сеть**: (орграф)

- **исток**: S — нет входящих рёбер!
- **сток/устье**: F — нет исходящих рёбер!

$\sigma: E \rightarrow \mathbb{R}_{\geq 0}$ (пропускная способность, пока на рёбрах).

Примеры (дополнительные).

1. Водопроводная сеть: исток — водонапорная башня, сток — потребитель, пропускные способности — диаметры труб.
2. Сеть передачи данных: рёбра — каналы связи с ограниченной пропускной способностью (бит/с).

Определение 45. Поток (см. стр. 76)

Поток: $f: E \rightarrow \mathbb{R}$, свойства:

1. $0 \leq f(e) \leq \sigma(e)$
2. **Условие балансировки**: $\forall v \in V \setminus \{S, F\}$:
$$\sum_{\substack{e: u \rightarrow v \\ u \in V}} f(e) = \sum_{\substack{e: v \rightarrow w \\ w \in V}} f(e)$$

(поток входящий = поток исходящий)

Примеры (дополнительные).

1. Реальное распределение воды по трубам, не превышающее их пропускных способностей, с балансом в узлах.
2. Нулевой поток ($f \equiv 0$) — всегда допустимый поток мощности 0.

Определение 46. Мощность потока (см. стр. 77)

Мощность потока $|f|$ = суммарное количество перемещаемого в единицу времени (всё, что вытекает из S).

$|f| \rightarrow \max$

Поток — функция, НЕ число!

Примеры (дополнительные).

1. Если из истока выходят два насыщенных ребра по 4 и 6, то мощность потока равна 10.
2. Мощность нулевого потока равна 0.

Определение 47. Остаточная пропускная способность (см. стр. 81)

Для ребра $e = \overline{uv}$ и текущего потока f **остаточная пропускная способность** $\tilde{\sigma}(e)$ показывает, сколько ещё можно пропустить по e . Вводится обратное (фиктивное) ребро $\text{inv}(e) = \overline{vu}$, и

$$\tilde{\sigma}(e) + \tilde{\sigma}(\text{inv}(e)) = \sigma(e).$$

Изначально $\tilde{\sigma}(e) := \sigma(e)$, $\tilde{\sigma}(\text{inv}(e)) := 0$.

Примеры (дополнительные).

1. Ребро с пропускной способностью 5 и потоком 3 имеет остаточную способность 2 вперёд и 3 назад.
2. Насыщенное ребро имеет остаточную способность 0 в прямом направлении.

Определение 48. Остаточная сеть (см. стр. 81)

Остаточная сеть — сеть на тех же вершинах, рёбрами которой служат все рёбра с положительной остаточной пропускной способностью $\tilde{\sigma}(e) > 0$ (включая обратные рёбра $\text{inv}(e)$).

Примеры (дополнительные).

1. После проталкивания потока по пути в остаточной сети появляются обратные дуги, позволяющие «отменить» часть потока.
2. Остаточная сеть нулевого потока совпадает с исходной сетью.

Определение 49. Исчерпанное (насыщенное) ребро (см. стр. 81)

Исчерпанное ребро — ребро с нулевой остаточной пропускной способностью $\tilde{\sigma}(e) = 0$ (поток по нему равен пропускной способности: $f(e) = \sigma(e)$).

Примеры (дополнительные).

1. В минимальном разрезе все рёбра, идущие из истоковой доли, насыщены.
2. Ребро пропускной способности 0 насыщено при любом потоке.

Определение 50. Разрез в сети (см. стр. 81)

Разрез в сети — такой набор рёбер $\tilde{E} \subseteq E$, что удаление его разрывает все пути $S \rightarrow F$, т. е. граф $G(V, E \setminus \tilde{E})$ теряет связность между S и F .

NB. Любой мост — разрез. Обычно рассматривают $\tilde{E}$, минимальный по включению.

Примеры (дополнительные).

1. Разрез, отделяющий исток от всех остальных вершин — совокупность всех рёбер, выходящих из истока.
2. В цепочке $S \rightarrow A \rightarrow F$ разрез $\{A, F\}$ против $\{S\}$ состоит из ребра $S \rightarrow A$.

Определение 51. Главный разрез (см. стр. 81)

Главный разрез — разрез, при котором граф делится ровно на две части.

Примеры (дополнительные).

1. Главный разрез $\{S\}$ против $V \setminus \{S\}$ — все рёбра из истока.
2. Главный разрез $V \setminus \{F\}$ против $\{F\}$ — все рёбра в сток.

Определение 52. Пропускная способность разреза (см. стр. 81)

Пропускная способность разреза $\tilde{E}$ — сумма пропускных способностей его рёбер, направленных из части истока в часть стока: $\sigma(\tilde{E}) = \sum_{e \in \tilde{E}} \sigma(e)$.

Примеры (дополнительные).

1. Если из истоковой доли выходят рёбра с пропускными способностями 3 и 2, ёмкость разреза равна 5.
2. По теореме Форда–Фалкерсона минимальная ёмкость разреза равна максимальному потоку.

Определение 53. Предпоток (см. стр. 85)

Пусть G — сеть, σ — пропускная способность рёбер.

Предпоток — это функция $\tilde{f}: V \times V \rightarrow \mathbb{R}$, удовлетворяющая условиям:

1. **Ограниченность:** $\forall e = uv: -\sigma(e) \leq \tilde{f}(u, v) \leq \sigma(e)$;

2. **Кососимметричность:** $\forall u, v \in V: \tilde{f}(u, v) = -\tilde{f}(v, u);$

3. $\forall \nexists e = uv, e = vu \Rightarrow \tilde{f}(u, v) = 0.$

В отличие от потока, условие балансировки *не требуется*: в вершину может втекать больше, чем вытекает.

Примеры (дополнительные).

1. На старте push-relabel рёбра из истока насыщены — это предпоток с ненулевыми эксцессами у соседей истока.
2. Любой поток — частный случай предпотока (все внутренние эксцессы нулевые).

Определение 54. Эксцесс вершины (см. стр. 85)

Эксцесс вершины $\forall v \in V$:

$$\varepsilon(v) := \sum_{w \in P(v)} \tilde{f}(wv) - \sum_{u \in \text{Ch}(v)} \tilde{f}(vu),$$

где $P(v)$ — родители, $\text{Ch}(v)$ — дети (т. е. эксцесс — это «втекло минус вытекло»).

Примеры (дополнительные).

1. Если в вершину втекает 5, а вытекает 3, её эксцесс равен 2.
2. У вершины-потока (с соблюждённым балансом) эксцесс равен 0.

Определение 55. Высота вершины (см. стр. 85)

Высота вершины — это функция $h: V \rightarrow \mathbb{N}_0$. Инициализируется как $h(S) = |V|$, $h(v) = 0$ для остальных, и в процессе работы только растёт. Проталкивать избыток разрешается только «вниз»: из вершины v в вершину u с $h(u) = h(v) - 1$.

Примеры (дополнительные).

1. В начале push-relabel высота истока равна $|V|$, всех остальных — 0.
2. Операция lift поднимает застрявшую активную вершину на единицу выше её низшего соседа.

Определение 56. Правильная раскраска графа (см. стр. 90)

Раскраска графа называется **правильной**, если любые две смежные вершины окрашены в разные цвета.

Примеры (дополнительные).

1. Раскраска политической карты так, чтобы соседние страны имели разные цвета (граф соседства).
2. Раскраска вершин шахматной доски в два цвета (как клеток).

Определение 57. Хроматическое число (см. стр. 90)

Хроматическое число графа $\chi(G)$ — минимальное число цветов, в которые можно правильно раскрасить граф G .

Примеры (дополнительные).

1. $\chi(C_4) = 2$, $\chi(C_5) = 3$ (чётный и нечётный циклы).
2. $\chi(\text{дерева}) = 2$ для любого дерева хотя бы с одним ребром.

Определение 58. Клика (см. стр. 91)

Клика — максимальный полный подграф графа G .

Примеры (дополнительные).

1. В графе знакомств клика — компания, где все знакомы друг с другом.
2. Треугольник K_3 внутри большего графа — клика размера 3.

Определение 59. Хроматический многочлен (см. стр. 92)

Пусть $G(V, E)$ — граф. **Хроматический многочлен** графа G :

$$P_G: \mathbb{N}_0 \rightarrow \mathbb{N}_0, \quad P_G(t) = a_1 t + a_2 t^2 + \dots + a_n t^n,$$

т. е. это многочлен, значение которого $P_G(k)$ равно числу правильных раскрасок G в k цветов ($\forall k \in \mathbb{N}_0$).

Примеры (дополнительные).

1. Для одного ребра K_2 : $P(t) = t(t-1)$.
2. Для треугольника K_3 : $P(t) = t(t-1)(t-2)$; $P(2) = 0$ (в два цвета не раскрасить).

Определение 60. Правильная рёберная раскраска (см. стр. 99)

Рёберная раскраска графа называется **правильной**, если $\forall e_1, e_2 \in E$, имеющие общую вершину, окрашены в разные цвета.

Примеры (дополнительные).

1. Составление расписания: рёбра-уроки, инцидентные одному учителю, должны идти в разное время (разные цвета).
2. Рёбра K_4 можно правильно раскрасить в 3 цвета (три совершенных паросочетания).

Определение 61. Хроматически двойственный граф (см. стр. 99)

Пусть $G(V, E)$. **Хроматически двойственный** граф $G_\chi(V_\chi, E_\chi)$ строится так:

1. $V_\chi = E$ (вершины G_χ — это рёбра G);
2. $\hat{u}, \hat{v} \in V_\chi$: $\exists \hat{e} = \hat{u}\hat{v} \Leftrightarrow$ рёбра $u, v \in E$ имеют общую вершину в G .

Правильные *рёберные* раскраски G — это в точности правильные *вершинные* раскраски G_χ , поэтому рёберное хроматическое число $\chi'(G) = \chi(G_\chi)$.

Примеры (дополнительные).

1. Рёберный граф пути P_4 (3 ребра) — это путь P_3 .
2. Рёберный граф звезды $K_{1,n}$ — полный граф K_n (все рёбра попарно смежны в центре).

Все теоремы и утверждения

Теорема 1. Теорема о классификации (см. стр.)

$M \neq \emptyset$, f — эквивалентность на M . Тогда:

1. $\forall a \in M: a \in H_a$.
2. $\forall a, b \in M: H_a \cap H_b = \emptyset$ или $H_a = H_b$.
3. $M = \bigcup_i H_{a_i}$.

Доказательство. Напомним, что класс элемента a — это $H_a = \{x \in M: f(a, x) = 1\}$.

1) Каждый элемент лежит в своём классе. В силу рефлексивности $f(a, a) = 1$, поэтому $a \in H_a$; в частности, $H_a \neq \emptyset$.

2) Два класса либо не пересекаются, либо совпадают. Пусть $H_a \cap H_b \neq \emptyset$; покажем, что тогда $H_a = H_b$. Возьмём $c \in H_a \cap H_b$, то есть $f(a, c) = 1$ и $f(b, c) = 1$. Докажем включение $H_a \subseteq H_b$. Пусть $x \in H_a$, т. е. $f(a, x) = 1$. По симметричности из $f(a, c) = 1$ получаем $f(c, a) = 1$; тогда по транзитивности $f(c, a) = 1 \wedge f(a, x) = 1 \Rightarrow f(c, x) = 1$. Наконец, $f(b, c) = 1$ и $f(c, x) = 1$ дают по транзитивности $f(b, x) = 1$, то есть $x \in H_b$. Значит $H_a \subseteq H_b$; симметрично $H_b \subseteq H_a$, поэтому $H_a = H_b$. Итак, два класса либо совпадают, либо не пересекаются.

3) Классы покрывают всё множество. По пункту 1 каждый элемент $a \in M$ лежит в своём классе H_a , поэтому $M \subseteq \bigcup_{a \in M} H_a \subseteq M$, откуда $M = \bigcup_{a \in M} H_a$. Выбрав по одному представителю из каждого попарно различного класса (по пункту 2 различные классы не пересекаются), получаем разбиение $M = \bigsqcup_i H_{a_i}$ на непустые непересекающиеся классы. Это и есть искомая классификация. ■

Теорема 2. Лемма о рукопожатиях (см. стр.)

$G(V, E)$ — неориентированный граф. Тогда:

$$\sum_{v \in V} \deg(v) = 2|E|.$$

Доказательство. Посчитаем пары (v, e) , где ребро e инцидентно вершине v , двумя способами. С одной стороны, каждая вершина v участвует ровно в $\deg(v)$ таких парах, всего $\sum_{v \in V} \deg(v)$. С другой стороны, каждое ребро $e = \{u, w\}$ инцидентно ровно двум вершинам, т. е. участвует ровно в двух парах; всего $2|E|$. Значит, $\sum_v \deg(v) = 2|E|$. ■

Теорема 3. Лемма о чётности (см. стр.)

$G(V, E)$ — неориентированный граф. Тогда количество вершин нечётной степени чётно:

$$\#\{v \in V \mid \deg(v) \not\equiv 2\} \equiv 0 \pmod{2}.$$

Доказательство. Разобьём вершины на два множества: $V_{\text{ч}}$ — вершины чётной степени и $V_{\text{н}}$

— вершины нечётной степени. По лемме о рукопожатиях

$$\underbrace{\sum_{v \in V_{\text{ч}}} \deg(v)}_{\text{чётно}} + \sum_{v \in V_{\text{н}}} \deg(v) = 2|E| \quad \text{— чётно,}$$

поэтому $\sum_{v \in V_{\text{н}}} \deg(v)$ тоже чётно. Но это сумма *нечётных* слагаемых, а сумма k нечётных чисел чётна тогда и только тогда, когда k чётно. Значит, $|V_{\text{н}}|$ чётно. ■

Теорема 4. Лемма о числе рёбер полного графа (см. стр.)

$$|E_{K_n}| = \frac{n(n-1)}{2}.$$

Доказательство. В K_n любые две различные вершины соединены ровно одним ребром, поэтому рёбра взаимно однозначно соответствуют неупорядоченным парам вершин: $|E_{K_n}| = \binom{n}{2} = \frac{n(n-1)}{2}$. Иначе: каждая из n вершин имеет степень $n-1$, и по лемме о рукопожатиях $|E| = \frac{1}{2} \sum_v \deg(v) = \frac{n(n-1)}{2}$. ■

Теорема 5. Лемма о связности (см. стр.)

$G(V, E)$ — неориентированный связный граф, $|V| = n$. Тогда:

$$(n-1) \leq |E| \leq \frac{n(n-1)}{2}.$$

Доказательство. Верхняя оценка. Рёбер не может быть больше, чем всех неупорядоченных пар вершин, т. е. $|E| \leq \binom{n}{2} = \frac{n(n-1)}{2}$ (максимум достигается на полном графе K_n).

Нижняя оценка (индукция по n). База: при $n = 1$ имеем $|E| \geq 0 = n - 1$. Переход: пусть утверждение верно для всех связных графов с числом вершин меньше n . Возьмём связный граф G на $n \geq 2$ вершинах и удалим из него вершину v минимальной степени вместе с инцидентными ей рёбрами (их $\deg(v) \geq 1$, так как изолированных вершин в связном графе при $n \geq 2$ нет). Граф может распасться на $k \leq \deg(v)$ компонент связности с числом вершин $n_1, \dots, n_k$, $n_1 + \dots + n_k = n - 1$. По предположению индукции в i -й компоненте не менее $n_i - 1$ рёбер, итого

$$|E| \geq \deg(v) + \sum_{i=1}^k (n_i - 1) = \deg(v) + (n - 1) - k \geq n - 1,$$

поскольку $k \leq \deg(v)$. Оценка точна: у дерева ровно $n - 1$ ребро. ■

Теорема 6. Критерий Э и ПЭ (см. стр.)

I. $G(V, E)$ — неориент., связный.

a) $G - \text{Э} \Leftrightarrow \forall v \in V: \deg(v) \geq 2$

b) $G - \text{ПЭ} \Leftrightarrow \exists u \neq v \in V: \deg(u) \not\geq 2, \deg(v) \not\geq 2,$
 $\forall w \in V \setminus \{u, v\}: \deg(w) \geq 2$

II. G — ориент., связный (слабо).

а) $G - \exists \Leftrightarrow \forall v \in V: \delta^+(v) = \delta^-(v)$

б) $G - \text{П}\exists \Leftrightarrow \exists u, v \in V: \delta^+(u) = \delta^-(u) + 1, \delta^-(v) = \delta^+(v) + 1,$
 $\forall w \in V \setminus \{u, v\}: \delta^+(w) = \delta^-(w)$

Доказательство. а) Необходимость ($\Rightarrow$). Пусть в G есть эйлеров цикл. Идя по циклу, мы в каждую вершину столько же раз «входим», сколько «выходим», и каждое инцидентное вершине ребро используется ровно один раз — либо для входа, либо для выхода. Значит, рёбра при каждой вершине разбиваются на пары «вход–выход», и все степени чётны.

а) Достаточность ($\Leftarrow$). Пусть все степени чётны и G связан; индукция по числу рёбер. Возьмём любую вершину v_0 и пойдём по непройденным рёбрам, никогда не останавливаясь: войдя в вершину $w \neq v_0$, мы заняли нечётное число её рёбер, а степень чётна — значит, есть свободное ребро для выхода. Застрясть можно только в v_0 , т.е. получаем цикл C . Удалим рёбра C из графа: степени снова чётны (в каждой вершине цикл занял чётное число рёбер). Каждая компонента оставшегося графа по предположению индукции имеет эйлеров цикл, причём из связности G каждая компонента пересекается с C по вершине. Вклеим эти циклы в C при прохождении общих вершин — получим эйлеров цикл всего графа.

б) Сводится к пункту а): если ровно две вершины u, v нечётны, добавим фиктивное ребро $u-v$ — все степени станут чётными, по пункту а) существует эйлеров цикл; удалив из него фиктивное ребро, получим эйлеров путь из u в v . Обратно: эйлеров путь из u в v плюс фиктивное ребро $u-v$ дают эйлеров цикл, поэтому по пункту а) все вершины, кроме u и v , чётны, а u и v нечётны.

Случай II (ориентированный) доказывается дословно так же с заменой степени на полустепени: внутри цикла в каждую вершину входим столько же раз, сколько выходим, поэтому $\delta^+(v) = \delta^-(v)$, и наоборот. ■

Теорема 7. Теорема Оре (см. стр.)

$|V| = n \geq 3$. Если $\forall u, v \in V$ т.ч. $\nexists e = uv$ (u, v — несмежные): $\deg(u) + \deg(v) \geq n$, то G — гамильтонов.

Доказательство. От противного. Пусть условие выполнено, но гамильтонова цикла нет. Будем добавлять в G рёбра, пока это возможно без появления гамильтонова цикла (условие Оре при добавлении рёбер тем более сохраняется); получим *максимальный* контрпример G' : гамильтонова цикла нет, но добавление любого ребра его создаёт.

В G' есть несмежные вершины u, v (иначе $G' = K_n$ — гамильтонов). Добавление ребра uv создаёт гамильтонов цикл, значит уже в G' есть гамильтонов *путь* $u = v_1, v_2, \dots, v_n = v$.

Рассмотрим множества индексов $S = \{i \mid u \sim v_{i+1}\}$ и $T = \{i \mid v \sim v_i\}$; $|S| = \deg(u)$, $|T| = \deg(v)$, причём $S, T \subseteq \{1, \dots, n-1\}$. Так как $|S| + |T| = \deg(u) + \deg(v) \geq n > n-1$, по принципу Дирихле $\exists i \in S \cap T$: одновременно $u \sim v_{i+1}$ и $v \sim v_i$. Тогда

$$u = v_1, v_2, \dots, v_i, v_n = v, v_{n-1}, \dots, v_{i+1}, v_1$$

— гамильтонов цикл (идём по пути до v_i , прыгаем по ребру $v_i v_n$ в конец, возвращаемся по пути назад до v_{i+1} и замыкаемся ребром $v_{i+1} u$). Противоречие с тем, что в G' цикла нет. ■

Теорема 8. Теорема Дирака (см. стр.)

$|V| = n \geq 3$. Если $\forall u \in V: \deg(u) \geq \frac{n}{2}$, то G — гамильтонов.

Доказательство. Частный случай теоремы Оре: для любых несмежных u, v имеем $\deg(u) + \deg(v) \geq \frac{n}{2} + \frac{n}{2} = n$. ■

Теорема 9. Теорема Редди (о турнирах) (см. стр.)

В любом турнире существует гамильтонов путь.

Доказательство. Индукция по числу вершин n . База $n = 1$ тривиальна. Переход: удалим из турнира вершину v ; на оставшихся $n - 1$ вершинах по предположению индукции есть гамильтонов путь $u_1 \rightarrow u_2 \rightarrow \dots \rightarrow u_{n-1}$. Вставим v :

- если $v \rightarrow u_1$ — ставим v в начало;
- иначе $u_1 \rightarrow v$; пусть i — наибольший индекс с $u_i \rightarrow v$. Если $i = n - 1$, ставим v в конец. Если же $i < n - 1$, то по максимальной $v \rightarrow u_{i+1}$, и v вставляется между u_i и u_{i+1} :
 $u_1 \rightarrow \dots \rightarrow u_i \rightarrow v \rightarrow u_{i+1} \rightarrow \dots \rightarrow u_{n-1}$.

В каждом случае получили гамильтонов путь на всех n вершинах (заметим: гамильтонов *цикл* в турнире существовать не обязан — в транзитивном турнире из последней вершины дуги не выходят). ■

Теорема 10. Графы де Брёйна всегда эйлеровы (см. стр.)

$\forall \Sigma (|\Sigma| = k), \forall m$: граф $B(\Sigma, m)$ — эйлеров.

Доказательство. Проверим критерий эйлеровости для ориентированного графа.

Полустепени равны. Из вершины $w = a_{i_1} \dots a_{i_m}$ выходит ровно k дуг — слова $w \cdot a$ для каждой буквы $a \in \Sigma$ (ведут в вершины $a_{i_2} \dots a_{i_m} a$). Входит в w тоже ровно k дуг — слова $a \cdot w$ для каждой $a \in \Sigma$. Значит, $\delta^-(v) = \delta^+(v) = k$ для всех вершин.

Связность. Из любой вершины $u = a_{i_1} \dots a_{i_m}$ можно дойти до любой вершины $v = b_{j_1} \dots b_{j_m}$: последовательно «вдвигаем» буквы слова v справа — за m шагов $u \rightarrow a_{i_2} \dots a_{i_m} b_{j_1} \rightarrow \dots \rightarrow b_{j_1} \dots b_{j_m} = v$. Граф даже сильно связан.

Оба условия критерия выполнены — эйлеров цикл существует. (Именно он даёт *последовательность де Брёйна* — кратчайшую циклическую строку, содержащую все слова длины $m + 1$.) ■

Теорема 11. Теорема Кэли (о количестве помеченных деревьев) (см. стр.)

Количество различных помеченных деревьев на n вершинах равно n^{n-2} .

Доказательство. Построим биекцию между помеченными деревьями на n вершинах и последовательностями длины $n - 2$ из элементов V — это в точности код Прюфера.

Код корректно определён. Прямой алгоритм (лекция 6) на каждом шаге удаляет минимальный лист (у дерева с ≥ 2 вершинами лист всегда есть) и делает ровно $n - 2$ записи — любому дереву отвечает ровно одна последовательность $\text{Pruf}(G) \in V^{n-2}$.

Инъективность и сюръективность. Обратный алгоритм (выше) по *любой* последовательности $p \in V^{n-2}$ строит граф: на каждом шаге к минимальной вершине, не встречающейся

в остатке кода и ещё не использованной, проводится ребро к первому элементу кода. Получается ровно $n - 1$ ребро, и построенный граф связан и ацикличесен (каждый шаг присоединяет «новый лист» к остальной части), т. е. дерево. Прямой проверкой видно, что обратный алгоритм восстанавливает исходное дерево по его коду, а прямой алгоритм, применённый к дереву, построенному по последовательности p , возвращает p . Значит, соответствие — биекция.

Последовательностей длины $n - 2$ из n символов ровно n^{n-2} , следовательно, столько же и помеченных деревьев. ■

Теорема 12. Критерий двудольности графа (см. стр.)

G — двудольный $\Leftrightarrow G$ не содержит циклов нечётной длины.

Доказательство. ($\Rightarrow$) Если граф двудольен, нечётных циклов нет. Пусть $V = V_1 \sqcup V_2$ — доли, и все рёбра идут между ними. Любой цикл $v_0 v_1 \dots v_k v_0$ при движении по рёбрам поочерёдно меняет долю: $v_0 \in V_1 \Rightarrow v_1 \in V_2 \Rightarrow v_2 \in V_1$ и так далее. Чтобы вернуться в исходную долю (в вершину v_0), число шагов должно быть чётным, поэтому длина любого цикла чётна — нечётных циклов нет.

($\Leftarrow$) Если нечётных циклов нет, граф двудольен. Достаточно доказать утверждение для каждой связной компоненты по отдельности. Зафиксируем в компоненте вершину s и для каждой вершины v рассмотрим расстояние $d(s, v)$ (длину кратчайшего пути, BFS-нумерация). Положим

$$V_1 = \{v: d(s, v) \text{ чётно}\}, \quad V_2 = \{v: d(s, v) \text{ нечётно}\}.$$

Покажем, что любое ребро uv соединяет вершины из разных долей. Предположим противное: пусть u, v лежат в одной доле, то есть $d(s, u)$ и $d(s, v)$ одной чётности. Рассмотрим замкнутый маршрут: кратчайший путь $s \rightarrow u$, ребро uv и кратчайший путь $v \rightarrow s$. Его длина равна $d(s, u) + 1 + d(s, v)$ — сумма двух чисел одной чётности плюс единица, то есть *нечётна*. Замкнутый маршрут нечётной длины обязательно содержит простой цикл нечётной длины, что противоречит условию. Значит концы каждого ребра попадают в разные доли, и разбиение $V = V_1 \sqcup V_2$ задаёт двудольность.

Иллюстрация. Цикл чётной длины (C_8 слева) раскрашивается чередованием $V_1, V_2, V_1, V_2, \dots$ — двудольен. В цикле нечётной длины (C_7 справа) при таком чередовании последняя вершина оказывается смежной с первой вершиной той же доли (красное ребро) — разбить на две доли невозможно.

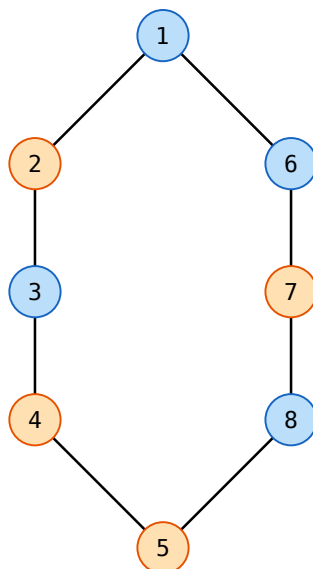


Рис. 69. Критерий: чётный цикл C_8 (двудольный)

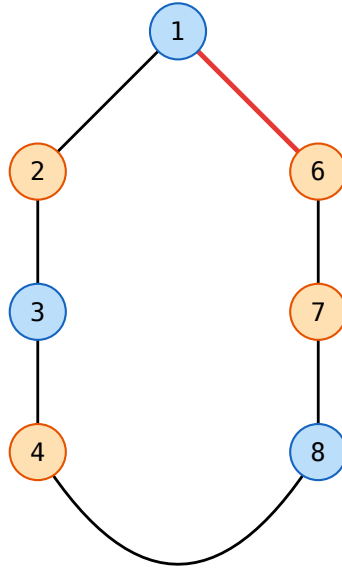


Рис. 70. Критерий: нечётный цикл C_7 (не двудольный)

Лемма 13. О клике (см. стр. 91)

Если K_n — наибольшая клика в G , то $\chi(G) \geq n$.

Доказательство. Любая правильная раскраска G является правильной и на подграфе K_n . Но в K_n все n вершин попарно смежны, поэтому уже им требуется n различных цветов: $\chi(G) \geq \chi(K_n) = n$. ■

Теорема 14. Теорема о редукции (см. стр.)

Пусть $G(V, E)$, и $u, v \in V$ — не смежные вершины. Тогда:

$$P_G(t) = P_{G \vee uv}(t) + P_{G * uv}(t),$$

где $G \vee uv$ — граф с добавленным ребром uv (приближаем к полному), а $G * uv$ — граф со стянутыми вершинами u и v .

Доказательство. Разобьём все правильные раскраски G в t цветов на два класса по тому, как покрашены u и v (они не смежны, поэтому возможно и то, и другое):

- **цвета u и v различны** — такие раскраски взаимно однозначно соответствуют правильным раскраскам графа $G \vee uv$: добавленное ребро uv как раз и требует различия цветов, остальные ограничения совпадают;
- **цвета u и v совпадают** — такие раскраски взаимно однозначно соответствуют правильным раскраскам графа $G * uv$: склеенная вершина получает этот общий цвет, и её смежности — объединение смежностей u и v .

Классы не пересекаются и исчерпывают все раскраски, поэтому $P_G = P_{G \vee uv} + P_{G * uv}$. ■

Лемма 15. Лемма 1 (висячая вершина) (см. стр. 95)

Если в графе G есть висячая вершина u , соединённая с подграфом G_1 , то:

$$P_G(t) = P_{G_1}(t) \cdot (t - 1).$$

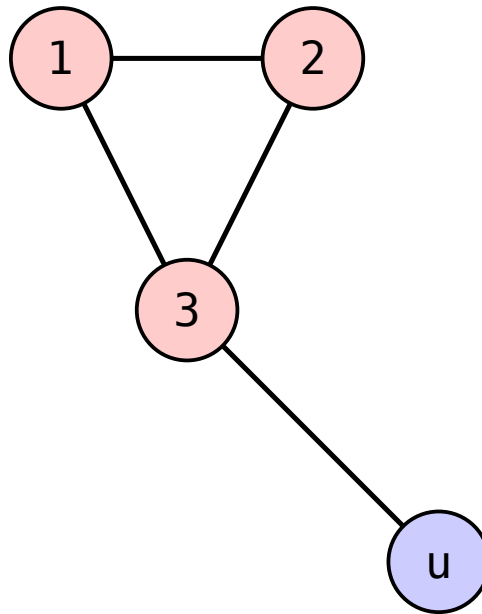


Рис. 71. Лемма 1: висячая вершина $\rightarrow P_G = P_{G_1} \cdot (t-1)$

Доказательство. Сначала правильно красим G_1 — $P_{G_1}(t)$ способами. У вершины u ровно один сосед, и какой бы цвет он ни получил, для u остаётся $t - 1$ допустимых цветов; выбор независим от остального. По правилу произведения $P_G(t) = P_{G_1}(t)(t - 1)$. ■

Лемма 16. Лемма 3 (вершина при ребре) (см. стр. 96)

Если вершина u присоединена к G_1 рёбрами к *обоим концам* некоторого ребра $vw \in G_1$ (т.е. через клику K_2), то:

$$P_G(t) = P_{G_1}(t) \cdot (t - 2).$$

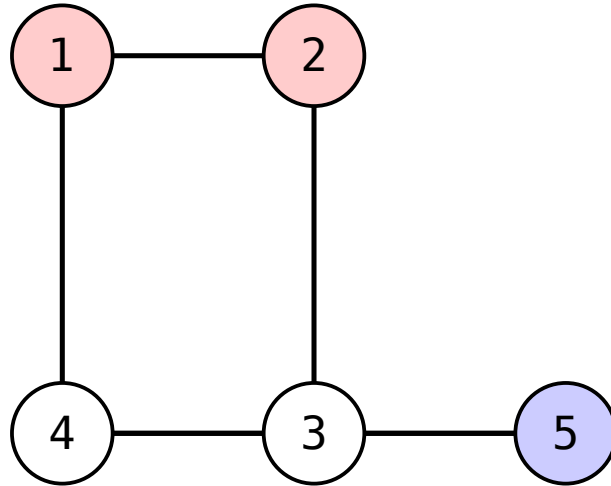


Рис. 72. Лемма 3: вершина при ребре $\rightarrow P_G = P_{\{G_1\}} \cdot (t-2)$

Доказательство. В любой правильной раскраске G_1 концы ребра vw покрашены в *два различных* цвета. Вершине u запрещены ровно эти два цвета, остаётся $t - 2$ вариантов — независимо от того, какая именно раскраска G_1 выбрана. ■

Лемма 17. Лемма 4 (вершина при K_n) (см. стр. 97)

Если вершина u с $\deg u = n$ присоединена к подграфу G_1 через K_n (все n соседей u попарно смежны в G_1), то:

$$P_G(t) = P_{G_1}(t) \cdot (t - n).$$

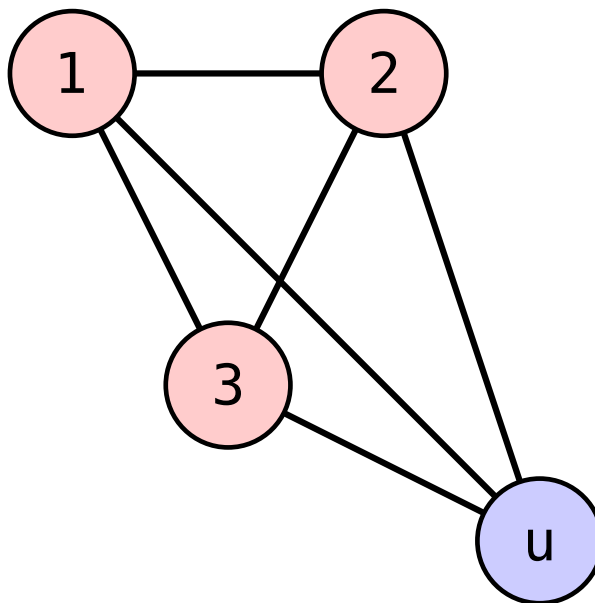


Рис. 73. Лемма 4: вершина при $K_n \rightarrow P_G = P_{\{G_1\}} \cdot (t-n)$

Доказательство. Соседи u образуют клику, поэтому в любой правильной раскраске G_1 они покрашены в n попарно различных цветов. Для u запрещены ровно n цветов — остаётся $t - n$ вариантов. Леммы 1 и 3 — частные случаи при $n = 1$ и $n = 2$. ■

Лемма 18. Лемма 5 (комбинация подвесок) (см. стр. 98)

Если к G_1 присоединены висячая вершина и вершина при ребре (как на рисунке), множители перемножаются:

$$P_G(t) = P_{G_1}(t) \cdot (t - 1)(t - 2).$$

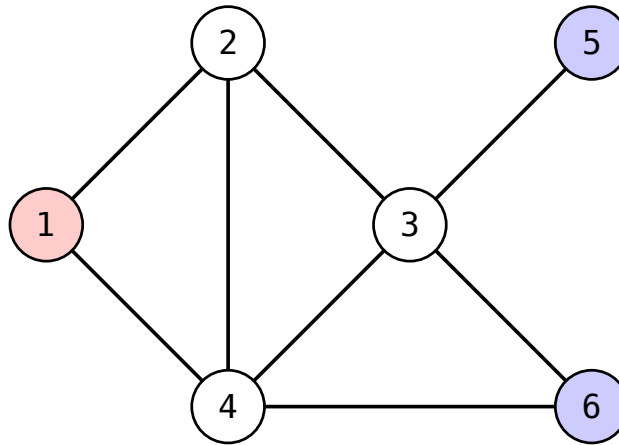


Рис. 74. Лемма 5: висячая вершина + вершина при ребре

Доказательство. Применяем леммы по очереди: сначала к $G' = G$ без висячей вершины 5 — по лемме 3 (вершина 6 при ребре 3–4) имеем $P_{G'} = P_{G_1}(t - 2)$; затем возвращаем висячую вершину 5 — по лемме 1 получаем $P_G = P_{G'}(t - 1) = P_{G_1}(t - 1)(t - 2)$. Порядок применения не важен: каждая подвеска даёт свой множитель независимо. ■

Лемма 19. Лемма 6 (общая, разделяющая K_n) (см. стр. 98)

Если граф G разделяется кликой K_n на подграфы G_1 и G_2 (т. е. $G = G_1 \cup G_2$, $G_1 \cap G_2 = K_n$), то:

$$P_G(t) = \frac{P_{G_1}(t) \cdot P_{G_2}(t)}{P_{K_n}(t)}.$$

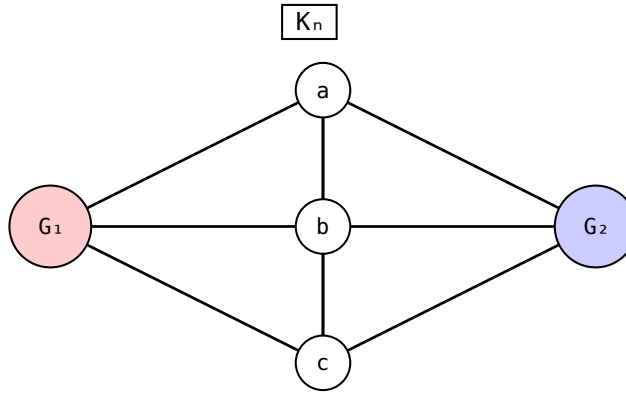


Рис. 75. Лемма 6: общая, $K_n \rightarrow P_G = P_{\{G_1\}} \cdot P_{\{G_2\}} / P_{\{K_n\}}$

Доказательство. Ключевое наблюдение: число продолжений фиксированной правильной раскраски клики K_n до правильной раскраски всего G_1 не зависит от того, какая именно раскраска клики выбрана. Действительно, любые две правильные раскраски K_n (n попарно различных цветов) переводятся друг в друга перестановкой цветов, а перестановка цветов — биекция на множестве продолжений. Поэтому продолжений ровно $P_{G_1}(t)/P_{K_n}(t)$ (всего раскрасок G_1 , делённое на число раскрасок клики); аналогично для G_2 .

Раскраска G — это раскраска клики ($P_{K_n}(t)$ способов) плюс независимые продолжения на G_1 и на G_2 (общих вершин вне клики нет, рёбер между $G_1 \setminus K_n$ и $G_2 \setminus K_n$ нет):

$$P_G = P_{K_n} \cdot \frac{P_{G_1}}{P_{K_n}} \cdot \frac{P_{G_2}}{P_{K_n}} = \frac{P_{G_1} \cdot P_{G_2}}{P_{K_n}}.$$

Леммы 1–5 — частные случаи (например, лемма 4: $G_2 = K_{n+1}$, $P_{G_2}/P_{K_n} = t - n$). ■

Теорема 20. Жадная оценка (см. стр.)

$$\chi(G) \leq \Delta(G) + 1.$$

Доказательство. Красим вершины в произвольном порядке, выделив $\Delta + 1$ цветов. Когда очередь доходит до вершины v , покрашено не более $\deg(v) \leq \Delta$ её соседей — они запрещают не более Δ цветов, и хотя бы один из $\Delta + 1$ цветов свободен. Оценка точна: для K_n и для нечётных циклов достигается равенство (теорема Брукса утверждает, что во всех остальных связных графах $\chi \leq \Delta$). ■

Теорема 21. Теорема Визинга (о рёберной раскраске) (см. стр.)

$$\Delta(G) \leq \chi'(G) \leq \Delta(G) + 1.$$

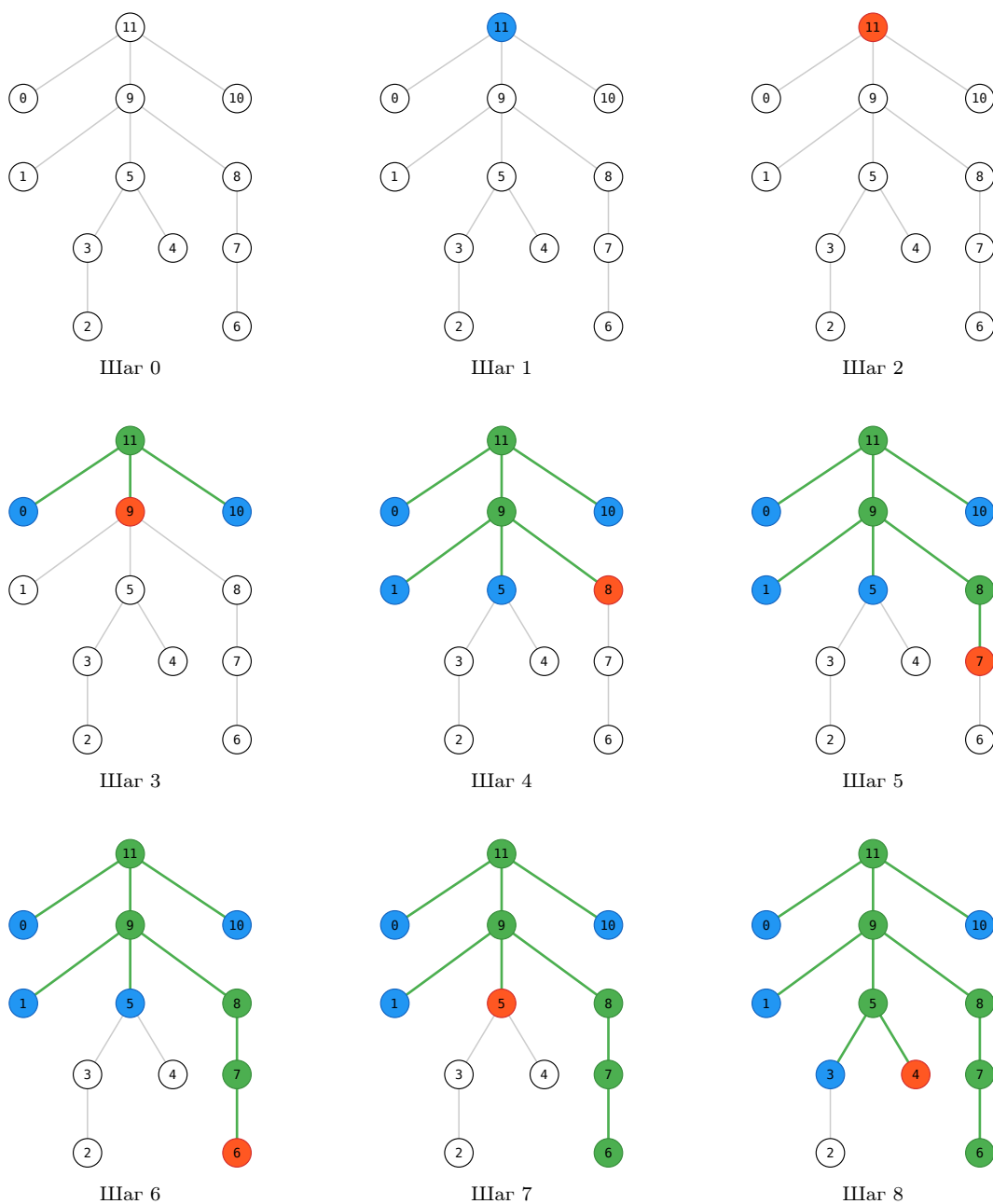
Все алгоритмы

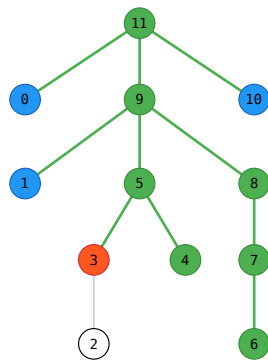
Алгоритм 1. Поиск в глубину (DFS) и поиск в ширину (BFS) (см. стр. 7)

Поиск в глубину (DFS) использует стек: из текущей вершины уходим как можно глубже, и только упёршись в тупик, возвращаемся назад. **Поиск в ширину (BFS)** использует очередь: сначала обрабатываются все соседи текущей вершины, затем соседи соседей — обход идёт «слоями».

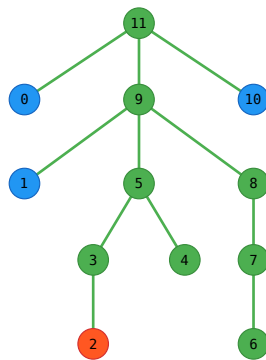
Цветовые обозначения на рисунках: текущая (активная) вершина — оранжевая, уже в стеке/очереди — синяя, обработанная — зелёная; рёбра дерева обхода выделены зелёным.

Пример работы DFS (старт из S):

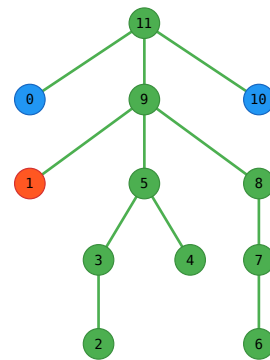




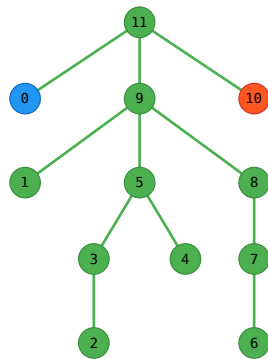
Шаг 9



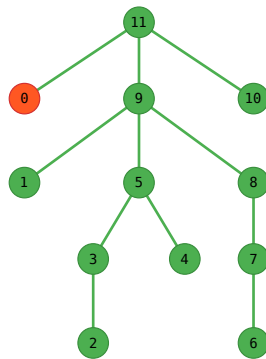
Шаг 10



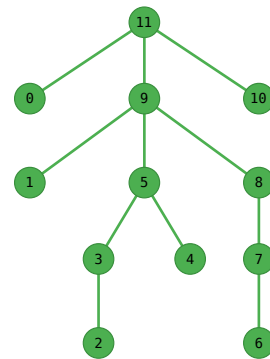
Шаг 11



Шаг 12

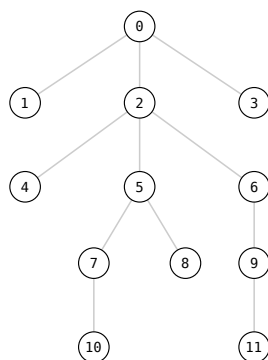


Шаг 13

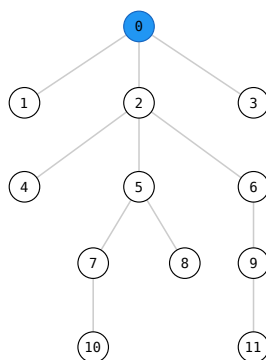


Шаг 14

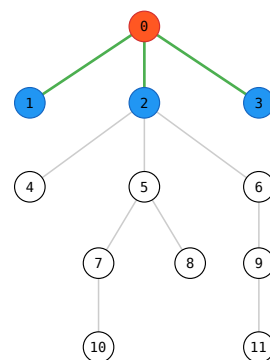
Пример работы BFS (старт из S):



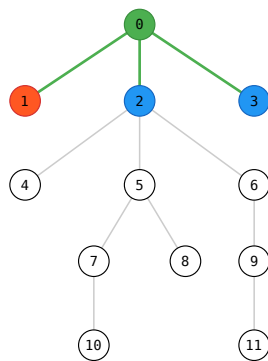
Шаг 0



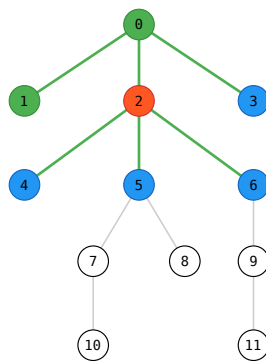
Шаг 1



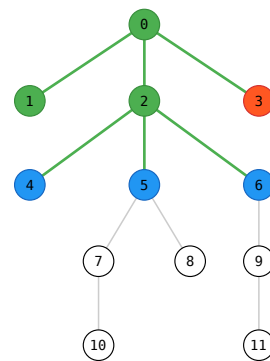
Шаг 2



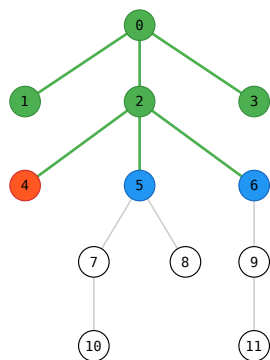
Шаг 3



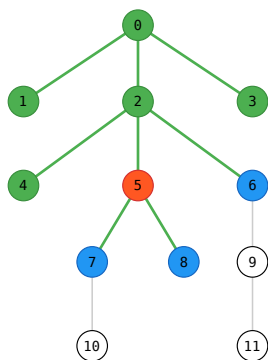
Шаг 4



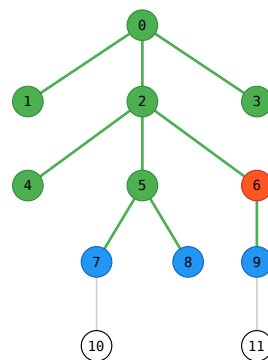
Шаг 5



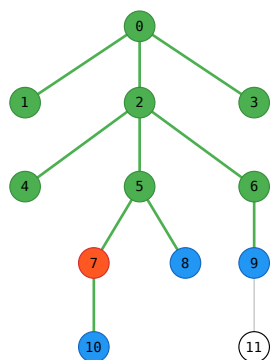
Шаг 6



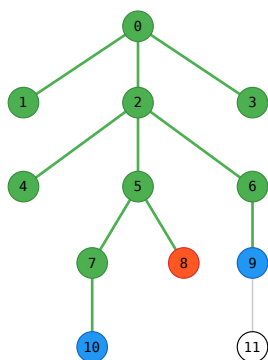
Шаг 7



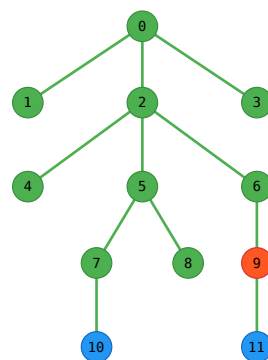
Шаг 8



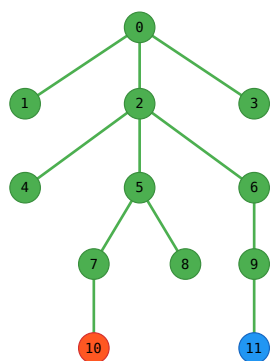
Шаг 9



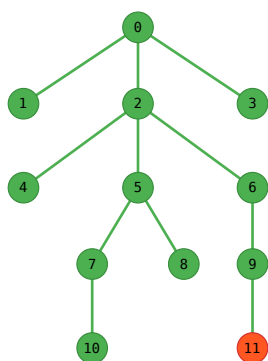
Шаг 10



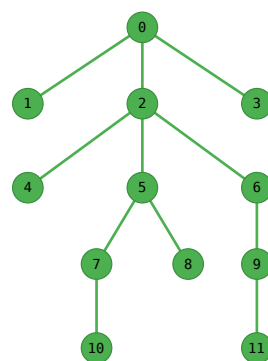
Шаг 11



Шаг 12



Шаг 13



Шаг 14

Алгоритм 2. Топологическая сортировка (Topsort) (см. стр. 9)

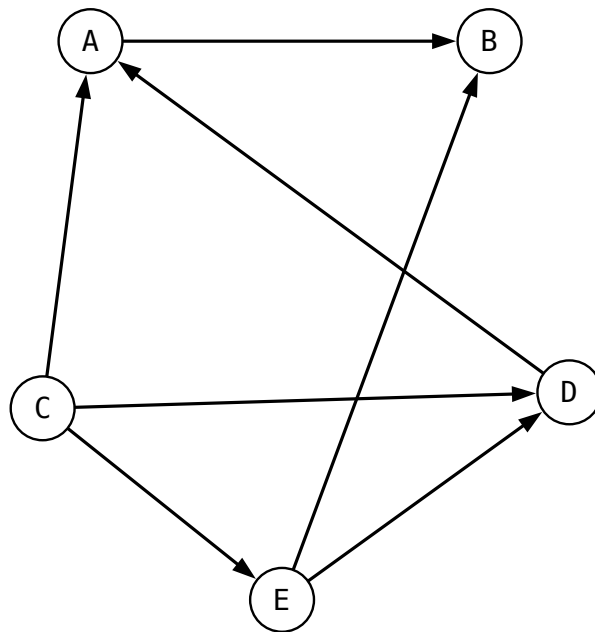
Дано: ориентированный граф без циклов, задающий частичный порядок. *Построить:* линейный порядок, согласованный с исходным.

Шаг I. Серия поисков в глубину (DFS). Когда вершина становится «тупиковой» (все её исходящие рёбра уже обработаны), она кладётся в стек.

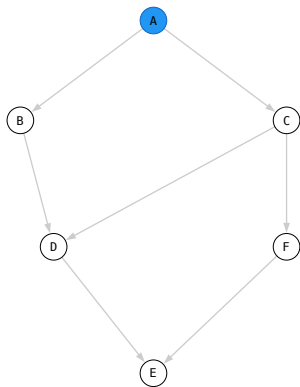
Шаг II. Нумеруем вершины по стеку *снизу вверх*: самая нижняя вершина стека получает номер 1, верхняя — номер n . После этого дорисовываем рефлексивные петли и недостающие рёбра: из каждой вершины с *большим* номером — во все вершины с *меньшим* номером.

Результат — граф, задающий отношение **линейного порядка**.

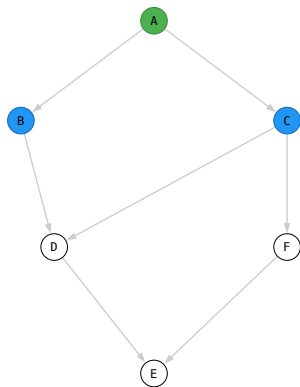
Работа алгоритма по шагам. Исходный граф:



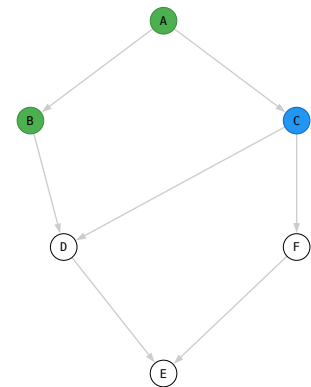
Активная ветка DFS — синяя, текущая вершина — оранжевая, тупиковые (уложенные в стек) — зелёные, рёбра дерева обхода — зелёные. Стек показан справа (верх стека — сверху).



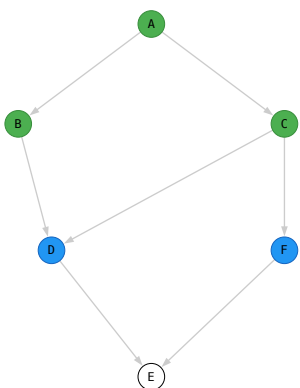
Шаг 0. Исходный граф, стек пуст



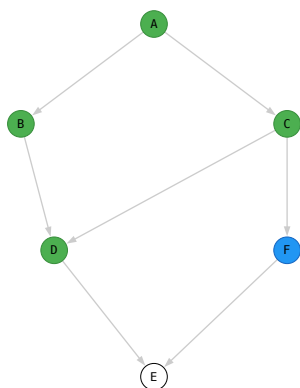
Шаг 1. DFS из A: идём $A \rightarrow B$



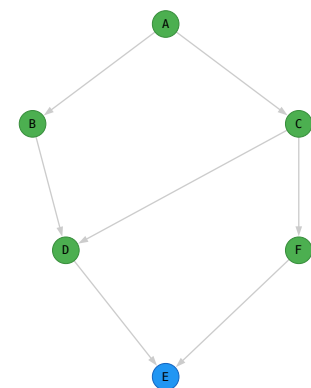
Шаг 2. B тупиковая $\rightarrow$ в стек



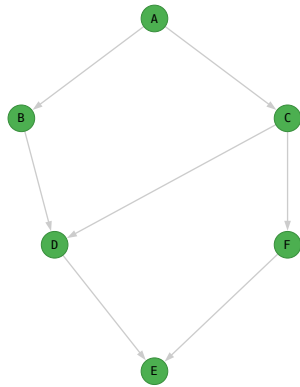
Шаг 3. A тупиковая $\rightarrow$ в стек



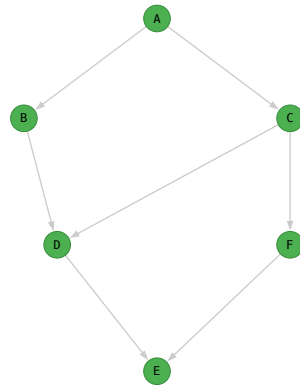
Шаг 4. DFS из C: $C \rightarrow D$; D $\rightarrow$ в стек



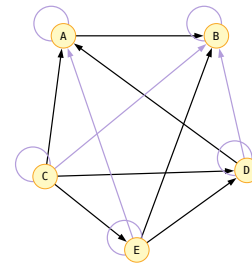
Шаг 5. $C \rightarrow E$; E $\rightarrow$ в стек



Шаг 6. C тупиковая; шаг I завершён



Шаг 7. Нумерация по стеку + петли



Стек:
$C = 5$
$E = 4$
$D = 3$
$A = 2$
$B = 1$

Шаг 8. Недостающие рёбра

Комментарии к шагам. На шаге 1 запускаем DFS из A : единственное исходящее ребро ведёт в B . У B исходящих рёбер нет — она тупиковая, кладём её в стек (шаг 2); возвращаемся в A — все её рёбра обработаны, A тоже уходит в стек (шаг 3). Запускаем DFS из следующей непосещённой вершины C : ребро $C \rightarrow D$ приводит в D , откуда идёт только ребро $D \rightarrow A$ в уже посещённую вершину, поэтому D тупиковая (шаг 4). Далее $C \rightarrow E$: оба ребра из E ($E \rightarrow D$, $E \rightarrow B$) ведут в посещённые вершины — E в стек (шаг 5). Наконец тупиковой становится и C (шаг 6).

На шаге 7 нумеруем вершины по стеку снизу вверх: $B = 1$, $A = 2$, $D = 3$, $E = 4$, $C = 5$ — и дорисовываем рефлексивные петли (светло-фиолетовые). На шаге 8 добавляем недостающие рёбра из вершин с большими номерами в вершины с меньшими (тоже светло-фиолетовые: $D \rightarrow B$, $E \rightarrow A$, $C \rightarrow B$ и т.д.).

Стек (низ $\rightarrow$ верх): B, A, D, E, C .

Нумерация вершин: $B = 1$, $A = 2$, $D = 3$, $E = 4$, $C = 5$.

Линейный порядок: $B \leq A \leq D \leq E \leq C$.

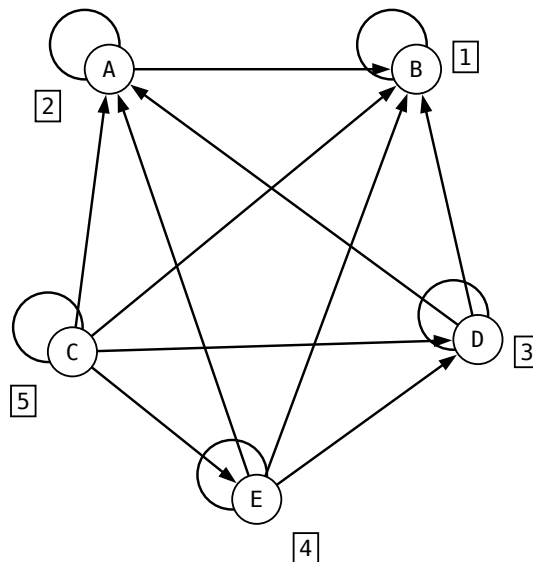


Рис. 76. Результат Topsort — линейный порядок $B \leq A \leq D \leq E \leq C$

Алгоритм 3. Алгоритм Флойда–Уоршелла (построение транзитивного замыкания) (см. стр. 12)

Вход: матрица смежности A_f б. о. f (размер $n \times n$).

Выход: матрица транзитивного замыкания $\tilde{A}$.

Шаг 1: $\forall v \in V$: рассматриваем v как «промежуточную» вершину.

Шаг 2: Для каждой пары (i, j) : если $a_{iv} = 1$ и $a_{vj} = 1$, то полагаем $a_{ij} := 1$.
Повторяем для всех вершин v .
После дорисовывания графа получаем транзитивное замыкание.

Алгоритм 4. Алгоритм поиска мостов (см. стр. 25)

Вход: $G(V, E)$ — связный, неориентированный.

Выход: полный список мостов.

Шаг 1. Серия DFS:

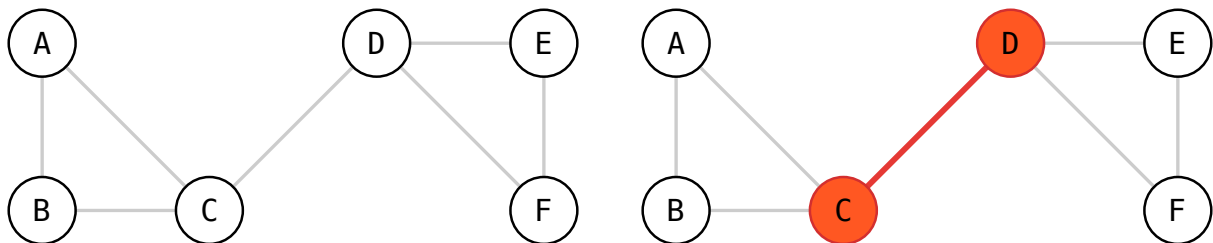
- а) при первом посещении вершины — добавить её в список;
- б) при первом посещении ребра — ориентировать его в направлении обхода.

Шаг 2. Инвертируем все рёбра: $\bar{E}$ — меняем направление каждого ребра на противоположное.

Шаг 3. Серия DFS по списку из шага 1 на графе $G(V, \bar{E})$; каждая серия (компонента) красится своим цветом.

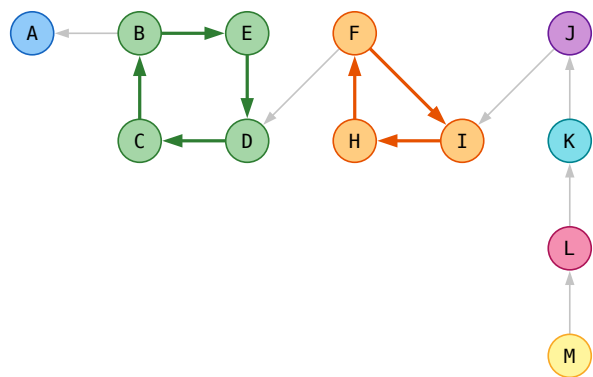
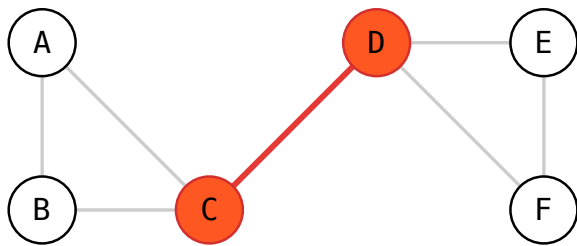
Результат: мосты — это в точности рёбра, концы которых оказались *разного цвета* (в разных компонентах обратного обхода).

Работа алгоритма по шагам (граф из 12 вершин):



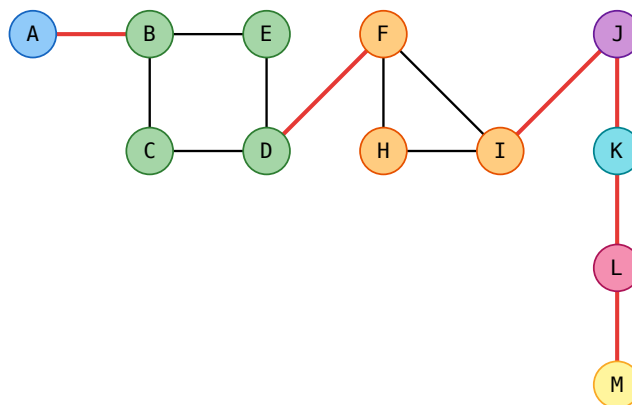
Шаг 0. Исходный граф

Шаг 1. DFS из A: ориентируем рёбра по обходу



Шаг 2. Инвертируем все рёбра

Шаг 3. Серия DFS: компоненты — цветами



Шаг 4. Мосты — рёбра между разными цветами (красные)

Комментарии к шагам. На шаге 1 поиск в глубину из A ориентирует рёбра в порядке обхода: $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$, ребро $E-B$ впервые посещается из E (ориентируем $E \rightarrow B$); далее $D \rightarrow F \rightarrow H \rightarrow I \rightarrow J \rightarrow K \rightarrow L \rightarrow M$ и ребро $I \rightarrow F$. На шаге 3 серия DFS на инвертированном графе (в порядке списка $A, B, C, \dots$) даёт компоненты $\{A\}$, $\{B, C, D, E\}$, $\{F, H, I\}$, $\{J\}$, $\{K\}$, $\{L\}$, $\{M\}$: из A по инвертированным рёбрам никуда не попасть; из B обходится цикл $B \rightarrow E \rightarrow D \rightarrow C \rightarrow B$; из F — цикл $F \rightarrow I \rightarrow H \rightarrow F$; вершины J, K, L, M остаются поодиночке.

Мосты данного графа (рёбра между разными компонентами): $A-B$, $D-F$, $I-J$, $J-K$, $K-L$, $L-M$.

Алгоритм 5. Алгоритм Косарайю (построение графа Герца) (см. стр. 29)

Алгоритм для выделения компонент сильной связности.

Шаг I. Серия DFS: «тупиковая» вершина (все исходящие рёбра которой уже обработаны) кладётся в стек. Стек заполняется снизу вверх по порядку завершения обработки вершин.

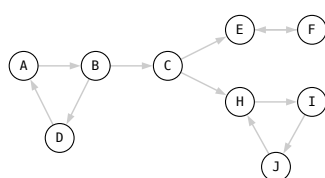
Зачем это нужно: если из компоненты X есть путь в компоненту Y (X «выше по течению»), то хотя бы одна вершина X окажется в стеке *выше* всех вершин Y . Поэтому на шаге III обход стартует из компонент, «выше по течению», и не «перетекает» в чужие компоненты.

Шаг II. $\text{inv } E$ — инвертируем все рёбра.

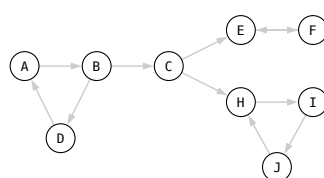
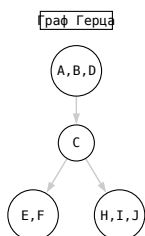
Шаг III. Серия DFS, согласованная со стеком: достаём вершины с верха стека; из каждой ещё не помеченной вершины запускаем DFS по *инвертированным* рёбрам — все достижимые непомеченные вершины образуют одну компоненту сильной связности (красим своим цветом).

Результат: компоненты сильной связности; стянув каждую в одну вершину, получаем граф Герца.

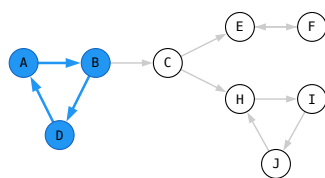
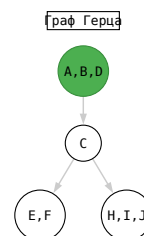
Работа алгоритма по шагам (граф из примера выше; стек показан справа, верх стека — сверху):



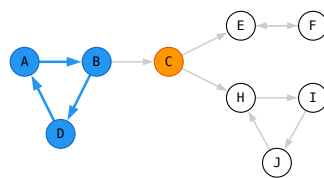
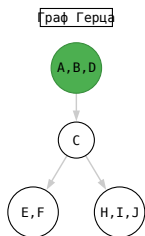
Шаг 0. Исходный граф



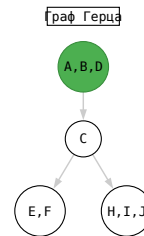
Шаг 1. Серия DFS из A : стек по временам выхода

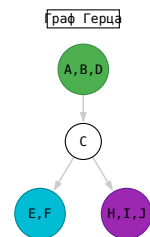
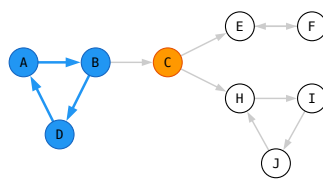
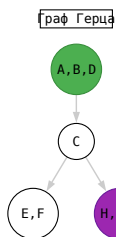
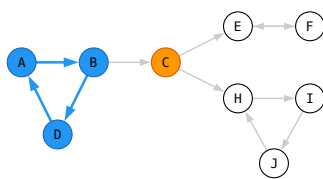


Шаг 2. Инвертируем все рёбра



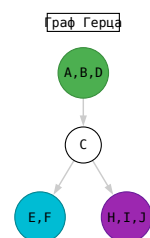
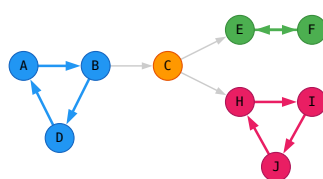
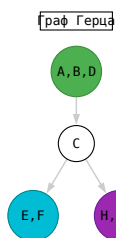
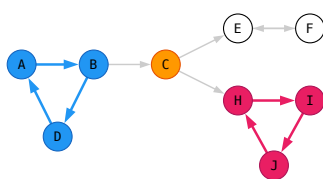
Шаг 3. Сверху стека A : DFS даёт $\{A, D, B\}$





Шаг 4. Следующая непомеченная C : $\{C\}$

Шаг 5. Из H : компонента $\{H, J, I\}$



Шаг 6. Из E : компонента $\{E, F\}$

Шаг 7. Итог: 4 компоненты сильной связности

Комментарии к шагам. На шаге 1 поиск в глубину из A обходит граф: $A \rightarrow B \rightarrow D$ (ребро $D \rightarrow A$ ведёт в посещённую вершину, D — тупиковая, кладём в стек); далее $B \rightarrow C \rightarrow E \rightarrow F$ (ребро $F \rightarrow E$ — в посещённую, F в стек, затем E); $C \rightarrow H \rightarrow I \rightarrow J$ ($J \rightarrow H$ — в посещённую, в стек уходят J, I, H); наконец в стек уходят C, B, A . Стек снизу вверх: $D, F, E, J, I, H, C, B, A$.

На шаге III достаём вершины сверху: из A по инвертированным рёбрам достижимы D и B — компонента $\{A, B, D\}$ (шаг 3); B уже помечена, следующая C — из неё по инвертированным рёбрам можно попасть только в помеченную B , компонента $\{C\}$ (шаг 4); из H достижимы J и I — $\{H, I, J\}$ (шаг 5); из E достижима F — $\{E, F\}$ (шаг 6). Все вершины помечены — алгоритм завершён (шаг 7).

Алгоритм 6. Алгоритм Флэри (см. стр. 36)

Вход: G — Э или ПЭ.

Выход: эйлеров цикл / эйлеров путь.

P.S. Граф не может быть Э и ПЭ одновременно.

Инициализация:

1. $\exists \Rightarrow \text{start} \in V$ — любая вершина.
2. $\text{П}\exists \Rightarrow \text{start} \in V: \deg(v) \not\equiv 2.$
3. (ориент.) $\Rightarrow \text{start} \in V: \delta^-(v) = \delta^+(v) + 1.$

Шаг (step): на каждом шаге идём по ребру, которое **не является мостом** в оставшемся графе (если есть выбор). По мосту идём, только когда другого ребра из текущей вершины нет. Пройденное ребро удаляем.

Пример.

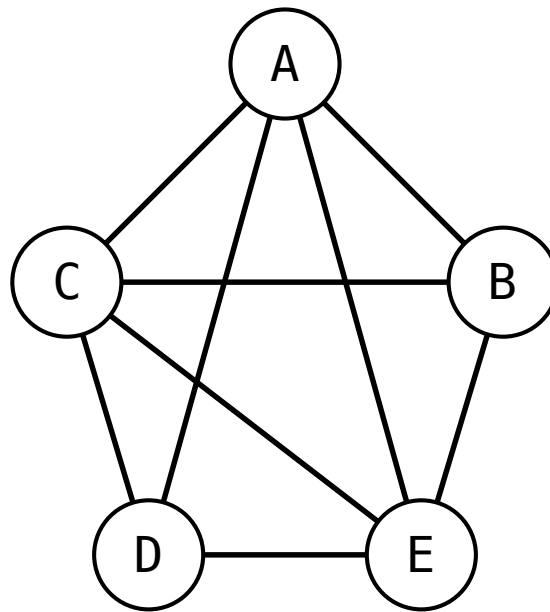


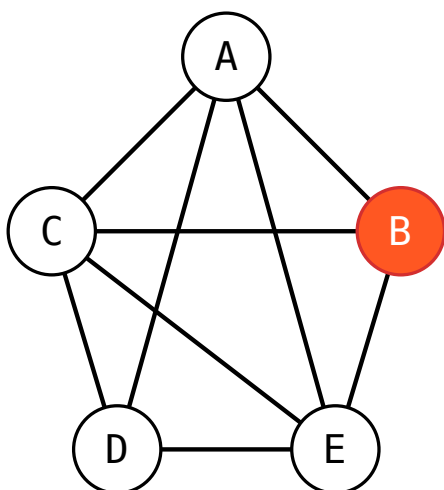
Рис. 77. Алг. Флёри: пример графа

Степени: $\deg A = \deg C = \deg E = 4$, $\deg B = \deg D = 3$ — ровно две нечётные вершины, граф полуэйлеров; стартуем из нечётной вершины B (закончим в D).

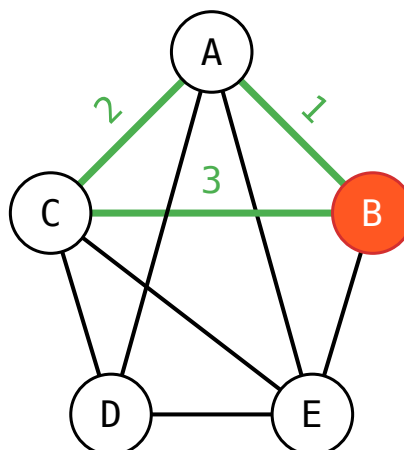
№	вершина	ребро	комментарий (мосты в оставшемся графе)
1	B	$B-A$	мостов нет, берём любое ребро
2	A	$A-C$	мостов нет
3	C	$C-B$	мостов нет
4	B	$B-E$	единственное оставшееся ребро из B
5	E	$E-C$	$E-C$ не мост (цикл $C-D-E$ цел)
6	C	$C-D$	единственное ребро из C (мост, но выбора нет)
7	D	$D-E$	$D-E$ не мост, а $D-A$ — мост: избегаем
8	E	$E-A$	единственное ребро из E
9	A	$A-D$	последнее ребро; финиш в D

Эйлеров путь: $B, A, C, B, E, C, D, E, A, D$ — все 9 рёбер ровно по одному разу, от одной нечётной вершины (B) до другой (D).

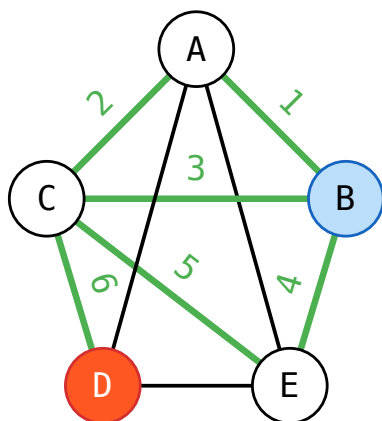
Работа алгоритма по шагам (номера на рёбрах — порядок прохождения; старт B — синий, текущая вершина — оранжевая):



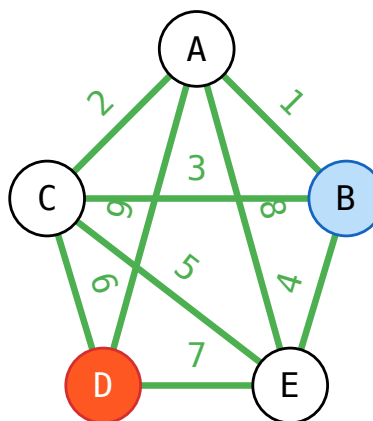
Шаг 0. Старт в нечётной вершине B



Шаги 1–3: $B-A-C-B$



Шаги 4–6: $B-E-C-D$



Шаги 7–9: $D-E-A-D$, финиш в D

Алгоритм 7. Алгоритм поиска эйлерова цикла (на списках инцидентности) (см. стр. 38)

Идея. Граф задан списками инцидентности (для каждой вершины — список исходящих рёбер). Используются стек `temp` и список `result`:

1. кладём стартовую вершину в стек;
2. пока стек не пуст, смотрим на вершину v с верха стека:
 - если в списке v осталось неиспользованное ребро $v \rightarrow u$ — вычёркиваем его из списка и кладём u в стек;
 - если список v исчерпан — снимаем v со стека и дописываем её в `result`;
3. в конце `result`, прочитанный в обратном порядке, — эйлеров цикл (путь).

Пример.

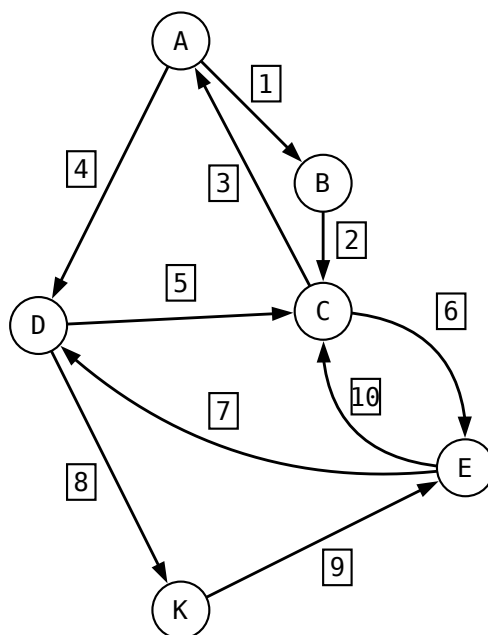


Рис. 78. Алг. на списках инцидентности

Полустепени: у вершины A выходов на один больше, чем входов ($\delta^-(A) = 2$, $\delta^+(A) = 1$), у C — наоборот ($\delta^-(C) = 2$, $\delta^+(C) = 3$), у остальных входы равны выходам — по критерию граф полуэйлеров; эйлеров путь идёт из A в C .

Списки инцидентности (исходящие рёбра):

A : B, D
 B : C
 C : A, E
 D : C, K
 E : D, C
 K : E

Работа алгоритма: сначала стек только растёт — $A \rightarrow B \rightarrow C \rightarrow A \rightarrow D \rightarrow C \rightarrow E \rightarrow D \rightarrow K \rightarrow E \rightarrow C$ (каждый раз вычёркиваем первое неиспользованное ребро из списка текущей вершины). К этому моменту все 10 рёбер израсходованы, и стек начинает разбираться: вершины по очереди исчерпаны и уходят в **result**.

Состояния (**temp** — стек после фазы роста, читается снизу вверх; **result** — порядок выгрузки, сверху вниз):

temp (низ → верх)	result
A	C
B	E
C	K
A	D
D	E
C	C
E	D
D	A
K	C
E	B
C	A

Эйлеров путь — это **result**, прочитанный снизу вверх:

$$A, B, C, A, D, C, E, D, K, E, C$$

— все 10 дуг ровно по одному разу, из A в C .

Алгоритм 8. Построение гамильтонова пути в условиях теоремы Дирака (см. стр. 45)

Инициализация: $v_1, v_2, \dots, v_n$ — произвольный порядок *всех* вершин графа.

Шаг (поворот): пока в последовательности есть несмежная соседняя пара (v_i, v_{i+1}) , ищем индекс $j > i + 1$, для которого *оба* ребра существуют:

$$\exists e_1 = v_i v_j \quad \text{и} \quad \exists e_2 = v_{i+1} v_{j+1};$$

(условие Дирака гарантирует, что такой j найдётся). Переворачиваем отрезок $v_{i+1} \dots v_j$:

$$v_1 \dots v_i \mid \underbrace{v_j v_{j-1} \dots v_{i+1}}_{\text{перевёрнут}} \mid v_{j+1} \dots v_n.$$

На стыках теперь стоят пары (v_i, v_j) и (v_{i+1}, v_{j+1}) — обе смежные, а внутри перевёрнутого отрезка смежность пар не изменилась. Число несмежных пар строго уменьшилось; повторяем, пока их не останется — получился гамильтонов путь.

Пример. Возьмём граф с рёбрами $A-C, C-E, E-F, F-D, D-B, B-A, A-D, C-F, B-E$ (к графу с рис. выше по тексту добавлено ребро $B-E$, чтобы выполнялось условие Дирака: теперь все степени равны $3 = \frac{6}{2} = \frac{n}{2}$).

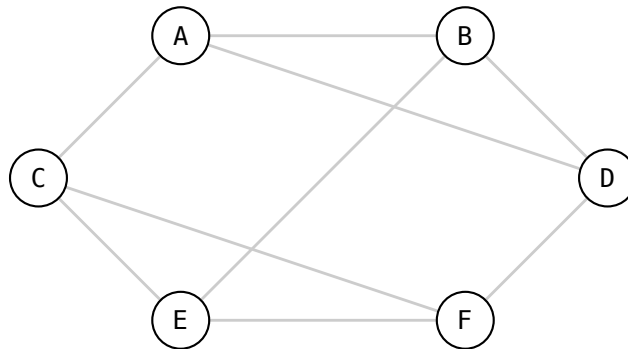


Рис. 79. Алг. Дирака: граф $A, \dots, F$, все степени = 3

Начальный порядок: E, A, B, D, F, C — т.е. $v_1 = E, v_2 = A, v_3 = B, v_4 = D, v_5 = F, v_6 = C$. Порядок *произвольный* (алгоритм стартует с любого); этот выбран для компактности примера — в нём всего одна несмежная соседняя пара. Проверяем пары: (E, A) — *несмежная* (ребра $E-A$ нет); остальные пары $(A, B), (B, D), (D, F), (F, C)$ — смежные.

Чиним пару $(v_i, v_{i+1}) = (v_1, v_2) = (E, A)$. По описанию шага ищем индекс $j > i + 1 = 2$, для которого существуют *оба* ребра $v_i v_j$ и $v_{i+1} v_{j+1}$, т.е. $E \sim v_j$ и $A \sim v_{j+1}$ — перебираем кандидатов $j = 3, 4, 5$ по порядку:

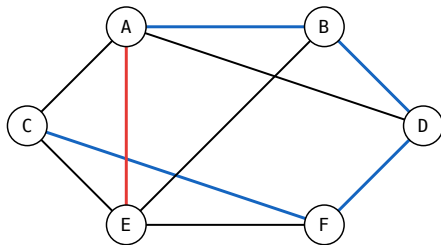
- $j = 3$: $v_3 = B$, $v_4 = D$. Проверяем: $E \sim B$? — да (ребро $B-E$); $A \sim D$? — да (ребро $A-D$). **Оба условия выполнены** — берём $j = 3$ (далее можно не перебирать; для сравнения: $j = 4$ не подошёл бы — уже первое условие $E \sim v_4 = D$ не выполнено, ребра $E-D$ в графе нет).

Переворачиваем отрезок $v_{i+1} \dots v_j = v_2 \dots v_3 = (A, B)$:

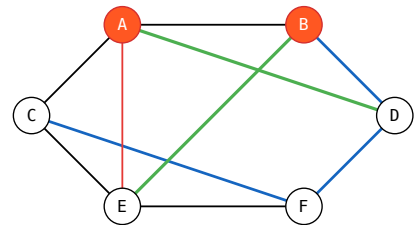
$$E, \underline{A}, \underline{B}, D, F, C \longrightarrow E, \underline{B}, \underline{A}, D, F, C.$$

На стыках перевёрнутого отрезка встали проверенные рёбра: слева — $v_1 v_3 = E-B$, справа — $v_2 v_4 = A-D$. Все соседние пары теперь смежные: $E-B$, $B-A$, $A-D$, $D-F$, $F-C$ — это **гамильтонов путь**. Более того, концы E и C тоже смежны ($C-E$ — ребро), так что путь замыкается в **гамильтонов цикл** E, B, A, D, F, C, E .

Работа алгоритма по шагам (синие рёбра — смежные соседние пары, красное — несмежная пара, зелёные — рёбра поворота и итоговый путь):



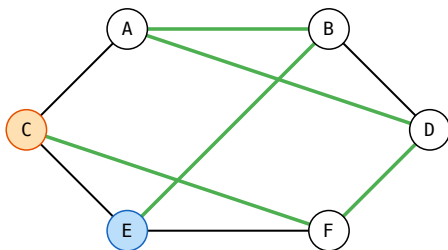
Последовательность: E, A, B, D, F, C
Несмежная пара: (E, A)



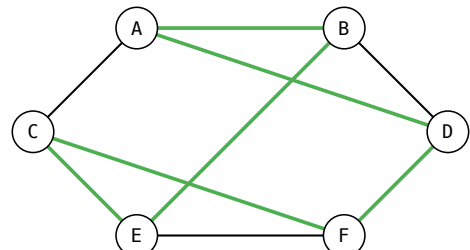
Поворот: $j = 3$, $v_j = B$, $v_{j+1} = D$
 $E-B$ и $A-D$ есть в графе → переворачиваем (A, B)

Шаг 0. Порядок E, A, B, D, F, C ; пара (E, A) несмежная

Шаг 1. Поворот: $j = 3$, рёбра $E-B$ и $A-D$ есть



Гамильтонов путь: E, B, A, D, F, C



C и E смежны → гамильтонов цикл

Шаг 2. Отрезок (A, B) перевёрнут: гамильтонов путь

Шаг 3. $C \sim E$: замыкаем в гамильтонов цикл

Алгоритм 9. Код Прюфера (прямая задача) (см. стр. 49)

Дано: $G(V, E)$ — дерево, $|V| = n$, порядок лексикографический.

Выход: $\text{Pruf}(G) = \{v_{i_1}, \dots, v_{i_{n-2}}\}$.

Шаг (step): удаление лексикографически минимального листа; в код записывается его родитель (единственный сосед). Повторяем, пока не останется 2 вершины: код имеет длину $n - 2$.

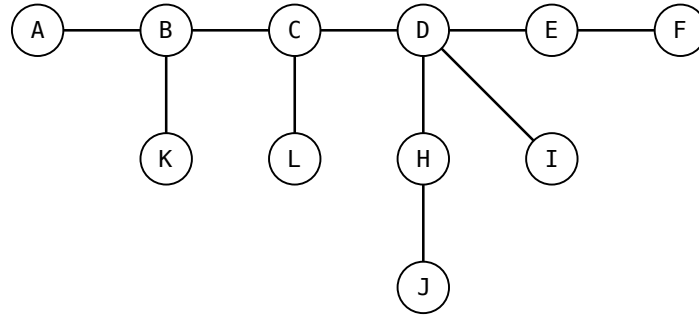


Рис. 80. Код Прюфера: дерево 11 вершин

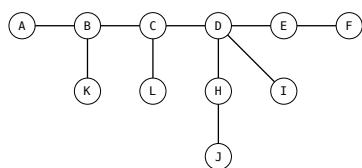
Пример. Дерево (11 вершин): $A-B, B-C, C-D, D-E, E-F, D-H, H-J, D-I, B-K, C-L$. Порядок: $A, B, C, D, E, F, H, I, J, K, L$.

По шагам (лист $\rightarrow$ в код пишем его соседа):

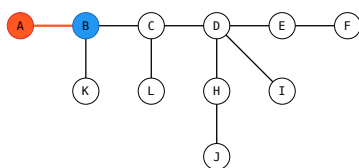
1. листья $\{A, F, I, J, K, L\}$; минимальный A , сосед $B \rightarrow$ код (B) ;
2. минимальный лист F , сосед $E \rightarrow$ код (B, E) ;
3. E стала листом; минимальный лист E , сосед $D \rightarrow (B, E, D)$;
4. минимальный лист I , сосед $D \rightarrow (B, E, D, D)$;
5. минимальный лист J , сосед $H \rightarrow (B, E, D, D, H)$;
6. H стала листом; сосед $D \rightarrow (B, E, D, D, H, D)$;
7. D стала листом; сосед $C \rightarrow (B, E, D, D, H, D, C)$;
8. минимальный лист K , сосед $B \rightarrow (B, E, D, D, H, D, C, B)$;
9. B стала листом; сосед $C \rightarrow (B, E, D, D, H, D, C, B, C)$; остались две вершины C и L — стоп.

$$\text{Pruf}(G) = (B, E, D, D, H, D, C, B, C), \quad |\text{Pruf}(G)| = 11 - 2 = 9.$$

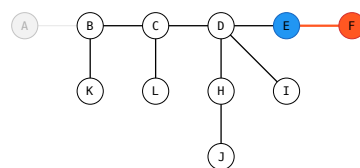
Работа алгоритма по шагам. На каждом шаге удаляется лист с минимальной меткой (оранжевый), в код дописывается его сосед (синий); удалённые вершины и рёбра показаны серым.



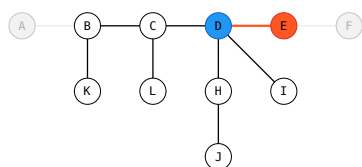
Шаг 0. Исходное дерево



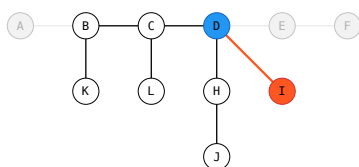
Шаг 1. $A \rightarrow$ код: B



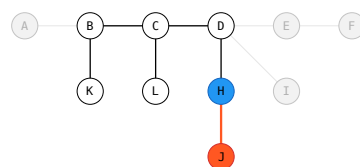
Шаг 2. $F \rightarrow$ код: E



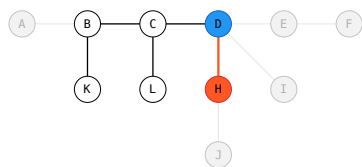
Шаг 3. $E \rightarrow$ код: D



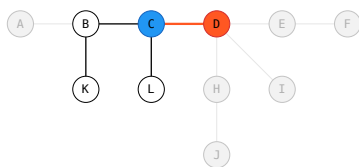
Шаг 4. $I \rightarrow$ код: D



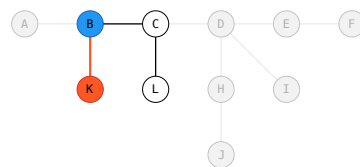
Шаг 5. $J \rightarrow$ код: H



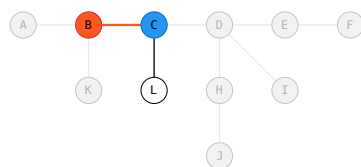
Шаг 6. $H \rightarrow$ код: D



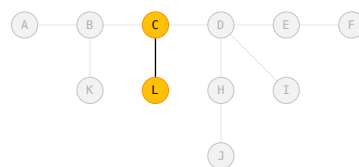
Шаг 7. $D \rightarrow$ код: C



Шаг 8. $K \rightarrow$ код: B



Шаг 9. $B \rightarrow$ код: C



Шаг 10. Остались C и L — стоп

Алгоритм 10. Алгоритм Прима (см. стр. 36)

Инициализация: $\text{start} \in V$ (любая вершина); дерево $T = \{\text{start}\}$.

Step: среди всех рёбер, соединяющих уже выбранную вершину с ещё не выбранной, берём ребро e с минимальным весом $\sigma(e)$; добавляем его (и новую вершину) в T . Новая вершина присоединяется не обязательно к последней — к *любой* из уже выбранных. Повторяем, пока T не охватит все вершины ($n - 1$ ребро).

Пример (старт из J):

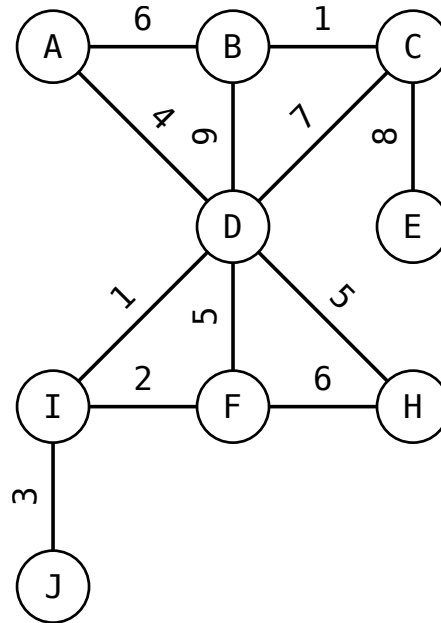
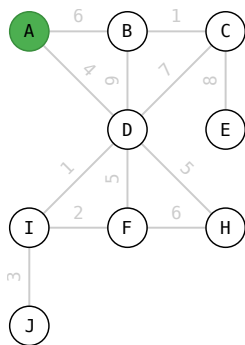


Рис. 81. Алг. Прима: взвешенный граф 9 вершин

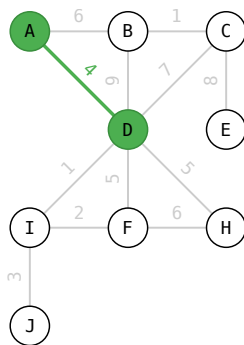
V	e	$\sum \sigma(e)$	комментарий
J	—	0	старт
I	$IJ(3)$	3	единственное ребро из J
D	$DI(1)$	4	$DI(1) < IF(2)$
F	$IF(2)$	6	$IF(2) < AD(4) < DH(5) = DF(5) < \dots$
A	$AD(4)$	10	$AD(4) < DH(5) < FH(6) < \dots$
H	$DH(5)$	15	$DH(5) < AB(6) = FH(6) < DC(7)$
B	$AB(6)$	21	$AB(6) < DC(7) < BD(9)$
C	$BC(1)$	22	после прихода B открылось ребро $BC(1)$
E	$CE(8)$	30	единственное ребро к последней вершине E

Минимальное остовное дерево: $IJ, DI, IF, AD, DH, AB, BC, CE$; суммарный вес $\sum \sigma = 30$.

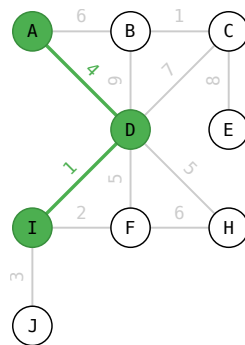
Работа алгоритма по шагам (выбранные рёбра — зелёные, текущее добавляемое — оранжевое, недоступные пока рёбра — серые):



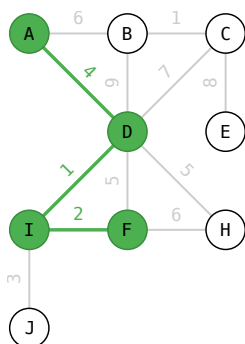
Шаг 0. Старт: $T = \{J\}$



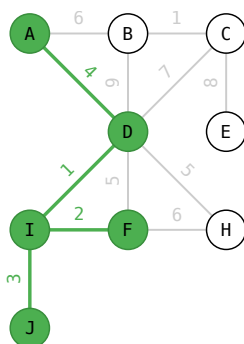
Шаг 1. $+IJ(3)$, $\Sigma = 3$



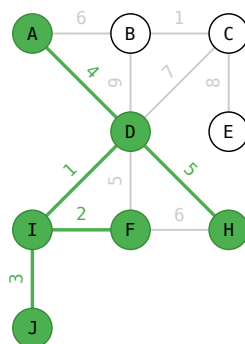
Шаг 2. $+DI(1)$, $\Sigma = 4$



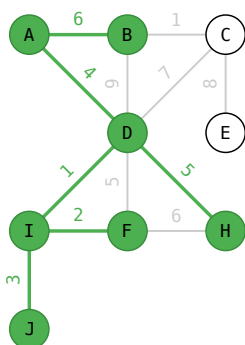
Шаг 3. $+IF(2)$, $\Sigma = 6$



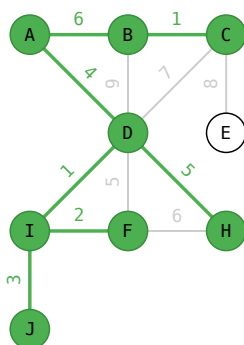
Шаг 4. $+AD(4)$, $\Sigma = 10$



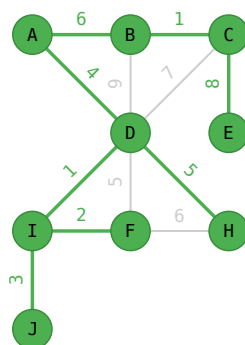
Шаг 5. $+DH(5)$, $\Sigma = 15$



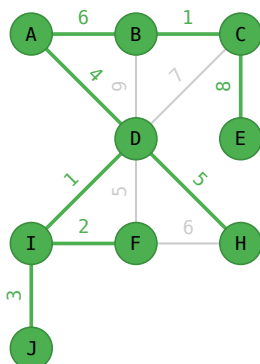
Шаг 6. $+AB(6)$, $\Sigma = 21$



Шаг 7. $+BC(1)$, $\Sigma = 22$



Шаг 8. $+CE(8)$, $\Sigma = 30$



Шаг 9. Остов построен, $\Sigma = 30$

Замечание. Если все веса рёбер различны, минимальное остовное дерево единственно — и алгоритм Прима находит именно его независимо от выбора стартовой вершины.

Алгоритм 11. Алгоритм Краскала (см. стр. 61)

Step: рёбра перебираются в порядке возрастания веса; очередное ребро e с минимальным $\sigma(e)$ добавляется в остов, если при этом *не появляется цикл* (т.е. концы e лежат в разных компонентах уже построенного леса); иначе ребро пропускается.

Пример (граф той же структуры, что у алгоритма Прима, с другими весами):

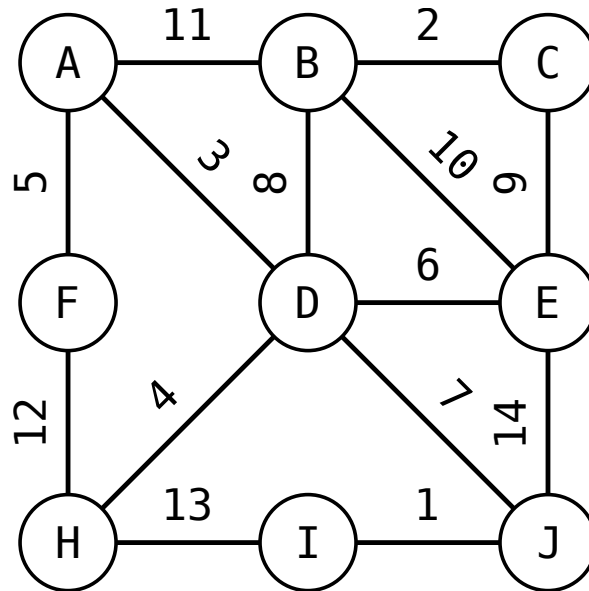


Рис. 82. Алг. Краскала: взвешенный граф 9 вершин

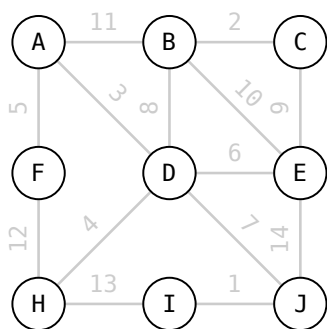
V	e	$\sum \sigma(e)$	комментарий
—	—	0	лес из 9 изолированных вершин
J, I	$JI(1)$	1	соединяем $\{J\}$ и $\{I\}$
B, C	$BC(2)$	3	соединяем $\{B\}$ и $\{C\}$
A, D	$AD(3)$	6	соединяем $\{A\}$ и $\{D\}$
H	$HD(4)$	10	H присоединяется к $\{A, D\}$
F	$AF(5)$	15	F присоединяется к $\{A, D, H\}$
E	$DE(6)$	21	E присоединяется к $\{A, D, H, F\}$
—	$JD(7)$	28	сливаем $\{J, I\}$ и $\{A, D, H, F, E\}$
—	$BD(8)$	36	сливаем $\{B, C\}$ с остальными; 8 рёбер — стоп

Оставшиеся рёбра $EC(9)$, $BE(10)$, $AB(11)$, $FH(12)$, $HI(13)$, $EJ(14)$ замыкали бы циклы и пропускаются. Минимальное остовное дерево: $JI, BC, AD, HD, AF, DE, JD, BD$, суммарный вес $\sum \sigma = 36$.

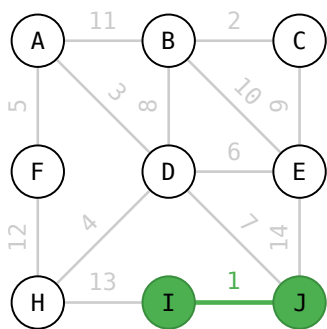
Упр. Почему алгоритм сойдётся (построит остов)?

Ответ. Каждое добавленное ребро уменьшает число компонент леса ровно на 1: начали с $|V|$ компонент, дерево получится после $|V| - 1$ добавлений. Застрясть алгоритм не может: пока компонент больше одной, в связном графе обязательно найдётся ребро между разными компонентами — и при просмотре списка оно не будет пропущено.

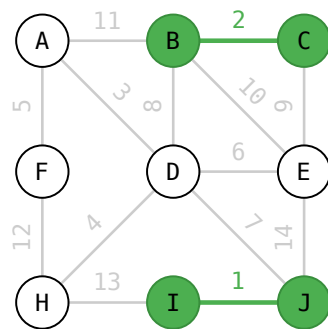
Работа алгоритма по шагам (добавленные рёбра — зелёные, текущее — оранжевое, пропущенные — бледно-серые):



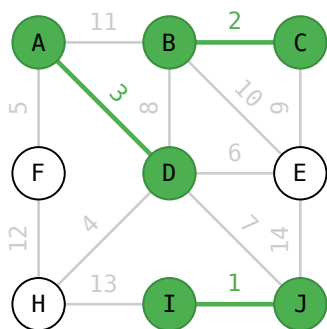
Шаг 0. Лес из изолированных вершин



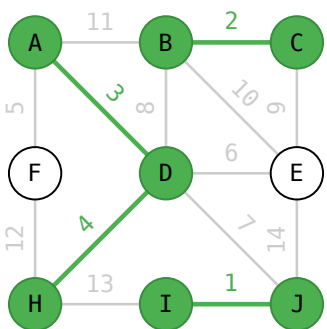
Шаг 1. $+JI(1)$



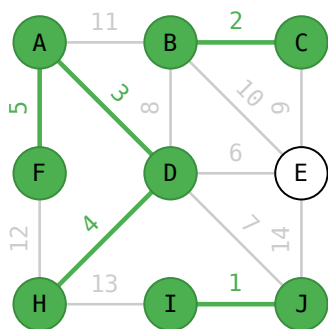
Шаг 2. $+BC(2)$



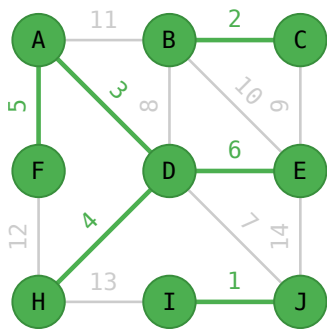
Шаг 3. $+AD(3)$



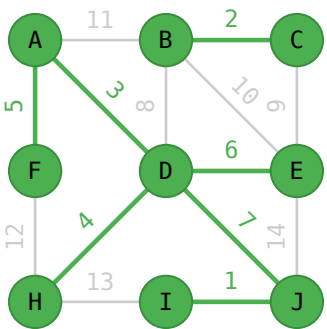
Шаг 4. $+HD(4)$



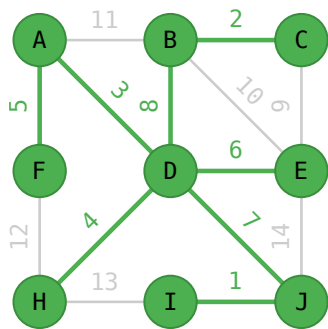
Шаг 5. $+AF(5)$



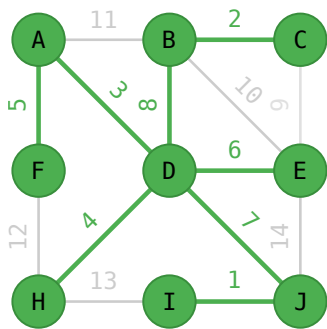
Шаг 6. $+DE(6)$



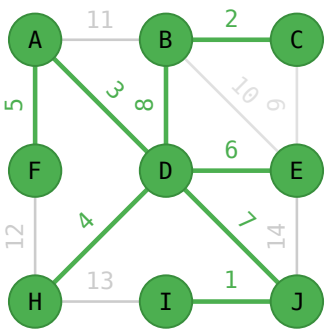
Шаг 7. $+JD(7)$



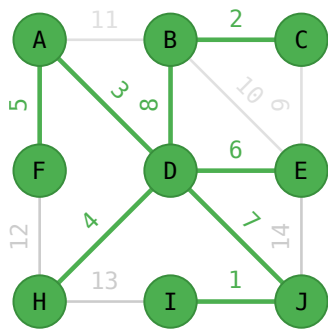
Шаг 8. $+BD(8)$: осто́в готов



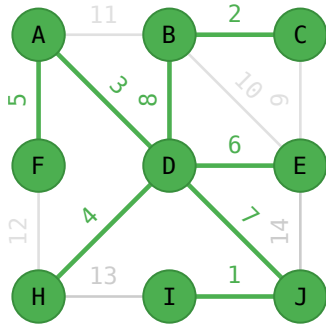
Шаг 9. $EC(9)$ — цикл, пропуск



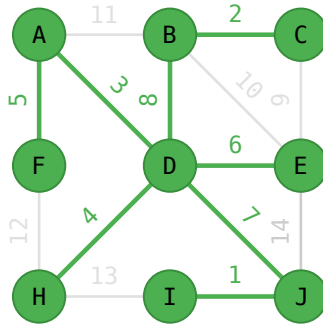
Шаг 10. $BE(10)$ — пропуск



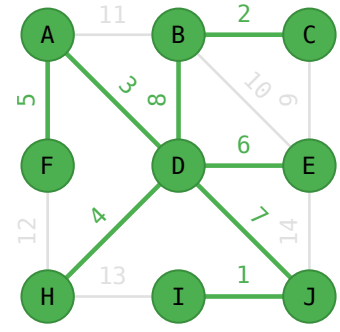
Шаг 11. $AB(11)$ — пропуск



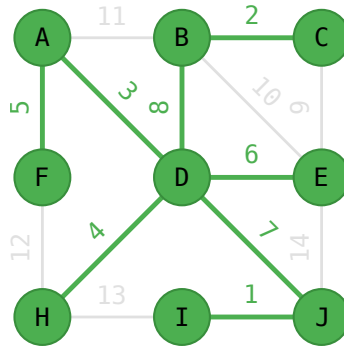
Шаг 12. $FH(12)$ — пропуск



Шаг 13. $HI(13)$ — пропуск



Шаг 14. $EJ(14)$ — пропуск



Шаг 15. Остов построен, $\Sigma = 36$

Алгоритм 12. Алгоритм поиска наибольшего паросочетания (см. стр. 71)

Дано: $G(V, E)$ — двудольный, $V = V_1 \sqcup V_2$.

Найти: наибольшее паросочетание $\hat{E}$.

Инициализация. Добавляем фиктивные вершины S и F :

$$\forall v_i \in V_1: \text{ дуга } S \rightarrow v_i; \quad \forall w_j \in V_2: \text{ дуга } w_j \rightarrow F,$$

а все рёбра графа ориентируем слева направо: $v_i \rightarrow w_j$.

Step. Пока существует путь $l: S \rightarrow \dots \rightarrow F$:

1. находим такой путь (любым обходом);
2. *инвертируем* на нём все рёбра между долями: пройденное ребро $v_i \rightarrow w_j$ разворачивается в $w_j \rightarrow v_i$ — развёрнутые рёбра и есть текущее паросочетание;
3. крайние дуги пути ($S \rightarrow v_i$ и $w_j \rightarrow F$) удаляем — вершины v_i и w_j насыщены.

Когда путей из S в F больше нет, развёрнутые рёбра $w_j \rightarrow v_i$ образуют наибольшее паросочетание.

Почему это работает: путь $S \rightarrow v_i \rightarrow w_j \rightarrow v_k \rightarrow \dots \rightarrow F$, проходящий по развёрнутому ребру $w_j \rightarrow v_k$, — это *увеличивающий путь*: он отбирает w_j у вершины v_k и тут же находит ей новую пару, а в конце насыщает одну новую вершину каждой доли. Число рёбер паросочетания каждый раз растёт на 1.

Пример. $V_1 = \{A, B, C, D\}$, $V_2 = \{\alpha, \beta, \gamma\}$, рёбра: $A\alpha, A\beta, B\alpha, B\beta, C\beta, C\gamma, D\alpha, D\gamma$.

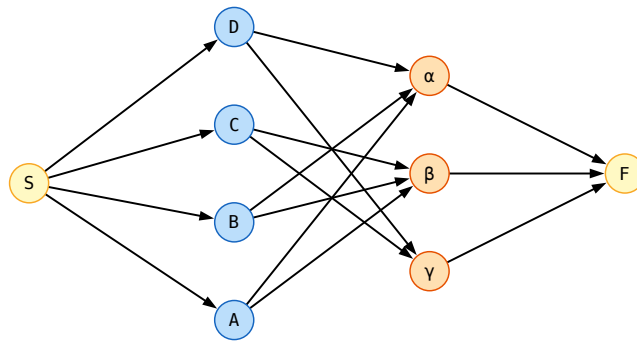


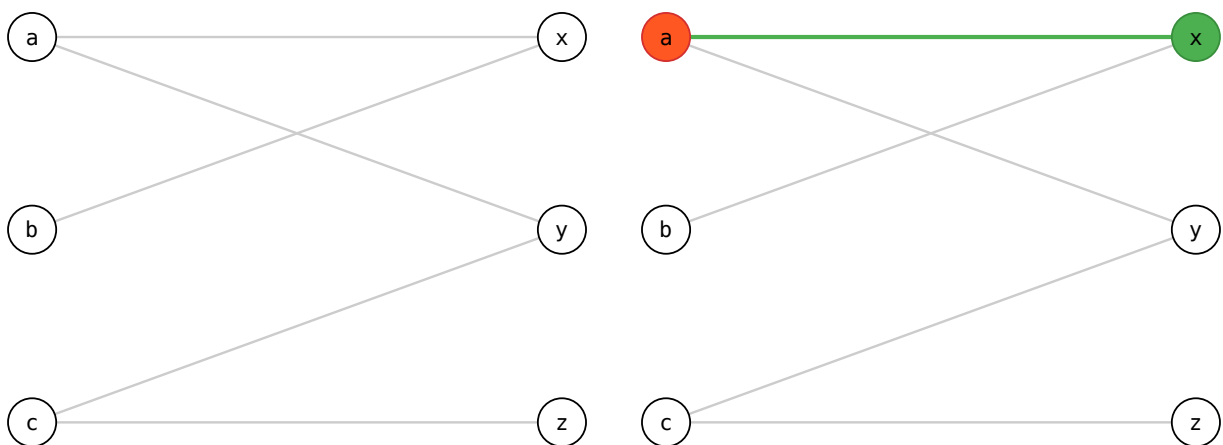
Рис. 83. Алг. наиб. паросочетания: S , доли $\{A, B, C, D\}$ и $\{\alpha, \beta, \gamma\}$, F

Находим пути и инвертируем (стрелки «туда-обратно»):

1. S, B, β, F — ребро $B\beta$ разворачивается: $\beta \rightarrow B$; паросочетание $\{B\beta\}$;
2. $S, C, \beta, B, \alpha, F$ — путь идёт по развёрнутому ребру $\beta \rightarrow B$ (отбираем β у B): теперь $\beta \rightarrow C$, $\alpha \rightarrow B$; паросочетание $\{C\beta, B\alpha\}$;
3. $S, A, \beta, C, \gamma, F$ — аналогично β переходит к A , а C получает γ ; паросочетание $\{A\beta, B\alpha, C\gamma\}$.

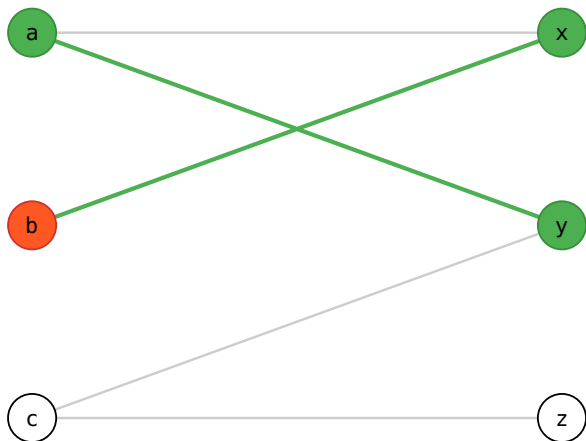
Итог: наибольшее паросочетание $\hat{E} = \{A\beta, B\alpha, C\gamma\}$, $|\hat{E}| = 3 = \min(|V_1|, |V_2|)$ — вершина D осталась свободной.

Работа алгоритма по шагам (текущий путь — оранжевый, развёрнутые рёбра паросочетания — зелёные, удалённые дуги — бледные):

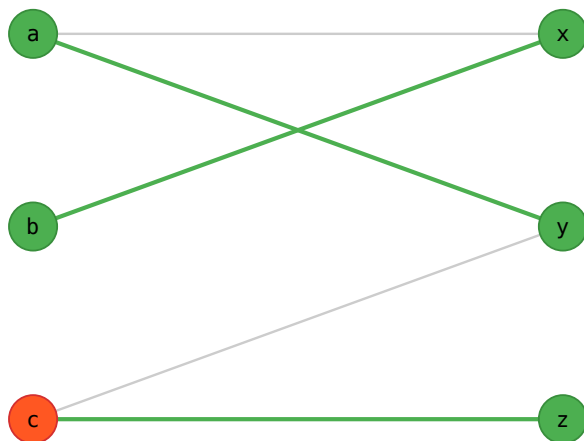


Шаг 0. Сеть с фиктивными S и F

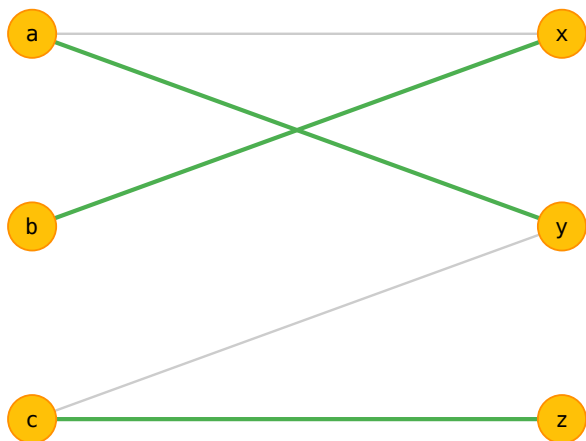
Шаг 1. Путь 1: S, B, β, F



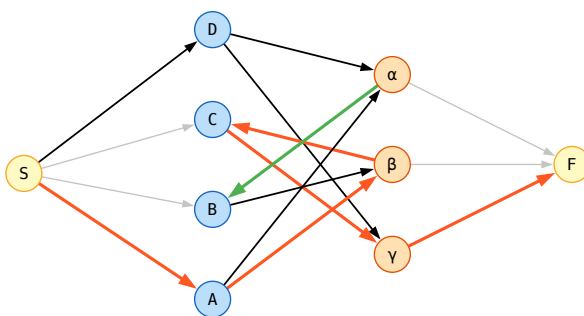
Шаг 2. $B\beta$ развёрнуто: $\{B\beta\}$



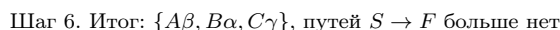
Шаг 3. Путь 2: $S, C, \beta, B, \alpha, F$



Шаг 4. β перешла к C , B получил α

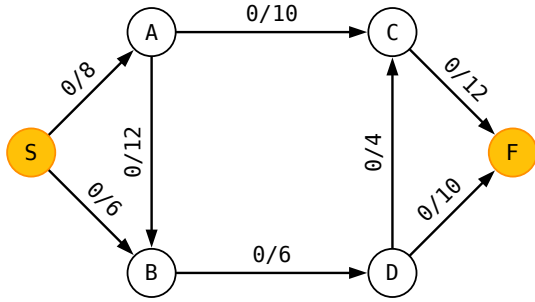


Шаг 5. Путь 3: $S, A, \beta, C, \gamma, F$

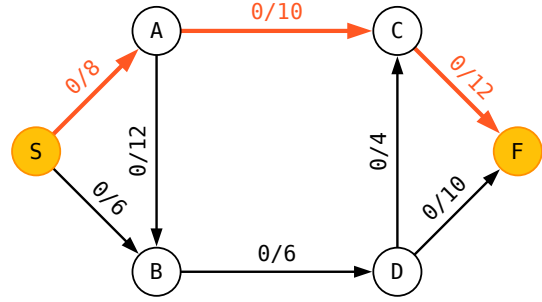


Ответ (идея, лемма Бержса). Паросочетание наибольшее $\Leftrightarrow$ для него нет увеличивающего пути (чередующегося пути между двумя свободными вершинами). Алгоритм останавливается ровно тогда, когда пути $S \rightarrow F$ нет, т. е. нет и увеличивающего пути. Если бы существовало паросочетание $\hat{E}'$ с $|\hat{E}'| > |\hat{E}|$, то в симметрической разности $\hat{E} \Delta \hat{E}'$ (каждая вершина задета не более чем двумя рёбрами — это набор путей и чётных циклов) нашлась бы компонента-путь, где рёбер $\hat{E}'$ больше, чем рёбер $\hat{E}$, — это и есть увеличивающий путь для $\hat{E}$. Противоречие.

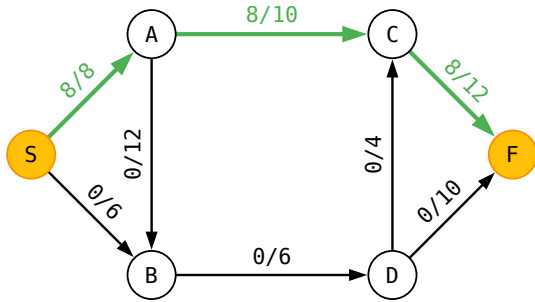
Пример. Сеть с вершинами S, A, B, C, D, F (та же, что в лекции 9). Подпись каждого ребра — f/σ (поток / пропускная способность). Дополняющий путь выделен оранжевым, рёбра с положительным потоком — зелёным. Пути ищем поиском в ширину (кратчайший по числу рёбер — вариант Эдмондса–Карпа).



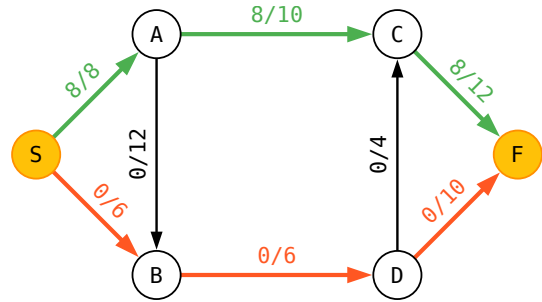
Шаг 0. Инициализация: $f \equiv 0$, $\tilde{\sigma} = \sigma$



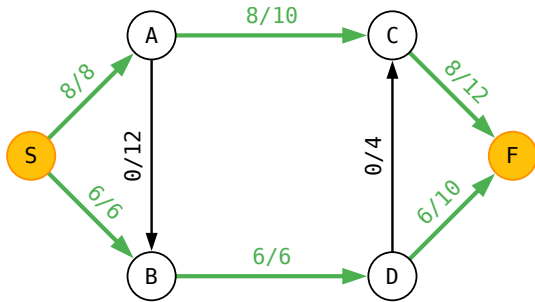
Шаг 1. Путь $S \rightarrow A \rightarrow C \rightarrow F$, $\delta = \min(2, 1, 3) = 1$



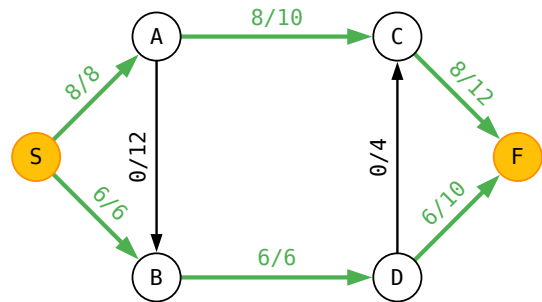
Шаг 2. Поток проведён: $|f| = 1$; $A \rightarrow C$ исчерпано



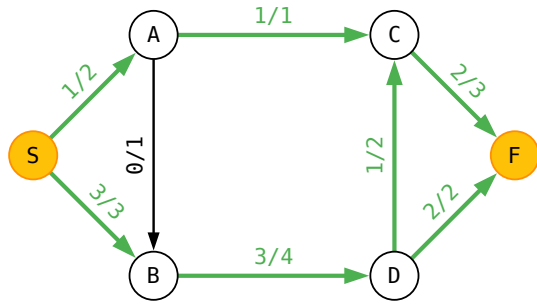
Шаг 3. Путь $S \rightarrow B \rightarrow D \rightarrow F$, $\delta = \min(3, 4, 2) = 2$



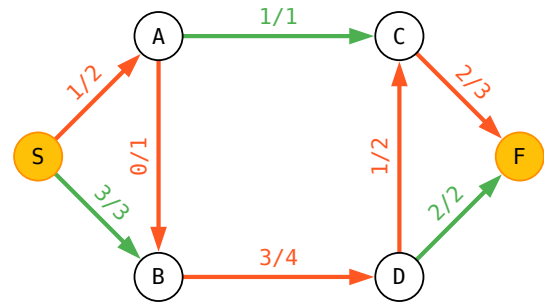
Шаг 4. $|f| = 3$; ребро $D \rightarrow F$ исчерпано



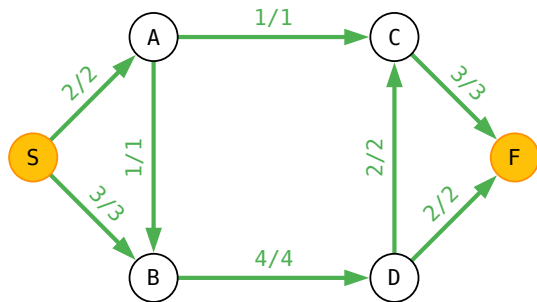
Шаг 5. Путь $S \rightarrow B \rightarrow D \rightarrow C \rightarrow F$, $\delta = 1$



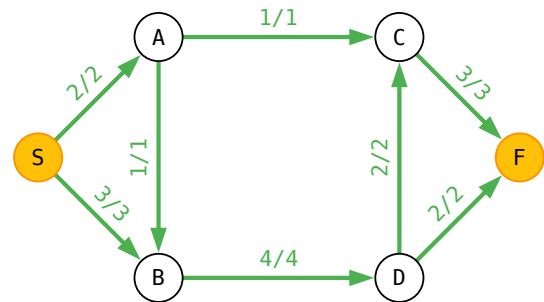
Шаг 6. $|f| = 4$; ребро $S \rightarrow B$ исчерпано



Шаг 7. Путь $S \rightarrow A \rightarrow B \rightarrow D \rightarrow C \rightarrow F$, $\delta = 1$



Шаг 8. $|f| = 5$; ребро $S \rightarrow A$ исчерпано



Шаг 9. Путь $S \rightarrow F$ в остаточной сети нет: $|f|_{\max} = 5$

Комментарии к шагам. На каждом шаге величина потока увеличивается на узкое место δ найденного пути: $1 + 2 + 1 + 1 = 5$. После шага 8 оба ребра из истока ($S \rightarrow A$ и $S \rightarrow B$) исчерпаны: в остаточной сети из S не выходит ни одного ребра, дополняющих путей нет — алгоритм останавливается.

Проверка через разрез. Набор рёбер $\tilde{E} = \{S \rightarrow A, S \rightarrow B\}$ — главный разрез ($\{S\}$ против остальных вершин) с пропускной способностью $\sigma(\tilde{E}) = 2 + 3 = 5$. Мощность любого потока не превосходит пропускной способности любого разреза, поэтому $|f| = 5 = \sigma(\tilde{E})$ — поток максимален, а разрез минимален (теорема Форда–Фалкерсона: $\max |f| = \min \sigma(\tilde{E})$).

Алгоритм 14. Метод проталкивания предпотока (Push-Relabel) (см. стр. 85)

Инициализация:

1. $h(S) := |V|$; $\forall u \in V \setminus \{S\}: h(u) := 0$.
2. $\tilde{f}_{\text{start}}: \forall v \in \text{Ch}(S): \tilde{f}(S, v) := \sigma(\overline{Sv}), \quad \tilde{f}(v, S) := -\sigma(\overline{Sv});$
 $\forall u, v \in V \setminus \{S\}: \tilde{f}(u, v) := 0$ (рёбра из истока насыщаются полностью).

Основной цикл: Пока $\exists v \in V \setminus \{S, F\}$ т. ч. $\varepsilon(v) > 0$ (*активная* вершина):

Push (проталкивание): если $\exists u \in \text{Ch}(v)$ (в остаточной сети) т. ч. $h(u) < h(v)$:

$$\delta_{\text{new}} := \min\{\varepsilon(v), \tilde{\sigma}(\overline{vu})\},$$

$$\tilde{f}_{\text{new}}(v, u) := \tilde{f}_{\text{old}}(v, u) + \delta_{\text{new}}, \quad \tilde{f}_{\text{new}}(u, v) := -\tilde{f}_{\text{new}}(v, u),$$

$$\tilde{\sigma}_{\text{new}}(v, u) := \tilde{\sigma}_{\text{old}}(v, u) - \delta_{\text{new}}, \quad \tilde{\sigma}_{\text{new}}(u, v) := \tilde{\sigma}_{\text{old}}(u, v) + \delta_{\text{new}}.$$

Здесь $\tilde{\sigma}(u, v) + \tilde{\sigma}(v, u) = \sigma(v, u)$ для $vu = e$ (остаточные пропускные способности, как в алгоритме Форда–Фалкерсона; проталкивание по обратному ребру означает *возврат* ранее протолкнутого потока).

Lift (подъём): если $\nexists u \in \text{Ch}(v)$ т. ч. $h(u) < h(v)$, то

$$h(v) := \min_{u \in \text{Ch}(v)} h(u) + 1$$

(минимум — по соседям в остаточной сети; после подъёма проталкивание из v снова возможно).

Выход: когда активных вершин не осталось, предпоток стал потоком (все внутренние эксцессы равны нулю), и этот поток — максимальный; $|f| = \varepsilon(F)$.

Пример. Сеть S, A, B, C, D, F с пропускными способностями (та же топология, что в лекциях 9–10):

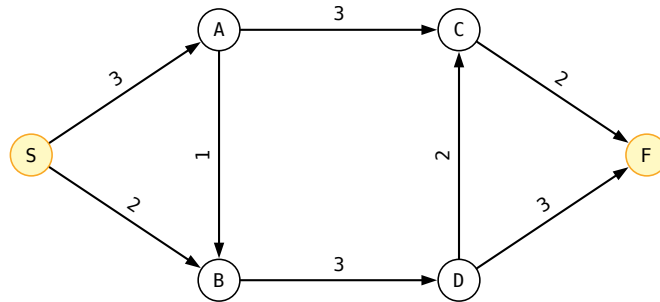


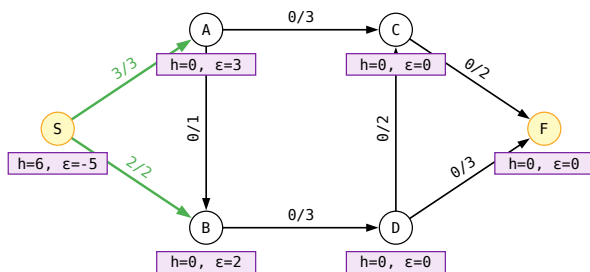
Рис. 84. Сеть Push-Relabel: S, A, B, C, D, F , σ на рёбрах

Протокол работы (каждая строка — одна операция; δ — объём проталкивания):

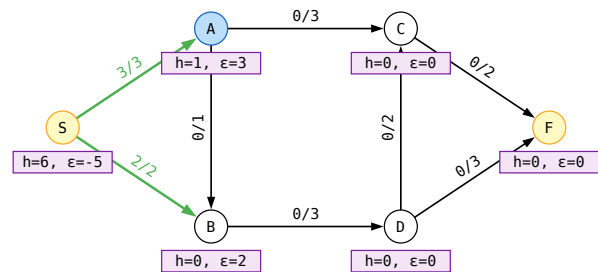
№	операция	δ	комментарий
0	инициализация	—	$S \rightarrow A, S \rightarrow B$ насыщены: $\varepsilon(A)=3, \varepsilon(B)=2$
1	lift $A: h(A):=1$	—	соседи A имеют $h = 0$, проталкивать некуда
2	push $A \rightarrow C$	3	$h(C) = 0 < 1; \varepsilon(A)=0, \varepsilon(C)=3$
3	lift $B: h(B):=1$	—	
4	push $B \rightarrow D$	2	$\varepsilon(B)=0, \varepsilon(D)=2$
5	lift $C: h(C):=1$	—	
6	push $C \rightarrow F$	2	ребро $C \rightarrow F$ насыщено; $\varepsilon(C)=1$
7	lift $C: h(C):=2$	—	вниз идти некуда: остаточный сосед — только A ($h=1$)
8	push $C \rightarrow A$	1	возврат по обратному ребру: $A \rightarrow C$ теперь $2/3; \varepsilon(A)=1$
9	lift $A: h(A):=2$	—	остаточные соседи: C ($h=2$), B ($h=1$), S ($h=6$)
10	push $A \rightarrow B$	1	$h(B) = 1 < 2; \varepsilon(B)=1$
11	push $B \rightarrow D$	1	$\varepsilon(D)=3$
12	lift $D: h(D):=1$	—	сосед F имеет $h = 0$
13	push $D \rightarrow F$	3	$\varepsilon(D)=0$; активных вершин нет — стоп

Итог: $|f| = \varepsilon(F) = 2 + 3 = 5$; поток по рёбрам: $S \rightarrow A = 3, S \rightarrow B = 2, A \rightarrow C = 2, A \rightarrow B = 1, B \rightarrow D = 3, C \rightarrow F = 2, D \rightarrow F = 3, D \rightarrow C = 0$ — условие балансировки выполнено во всех внутренних вершинах.

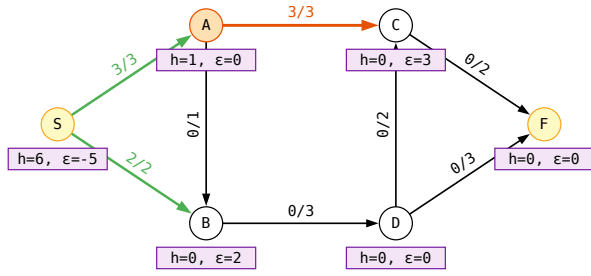
Работа алгоритма по шагам. Подпись ребра — $\tilde{f}/\sigma$; под каждой вершиной — её h и ε . Активная вершина операции — оранжевая, поднимаемая — синяя, ребро проталкивания — оранжевое, рёбра с положительным потоком — зелёные.



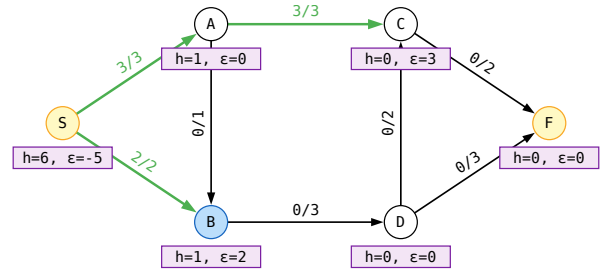
Шаг 0. Инициализация: рёбра из S насыщены



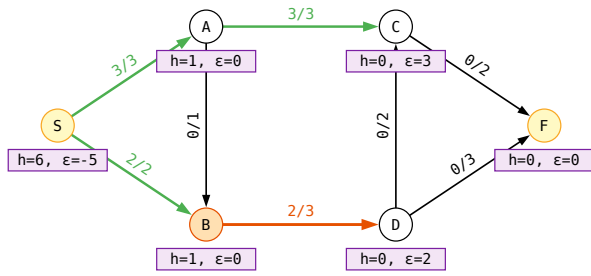
Шаг 1. Lift $A: h(A) := 1$



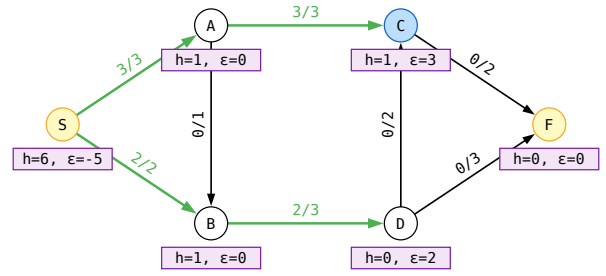
IIIar 2. Push $A \rightarrow C$, $\delta = 3$



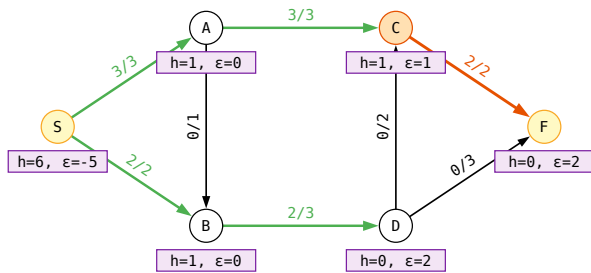
IIIar 3. Lift B : $h(B) := 1$



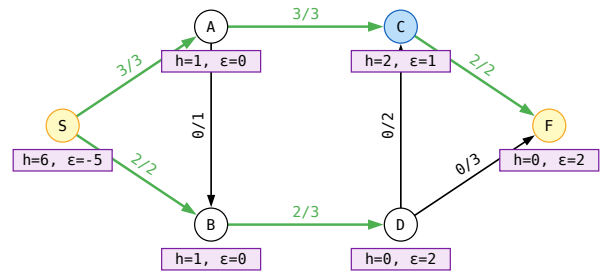
IIIar 4. Push $B \rightarrow D$, $\delta = 2$



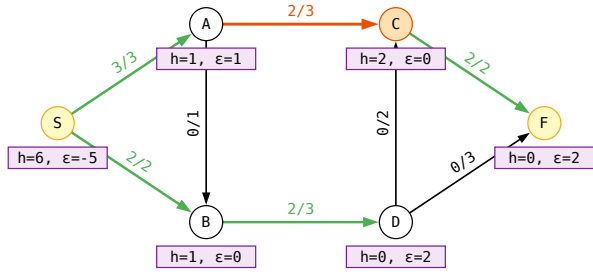
IIIar 5. Lift C : $h(C) := 1$



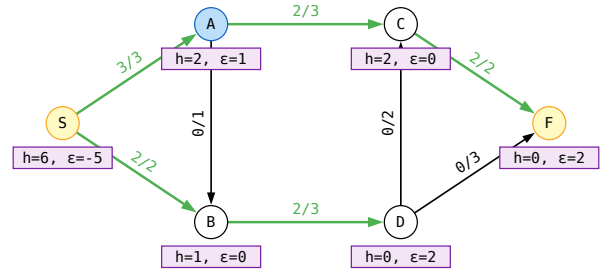
IIIar 6. Push $C \rightarrow F$, $\delta = 2$: ребро насыщено



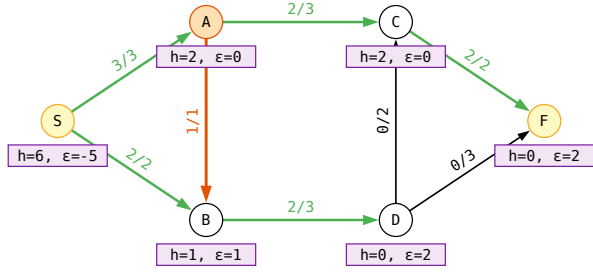
IIIar 7. Lift C : $h(C) := 2$ (выше A)



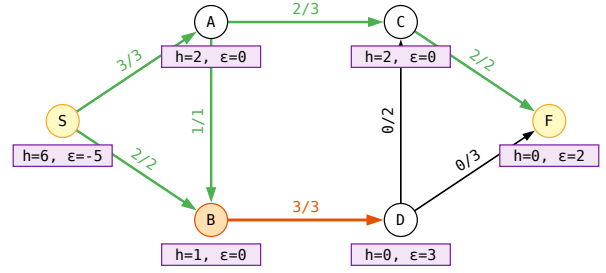
Шаг 8. Push $C \rightarrow A$, $\delta = 1$: возврат, $A \rightarrow C$ стало $2/3$



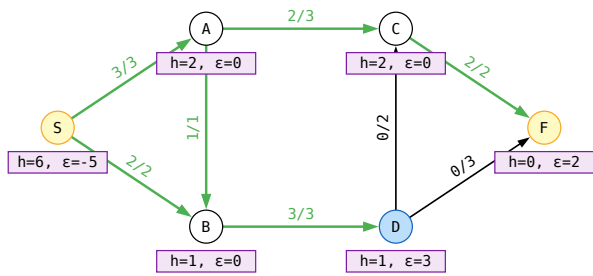
Шаг 9. Lift A: $h(A) := 2$



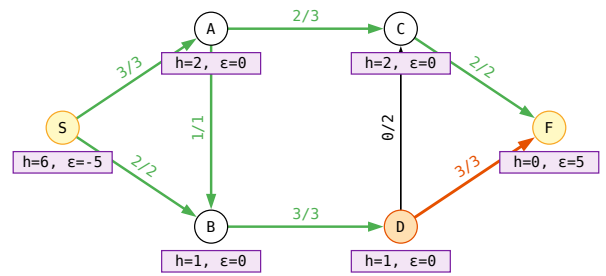
Шаг 10. Push $A \rightarrow B$, $\delta = 1$



Шаг 11. Push $B \rightarrow D$, $\delta = 1$



Шаг 12. Lift D: $h(D) := 1$



Шаг 13. Push $D \rightarrow F$, $\delta = 3$: $|f| = 5$, конец

Алгоритм 15. Алгоритм Дейкстры (см. стр. 103)

Требование: $\sigma: E \rightarrow \mathbb{R}_{\geq 0}$ (только неотрицательные веса!).

Инициализация: $\mu(\text{start}) := 0$, $\mu(v) := \infty$ для всех остальных вершин.

Шаг: Выбираем v с минимальной меткой $\mu(v)$ среди непосещённых.

Для каждого $w \in \text{Ch}(v)$:

$$\text{если } \mu(w) > \mu(v) + \sigma(\overline{vw}), \text{ то } \begin{cases} \mu(w) := \mu(v) + \sigma(\overline{vw}), \\ P(w) := v. \end{cases}$$

Это **динамическое программирование** (жадный выбор + оптимальная подструктура).

Почему жадный выбор корректен: пусть v — непосещённая вершина с минимальной меткой $\mu(v)$. Любой путь из старта в v обязан в какой-то момент впервые покинуть множество посещённых вершин по ребру $x \rightarrow y$ (y не посещена); его длина не меньше $\mu(y) \geq \mu(v)$, а дальнейшие рёбра *неотрицательны* — короче $\mu(v)$ не получится. Значит, метку $\mu(v)$ можно зафиксировать как окончательную. (Именно здесь существенно $\sigma \geq 0$: с отрицательными рёбрами «дешёвый обход позже» возможен, и жадность ломается — нужен Форд–Беллман.)

Пример (рис. p29-dijkstra-example-1): граф A, B, C, D . Start = A . Рёбра: BA (2), CB (5), AC (10), DC (3), AD (2), BD (7).

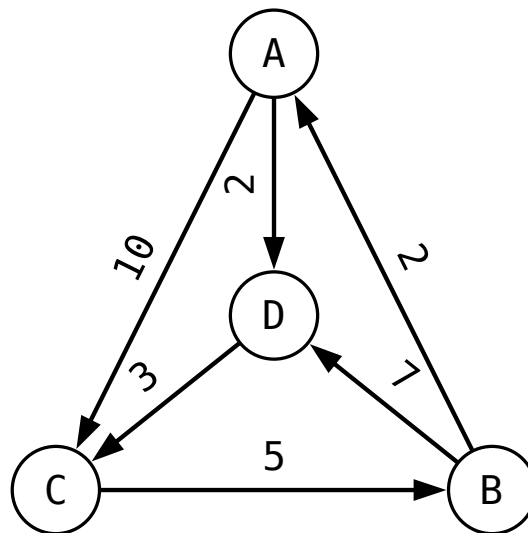


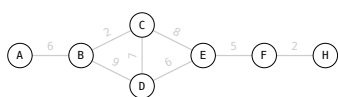
Рис. 85. Алг. Дейкстры: A, B, C, D ($AD=2, AC=10, DC=3, BD=7, CB=5, BA=2$)

	A	B	C	D
Иниц.	0	∞	∞	∞
A	—	∞	10, A	2, A
D	—	∞	5, D	—
C	—	10, C	—	—

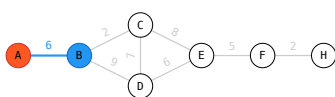
$D \rightarrow C$: $\mu(D) = 2$, $\sigma(DC) = 3$, итого $2 + 3 = 5 < 10$, обновляем.

Дерево кратчайших путей: $A \xrightarrow{2} D \xrightarrow{3} C \xrightarrow{5} B$.

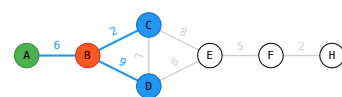
Работа алгоритма по шагам. Для разнообразия демонстрация выполнена на другом графе (7 вершин, рис. p9-example-0, веса проставлены детерминированно), старт — A .



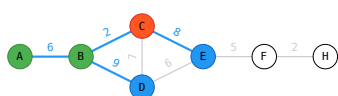
Шаг 0



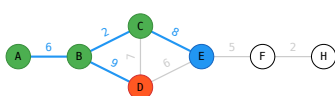
Шаг 1



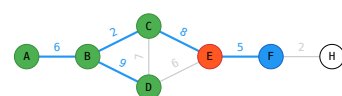
Шаг 2



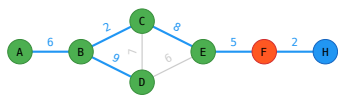
Шаг 3



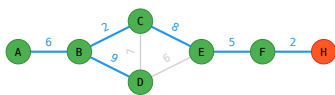
Шаг 4



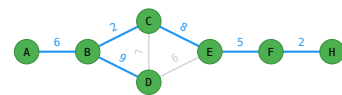
Шаг 5



Шаг 6



Шаг 7



Шаг 8

Алгоритм 16. Алгоритм Форда-Беллмана (см. стр. 104)

Допускается: $\sigma: E \rightarrow \mathbb{R}$ (в том числе отрицательные веса).

Инициализация: Произвольное упорядочивание рёбер. Стартовая вершина v : $\mu(v) := 0$, $\mu(w) := \infty$ для остальных.

Шаг (1 шаг = полный просмотр списка рёбер):

$\forall e_i \in \text{list}(E)$, $e_i = uv$:

$$\text{если } \mu(v) > \mu(u) + \sigma(e_i), \text{ то } \begin{cases} \mu(v) := \mu(u) + \sigma(e_i), \\ P(v) := u. \end{cases}$$

Конец алгоритма: шаги k и $k + 1$ дают одинаковый результат. Если n вершин, то алгоритм сойдётся не позже чем за $n - 1$ шагов.

Почему работает: индукцией по k проверяется инвариант — после k -го прохода по списку рёбер метка $\mu(w)$ не превосходит длины кратчайшего пути из старта в w , содержащего $\leq k$ рёбер (релаксация последнего ребра такого пути произойдёт не позже k -го прохода). Кратчайшие пути просты, т.е. содержат не более $n - 1$ ребра, поэтому $n - 1$ проходов

достаточно. Если же обновления случаются и на n -м проходе — в графе есть достижимый отрицательный цикл (так его и детектируют).

Пример (рис. p30-fb-example-1): граф A, B, C, D . $\text{Start} = B$. Отличие от Дейкстры: ребро BA имеет вес -2 .

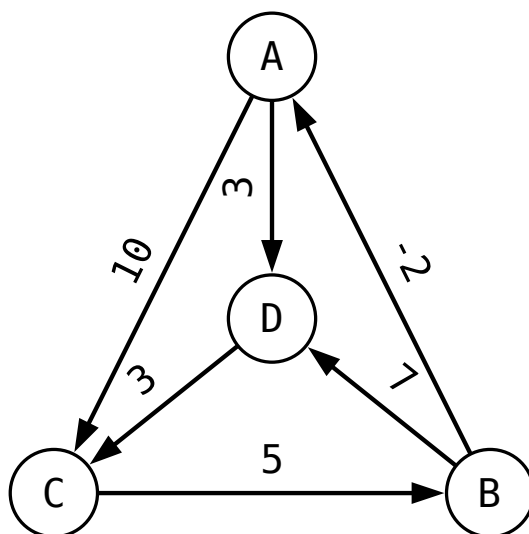


Рис. 86. Алг. Форда-Беллмана: тот же граф, $BA = -2$

Список рёбер с весами:

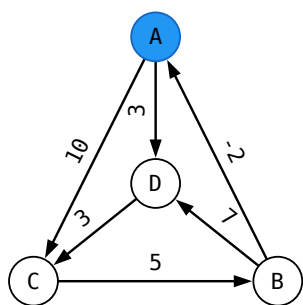
Ребро	Вес
AC	10
AD	2
BA	-2
BD	7
CB	5
DC	3

Протокол:

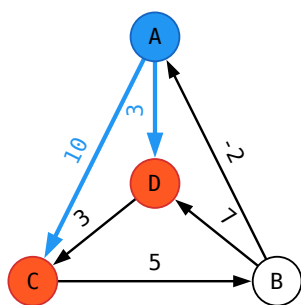
	A	B	C	D
Иниц.	∞	0	∞	∞
Шаг 1	-2, B	0	$7 + 3 = 10, D$	7, B
Шаг 2	-2, B	0	8, $A \rightarrow 3, D$	0, A

Корневое дерево кратчайших путей: $B \xrightarrow{-2} A \xrightarrow{2} D \xrightarrow{3} C$.

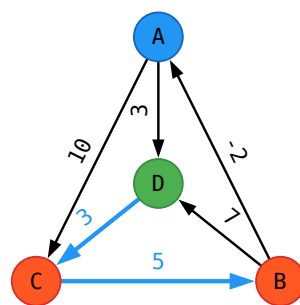
Работа алгоритма по шагам:



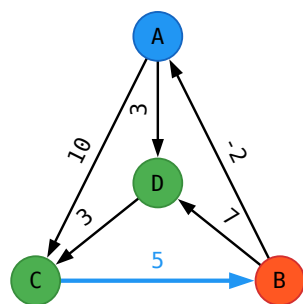
Шаг 0



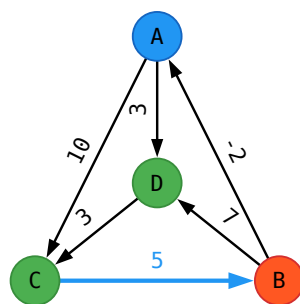
Шаг 1



Шаг 2



Шаг 3



Шаг 4

Алгоритм 17. Алгоритм Флойда-Уоршелла (см. стр. 106)

Задача $\odot$: кратчайшие пути для каждой пары $u, v \in V$.

Идея: аналог алгоритма Уоршелла (транзитивное замыкание), но для взвешенного графа.

Инициализация: Матрица $\mu(u, w)$ — по весам рёбер. Диагональ: $\mu(v, v) = 0$. Нет рёбра: $\mu(u, w) = \infty$.

Шаг: v — ведущая вершина. На i -м шаге выделяем i -ю строку и i -й столбец и смотрим пересечения:

$\forall (u, w)$, где $u \neq v, w \neq v$:

$$\text{если } \mu(u, w) > \mu(u, v) + \mu(v, w), \text{ то } \begin{cases} \mu(u, w) := \mu(u, v) + \mu(v, w), \\ P(w) := \begin{cases} v, & \text{если } P(v) = \emptyset, \\ P(v), & \text{если } P(v) \neq \emptyset. \end{cases} \end{cases}$$

Пример (рис. p31-fu-example-1): тот же граф A, B, C, D .

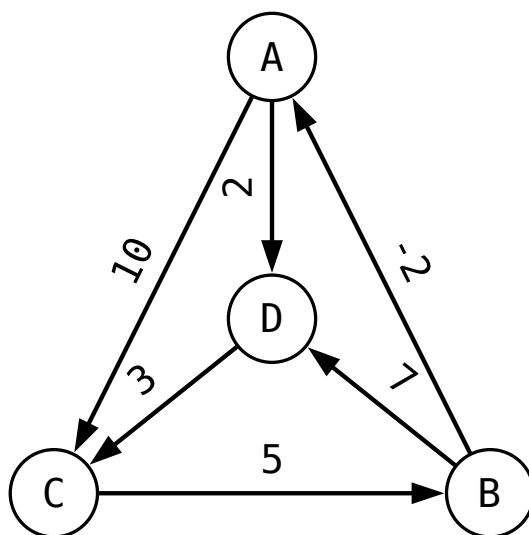


Рис. 87. Алг. Флойда-Уоршелла: тот же граф ($BA = -2$)

Шаг 0 (инициализация):

	A	B	C	D
A	0	∞	10	2
B	-2	0	∞	7
C	∞	5	0	∞
D	∞	∞	3	0

Шаг 1 (ведущая A): $\mu(B, C): \infty > (-2) + 10 = 8(A)$; $\mu(B, D): 7 > (-2) + 2 = 0(A)$.

	A	B	C	D
A	0	∞	10	2
B	-2	0	8, A	0, A
C	∞	5	0	∞
D	∞	∞	3	0

Шаг 2 (ведущая B): $\mu(C, A): \infty > 5 + (-2) = 3(B)$; $\mu(C, D): \infty > 5 + 0 = 5(B)$.

	A	B	C	D
A	0	∞	10	2
B	-2	0	8, A	0, A
C	3, B	5	0	5, B
D	∞	∞	3	0

Шаг 3 (ведущая C): $\mu(A, B): \infty > 10 + 5 = 15(C)$; $\mu(D, A): \infty > 3 + 3 = 6(C)$; $\mu(D, B): \infty > 3 + 5 = 8(C)$.

	A	B	C	D
A	0	15, C	10	2
B	-2	0	8, A	0, A
C	3, B	5	0	5, B
D	6, C	8, C	3	0

Шаг 4 (ведущая D): $\mu(A, B): 15 > 2 + 8 = 10(D)$ — путь $A \rightarrow D \rightarrow C \rightarrow B$; $\mu(A, C): 10 > 2 + 3 = 5(D)$; $\mu(B, C): 8 > 0 + 3 = 3(A)$ — путь $B \rightarrow A \rightarrow D \rightarrow C$, первая путевая точка A .

	A	B	C	D
A	0	10, D	5, D	2
B	-2	0	3, A	0, A
C	3, B	5	0	5, B
D	6, C	8, C	3	0

Восстановление: $A \rightarrow D$: путевая точка не записана (прямое ребро), путь $A \rightarrow D$, длина = 2. $C \rightarrow B$: путевая точка тоже не записана — кратчайший путь так и остался прямым ребром $C \rightarrow B$ длины 5 (обходы $C \rightarrow A \rightarrow \dots$ дороже: $\mu(C, A) = 3$, а ребра $A \rightarrow B$ нет, $\mu(A, B) = 10$). $A \rightarrow B$: $P_{AB} = D$, далее $\mu(D, B): P_{DB} = C$, далее прямое ребро $C \rightarrow B$; итог $A \xrightarrow{2} D \xrightarrow{3} C \xrightarrow{5} B$, длина = 10.

Циклы отрицательного суммарного веса $\Leftrightarrow$ отрицательные значения на главной диагонали.

Алгоритм 18. Алгоритм Флойда (см. стр. 110)

$G(V, E)$ — слабо связный ориентированный граф, $\sigma: E \rightarrow \mathbb{R}$.

Алгоритм аналогичен Флойду–Уоршеллу. На каждом шаге выделяем ведущую вершину (строку и столбец), проверяем все пересечения.

Хранение пути: для пары (u, w) храним P_{uw} — первую путевую точку. Для прямого ребра $A \rightarrow B$ — ничего хранить не надо. Остаток пути: $B \rightarrow \dots \rightarrow L$.

Если $\mu(u, w) > \mu(u, v) + \mu(v, w)$, то $\mu(u, w) := \mu(u, v) + \mu(v, w)$, $P_{uw} := P_{uv}$ (или v , если P_{uv} пуст).

Пример (рис. p32-f-example-1): граф A, B, C, D .

Список рёбер с весами:

Ребро	Вес
AB	1
BC	1
BD	3
CA	-2
CD	1
DA	4

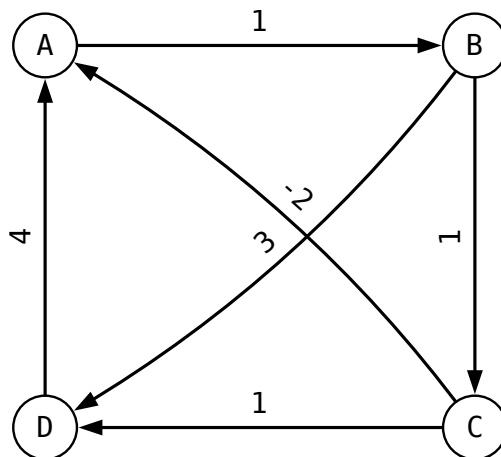


Рис. 88. Алг. Флойда: A, B, C, D ($AB=1$, $BC=1$, $BD=3$, $CA=-2$, $CD=1$, $DA=4$)

Шаг 0 (инициализация):

	A	B	C	D
A	0	1	∞	∞
B	∞	0	1	3
C	-2	∞	0	1
D	4	∞	∞	0

Шаг 1 (ведущая A): выделяем строку A и столбец A . Обновления: $\mu(C, B): \infty > (-2) + 1 = -1$; $\mu(D, B): \infty > 4 + 1 = 5$.

	A	B	C	D
A	0	1	∞	∞
B	∞	0	1	3
C	-2	-1, A	0	1
D	4	5, A	∞	0

Шаг 2 (ведущая B): обновления: $\mu(A, C): \infty > 1 + 1 = 2$; $\mu(A, D): \infty > 1 + 3 = 4$; $\mu(D, C): \infty > 5 + 1 = 6$.

	A	B	C	D
A	0	1	2, B	4, B
B	∞	0	1	3
C	-2	-1, A	0	1
D	4	5, A	6, A	0

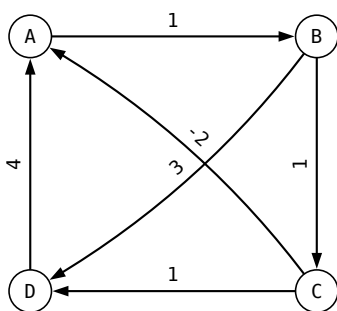
Шаг 3 (ведущая C): обновления: $\mu(A, D): 4 > 2 + 1 = 3$; $\mu(B, A): \infty > 1 + (-2) = -1$; $\mu(B, D): 3 > 1 + 1 = 2$.

	A	B	C	D
A	0	1	2, B	3, B
B	-1, C	0	1	2, C
C	-2	-1, A	0	1
D	4	5, A	6, A	0

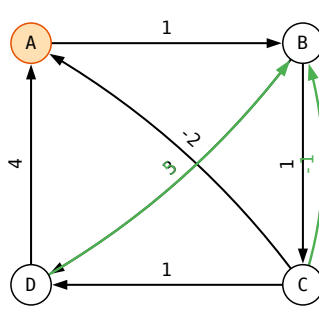
Шаг 4 (ведущая D): нет изменений (все пути через D дороже найденных).

	A	B	C	D
A	0	1	2, B	3, B
B	-1, C	0	1	2, C
C	-2	-1, A	0	1
D	4	5, A	6, A	0

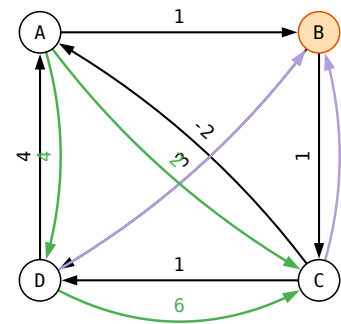
Работа алгоритма по шагам. Ведущая вершина — оранжевая; найденные на текущем шаге кратчайшие обходы — зелёные дуги с новой длиной, найденные ранее — светло-фиолетовые:



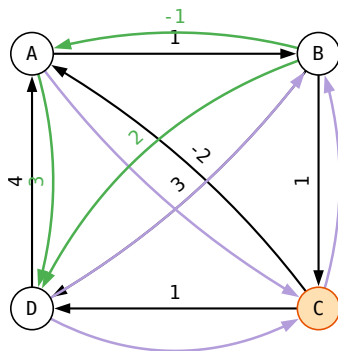
Шаг 0. Исходный граф



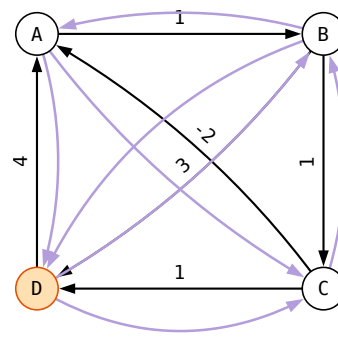
Шаг 1. Через A: $CB=-1$, $DB=5$



Шаг 2. Через B: $AC=2$, $AD=4$, $DC=6$



Шаг 3. Через C: $AD=3$, $BA=-1$, $BD=2$



Шаг 4. Через D обновлений нет — итог

Восстановление путей:

$A \rightarrow D$: $P_{AD} = B$. $A \xrightarrow{1} B$, $P_{BD} = C$, $B \xrightarrow{1} C$, P_{CD} пуста, $C \xrightarrow{1} D$. Итого: $A \xrightarrow{1} B \xrightarrow{1} C \xrightarrow{1} D$, длина = 3.

$C \rightarrow B$: $P_{CB} = A$. $C \xrightarrow{-2} A$, P_{AB} пуста, $A \xrightarrow{1} B$. Итого: $C \xrightarrow{-2} A \xrightarrow{1} B$, длина = -1.

Циклы отрицательного суммарного веса $\Leftrightarrow$ отрицательные значения на главной диагонали.

Почему: диагональный элемент $\mu(v, v)$ — длина кратчайшего замкнутого маршрута через v ; он станет отрицательным в точности тогда, когда через v проходит цикл отрицательного веса. Здесь диагональ нулевая: например, цикл $A \rightarrow B \rightarrow C \rightarrow A$ весит $1 + 1 - 2 = 0$, отрицательных циклов нет.

Алгоритм 19. Алгоритм Джонсона (см. стр. 112)

Преобразование графа к виду с неотрицательными весами рёбер с сохранением набора кратчайших путей.

$G(V, E)$ — слабо связный, $\sigma: E \rightarrow \mathbb{R}$.

Идея. Дейкстра быстрее Флойда ($|V|$ запусков Дейкстры дешевле при разреженных графах), но не работает с отрицательными рёбрами. Джонсон «выпрямляет» веса потенциалами так, что все веса становятся неотрицательными, а кратчайшие пути не меняются.

Шаг 1. Джонсоновский потенциал.

Добавляем фиктивную вершину S : $\forall u \in V: e_u = Su, \sigma(e_u) = 0$.

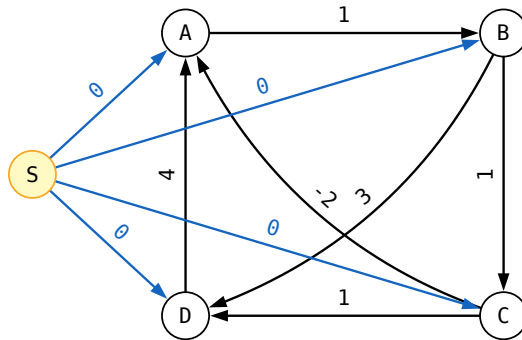


Рис. 89. Алг. Джонсона: тот же граф + фиктивная S (синие дуги веса 0)

Запускаем **алгоритм Форда–Беллмана** от S (Дейкстру пока нельзя — веса ещё отрицательные); $h(u) :=$ найденное расстояние от S до u .

Результат Форда–Беллмана (потенциалы):

$$h(A) = -2, \quad h(B) = -1, \quad h(C) = h(D) = 0.$$

(Проверка: до C и D дешевле всего прямые нулевые дуги из S , т.е. $h(C) = h(D) = 0$; до A выгоден путь $S \rightarrow C \rightarrow A$ весом $0 + (-2) = -2$; до B — путь $S \rightarrow C \rightarrow A \rightarrow B$ весом $0 - 2 + 1 = -1$.)

Шаг 2. Пересчёт весов.

Новая весовая функция:

$$\tilde{\sigma}(\overline{uv}) = \sigma(\overline{uv}) + h(u) - h(v) \geq 0.$$

Почему $\tilde{\sigma} \geq 0$: h — корректные расстояния от S , поэтому для любого ребра uv выполнено неравенство треугольника $h(v) \leq h(u) + \sigma(uv)$, что и означает $\sigma(uv) + h(u) - h(v) \geq 0$.

Ребро	σ	$\tilde{\sigma}$	Вычисление
AB	1	0	$1 + (-2) - (-1) = 0$
BC	1	0	$1 + (-1) - 0 = 0$
BD	3	2	$3 + (-1) - 0 = 2$
CA	-2	0	$-2 + 0 - (-2) = 0$
CD	1	1	$1 + 0 - 0 = 1$
DA	4	6	$4 + 0 - (-2) = 6$

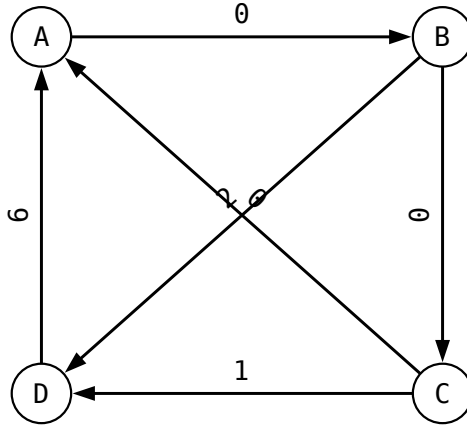


Рис. 90. Перевешенный граф: все $\tilde{\sigma} \geq 0$

Все $\tilde{\sigma} \geq 0$ — можно применять Дейкстру!

Шаг 3. Дейкстра от каждой вершины с весами $\tilde{\sigma}$.

Матрица расстояний в весах $\tilde{\sigma}$:

$\text{dist}_{\tilde{\sigma}}$	A	B	C	D
A	0	0	0	1
B	0	0	0	1
C	0	0	0	1
D	6	6	6	0

(Например, из D : единственный выход — дуга DA веса 6, дальше всё бесплатно по нулевым дугам $A \rightarrow B \rightarrow C$; поэтому вся строка D равна 6.)

Шаг 4. Обратный пересчёт.

Для получения истинных расстояний:

$$\text{dist}_{\sigma}(u, v) = \text{dist}_{\tilde{\sigma}}(u, v) - h(u) + h(v).$$

Почему пути не изменились: для любого пути $u = x_0 \rightarrow x_1 \rightarrow \dots \rightarrow x_k = v$ его новая длина равна

$$\sum_i \tilde{\sigma}(x_i x_{i+1}) = \sum_i \sigma(x_i x_{i+1}) + h(u) - h(v)$$

— телескопическая сумма: все промежуточные потенциалы сокращаются. Все пути из u в v сдвинулись на *одну и ту же* константу $h(u) - h(v)$, поэтому порядок путей по длине (и сами кратчайшие пути) сохранился, а формула пересчёта очевидна.

Итоговая матрица истинных расстояний ($\text{dist}_{\sigma} = \text{dist}_{\tilde{\sigma}} - h(u) + h(v)$):

dist_{σ}	A	B	C	D
A	0	1	2	3
B	-1	0	1	2
C	-2	-1	0	1
D	4	5	6	0

Например, $\text{dist}_{\sigma}(D, A) = 6 - h(D) + h(A) = 6 - 0 - 2 = 4$; $\text{dist}_{\sigma}(B, A) = 0 - (-1) + (-2) = -1$. Матрица совпала с результатом алгоритма Флойда выше — проверка пройдена. ✓

Свойства:

1. Набор кратчайших путей для G в терминах σ и $\tilde{\sigma}$ совпадает (пути не меняются, только длины) — телескопическая сумма выше.

2. $\forall e \in E: \tilde{\sigma}(e) \geq 0$ (неравенство треугольника для корректных потенциалов).
3. Если $\tilde{\sigma}(e) < 0$ для какого-то ребра — значит, потенциалы найдены неправильно.

Все задачи

Задача 1. (см. стр. 12)

Доказать, что асимметричность $\Rightarrow$ арефлексивность.

Решение. Асимметричность: $\forall a, b: f(a, b) = 1 \Rightarrow f(b, a) = 0$. Возьмём $b = a$: если бы нашлось a с $f(a, a) = 1$, то по асимметричности (при $b = a$) получили бы $f(a, a) = 0$ — противоречие. Значит $\forall a: f(a, a) = 0$, т.е. отношение арефлексивно. ■

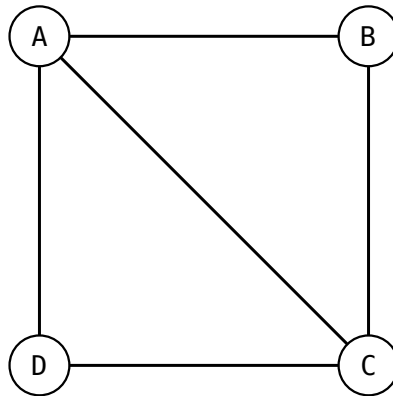
Задача 2. (см. стр. 17)

Доказать: если $G(V, E)$ — связный граф, $|V| = n$, $|E| = n - 1$, то G — дерево.

Решение. Дерево — связный ациклический граф, и связность дана, поэтому достаточно показать, что в G нет циклов. Предположим противное: в G есть цикл. Удалим любое ребро e этого цикла — граф $G' = G - e$ останется связным (любой путь, проходивший через e , можно провести в обход по оставшейся части цикла), и в нём $n - 2$ ребра. Будем повторять: пока в графе остаются циклы, удаляем по одному ребру цикла, сохраняя связность. Процесс конечен и закончится связным ациклическим графом (деревом) T на тех же n вершинах, в котором меньше чем $n - 1$ рёбер. Но любой связный граф на n вершинах содержит не менее $n - 1$ ребра (см. лемму о связности ниже: каждая новая вершина при построении связного графа требует хотя бы одного нового ребра) — противоречие. Значит, циклов в G не было, и G — дерево. ■

Задача 3. Эйлеровость графа (см. стр. 39)

Условие. Дан связный граф на вершинах A, B, C, D с рёбрами AB, BC, CD, DA, AC . Выясните, является ли он эйлеровым или полуэйлеровым, и при возможности укажите эйлеров путь/цикл.



Решение. Применим критерий Эйлера: связный граф эйлеров (есть эйлеров цикл) $\Leftrightarrow$ все степени вершин чётны; полуэйлеров (есть эйлеров путь, но не цикл) $\Leftrightarrow$ ровно две вершины нечётной степени.

Подсчитаем степени:

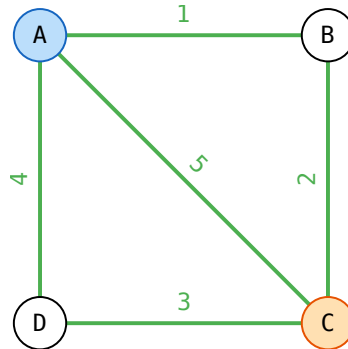
$$\deg A = 3 \text{ (} B, D, C \text{)}, \quad \deg B = 2 \text{ (} A, C \text{)}, \quad \deg C = 3 \text{ (} B, D, A \text{)}, \quad \deg D = 2 \text{ (} C, A \text{)}.$$

Нечётную степень имеют ровно две вершины — A и C . Следовательно, граф *полуэйлеров*: эйлерова цикла нет, но есть эйлеров путь, причём он обязан начинаться и заканчиваться в нечётных вершинах A и C .

Построим путь (например, алгоритмом Флёрри, не выбирая мост, пока есть альтернатива):

$$A \rightarrow B \rightarrow C \rightarrow D \rightarrow A \rightarrow C.$$

Он проходит по всем пяти рёбрам AB, BC, CD, DA, AC ровно по одному разу (номера на рёбрах — порядок прохождения):



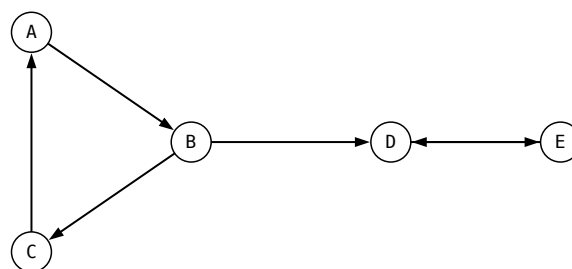
Ответ. Граф полуэйлеров; эйлеров путь $A-B-C-D-A-C$ (из A в C). Эйлерова цикла нет, так как есть вершины нечётной степени.

Задача 4. Компоненты сильной связности: алгоритм Косарайю (см. стр. 40)

Условие. Дан ориентированный граф на вершинах A, B, C, D, E с дугами

$$A \rightarrow B, B \rightarrow C, C \rightarrow A, B \rightarrow D, D \rightarrow E, E \rightarrow D.$$

Найдите компоненты сильной связности алгоритмом Косарайю.



Решение. Алгоритм Косарайю состоит из трёх шагов: (1) обход в глубину исходного графа G с записью времён выхода; (2) построение транспонированного графа G^T (все дуги развёрнуты); (3) обход G^T в порядке убывания времён выхода — каждое дерево обхода даёт одну компоненту.

Шаг 1. DFS по G из A . Порядок выхода (по возрастанию):

$$C(1), E(2), D(3), B(4), A(5).$$

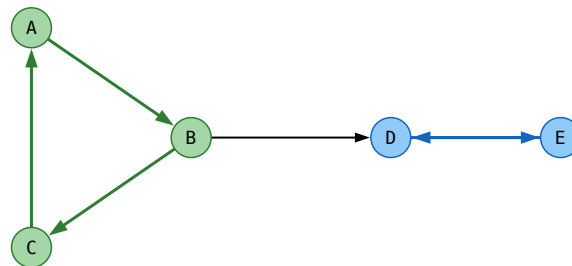
Порядок убывания времён выхода: A, B, D, E, C .

Шаг 2. Транспонированный граф G^T :

$$B \rightarrow A, C \rightarrow B, A \rightarrow C, D \rightarrow B, E \rightarrow D, D \rightarrow E.$$

Шаг 3. DFS по G^T в порядке A, B, D, E, C .

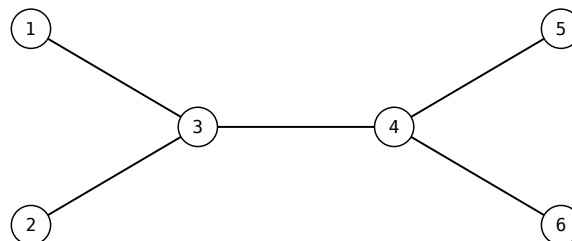
- Из A : $A \rightarrow C \rightarrow B \rightarrow A$ — собрана компонента $\{A, B, C\}$.
- Следующая непосещённая (по порядку) — D : $D \rightarrow E \rightarrow D$ — компонента $\{D, E\}$.



Ответ. Две компоненты сильной связности: $\{A, B, C\}$ (цикл $A \rightarrow B \rightarrow C \rightarrow A$) и $\{D, E\}$ (цикл $D \rightarrow E \rightarrow D$). Дуга $B \rightarrow D$ соединяет компоненты в одном направлении, поэтому в одну компоненту они не объединяются.

Задача 5. Код Прюфера: прямая и обратная задача (см. стр. 51)

Условие. (а) Постройте код Прюфера для дерева на вершинах $1, \dots, 6$ с рёбрами $1-3, 2-3, 3-4, 4-5, 4-6$. (б) Восстановите дерево по коду Прюфера $(3, 3, 4, 4)$.



Решение (а). На каждом шаге удаляем лист с наименьшим номером и записываем

номер его единственного соседа; повторяем, пока не останется 2 вершины (для дерева на n вершинах код имеет длину $n - 2 = 4$).

1. Листья $\{1, 2, 5, 6\}$. Наименьший — 1, сосед 3. Записываем 3.
2. Листья $\{2, 5, 6\}$. Наименьший — 2, сосед 3. Записываем 3.
3. Теперь 3 стала листом (соединена только с 4). Листья $\{3, 5, 6\}$. Наименьший — 3, сосед 4. Записываем 4.
4. Листья $\{5, 6\}$. Наименьший — 5, сосед 4. Записываем 4. Осталось 2 вершины (4 и 6) — стоп.

Код Прюфера: $(3, 3, 4, 4)$.

Решение (б). Обратный алгоритм. Список степеней: каждая вершина встречается в коде $(\deg - 1)$ раз, поэтому степень = (число вхождений в код) + 1. Для кода $(3, 3, 4, 4)$ и вершин $1 \dots 6$:

$$\deg 3 = 3, \deg 4 = 3, \deg 1 = \deg 2 = \deg 5 = \deg 6 = 1.$$

На каждом шаге берём наименьший лист (вершину степени 1, не входящую в оставшийся код) и соединяем с первым элементом кода:

1. Код $(3, 3, 4, 4)$; наименьший лист 1 $\rightarrow$ ребро $1-3$; убираем 3 из кода.
2. Код $(3, 4, 4)$; наименьший лист 2 $\rightarrow$ ребро $2-3$; убираем 3.
3. Код $(4, 4)$; теперь 3 стала листом, наименьший лист 3 $\rightarrow$ ребро $3-4$; убираем 4.
4. Код (4) ; наименьший лист 5 $\rightarrow$ ребро $5-4$; убираем 4.
5. Код пуст; остались вершины 4 и 6 $\rightarrow$ ребро $4-6$.

Восстановленные рёбра: $1-3$, $2-3$, $3-4$, $4-5$, $4-6$ — совпадают с исходным деревом из пункта (а) (см. рисунок выше), как и должно быть.

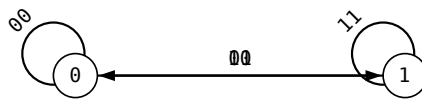
Ответ. Код Прюфера дерева — $(3, 3, 4, 4)$; обратное восстановление даёт то же дерево.

Задача 6. Последовательность де Брёйна над алфавитом 0 и 1 (см. стр. 52)

Условие. Постройте граф де Брёйна для слов длины 2 над алфавитом $\{0, 1\}$ и найдите кратчайшую циклическую последовательность, содержащую все 2-буквенные слова в качестве подстрок.

Решение. В графе де Брёйна $B(\{0, 1\}, 2)$ вершины — слова длины 1 (то есть 0 и 1), а дуги — слова длины 2: дуга из u в v помечена словом uv . Получаем 4 дуги:

$$0 \xrightarrow{00} 0, \quad 0 \xrightarrow{01} 1, \quad 1 \xrightarrow{11} 1, \quad 1 \xrightarrow{10} 0.$$



Искомая последовательность — это *эйлеров цикл* (он проходит по каждой дуге-слову ровно один раз). У каждой вершины полустепени захода и исхода равны 2, граф связан — эйлеров цикл существует. Один из них:

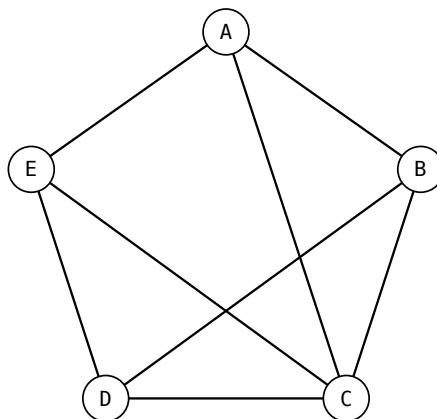
$$0 \xrightarrow{00} 0 \xrightarrow{01} 1 \xrightarrow{11} 1 \xrightarrow{10} 0,$$

что даёт циклическую последовательность **0011**. Читая её по кругу, получаем все 2-буквенные слова: 00, 01, 11, 10.

Ответ. Кратчайшая циклическая последовательность — 0011 (длина $4 = 2^2$), содержит все четыре слова длины 2.

Задача 7. Гамильтоновость: теорема Дирака (см. стр. 52)

Условие. Граф на 5 вершинах A, B, C, D, E задан как полный граф K_5 без двух рёбер AD и BE . Является ли он гамильтоновым? Постройте гамильтонов цикл.



Решение. Теорема Дирака: если в графе на $n \geq 3$ вершинах степень каждой вершины не меньше $n/2$, то граф гамильтонов. Здесь $n = 5$, порог $n/2 = 2,5$. Подсчитаем степени (в K_5 каждая вершина имела степень 4; удаление AD и BE уменьшает на единицу степени A, D, B, E):

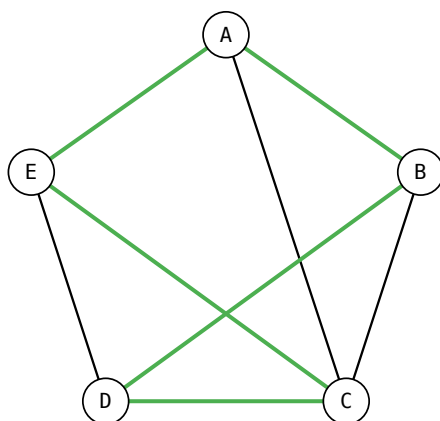
$$\deg A = \deg B = \deg D = \deg E = 3, \quad \deg C = 4.$$

Все степени $\geq 3 > 2,5$, поэтому по теореме Дирака граф гамильтонов.

Построим гамильтонов цикл, обходя все вершины по одному разу и избегая отсутствующих рёбер AD, BE :

$$A - B - D - C - E - A.$$

Проверим рёбра: AB , BD , DC , CE , EA — все присутствуют (среди них нет ни AD , ни BE).



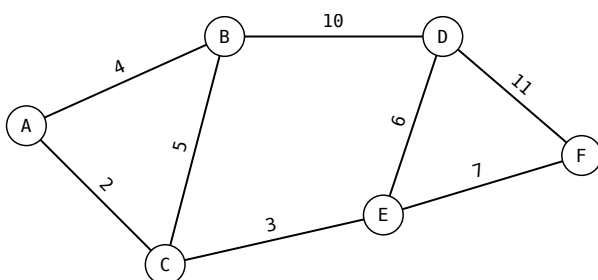
Ответ. Граф гамильтонов (условие Дирака выполнено); гамильтонов цикл $A-B-D-C-E-A$.

Задача 8. Минимальное остовное дерево: алгоритм Прима (см. стр. 58)

Условие. Дан неориентированный взвешенный граф на вершинах A, B, C, D, E, F с рёбрами

$$AB = 4, \quad AC = 2, \quad BC = 5, \quad BD = 10, \quad CE = 3, \quad ED = 6, \quad EF = 7, \quad DF = 11.$$

Постройте минимальное остовное дерево алгоритмом Прима, начиная с вершины A . Укажите порядок включения рёбер и суммарный вес.



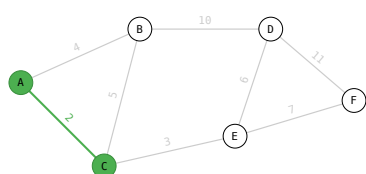
Решение. Алгоритм Прима вырабатывает дерево из одной вершины, добавляя на каждом шаге ребро наименьшего веса, ведущее из уже построенного дерева в ещё не включённую вершину.

1. Дерево $\{A\}$. Из A выходят $AC = 2$ и $AB = 4$; минимум — $AC = 2$, включаем вершину C .

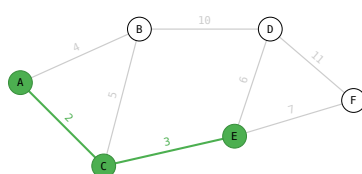
2. Дерево $\{A, C\}$. Доступны $CE = 3$, $AB = 4$, $BC = 5$; минимум — $CE = 3$, включаем E .
3. Дерево $\{A, C, E\}$. Доступны $AB = 4$, $BC = 5$, $ED = 6$, $EF = 7$; минимум — $AB = 4$, включаем B .
4. Дерево $\{A, C, E, B\}$. Доступны $ED = 6$, $EF = 7$, $BD = 10$; минимум — $ED = 6$, включаем D .
5. Дерево $\{A, C, E, B, D\}$. Доступны $EF = 7$, $DF = 11$; минимум — $EF = 7$, включаем F . Дерево охватило все вершины.

Порядок включения рёбер: $AC(2)$, $CE(3)$, $AB(4)$, $ED(6)$, $EF(7)$.

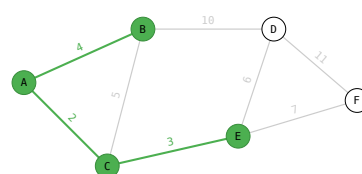
Шаги построения:



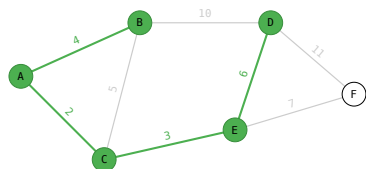
+AC(2)



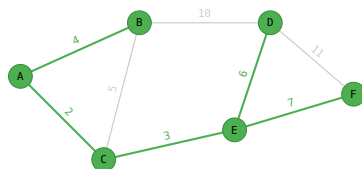
+CE(3)



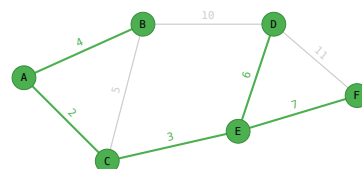
+AB(4)



+ED(6)



+EF(7)



Остов построен

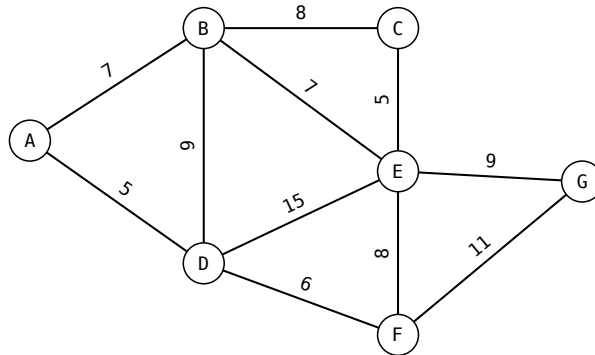
Ответ. Минимальное остовное дерево состоит из рёбер AC, CE, AB, ED, EF ; его суммарный вес равен $2 + 3 + 4 + 6 + 7 = 22$.

Задача 9. Минимальное остовное дерево: алгоритм Краскала (см. стр. 65)

Условие. Дан неориентированный взвешенный граф на вершинах A, B, C, D, E, F, G с рёбрами

$$AB = 7, \quad AD = 5, \quad BC = 8, \quad BD = 9, \quad BE = 7, \quad CE = 5, \\ DE = 15, \quad DF = 6, \quad EF = 8, \quad EG = 9, \quad FG = 11.$$

Постройте минимальное остовное дерево алгоритмом Краскала. Укажите порядок включения рёбер и суммарный вес.



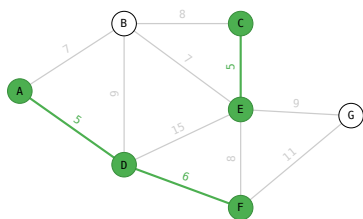
Решение. Краскал сортирует рёбра по возрастанию веса и добавляет ребро, если его концы лежат в разных компонентах (что отслеживается системой непересекающихся множеств) — иначе ребро замкнуло бы цикл.

Отсортированный список рёбер:

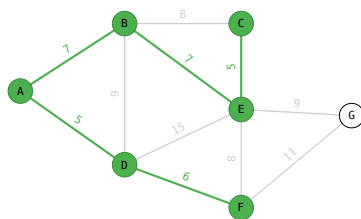
$AD(5), CE(5), DF(6), AB(7), BE(7), BC(8), EF(8), BD(9), EG(9), FG(11), DE(15)$.

1. $AD(5)$ — компоненты $\{A\}, \{D\}$ различны, добавляем $\Rightarrow \{A, D\}$.
2. $CE(5)$ — добавляем $\Rightarrow \{C, E\}$.
3. $DF(6)$ — F присоединяется к $\{A, D\}$, добавляем $\Rightarrow \{A, D, F\}$.
4. $AB(7)$ — B присоединяется, добавляем $\Rightarrow \{A, B, D, F\}$.
5. $BE(7)$ — сливаем $\{A, B, D, F\}$ и $\{C, E\}$, добавляем $\Rightarrow \{A, B, C, D, E, F\}$.
6. $BC(8)$ — B и C уже в одной компоненте: *цикл, пропускаем*.
7. $EF(8)$ — цикл, пропускаем.
8. $BD(9)$ — цикл, пропускаем.
9. $EG(9)$ — G присоединяется, добавляем. Все 7 вершин в одной компоненте, набрано $6 = 7 - 1$ рёбер — остов построен.
10. $FG(11), DE(15)$ — цикл, пропускаем.

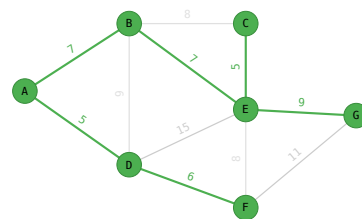
Ключевые шаги построения:



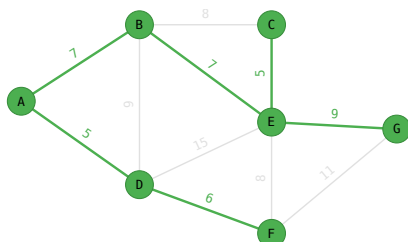
$+AD, +CE, +DF$



$+AB, +BE$



пропуски BC, EF, BD ; $+EG$



Остов построен, $\Sigma = 39$

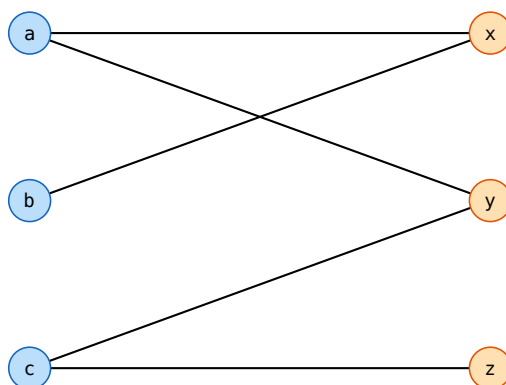
Ответ. Остовное дерево: рёбра AD, CE, DF, AB, BE, EG ; суммарный вес $5 + 5 + 6 + 7 + 7 + 9 = 39$.

Задача 10. Наибольшее паросочетание в двудольном графе (см. стр. 79)

Условие. Дан двудольный граф с долями $\{a, b, c\}$ и $\{x, y, z\}$ и рёбрами

$$a-x, \quad a-y, \quad b-x, \quad c-y, \quad c-z.$$

Найдите наибольшее паросочетание (алгоритм Куна, поиск увеличивающих путей).

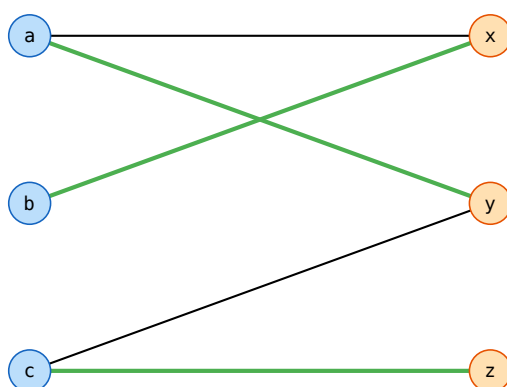


Решение. Паросочетание — набор попарно несмежных рёбер. Алгоритм Куна по очереди

пытается насытить каждую вершину левой доли, ища увеличивающий путь (чередующий путь из свободной вершины в свободную).

1. a : свободно ребро $a-x \Rightarrow$ паросочетание $\{a-x\}$.
2. b : единственное ребро $b-x$, но x занят a . Ищем увеличивающий путь: $b-x$, x занят a , у a есть альтернатива $a-y$ (y свободна). Переключаем: $a-y$, $b-x$. Паросочетание $\{a-y, b-x\}$.
3. c : ребро $c-y$ ведёт к занятому y ; пробуем $c-z$ (z свободна) $\Rightarrow$ добавляем $c-z$. Паросочетание $\{a-y, b-x, c-z\}$.

Все три вершины левой доли насыщены — паросочетание совершенное, его размер 3 (больше быть не может, так как доли по 3 вершины).



Ответ. Наибольшее паросочетание: $\{a-y, b-x, c-z\}$, размер 3 (совершенное).

Задача 11. Хроматический многочлен цикла и графа с подвеской (см. стр. 101)

Условие. Найдите хроматические многочлены: (а) цикла C_4 на вершинах $A-B-C-D-A$; (б) графа G на вершинах A, B, C, D с рёбрами AB, BC, CA (треугольник) и подвешенным ребром CD .

Решение (а). По доказанной формуле для цикла $P_{C_n}(t) = (t-1)^n + (-1)^n(t-1)$. При $n = 4$:

$$P_{C_4}(t) = (t-1)^4 + (t-1) = t^4 - 4t^3 + 6t^2 - 4t + 1 + t - 1 = t^4 - 4t^3 + 6t^2 - 3t.$$

Проверка: $P_{C_4}(2) = 16 - 32 + 24 - 6 = 2$ (чётный цикл имеет ровно 2 правильные раскраски в 2 цвета); $P_{C_4}(3) = 81 - 108 + 54 - 9 = 18$.

Решение (б). Треугольник — это K_3 , поэтому $P_{K_3}(t) = t(t-1)(t-2)$. Подвешенная вершина D соединена единственным ребром CD : по лемме 1 (висячая вершина) добавление висячего ребра умножает многочлен на $(t-1)$:

$$P_G(t) = P_{K_3}(t) \cdot (t-1) = t(t-1)(t-2)(t-1) = t(t-1)^2(t-2).$$

Проверка: $P_G(3) = 3 \cdot 4 \cdot 1 = 12$ — треугольник красится $3! = 6$ способами, а D — 2 способами, итого $6 \cdot 2 = 12$.

Ответ. (а) $P_{C_4}(t) = t^4 - 4t^3 + 6t^2 - 3t$; (б) $P_G(t) = t(t-1)^2(t-2)$.

Задача 12. Кратчайшие пути: алгоритм Дейкстры (см. стр. 107)

Условие. Дан неориентированный взвешенный граф на вершинах S, A, B, C, D, F с рёбрами

$$SA = 2, SB = 5, AB = 1, AC = 4, BD = 3, CD = 1, CF = 6, DF = 2.$$

Найдите кратчайшие расстояния от вершины S до всех остальных алгоритмом Дейкстры.

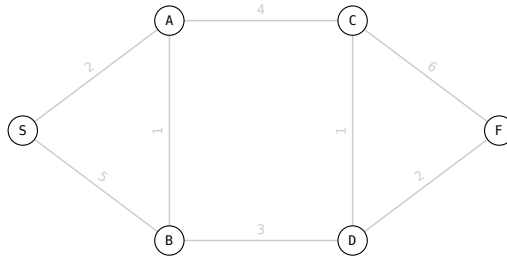


Рис. 91. Граф задачи (условие)

Решение. Дейкстра поддерживает множество вершин с окончательно найденным расстоянием. На каждом шаге фиксируется вершина с наименьшей текущей оценкой d , после чего релаксируются её рёбра.

1. Старт: $d[S] = 0$, остальные $= \infty$. Фиксируем S : $d[A] = 2$, $d[B] = 5$.
2. Минимум среди нефиксированных — $A(2)$. Фиксируем A : $d[B] = \min(5, 2 + 1) = 3$, $d[C] = 2 + 4 = 6$.
3. Минимум — $B(3)$. Фиксируем B : $d[D] = 3 + 3 = 6$.
4. Минимум — $C(6)$. Фиксируем C : $D(6 + 1 = 7 > 6$ — без изменений), $d[F] = 6 + 6 = 12$.
5. Минимум — $D(6)$. Фиксируем D : $d[F] = \min(12, 6 + 2) = 8$.
6. Фиксируем $F(8)$. Все вершины обработаны.

Ответ. $d[S] = 0$, $d[A] = 2$, $d[B] = 3$, $d[C] = 6$, $d[D] = 6$, $d[F] = 8$.

Задача 13. Кратчайшие пути с отрицательными рёбрами: Форд—Беллман (см. стр. 108)

Условие. Дан *ориентированный* взвешенный граф на вершинах S, A, B, C, D, F с дугами $S \rightarrow A = 3$, $S \rightarrow B = 2$, $A \rightarrow C = 2$, $B \rightarrow A = -2$, $C \rightarrow D = 1$, $B \rightarrow D = 5$, $D \rightarrow F = 3$, $C \rightarrow F = 6$.

Найдите кратчайшие расстояния от S алгоритмом Форда—Беллмана.

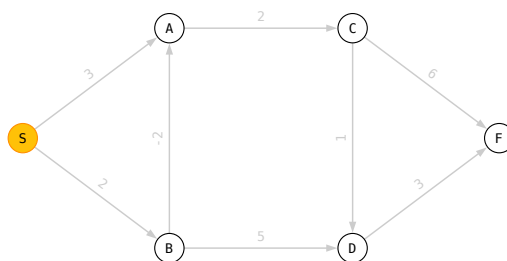


Рис. 92. Граф задачи (условие)

Решение. Форд—Беллман выполняет до $|V| - 1$ итераций, на каждой релаксируя все дуги; отрицательные веса допустимы (при отсутствии отрицательных циклов).

- **Итерация 1.** $d[A] = 3$, $d[B] = 2$, $d[C] = 5$; затем дуга $B \rightarrow A$ улучшает $d[A] = 2 - 2 = 0$; далее $d[D] = 6$, $d[F] = 9$.
- **Итерация 2.** Так как $d[A]$ упало до 0, пересчёт даёт $d[C] = 0 + 2 = 2$, $d[D] = 2 + 1 = 3$, $d[F] = 3 + 3 = 6$.
- **Итерация 3.** Изменений нет — расстояния стабилизировались.

Заметим: кратчайший путь до A идёт *не* напрямую $S \rightarrow A$ (вес 3), а через $S \rightarrow B \rightarrow A$ (вес $2 + (-2) = 0$). Именно из-за отрицательной дуги здесь нужен Форд—Беллман, а не Дейкстра.

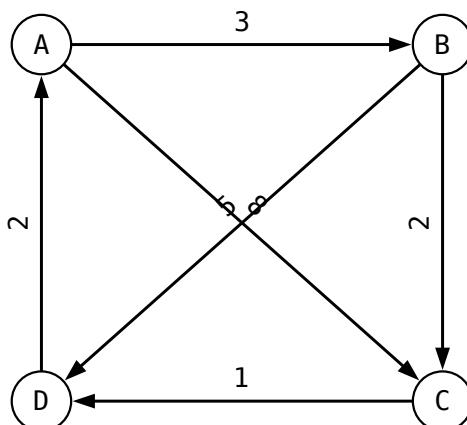
Ответ. $d[S] = 0$, $d[A] = 0$, $d[B] = 2$, $d[C] = 2$, $d[D] = 3$, $d[F] = 6$.

Задача 14. Кратчайшие пути между всеми парами: алгоритм Флойда (см. стр. 114)

Условие. Дан ориентированный взвешенный граф на вершинах A, B, C, D с дугами

$$A \rightarrow B = 3, A \rightarrow C = 8, B \rightarrow C = 2, B \rightarrow D = 5, C \rightarrow D = 1, D \rightarrow A = 2.$$

Найдите кратчайшие расстояния между всеми парами вершин алгоритмом Флойда.



Решение. Флойд последовательно разрешает в качестве промежуточных вершины A, B, C, D . На шаге k обновляем $d[i][j] \leftarrow \min(d[i][j], d[i][k] + d[k][j])$.

Начальная матрица $D^{(0)}$ (на диагонали 0, отсутствующие дуги — ∞):

$D^{(0)}$	A	B	C	D
A	0	3	8	∞
B	∞	0	2	5
C	∞	∞	0	1
D	2	∞	∞	0

Через A: обновляются пути из D ($D \rightarrow A = 2$): $D \rightarrow B = 2 + 3 = 5$, $D \rightarrow C = 2 + 8 = 10$.

Через B: $A \rightarrow C = \min(8, 3 + 2) = 5$, $A \rightarrow D = 3 + 5 = 8$; $D \rightarrow C = \min(10, 5 + 2) = 7$.

Через C: $A \rightarrow D = \min(8, 5 + 1) = 6$, $B \rightarrow D = \min(5, 2 + 1) = 3$.

Через D: замыкаются обратные пути: $B \rightarrow A = 3 + 2 = 5$, $C \rightarrow A = 1 + 2 = 3$, $C \rightarrow B = 1 + 2 + 3 = 6$ и т. д.

Итоговая матрица кратчайших расстояний $D^{(4)}$:

$D^{(4)}$	A	B	C	D
A	0	3	5	6
B	5	0	2	3
C	3	6	0	1
D	2	5	7	0

Проверка: $B \rightarrow A$ идёт по маршруту $B \rightarrow C \rightarrow D \rightarrow A = 2 + 1 + 2 = 5$; $C \rightarrow B = C \rightarrow D \rightarrow A \rightarrow B = 1 + 2 + 3 = 6$ — совпадает с таблицей.

Ответ. Матрица кратчайших расстояний $D^{(4)}$ приведена выше.